

BUILD IT TO UNDERSTAND IT

Build an LLM Inference Engine in C++

Through Challenges

From a blank project to a working Transformer that loads a real model and generates text — forged one challenge at a time, in modern C++.

Hatem M.

Build an LLM Inference Engine in C++ — Through Challenges. Hatem M.

This is an educational text for programmers already fluent in C++ and comfortable with algorithms. It develops a complete CPU-first inference engine for Llama-family Transformer models, from project scaffolding to a full forward pass and a running, model-loading engine, entirely through hands-on challenges.

Reference solutions are provided for learning and verification. Type the code, run the tests, and treat each green test suite as a save point.

Typeset with Xe_ΛT_EX using TeX Gyre Pagella, TeX Gyre Heros, and DejaVu Sans Mono. Code is set with `listings`; boxes with `tcolorbox`; diagrams with `TikZ`.

First Edition, 2026.

Contents

Preface	vii
How to Read This Book	x
0 Foundation & Setup	1
0.1 The Project Skeleton	2
0.2 Proof of Life: a Test Framework	7
0.3 Errors and Logging: the Engine’s Nervous System	12
0.4 One Command, Green and Clean	17
Unit 0 — Milestone check	20
1 Tensor & Memory	21
1.1 The Tensor	21
1.2 Aligned Memory and the Arena	32
1.3 Views and Reshape (Zero-Copy)	36
1.4 Zero-Copy Discipline	43
Unit 1 — Milestone check	47
2 Math Kernels	48
2.1 Elementwise Operations	49
2.2 Activations: SiLU, GELU, SwiGLU	53
2.3 RMSNorm	58
2.4 Numerically-Stable Softmax	63
Unit 2 — Milestone check	68

3	Matrix Multiplication	70
3.1	Naive GEMM, a Reference Oracle, and a GFLOP/s Harness	71
3.2	Loop Reordering: ijk to ikj	76
3.3	Cache Blocking (Tiling)	79
3.4	Multi-threading Across Cores	83
3.5	SIMD and Packing	88
	Unit 3 — Milestone check	94
4	The Tokenizer	96
4.1	Bytes and the Vocabulary	97
4.2	BPE Merges and Encoding	101
4.3	Pre-tokenization and the Full Encoder	106
4.4	Decoding	110
4.5	Round-trip Fidelity	113
	Unit 4 — Milestone check	117
5	The Transformer	119
5.1	Token Embeddings and the Model Skeleton	120
5.2	Rotary Position Encoding (RoPE)	126
5.3	Attention with Grouped-Query Heads	130
5.4	The Transformer Block and Feed-Forward Network	136
5.5	The Full Forward Pass	141
	Unit 5 — Milestone check	146
6	Mixture of Experts	147
6.1	The Router	148
6.2	The Mixture-of-Experts FFN	152
6.3	Grouped Dispatch and Proven Sparsity	158
	Unit 6 — Milestone check	164

7	The KV Cache	165
7.1	The Cache Structure	166
7.2	Cached Attention	170
7.3	Prefill, Decode, and Exact Equivalence	176
	Unit 7 — Milestone check	181
8	Flash Attention	182
8.1	Online Softmax	183
8.2	Tiled Flash Attention	188
8.3	Exactness at Scale	193
	Unit 8 — Milestone check	197
9	Quantization	198
9.1	Block Quantization (Q8_0)	199
9.2	4-bit Packing and Quantized Linear	203
9.3	The Compression–Fidelity Trade	209
	Unit 9 — Milestone check	213
10	Model Loading	215
10.1	Reading GGUF: Header and Metadata	216
10.2	Tensor Descriptors and Memory Mapping	222
10.3	Assemble and Run a Model	228
	Unit 10 — Milestone check	235
11	Sampling and Generation	236
11.1	Greedy and Temperature Sampling	237
11.2	Top- k and Top- p Sampling	241
11.3	The Generation Loop	246
	Unit 11 — Milestone check	250

12 Speculative Decoding	252
12.1 Drafting and Parallel Verification	253
12.2 Distribution-Preserving Acceptance	258
12.3 The Speculative Loop	262
Unit 12 — Milestone check	266
13 Serving and Batching	268
13.1 Batched Decode	269
13.2 Paged KV Cache	275
13.3 Continuous Batching	280
Unit 13 — Milestone check	286
14 Performance	287
14.1 Profiling and the Roofline	288
14.2 Threading the Hot Loops	292
14.3 Engine Alive	298
Unit 14 — Milestone check	303
15 GPU and CUDA	305
15.1 Your First Kernel: Device Memory and Elementwise Ops	306
15.2 Tiled GEMM on the GPU	310
15.3 The GPU Forward Pass	315
Unit 15 — Milestone check	320
Afterword	321

Preface

The pitch

There are two ways to learn how large language models actually run.

The first is to read. You read about attention, about kv caches, about quantization, and you nod along. You can draw the diagrams. You can explain RoPE at a whiteboard. And then someone hands you a two-gigabyte file full of 4-bit-packed weights and asks, “*make this generate a sentence*,” and you discover that the gap between understanding a Transformer and **running** one is enormous — and that almost nobody talks about what lives in that gap.

The second way is the one this book takes. You **build the engine**. Not a notebook that calls a library. Not a 200-line toy that multiplies a couple of matrices. A genuine, modular, CPU-first inference engine in modern C++ that:

- represents tensors with shapes, strides, dtypes, and shared storage;
- implements its own matrix multiplication and pushes it from “naive triple loop” toward something that actually uses your cache and your vector units;
- tokenizes text with byte-level BPE, exactly the way real models expect;
- assembles a full Llama-family Transformer — embeddings, RoPE, causal multi-head attention with GQA, SwiGLU feed-forward, RMSNorm — and runs a forward pass;
- caches keys and values so generation does not re-read the whole context every step;
- quantizes weights to int8 and int4 and multiplies the quantized blocks directly;
- memory-maps a GGUF model file, wires the weights up by name, and produces logits that match a reference;
- samples tokens, streams output, and — by the end — serves multiple concurrent requests with continuous batching.

By **Unit 14**, you have the whole thing working end-to-end on a CPU. The advanced machinery that everyone wants to understand — Mixture of Experts,

Flash Attention, speculative decoding, continuous batching and PagedAttention — is not bolted on at the end as a curiosity. Each of those topics lives *inside the unit that earns it*, right after you have built the thing it improves. **Unit 15**, GPU/CUDA, is a clearly-scoped optional capstone: we offload only the heaviest kernels, and we are honest that it is a bonus, not the point.

This is a hard book for a narrow audience, on purpose. If that audience is you, there is very little else like it.

Who this book is for

This book is written for a **programmer who is already fluent in C++ and comfortable with algorithms and data structures**. You will have a much better time if most of the following are true:

- You read and write modern C++ (we use **C++20**) without reaching for a reference every other line: templates, `RAII`, smart pointers, move semantics, `constexpr`, lambdas. Pointers and memory layout do not scare you.
- You know your way around a build. You have used **CMake** or can pick it up quickly, and you are comfortable on a command line.
- You remember enough linear algebra to know what a matrix–vector product *is* and why the dimensions must line up. We re-derive what we need, but we move fast.
- You have a rough mental model of a neural network — layers, weights, activations — even if you have never implemented one.
- Crucially: **you are willing to be stuck**. Some challenges will not fall on the first try. That discomfort is the mechanism by which this works.

You do not need prior experience with machine-learning frameworks, CUDA, or any LLM internals. That is what we are here to build. If you have never heard of RoPE or GQA, good — you will implement them and they will stop being acronyms. A book that demands strong C++ *and* a tolerance for systems detail *and* a willingness to build a Transformer by hand is not for everyone, and we are not pretending otherwise. That is the point.

A word on mindset

You will be stuck, and that is the product working. The challenges are calibrated so the answer is reachable but not obvious. When you hit the wall, resist jumping straight to the reference solution. Reread the background. Read *one* hint. Write something wrong and run the test to see *how* it is wrong. The understanding you build climbing out of a hole is the one that stays.

Numbers do not lie, and that is the point. Much of LLM systems work is a fight against vagueness — “it seems faster,” “the output looks reasonable.” This book pushes you toward *measuring*: GFLOP/s, bytes copied, bit-exact matches, tokens per second. The boss challenges convert “I think it works” into “the meter says it works.”

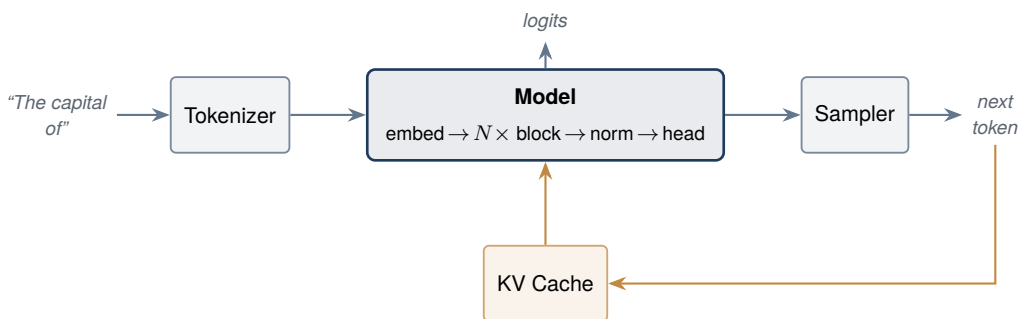
Correct first, then fast. We always build the obvious, slow, obviously correct version first and keep it forever as a reference oracle, then optimize against it. An optimization you cannot check against a reference is a bug you have not found yet.

Respect what you already know. This book moves quickly through what a strong programmer already understands and slows way down on what is genuinely new — memory layout for SIMD, the numerics of softmax, the bit-packing of int4, the bookkeeping of a kv cache. It is tuned for someone who finds “here is a for-loop” insulting and “here is why this for-loop’s memory access pattern halves your throughput” delightful.

How to Read This Book

What you will build — the engine

It helps to hold the whole machine in your head before forging its parts. For a single generated token: the prompt becomes token IDs, the model turns those into logits, a sampler picks the next token, and a cache lets the next step skip recomputing the past.



The lifecycle of a full generation is the **prefill / decode** split that organizes everything we do:

1. **Encode.** The tokenizer turns your prompt into integer token IDs.
2. **Prefill.** The model runs a forward pass over *all* prompt tokens at once — compute-heavy (a big matrix–matrix multiply per layer) — and, as a side effect, fills the kv cache for every prompt position.
3. **Sample.** From the logits at the last position, the sampler picks the next token (greedy, or temperature / top- k / top- p).
4. **Decode.** Feed that one token back in. The forward pass now processes a *single* position but attends to *all* cached keys and values, so it never recomputes the past. This step is memory-bandwidth-bound — where the kv cache, quantization, Flash Attention, and speculative decoding earn their keep.
5. **Loop.** Append, sample, repeat — until end-of-sequence or a length limit.
6. **Decode (the other kind).** The tokenizer turns the produced IDs back into text.

We build this **from the bottom up**; each unit assumes the one beneath it works.

Unit 14	Performance & optimization — ENGINE COMPLETE
Unit 13	Serving & batching: continuous batching; PagedAttention
Unit 12	Speculative decoding
Unit 11	Sampling & generation: greedy/temp/top-k/top-p; streaming
Unit 10	Model loading: GGUF; mmap; weights-by-name
Unit 9	Quantization: int8 / int4 (Q8_0, Q4_0, K-quants)
Unit 8	Flash Attention
Unit 7	KV cache
Unit 6	Mixture of Experts
Unit 5	Transformer: embeddings; RoPE; attention; GQA/MQA; SwiGLU FFN
Unit 4	Tokenizer: byte-level BPE
Unit 3	Matrix multiplication: naive → blocked → parallel → SIMD
Unit 2	Math kernels: elementwise; SiLU/GELU/SwiGLU; RMSNorm; softmax
Unit 1	Tensor & memory: strides, arena, zero-copy views
Unit 0	Build system · tests · conventions

The “engine is alive” milestone is **Unit 14**, on CPU. Unit 15 (CUDA) offloads only the two heaviest kernels and is explicitly *not* a complete CUDA course.

How the challenges work

This is a book of **challenges**, and the challenge *is* the chapter. You learn by building the next piece, not by reading about it and maybe trying. Each challenge — without exception — has the same seven parts, in order:

1. **Goal** — what the engine gains, as capability, not code.
2. **Background** — the concepts and the *why*: the math, the systems insight, the trade-off, the gotcha that bit everyone before you.
3. **The Task** — a precise specification: interface, behavior, constraints. The contract you fulfill.

4. **Hints** — graduated nudges, from “shape of the approach” to “the specific trick,” so you take exactly as much help as you need.
5. **Tests** — the executable definition of *done*, given up front.
6. **Reference Solution** — a complete, correct, idiomatic implementation with commentary. A *reference*, not *the* answer.
7. **Going Further** — harder variants, real-world considerations, threads to later units.

On the reference solutions. The fastest way to waste this book is to read each reference solution as if it were the assigned reading. The code is here so you are never permanently blocked and so you can compare your instincts against a known-good design — but the learning is in the struggle that precedes it. Write your version first. *Then* read ours. The delta between the two is the most valuable page in each challenge, and it is one only you can see.

Two layers. *Build challenges* carry construction forward in digestible steps; each adds a real, working piece. The *boss challenge* ends the unit — exactly one, hard, and **measured against an objective bar**: not “implement *X*” but “make *X* fast enough,” “match the reference byte-for-byte,” “prove this copied zero bytes.” You pass a boss because the measurement says so, not because you wrote code.

The advanced units. Units 6, 8, 12, and 13 (MoE, Flash Attention, speculative decoding, serving) are not quarantined in a final section, because that is pedagogically backwards: Flash Attention only makes sense once you have felt the pain of materializing a full attention matrix — right after you build naive attention. Each advanced unit sits immediately after the foundation it optimizes.

The reference model and test assets

We target the **Llama family** of decoder-only Transformers — the architecture behind Llama 2/3, TinyLlama, Mistral, and Qwen2. They share the ingredients we implement: RoPE, RMSNorm, SwiGLU feed-forward, grouped-query attention (GQA), and byte-level bPE. Once your engine runs one, running its cousins is mostly a matter of reading different metadata.

For most of the book you do *not* need a real model: building bottom-up lets us test each kernel against a mathematical reference using random or hand-constructed weights. Your matrix multiply is correct or it is not, regardless of where the numbers came from. When we reach **Unit 10 (Model Loading)** you will want a small GGUF file — TinyLlama-1.1B is an excellent first real model; Llama-3.2-1B/3B or

Qwen2.5-0.5B also work. Until then there is nothing to download.

Our north star, named honestly. This book is, in spirit, *“build your own llama.cpp, and understand every line.”* We borrow that project’s hard-won vocabulary — GGUF, Q4_0, block quantization, mmap’d weights — because it has become the lingua franca of CPU LLM inference. We are not reproducing its code; we are re-deriving its ideas so they are yours.

Tooling and conventions

Language: modern C++20 — the genuinely useful parts (`std::span`, `std::bit_cast`, light concepts, designated initializers, `std::source_location`), avoiding what hurts compile times. Any gcc 11+, Clang 14+, or msvc 2022+ will do. **Build: CMake** (3.20+), target-based and presets-driven. **The bar:** it compiles warning-clean under `-Wall -Wextra -Wpedantic`; hot paths do not allocate; correctness checks compile out of release but scream in debug; tests are fast and dependency-free. **Code in this book** is tagged with the file it belongs in (`// include/llm/tensor.hpp`); engine code lives in namespace `llm`, tests in namespace `llmtest`. **Platforms:** Linux, macOS, Windows; where they diverge — aligned allocation is the first place — we say so and guard it cleanly.

The road ahead

Unit	Title	What you walk away able to do
0	Foundation & Setup	Build, test, and structure the project to engine standards
1	Tensor & Memory	Strided tensors over shared, aligned storage; zero-copy views
2	Math Kernels	Elementwise ops, SiLU/GELU/SwiGLU, RMSNorm, stable softmax
3	Matrix Multiplication	GEMM from naive to cache-blocked, multi-threaded, SIMD
4	Tokenizer	Encode/decode with byte-level BPE, as models expect
5	Transformer	Embeddings, RoPE, causal MHA with GQA, SwiGLU — a full forward pass
6	Mixture of Experts	Route to top- k experts and run sparse FFNS
7	KV Cache	Make generation fast by never recomputing the past
8	Flash Attention	Online softmax and tiling; never materialize the score matrix
9	Quantization	Pack weights to int8 and int4 and multiply them directly
10	Model Loading	mmap a GGUF file, parse metadata, bind weights by name
11	Sampling & Generation	Greedy/temperature/top- k /top- p , the loop, streaming
12	Speculative Decoding	A draft model proposes; a target accepts or rejects
13	Serving & Batching	Concurrent requests, continuous batching, PagedAttention
14	Performance	Profile the path; hand-tune SIMD and threading — engine complete
15	Capstone: GPU/CUDA	Offload the heaviest kernels to CUDA — a bonus

Enough preamble. Let us build the foundation.

UNIT 0

Foundation & Setup

What you will build. The project skeleton: a CMake build producing a library, a CLI, and a test binary; a tiny, zero-dependency test framework used to verify every later challenge; and the error-handling and logging conventions the whole engine relies on. By the end, **one command** builds everything and runs a green test suite — warning-clean, reproducible, from a clean checkout.

Where this fits. This is the floor. It does nothing intelligent on its own, but every later unit stands on it. A messy foundation taxes every challenge that follows; a clean one disappears and lets you think about tensors instead of toolchains.

A reasonable instinct, for a strong programmer, is to skim a setup chapter. Skim if you like — but do not skip the two decisions we make here, because they shape the rest of the book: **how the engine handles errors** (an assert that is free in release but loud in debug, plus an always-on check) and **how we test** (a harness with floating-point-aware comparison, because *every* numerical kernel we write needs $\approx$, not $=$). Those two choices recur in literally every unit.

Here is the project layout we build toward; create it now, and the challenges fill it in.

```
llm-engine/  
|-- CMakeLists.txt           # the build  
|-- CMakePresets.json       # named configs (debug / release / ci)  
|-- include/  
|   |-- llm/  
|   |-- common.hpp          # types, logging, assertions (Challenge 0.3)  
|-- src/
```

```
|  `-- version.cpp          # engine .cpp files arrive from Unit 1 on
|-- tools/
|  `-- cli/
|      `-- main.cpp        # the `llm` command-line tool (a stub for now)
|-- tests/
|   |-- test_framework.hpp  # our tiny test harness          (Challenge 0.2)
|   |-- test_main.cpp       # the test runner's entry point
|   `-- test_smoke.cpp      # a first trivial test
```

We add `include/llm/tensor.hpp`, `src/tensor.cpp`, and friends starting in Unit 1. For now, three build challenges and a boss.

0.1 The Project Skeleton

GOAL

A reproducible, cross-platform build: one CMake project that compiles a **core library**, a **command-line executable** that links it, and a **test executable**, on Linux, macOS, and Windows, with a single configure-and-build invocation.

BACKGROUND

Before any interesting code, you need a build that *gets out of your way*. For a project that will grow to dozens of source files, ad-hoc compilation (`g++ *.cpp`) collapses almost immediately. We use **CMake** because it is the de-facto standard for C++ and because its target-based style keeps dependencies honest as the project grows.

Two ideas worth internalizing. **Library plus thin executables**: the intelligence of the engine lives in a **library target** (`llm_core`); the CLI and the tests are *thin* executables that link against it. This lets the test binary exercise the exact same code the CLI runs. A common beginner mistake is to put logic in `main.cpp`; resist it. `main.cpp` parses arguments and calls into the library, nothing more.

Modern, target-based CMake: old CMake was a soup of global variables that leaked settings everywhere. Modern CMake attaches properties to *targets* and propagates them along dependency edges with `PUBLIC` / `PRIVATE` / `INTERFACE`. When `llm_core` declares its headers `PUBLIC`, anything that links it gets the include

path automatically. Describe each target's needs; let CMake compute the transitive closure.

We also pin a few things: the standard (**C++20**, required, extensions off), a sane default build type (**Release** — “why is my code 30× slower” is almost always “you built Debug”), and a centralized **warnings** target so every part of the project is held to one standard.

THE TASK

Create the directory structure above and a `CMakeLists.txt` that declares a versioned project `llm_engine`; sets **C++20** (required, extensions off); defaults to **Release** when unset (single-config generators); defines an `INTERFACE` library `llm_warnings` carrying `-Wall -Wextra -Wpedantic` (or `/W4 /permissive-` on `msvc`) plus an option(`LLM_WERROR ...`); defines a static library `llm_core` (with a placeholder source) exposing `include/` as `PUBLIC`; defines executable `llm` linking both; and enables testing with `llm_tests` registered via `add_test`. Write a `main.cpp` stub that prints the engine name and version.

HINTS

Hint 1 — the shape of a target-based CMakeLists

Think in three blocks: *project & global settings*, then *the warnings interface target*, then *one block per real target* (`llm_core`, `llm`, `llm_tests`). Each gets its `target_link_libraries`; the library gets `target_include_directories(... PUBLIC include)`.

Hint 2 — a library with no sources yet

CMake dislikes `add_library` with zero sources. Give it a placeholder `src/version.cpp` defining `const char* llm::version()`; less surprising later than an `INTERFACE` library you must convert.

Hint 3 — Debug-only asserts via the build type

Make a compile definition conditional on the configuration:
`target_compile_definitions(llm_core PUBLIC $<$<CONFIG:Debug>:LLM_ENABLE_ASSERTS>)`. We use it in Challenge 0.3.

TESTS

This challenge's "test" is the build itself plus a smoke run.

```
# from the project root
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/llm          # should print: llm-engine 0.1.0
ctest --test-dir build # meaningful after 0.2; configures cleanly now
```

REFERENCE SOLUTION

CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.20)
project(llm_engine LANGUAGES CXX VERSION 0.1.0)

# ---- Global language settings ----
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON) # for clangd / tooling

# Default to Release for single-config generators if unset.
get_property(_multi GLOBAL PROPERTY GENERATOR_IS_MULTI_CONFIG)
if(NOT _multi AND NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
    set_property(CACHE CMAKE_BUILD_TYPE PROPERTY STRINGS
        Debug Release RelWithDebInfo MinSizeRel)
endif()

# ---- Centralized warnings ----
add_library(llm_warnings INTERFACE)
option(LLM_WERROR "Treat warnings as errors" OFF)
if(MSVC)
    target_compile_options(llm_warnings INTERFACE /W4 /permissive-)
    if(LLM_WERROR)
        target_compile_options(llm_warnings INTERFACE /WX)
    endif()
else()
    target_compile_options(llm_warnings INTERFACE -Wall -Wextra -Wpedantic)
    if(LLM_WERROR)
        target_compile_options(llm_warnings INTERFACE -Werror)
    endif()
endif()
```

```

endif()

# ---- Core library ----
add_library(llm_core STATIC
    src/version.cpp
    # Unit 1 onward appends sources here (src/tensor.cpp, ...).
)
target_include_directories(llm_core PUBLIC include)
target_compile_features(llm_core PUBLIC cxx_std_20)
target_link_libraries(llm_core PRIVATE llm_warnings)
target_compile_definitions(llm_core PUBLIC
    $$<CONFIG:Debug>;LLM_ENABLE_ASSERTS)

# ---- Command-line tool ----
add_executable(llm tools/cli/main.cpp)
target_link_libraries(llm PRIVATE llm_core llm_warnings)

# ---- Tests ----
enable_testing()
add_executable(llm_tests
    tests/test_main.cpp
    tests/test_smoke.cpp
)
target_include_directories(llm_tests PRIVATE tests)
target_link_libraries(llm_tests PRIVATE llm_core llm_warnings)
add_test(NAME unit COMMAND llm_tests)

```

src/version.cpp gives the library a real source and a version symbol:

```

// src/version.cpp
namespace llm {
const char* version() { return "0.1.0"; }
}

```

tools/cli/main.cpp:

```

// tools/cli/main.cpp
#include <cstdio>

namespace llm { const char* version(); }

int main(int argc, char** argv) {
    (void)argc; (void)argv;
}

```

```
std::printf("llm-engine %s\n", llm::version());
// Future subcommands: run, tokenize, perplexity, serve, ...
return 0;
}
```

A minimal CMakePresets.json so nobody memorizes flags:

```
{
  "version": 3,
  "cmakeMinimumRequired": { "major": 3, "minor": 20, "patch": 0 },
  "configurePresets": [
    { "name": "debug", "binaryDir": "${sourceDir}/build/debug",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "Debug" } },
    { "name": "release", "binaryDir": "${sourceDir}/build/release",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "Release" } },
    { "name": "ci", "binaryDir": "${sourceDir}/build/ci",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "Release", "LLM_WERROR": "ON" } }
  ],
  "buildPresets": [
    { "name": "debug", "configurePreset": "debug" },
    { "name": "release", "configurePreset": "release" },
    { "name": "ci", "configurePreset": "ci" }
  ]
}
```

With this, `cmake --preset release` then `cmake --build --preset release` and you are off. (`llm_tests` will not link until Challenge 0.2 adds `tests/test_main.cpp`; that is next.)

Why these choices. The `INTERFACE` warnings library is the most useful pattern here: every target links it, every target gets identical flags, and `-Werror` for `ci` is a one-line flip. Defaulting to `Release` prevents the most common “my engine is slow” question. Exporting `compile_commands.json` makes your editor’s language server understand the project.

GOING FURTHER

- **Sanitizers.** A preset compiling `Debug` with `-fsanitize=address,undefined`. For a memory-heavy engine, `ASAN` / `UBSAN` catch out-of-bounds reads in stride math and signed overflow in index computations before they become mysterious wrong numbers.

- **configure_file for the version**, so the version lives in one place.
- A **build.sh** wrapper for configure + build + test — the healthy path as one command. (We make this the boss.)

0.2 Proof of Life: a Test Framework

GOAL

A tiny, zero-dependency test framework — `TEST(...)`, `CHECK(...)`, `REQUIRE(...)`, and crucially `CHECK_CLOSE(...)` for floating-point — plus a runner that reports pass/fail counts and exits non-zero on any failure. This is the instrument you will use to verify *every* challenge from here on.

BACKGROUND

You could pull in **doctest** or **Catch2** (both excellent, and in a production engine you probably would). We write our own ~100-line harness instead, for three reasons: **zero dependencies** (the project stays trivially buildable anywhere), **full control** (we add exactly the assertions numerical code needs), and the fact that writing it demystifies what a test framework even *is*. If you prefer doctest, every “Tests” section maps onto it one-to-one.

The one feature we cannot live without — and why a generic assert will not do — is **approximate floating-point comparison**. Almost every kernel here produces floating-point numbers, and floating-point arithmetic is not associative: summing a vector left-to-right and summing it in blocks (as a SIMD or multi-threaded version will) gives *slightly* different results. Both are “correct.” So our tests must ask “is this close enough?”, not “is this bit-identical?”. The standard tool uses both an **absolute** and a **relative** tolerance:

$$\text{pass} \iff |a - b| \leq \text{atol} + \text{rtol} \cdot |b|.$$

The absolute term handles values near zero (where relative error is meaningless); the relative term handles large values (where a fixed absolute tolerance would be absurdly strict). Get this right once, here, and every numerical test becomes trustworthy.

We also want two *severities*. `CHECK` records a failure but keeps going, so one test can report all the ways it is broken in a single run. `REQUIRE` records a failure and

aborts that test (continuing is pointless or unsafe), but does not kill the suite; the runner moves on.

THE TASK

Implement `tests/test_framework.hpp` providing: `TEST(name) { ... }` (defines and auto-registers a test — no central list); `CHECK(cond)` (record-and-continue); `REQUIRE(cond)` (record-and-abort-this-test); `CHECK_CLOSE(a, b, tol)` passing iff $|a - b| \leq \text{tol} + \text{tol} \cdot |b|$; and `run_all()` executing every test, printing per-test OK/FAIL and a summary, catching exceptions (reporting rather than crashing), and returning 0 iff all passed. Add `tests/test_main.cpp` and a first `tests/test_smoke.cpp`.

HINTS

Hint 1 — auto-registration without a central list

A test is a function. Register it via *static initialization*: a file-scope object whose constructor runs before `main` and pushes the function into a global registry. `TEST` emits a forward-declared function, a static registrar, and the function's opening brace.

Hint 2 — the registry must be a function-local static

A namespace-scope registry can hit the *static initialization order fiasco*. Wrap it in a function returning a static `std::vector<...>&`; it is then constructed on first use, before any registrar touches it.

Hint 3 — REQUIRE aborts one test, not the suite

Make `REQUIRE` throw a private sentinel after recording the failure. In the runner, catch that sentinel silently (already recorded) and other exceptions as failures. A distinct sentinel avoids double-counting.

TESTS

The harness's own smoke test (these should *pass*):

```
// tests/test_smoke.cpp
#include "test_framework.hpp"

TEST(smoke_basic_checks) {
    CHECK(1 + 1 == 2);
    REQUIRE(true);           // does not abort
    CHECK_CLOSE(0.1 + 0.2, 0.3, 1e-9); // != in binary fp, but close
}

TEST(smoke_close_handles_zero) {
    CHECK_CLOSE(0.0, 1e-12, 1e-6); // absolute term saves us near zero
}
```

Then verify the *failure* path manually: temporarily add a `CHECK(false)`, confirm it is reported and the process exits non-zero, and remove it.

REFERENCE SOLUTION

```
// tests/test_framework.hpp
#pragma once
#include <cmath>
#include <cstdio>
#include <exception>
#include <functional>
#include <string>
#include <vector>

namespace llmtest {

// ---- Registry ----
struct TestCase { std::string name; std::function<void()> fn; };

inline std::vector<TestCase>& registry() {
    static std::vector<TestCase> r; // first-use init: no order fiasco
    return r;
}

struct Registrar {
    Registrar(std::string name, std::function<void()> fn) {
        registry().push_back({std::move(name), std::move(fn)});
    }
};
};
```

```

// ---- Failure bookkeeping ----
struct Stats { int checks = 0; int check_failures = 0; };
inline Stats& stats() { static Stats s; return s; }

struct RequireFailed {}; // thrown to abort the current test only

inline void report_failure(const char* file, int line, const std::string& what) {
    ++stats().check_failures;
    std::fprintf(stderr, "    FAIL  %s:%d  %s\n", file, line, what.c_str());
}

// ---- Runner ----
inline int run_all() {
    int tests_failed = 0;
    for (const auto& tc : registry()) {
        const int before = stats().check_failures;
        std::printf("[ RUN ] %s\n", tc.name.c_str());
        try {
            tc.fn();
        } catch (const RequireFailed&) {
            // failure already recorded; just stop this test
        } catch (const std::exception& e) {
            report_failure("<exception>", 0, std::string("uncaught: ") + e.what());
            ➞
        } catch (...) {
            report_failure("<exception>", 0, "uncaught non-std exception");
        }
        if (stats().check_failures > before) {
            std::printf("[ FAIL ] %s\n", tc.name.c_str());
            ++tests_failed;
        } else {
            std::printf("[ OK ] %s\n", tc.name.c_str());
        }
    }
    std::printf("\n%zu test(s) run, %d failed | %d checks, %d failed\n",
                registry().size(), tests_failed,
                stats().checks, stats().check_failures);
    return tests_failed == 0 ? 0 : 1;
}

} // namespace llmtest

// ---- Macros ----
#define TEST(test_name) \
    static void test_name(); \
    static ::llmtest::Registrar registrar_##test_name(#test_name, test_name); \
    static void test_name()

```



```

#define CHECK(cond)                                     \
do {                                                    \
    ++::llmtest::stats().checks;                        \
    if (!(cond)) ::llmtest::report_failure(__FILE__, __LINE__, #cond); \
} while (0)

#define REQUIRE(cond)                                   \
do {                                                    \
    ++::llmtest::stats().checks;                        \
    if (!(cond)) {                                      \
        ::llmtest::report_failure(__FILE__, __LINE__, #cond); \
        throw ::llmtest::RequireFailed{};                \
    }                                                    \
} while (0)

// pass iff |a - b| <= tol + tol*|b|  (absolute + relative tolerance)
#define CHECK_CLOSE(a, b, tol)                         \
do {                                                    \
    ++::llmtest::stats().checks;                        \
    const double _a = (a), _b = (b), _t = (tol);        \
    if (!(std::fabs(_a - _b) <= _t + _t * std::fabs(_b))) \
        ::llmtest::report_failure(__FILE__, __LINE__, \
                                   #a " ~= " #b " (tol " #tol ")"); \
} while (0)

```

```

// tests/test_main.cpp
#include "test_framework.hpp"

int main() {
    return llmtest::run_all();
}

```

Add the smoke test as `tests/test_smoke.cpp`, build, and run `./build/llm_tests`: two [OK] lines and 2 test(s) run, 0 failed. `cctest --test-dir build` now reports the unit test passing.

Why it is built this way. Auto-registration via static constructors means adding a test is *just writing a TEST* — no central list to forget, which matters across the ~100 tests this book accumulates. The function-local-static registry sidesteps initialization-order bugs. The relative-plus-absolute tolerance in `CHECK_CLOSE` is the single most important line for everything numerical that follows; treat it as load-bearing.

GOING FURTHER

- **CHECK_THROWS(expr, ExceptionType)** — assert an expression throws (handy for the bounds-checks in Unit 1).
- **Per-test timing** via `std::chrono` — notice when a “unit test” became a benchmark.
- **Filtering:** `run_all(substring)` wired to `argv[1]`, so `./llm_tests tensor` runs the tensor tests.
- **A vector-closeness helper** `check_all_close(got, want, tol)` reporting the worst mismatch. You will reach for this constantly once kernels return arrays.

0.3 Errors and Logging: the Engine’s Nervous System

GOAL

Establish project-wide conventions for **failing loudly** and **observing what is happening**: an assertion that is free in release but aborts with a clear message in debug (`LLM_ASSERT`), an always-on check (`LLM_CHECK`), an `LLM_UNREACHABLE()` marker, and a minimal leveled logger.

BACKGROUND

In a high-performance numerical library, error handling is a *design decision with a performance cost*, not an afterthought. The tension: you want aggressive internal checks (is this index in bounds? do these shapes match? is this pointer aligned?) to catch bugs early — but you cannot afford to run them inside a matrix-multiply inner loop that executes billions of times. The resolution, used by essentially every serious engine, is **two tiers**.

Debug-only assertions (`LLM_ASSERT`) encode *internal invariants* — things true if your code is correct. They run during development (Debug builds), turning “silently wrong number three layers down” into “aborted exactly here with a message.” In Release they compile to *nothing*, so the hot loop pays zero cost. Same philosophy as C’s `assert`, but we own the message format and the enable flag (tied to the Debug configuration back in 0.1).

Always-on checks (`LLM_CHECK`) encode conditions that depend on *external input* and must hold even in Release — a corrupt model file, an allocation that failed,

a nonsensical user-supplied dimension. You cannot compile these away, because the bad input can arrive in production. They are not in hot loops, so their cost is irrelevant.

Why not exceptions everywhere? You *can* throw, and `LLM_CHECK` could be a throw. But for an engine a failed invariant usually means a programming bug, and the most useful behavior is to abort *immediately* with file/line/function so the stack is intact under a debugger — not to unwind through code that was not written to be exception-safe. We abort by default and note where throwing is the better choice (Unit 10, where a malformed model file is genuinely recoverable).

C++20 gives us `std::source_location`, which captures the caller's file, line, and function *automatically* as a default argument — no `__FILE__`/`__LINE__` threaded through every function. The logger is deliberately minimal: levels, a global threshold, output to `stderr` — enough to narrate what the engine is doing without a logging dependency.

THE TASK

Fill in `include/llm/common.hpp` with: `LLM_ASSERT(cond, msg)` (aborts with a located message when `LLM_ENABLE_ASSERTS` is defined; otherwise a no-op that still *compiles* `cond`); `LLM_CHECK(cond, msg)` (always aborts on false); `LLM_UNREACHABLE()`; a `[[noreturn]] panic(message, source_location)`; and a leveled logger (enum `LogLevel`, a settable global threshold, `log_message(level, msg)`, and convenience macros).

HINTS

Hint 1 — a disabled assert that still type-checks its condition

A no-op that expands to nothing lets the condition rot. Expand the disabled form to do `{ (void)sizeof(cond); } while (0)`: `sizeof` does not evaluate its operand, so there is zero runtime cost, but the expression must still be well-formed.

Hint 2 — capturing the caller's location automatically

Give `panic` a parameter `std::source_location loc = std::source_location::current()`. The default argument is evaluated at the *call site*, so `loc` points at the caller.

Hint 3 — why LLM_UNREACHABLE helps beyond documentation

In a switch that returns from every case, the compiler may still warn “control reaches end of non-void function.” Ending such a function with `LLM_UNREACHABLE();` silences that correctly and, routed to a `[[noreturn]]` function, tells the optimizer the path is dead.

TESTS

```
// tests/test_common.cpp
#include "test_framework.hpp"
#include "llm/common.hpp"

TEST(check_passes_silently_on_true) {
    LLM_CHECK(2 + 2 == 4, "arithmetic still works");
    CHECK(true); // if we got here, LLM_CHECK did not abort
}

TEST(logger_respects_threshold) {
    llm::set_log_level(llm::LogLevel::Warn);
    LLM_LOG_INFO("this should be suppressed"); // should NOT appear
    LLM_LOG_WARN("this should appear");        // should appear
    CHECK(true);
    llm::set_log_level(llm::LogLevel::Info);    // restore for other tests
}
```

The failure paths (`LLM_CHECK(false, ...)` aborts; `LLM_ASSERT(false, ...)` aborts in Debug) are verified by tripping them once and observing the abort, then removing the line — aborting cannot be wrapped in a normal CHECK.

REFERENCE SOLUTION

```
// include/llm/common.hpp
#pragma once
#include <cstdint>
#include <cstdio>
#include <stdlib.h>
#include <source_location>
#include <string_view>
```

```

namespace llm {

const char* version();

// ---- Panic & checks ----
[[noreturn]] inline void panic(
    std::string_view msg,
    const std::source_location loc = std::source_location::current()) {
    std::fprintf(stderr, "\n[llm:PANIC] %s:%u in %s\n          %.*s\n",
        loc.file_name(), loc.line(), loc.function_name(),
        static_cast<int>(msg.size()), msg.data());
    std::abort();
}

// ---- Logging ----
enum class LogLevel : int { Trace = 0, Debug = 1, Info = 2, Warn = 3, Error = 4 };

inline LogLevel& log_threshold() { static LogLevel level = LogLevel::Info; return
    ↪ level; }
inline void set_log_level(LogLevel l) { log_threshold() = l; }

inline const char* level_name(LogLevel l) {
    switch (l) {
        case LogLevel::Trace: return "TRACE";
        case LogLevel::Debug: return "DEBUG";
        case LogLevel::Info:  return "INFO";
        case LogLevel::Warn:  return "WARN";
        case LogLevel::Error: return "ERROR";
    }
    return "?";
}

inline void log_message(LogLevel l, std::string_view msg) {
    if (static_cast<int>(l) < static_cast<int>(log_threshold())) return;
    std::fprintf(stderr, "[llm:%s] %.*s\n", level_name(l),
        static_cast<int>(msg.size()), msg.data());
}

} // namespace llm

// ---- Macros (outside any namespace) ----

// Always-on: enforces conditions that depend on external input.
#define LLM_CHECK(cond, msg) \
    do { if (!(cond)) ::llm::panic(msg); } while (0)

// Debug-only invariant checks; zero cost in Release.

```

```

#if defined(LLM_ENABLE_ASSERTS)
    #define LLM_ASSERT(cond, msg)                                \
        do { if (!(cond)) ::llm::panic(msg); } while (0)
#else
    #define LLM_ASSERT(cond, msg)                                \
        do { (void)sizeof(cond); /* type-checks without evaluating */ } while (0)
#endif

#define LLM_UNREACHABLE() ::llm::panic("unreachable code reached")

#define LLM_LOG_TRACE(msg) ::llm::log_message(::llm::LogLevel::Trace, msg)
#define LLM_LOG_DEBUG(msg) ::llm::log_message(::llm::LogLevel::Debug, msg)
#define LLM_LOG_INFO(msg)  ::llm::log_message(::llm::LogLevel::Info,  msg)
#define LLM_LOG_WARN(msg)  ::llm::log_message(::llm::LogLevel::Warn,  msg)
#define LLM_LOG_ERROR(msg) ::llm::log_message(::llm::LogLevel::Error, msg)

```

Register tests/test_common.cpp in the llm_tests sources, build, and run: both tests pass, and (threshold at Warn) only the WARN line prints during the logger test.

Design notes. Everything is inline in the header, so there is no separate .cpp to keep in sync; if panic/log_message grow, move them to src/common.cpp. Tying LLM_ENABLE_ASSERTS to the build *type* means the right behavior happens automatically: develop and test in Debug with every invariant live, ship Release with the hot paths unburdened. The (void)sizeof(cond) trick separates a robust disabled-assert from one that silently lets conditions become uncompileable.

GOING FURTHER

- **Formatted messages** via std::format, wrapped so formatting happens only on failure.
- **Throwing variants** (LLM_CHECK_OR_THROW) for the genuinely recoverable cases (model loading).
- **A scoped trace timer** — an RAII ScopedTimer{"prefill"} logging elapsed time on scope exit. A free, always-available profiler once sprinkled through the forward pass.
- **Compile-time log floor** (LLM_LOG_MIN_LEVEL) so verbose logging costs nothing in release.

0.4 One Command, Green and Clean

BOSS CHALLENGE

From a clean checkout, one command builds everything and runs the whole suite — green, with zero warnings under `-Werror`. The bar is objective: no warnings, no failures, reproducible from scratch.

GOAL

From a **clean checkout**, a **single command** configures, builds, and runs the entire test suite — and the result is **green with zero warnings**. This is the standard every later unit's code is held to, so we make achieving it a one-liner.

BACKGROUND

The difference between a project pleasant to work in for months and one that rots is, more than anything, whether “build it and check it is healthy” is *frictionless*. If it takes six remembered flags, people stop doing it, warnings accumulate, and one day a warning that was actually a bug hides in the noise. The fix: make the healthy path the *easy* path — one command, warnings as errors, tests required to pass. That is also exactly what a continuous-integration server runs, so “passes locally” and “passes ci” become the same statement.

The **measurement** is unusually crisp for a setup unit, and that is intentional. **Zero warnings** under `-Wall -Wextra -Wpedantic` with `-Werror`: a warning is the compiler telling you about a likely bug, and in an engine doing pointer and integer arithmetic all day you cannot afford to ignore them. **Zero test failures**: `ctest` returns success. **Reproducible from clean**: delete `build/`, run the one command, and it all happens again.

THE TASK

Provide a single entry point — `scripts/build.sh` (and/or `build.ps1`), or a documented preset invocation — that configures with the `ci` preset (Release, `LLM_WERROR=ON`), builds all targets, and runs `ctest`. Then make it pass: everything from 0.1–0.3 must compile **warning-clean** with `-Werror` and all tests must pass. Verify reproducibility with `rm -rf build && ./scripts/build.sh`.

The bar (the boss): clean checkout → one command → build succeeds with *zero* warnings (`-Werror on`) → all tests pass → exit code 0.

HINTS

Hint 1 — fail fast in the script

Start a bash script with `set -euo pipefail` so any failing step aborts the whole script with a non-zero status. A script that plows on after a failed compile and reports “done” is worse than none.

Hint 2 — turning on `-Werror` via the preset

You already wired `option(LLM_WERROR ...)` and a ci configure preset that sets it ON. The script just needs `cmake --preset ci, cmake --build --preset ci`, then `ctest` pointed at that binary dir.

Hint 3 — the warnings you are most likely to hit first

With `-Wextra`, unused parameters (`argc, argv`) and sign-conversion in size/index arithmetic are the usual first offenders. Cast-to-void the genuinely-unused, and be deliberate about signed vs unsigned in size math — a habit that prevents real bugs in the stride arithmetic coming next unit.

TESTS

The boss *is* the test. Run it and read the exit code:

```
rm -rf build
./scripts/build.sh
echo "exit: $?"      # must be 0; no 'warning:' lines; ends with all tests passed
```

REFERENCE SOLUTION

`scripts/build.sh:`

```
#!/usr/bin/env bash
# One command: configure (CI preset), build everything, run all tests.
```



```

set -euo pipefail

cd "$(dirname "$0")/.."      # project root, wherever we're called from

cmake --preset ci           # Release + LLM_WERROR=ON
cmake --build --preset ci -j
ctest --test-dir build/ci --output-on-failure

echo "build + tests green"

```

Make it executable (`chmod +x scripts/build.sh`). A PowerShell twin would run the same three `cmake/ctest` commands with `$ErrorActionPreference = "Stop"`.

Now make it pass. The most likely fix, given the stubs from 0.1, is the unused-parameter warning in the CLI main — handled by the `(void)argc; (void)argv;` already in the reference. A clean run looks like:

```

$ rm -rf build && ./scripts/build.sh
... cmake configure output ...
... compiling llm_core, llm, llm_tests ...
[ RUN ] smoke_basic_checks
[ OK ] smoke_basic_checks
[ RUN ] smoke_close_handles_zero
[ OK ] smoke_close_handles_zero
[ RUN ] check_passes_silently_on_true
[ OK ] check_passes_silently_on_true
[ RUN ] logger_respects_threshold
[llm:WARN] this should appear
[ OK ] logger_respects_threshold
...
N test(s) run, 0 failed | M checks, 0 failed
build + tests green

```

Exit code 0, no warning: anywhere, reproducible after `rm -rf build`. **Boss cleared.**

Why this is the right bar to set first. Every challenge after this ends with “the tests pass.” By making “build everything warning-clean and run all tests” a single reproducible command *now*, the verification step in every future challenge is trivial to run — which means you will actually run it, which means bugs get caught the moment they are introduced rather than three units later as a wrong logit and a long debugging night.

GOING FURTHER

- **A real ci file** (e.g. GitHub Actions) running `./scripts/build.sh` on Linux, macOS, and Windows — “is it healthy?” answered automatically on every push.
- **Add the sanitizer build to the gate.** Also run a `Debug+ASAN/UBSAN` build of the tests; from Unit 1 on, this catches memory bugs in your tensor code the instant they appear.
- **Build timing budget.** Print total wall-clock time. A foundation that takes 90 seconds to build clean is one you will stop rebuilding; watch it.

Unit 0 — Milestone check

MILESTONE CHECK

You now have:

- a **target-based CMake build** producing a core library, a CLI, and a test binary, defaulting to Release, with centralized warnings and a ci preset that treats warnings as errors;
- a **zero-dependency test framework** with `TEST/CHECK/ REQUIRE` and — the load-bearing piece — `CHECK_CLOSE` for floating-point, the instrument you will use to verify every kernel in the book;
- project-wide **error and logging conventions**: `LLM_ASSERT` (free in release, loud in debug), `LLM_CHECK` (always on), `LLM_UNREACHABLE`, and a leveled logger;
- and a **one-command, green-and-clean** boss result proving it all fits together reproducibly.

The engine still does nothing — but the workshop is built, the tools are sharp, and the safety equipment is on. In **Unit 1** we forge the first real component: the **tensor**, with its shape, strides, dtype, shared storage, an aligned arena allocator, and zero-copy views — the data structure every other unit manipulates.

UNIT 1

Tensor & Memory

What you will build. The data structure the entire engine manipulates: a **Tensor** with a shape, strides, a dtype, and a reference to shared storage; an **aligned arena allocator** for the scratch memory a forward pass churns through; and **zero-copy views** — reshape, transpose, permute, slice — that re-describe data without touching a byte of it. The boss proves your views are honest with a hard number: **zero bytes copied**.

Where this fits. Everything above this line — every kernel, every layer, the whole model — is operations on tensors. If the tensor abstraction is wrong or slow, the whole engine is wrong or slow. This is the most-used code you will write, so it is worth getting genuinely right.

A tensor, stripped of mystique, is a **flat buffer of numbers plus a recipe for interpreting it as a multi-dimensional grid**. The buffer is just bytes. The recipe — shape, strides, dtype, offset — is what turns those bytes into “a $2 \times 3 \times 4$ array of 32-bit floats” or “the transpose of that array” or “rows 1 through 3 of it.” The deep idea of this unit, the one that makes high-performance tensor code possible, is that *the recipe and the bytes are separate*, so you can change the recipe (reshape, transpose, slice) without rewriting the bytes. Hold onto that; it is the whole game.

We build it in three steps — define the tensor, give it good memory, give it free views — then a boss that holds the views to account.

1.1 The Tensor

GOAL

Define the core Tensor type — shape, strides, dtype, and a shared, aligned storage buffer — with the ability to allocate a contiguous tensor, query its layout, and read/write individual elements for testing.

BACKGROUND

Four pieces of metadata turn a byte buffer into a tensor. Understand each and you understand the abstraction.

Shape. The size along each dimension. A shape of $[2, 3, 4]$ is a 3-dimensional tensor with 2 “matrices” of 3 rows by 4 columns — 24 elements total. The number of dimensions is the *rank* (here, 3). Rank 0 is a scalar; rank 1 a vector; rank 2 a matrix. The total element count is the product of the shape.

Dtype. What kind of number each element is, and therefore how many bytes it occupies. We start with F32 (4 bytes), F16 (2 bytes), and I8 (1 byte). This matters enormously later: the entire point of Unit 9 is to store weights as 4-bit integers to fit big models in small memory. For now, F32 is the working type; the others are declared so the machinery is dtype-aware from day one. (Block-quantized types like Q4_0 break the “one element = N bytes” model — their bytes-per-element is defined per *block* — so we handle them specially when we get there. Designing for that now would be premature; designing so it is *possible* is just keeping a dtype field.)

Strides — the key idea. A stride is *how many elements you skip in the flat buffer to advance one step along a given dimension*. This is the mechanism that decouples logical shape from physical layout. For a contiguous, **row-major** tensor (the default, and what C and PyTorch use), the last dimension is the fastest-varying: stepping one column moves one element in memory, stepping one row moves a whole row’s worth. The strides of a contiguous shape are computed back-to-front: the last dimension has stride 1, and each earlier stride is the product of all the dimensions to its right.

```
shape = [2, 3, 4]
strides = [12, 4, 1]
|   | +- step 1 element  -> move one column
|   +---- step 4 elements -> move one row (4 columns)
+----- step 12 elements -> move one "matrix" (3x4)
```

```
flat index of element (i, j, k) = i*12 + j*4 + k*1    (+ the view's offset)
```

Once indexing goes through strides, a startling amount becomes *free*. Transpose? Swap two entries in the shape and the strides — the data never moves. Slice off some rows? Adjust the starting offset and shrink one shape entry. We exploit this relentlessly in Challenge 1.3. For now, internalize the indexing formula: **flat index** = *offset* + $\sum_i (\text{coordinate}_i \times \text{stride}_i)$.

Storage and the storage/view split. The bytes live in a Storage object — a reference-counted, aligned buffer. A Tensor does not *own* a buffer outright; it holds a `shared_ptr<Storage>` plus its own shape/strides/offset. Two tensors can point at the *same* Storage while describing it differently (one as a matrix, the other as its transpose). The buffer is freed when the last tensor referencing it is destroyed — RAII via `shared_ptr`, no manual lifetime juggling. This shared-storage design is precisely what makes zero-copy views possible: a view is just another Tensor pointing at the same Storage with a different recipe.

Contiguity. A tensor is *contiguous* when its elements sit in memory in the exact order row-major iteration would visit them — i.e. its strides equal the freshly-computed contiguous strides for its shape. Freshly-allocated tensors are contiguous; transposed views generally are not. Contiguity matters because the fast kernels (and some operations like reshape) assume it. (One subtlety, standard across frameworks: dimensions of size 1 impose no constraint on their stride — there is only one position along them — so a correct contiguity check skips size-1 dims.)

THE TASK

Implement, across `include/llm/dtype.hpp`, `include/llm/tensor.hpp`, and `src/tensor.cpp`:

1. enum class `DType` { `F32`, `F16`, `I8` } with `dtype_size(DType)` and `dtype_name(DType)`.
2. Free helpers `int64_t numel(const Shape&)` (product of dims; 1 for a scalar) and `Strides contiguous_strides(const Shape&)` (row-major).
3. A Storage class: an aligned byte buffer (use the `aligned_malloc/aligned_free` helpers — we motivate the alignment in 1.2), non-copyable, that frees itself. Include a static `alloc_count()` counter (incremented per construction) — instrumentation the boss will use to prove views do not allocate.

4. A `Tensor` class holding `shared_ptr<Storage>`, an element offset, a `Shape`, a `Strides`, and a `DType`, with: `static Tensor empty(Shape, DType, alignment=64)`; layout queries (`shape`, `strides`, `dtype`, `ndim`, `numel`, `nbytes`, `is_contiguous`); a templated `data_ptr<T>()` returning a pointer to this tensor's first element (including the offset); and a templated `at<T>({i, j, ...})` for element read/write (for tests), bounds-checked via `LLM_ASSERT`.

Use `Shape = std::vector<int64_t>` and `Strides = std::vector<int64_t>` for clarity (the production alternative is in *Going Further*).

HINTS

Hint 1 — why offset is in elements, and what `data_ptr` returns

Store the view's `offset_` as a count of *elements* (not bytes). Then `data_ptr<T>()` is simply `reinterpret_cast<T*>(storage->data()) + offset_`. Because `data_ptr` already folds in the offset, your `at()` indexing should add only $\sum_i \text{coord}_i \cdot \text{stride}_i$ *relative to* `data_ptr` — do not add the offset twice.

Hint 2 — computing contiguous strides

Walk the shape from the last dimension to the first with a running product starting at 1: keep `acc = 1`, and for `i` from `ndim-1` down to 0 set `stride[i] = acc; acc *= shape[i];`.

Hint 3 — the contiguity check and size-1 dimensions

Reconstruct the expected contiguous stride on the fly while scanning right-to-left with a running product expected (starting at 1). For each dim, if `shape[i] == 1` skip it; otherwise require `strides[i] == expected`, then multiply expected by `shape[i]`. If you never mismatch, it is contiguous.

Hint 4 — keep templated methods in the header

`data_ptr<T>` and `at<T>` are templates, so their definitions must live in the header. The non-template methods (`empty`, `is_contiguous`, the views in 1.3) go in `src/tensor.cpp`, which your `CMakeLists.txt` already lists under `llm_core`.

TESTS

```
// tests/test_tensor.cpp
#include "test_framework.hpp"
#include "llm/tensor.hpp"
using namespace llm;

TEST(tensor_shape_strides_numel) {
    Tensor t = Tensor::empty({2, 3, 4}, DType::F32);
    CHECK(t.ndim() == 3);
    CHECK(t.numel() == 24);
    CHECK(t.nbytes() == 24u * 4u);
    CHECK(t.strides()[0] == 12); // row-major strides for [2,3,4] are [12,4,1]
    CHECK(t.strides()[1] == 4);
    CHECK(t.strides()[2] == 1);
    CHECK(t.is_contiguous());
    CHECK(t.dtype() == DType::F32);
}

TEST(tensor_set_get_roundtrip) {
    Tensor t = Tensor::empty({2, 2}, DType::F32);
    t.at<float>({0, 0}) = 1.0f;  t.at<float>({0, 1}) = 2.0f;
    t.at<float>({1, 0}) = 3.0f;  t.at<float>({1, 1}) = 4.0f;
    CHECK_CLOSE(t.at<float>({1, 0}), 3.0f, 1e-6);
    const float* p = t.data_ptr<float>(); // contiguous order is 1,2,3,4
    CHECK_CLOSE(p[0], 1.0f, 1e-6);
    CHECK_CLOSE(p[3], 4.0f, 1e-6);
}

TEST(dtype_sizes) {
    CHECK(dtype_size(DType::F32) == 4);
    CHECK(dtype_size(DType::F16) == 2);
    CHECK(dtype_size(DType::I8) == 1);
}
```

Add `tests/test_tensor.cpp` to the `llm_tests` sources and `src/tensor.cpp` to the `llm_core` sources in `CMakeLists.txt`.

REFERENCE SOLUTION

`include/llm/dtype.hpp`:

```

// include/llm/dtype.hpp
#pragma once
#include <cstdint>
#include <cstdint>
#include "llm/common.hpp"

namespace llm {

// Dense element types for now. Block-quantized types (Q8_0, Q4_0, ...) arrive in
// Unit 9 and need special handling: their bytes-per-element is per block, not
// per scalar, so they don't fit this simple size model.
enum class DType : uint8_t { F32, F16, I8 };

inline size_t dtype_size(DType dt) {
    switch (dt) {
        case DType::F32: return 4;
        case DType::F16: return 2;
        case DType::I8: return 1;
    }
    LLM_UNREACHABLE();
}

inline const char* dtype_name(DType dt) {
    switch (dt) {
        case DType::F32: return "f32";
        case DType::F16: return "f16";
        case DType::I8: return "i8";
    }
    LLM_UNREACHABLE();
}

} // namespace llm

```

include/llm/memory.hpp (the aligned-allocation helpers; we add the Arena in 1.2):

```

// include/llm/memory.hpp
#pragma once
#include <cstdint>
#include <cstdint>
#include <stdlib>
#include <malloc.h> // _aligned_malloc / _aligned_free
#ifdef _WIN32
#include <malloc.h> // _aligned_malloc / _aligned_free
#endif
#include "llm/common.hpp"

```



```

namespace llm {

inline bool is_power_of_two(size_t x) { return x != 0 && (x & (x - 1)) == 0; }

// Allocate `nbytes` with the given (power-of-two) alignment. Pair with
// ↪ aligned_free.
// (Why 64 bytes by default? See Challenge 1.2 -- cache lines and SIMD.)
inline void* aligned_malloc(size_t nbytes, size_t alignment = 64) {
    LLM_ASSERT(is_power_of_two(alignment), "alignment must be a power of two");
    #if defined(_WIN32)
        return _aligned_malloc(nbytes ? nbytes : 1, alignment);
    #else
        // NOTE: std::aligned_alloc (C++17) requires size be a multiple of alignment,
        // so we round up. (It is also not provided by MSVC's runtime, which is why
        // Windows uses _aligned_malloc above.)
        const size_t rounded = (nbytes + alignment - 1) & ~(alignment - 1);
        return std::aligned_alloc(alignment, rounded ? rounded : alignment);
    #endif
}

inline void aligned_free(void* p) noexcept {
    #if defined(_WIN32)
        _aligned_free(p);
    #else
        std::free(p);
    #endif
}

} // namespace llm

```

include/llm/tensor.hpp:

```

// include/llm/tensor.hpp
#pragma once
#include <stdint>
#include <initializer_list>
#include <memory>
#include <vector>

#include "llm/common.hpp"
#include "llm/dtype.hpp"
#include "llm/memory.hpp"

namespace llm {

```

```

using Shape    = std::vector<int64_t>;
using Strides  = std::vector<int64_t>;

int64_t numel(const Shape& shape);           // product of dims; 1 for a scalar
    ↪
Strides contiguous_strides(const Shape& shape); // row-major strides

// Raw, reference-counted, aligned byte buffer. Tensors hold a shared_ptr to one,
// so views share storage and the buffer is freed when the last view dies.
class Storage {
public:
    explicit Storage(size_t nbytes, size_t alignment = 64);
    ~Storage();
    Storage(const Storage&) = delete;
    Storage& operator=(const Storage&) = delete;

    uint8_t* data() noexcept { return data_; }
    const uint8_t* data() const noexcept { return data_; }
    size_t nbytes() const noexcept { return nbytes_; }

    // Instrumentation (single-threaded): how many Storages have ever been built.
    // The boss challenge uses this to prove view ops allocate nothing.
    static uint64_t alloc_count() noexcept { return s_alloc_count; }

private:
    uint8_t* data_ = nullptr;
    size_t nbytes_ = 0;
    inline static uint64_t s_alloc_count = 0;
};

class Tensor {
public:
    Tensor() = default;
    static Tensor empty(Shape shape, DType dtype, size_t alignment = 64);

    // ---- layout queries ----
    const Shape& shape() const noexcept { return shape_; }
    const Strides& strides() const noexcept { return strides_; }
    DType dtype() const noexcept { return dtype_; }
    size_t ndim() const noexcept { return shape_.size(); }
    int64_t numel() const noexcept { return llm::numel(shape_); }
    size_t nbytes() const noexcept {
        return static_cast<size_t>(numel()) * dtype_size(dtype_);
    }
    bool is_contiguous() const noexcept;
    bool defined() const noexcept { return storage_ != nullptr; }

```

```

// ---- data access ----
// Pointer to *this tensor's* first element (already includes the view offset).
↪
template <typename T> T* data_ptr() {
    return reinterpret_cast<T*>(storage_->data()) + offset_;
}
template <typename T> const T* data_ptr() const {
    return reinterpret_cast<const T*>(storage_->data()) + offset_;
}

// Element access for tests/debugging only (strided, branchy -- not hot paths).
↪
template <typename T> T& at(std::initializer_list<int64_t> idx) {
    return data_ptr<T>()[rel_offset(idx)];
}
template <typename T> const T& at(std::initializer_list<int64_t> idx) const {
    return data_ptr<T>()[rel_offset(idx)];
}

// ---- zero-copy views (Challenge 1.3) ----
Tensor reshape(Shape new_shape) const; // requires contiguous; shares
↪ storage
Tensor view(Shape new_shape) const { return reshape(std::move(new_shape)); }
Tensor permute(const std::vector<int>& dims) const;
Tensor transpose(int dim0, int dim1) const;
Tensor slice(int dim, int64_t start, int64_t stop) const;

// Materialize a contiguous copy (allocates iff not already contiguous).
Tensor contiguous() const;

// Identity helpers for tests: tensors sharing storage return the same pointer.
↪
const void* storage_id() const noexcept { return storage_.get(); }
int64_t offset() const noexcept { return offset_; }

private:
    int64_t rel_offset(std::initializer_list<int64_t> idx) const;

    std::shared_ptr<Storage> storage_;
    int64_t offset_ = 0; // in elements, from start of storage
    Shape shape_;
    Strides strides_;
    DType dtype_ = DType::F32;
};

} // namespace llm

```

src/tensor.cpp (the non-template methods; the view methods are added in 1.3):

```
// src/tensor.cpp
#include "llm/tensor.hpp"
#include <cstring>
#include <utility>

namespace llm {

int64_t numel(const Shape& shape) {
    int64_t n = 1;
    for (int64_t d : shape) n *= d;
    return n;    // empty shape -> 1 (a scalar)
}

Strides contiguous_strides(const Shape& shape) {
    Strides s(shape.size());
    int64_t acc = 1;
    for (int i = static_cast<int>(shape.size()) - 1; i >= 0; --i) {
        s[i] = acc;
        acc *= shape[i];
    }
    return s;
}

// ---- Storage ----
Storage::Storage(size_t nbytes, size_t alignment) : nbytes_(nbytes) {
    data_ = static_cast<uint8_t*>(aligned_malloc(nbytes, alignment));
    LLM_CHECK(data_ != nullptr, "Storage: allocation failed");
    ++s_alloc_count;
}

Storage::~Storage() { aligned_free(data_); }

// ---- Tensor ----
Tensor Tensor::empty(Shape shape, DType dtype, size_t alignment) {
    Tensor t;
    t.dtype_ = dtype;
    t.shape_ = std::move(shape);
    t.strides_ = contiguous_strides(t.shape_);
    t.offset_ = 0;
    const size_t bytes =
        static_cast<size_t>(llm::numel(t.shape_) * dtype_size(dtype));
    t.storage_ = std::make_shared<Storage>(bytes, alignment);
    return t;
}
```

```

bool Tensor::is_contiguous() const noexcept {
    int64_t expected = 1;
    for (int i = static_cast<int>(ndim()) - 1; i >= 0; --i) {
        if (shape_[i] == 1) continue;           // size-1 dims impose no constraint
        if (strides_[i] != expected) return false;
        expected *= shape_[i];
    }
    return true;
}

int64_t Tensor::rel_offset(std::initializer_list<int64_t> idx) const {
    LLM_ASSERT(idx.size() == ndim(), "at(): wrong number of indices");
    int64_t off = 0;
    size_t d = 0;
    for (int64_t i : idx) {
        LLM_ASSERT(i >= 0 && i < shape_[d], "at(): index out of range");
        off += i * strides_[d];
        ++d;
    }
    return off;    // relative to data_ptr(), which already includes offset_
}

} // namespace llm

```

Why these decisions. `int64_t` for shapes/strides/offsets is deliberate: real models have tensors that overflow 32-bit element counts (a vocabulary of 128k times a hidden size of 4096 is already half a billion elements, and intermediate products go higher). Storing the offset in *elements* and having `data_ptr` fold it in keeps `at()` indexing simple and prevents the classic “added the offset twice” bug. The `s_alloc_count` instrumentation looks like a curiosity now, but it is the meter the boss reads to *prove* — not assert by inspection — that views do not copy.

GOING FURTHER

- **Inline-capacity shape/strides.** `std::vector<int64_t>` heap-allocates, and every view copies two of them. Tensors rarely exceed ~6 dimensions, so production engines use a small-vector with inline storage (a fixed array plus a size, spilling to the heap only past the cap). Swapping `Shape/Strides` for such a type later removes a real allocation from the view path — and your boss challenge already isolates *data* allocations from these, so you can measure the win cleanly.
- **F16 conversions.** Implement `float f16_to_f32(uint16_t)` and `uint16_t`

`f32_to_f16(float)` via bit manipulation (or `std::bit_cast` plus the IEEE-754 half format). You will want them the moment a real model stores anything in half precision.

- **A fill/arrange helper.** A tiny `Tensor::arange(n)` filling $0, 1, 2, \dots$ makes every later test more readable.
- **Negative indices.** PyTorch-style negative indexing ($-1 = \text{last}$) is a five-line change to `rel_offset` and surprisingly nice when debugging.

1.2 Aligned Memory and the Arena

GOAL

Add a **bump (arena) allocator** for the transient memory a forward pass produces and discards, and understand *why* we align everything to 64 bytes. After this, allocating scratch during inference is a pointer increment, and resetting it between tokens is setting one integer to zero.

BACKGROUND

A forward pass is an allocation storm. Every layer produces intermediate activations — the output of a `matmul`, a normalized vector, an attention score buffer — that are consumed by the next step and then thrown away. If each went through `malloc/free`, you would pay two costs: the raw overhead of the allocator (locking, bookkeeping, searching free lists) on a path you run thousands of times per generated token, and **fragmentation** as differently-sized transient buffers churn. Neither is acceptable in a hot loop.

The fix is an **arena** (a.k.a. bump or linear allocator), beautiful in its simplicity. You allocate one big block up front. To “allocate” N bytes, you return the current position and advance a cursor by N . That is it — no free list, no search, no per-allocation header. You never free individual allocations; instead, when the whole batch of transients is no longer needed (e.g. at the end of a forward pass), you **reset** the arena by setting the cursor back to zero, reclaiming everything at once. The allocation cost is a few instructions; the deallocation cost is *one* instruction for the entire forward pass.

This maps perfectly onto inference’s structure: allocate an arena once when the engine starts; during each forward pass, carve activations out of it; `reset()` be-

tween passes. The same physical memory is reused token after token, hot in cache, never returned to the os. We wire the forward pass to an arena in Unit 5; here we build the allocator and prove it behaves.

Now, alignment — why 64 bytes? Two hardware realities. *Cache lines*: CPUs move memory in fixed-size chunks called cache lines — **64 bytes** on essentially all modern x86-64 and ARM64. When a buffer's start is aligned to 64, each logical "row" tends to begin at a cache-line boundary, avoiding a subtle penalty: a value straddling two cache lines forces the CPU to touch *both*, doubling memory traffic for that access. *SIMD*: the vector units of Unit 3 (SSE, AVX, AVX-512, NEON) load and store many elements at once — AVX-512 moves a full **64 bytes** (sixteen floats) per instruction. These wide loads are happiest, and on some microarchitectures *only correct* for the aligned-load instructions, when the address is a multiple of the vector width. 64 bytes satisfies both the cache line and the widest common SIMD register, which is why it is the alignment you see throughout serious inference engines.

One implementation detail, stated plainly: aligning a *size* up to a multiple of *A* is $(n + A - 1) \& \sim(A - 1)$ when *A* is a power of two — clear the low bits after rounding up. We use it both to satisfy `std::aligned_alloc`'s multiple-of-alignment requirement and to align each bump-allocation's starting offset.

THE TASK

Add an Arena class to include `llm/memory.hpp`: construct with a capacity (bytes) and an alignment (default 64), allocating one aligned backing block; `void* allocate(size_t)` aligns the cursor up, checks it still fits (LLM_CHECK on overflow), returns the pointer, advances the cursor; `reset()` sets the cursor to zero (remembering the peak via a high-water mark); and `used()`, `high_water()`, `capacity()` queries. Non-copyable; frees its block in the destructor. Every pointer `allocate` returns is aligned, and successive allocations do not overlap.

HINTS

Hint 1 — the bump

Keep `uint8_t* base_`, `size_t capacity_`, and a `size_t offset_` cursor. In `allocate`: compute `aligned_off = (offset_ + alignment_ - 1) & ~(alignment_ - 1)`, check `aligned_off + nbytes <= capacity_`, set the result to `base_ + aligned_off`, then `offset_ = aligned_off + nbytes`.

Hint 2 — high-water mark

After advancing, `high_water_ = max(high_water_, offset_)`. `reset()` zeroes `offset_` but leaves `high_water_`. This tells you the largest amount the arena ever needed simultaneously — invaluable for sizing the arena so it never overflows but does not waste memory.

Hint 3 — alignment must be a power of two

The `& ~(alignment-1)` trick only works for power-of-two alignments. Assert it in the constructor (`reuse` is `power_of_two`). Reuse `aligned_malloc/aligned_free` for the backing block so the arena's base is itself aligned.

TESTS

```
// add to tests/test_tensor.cpp (or a new tests/test_memory.cpp)
#include "llm/memory.hpp"

TEST(aligned_alloc_returns_aligned_pointer) {
    void* p = aligned_malloc(1000, 64);
    REQUIRE(p != nullptr);
    CHECK(reinterpret_cast<uintptr_t>(p) % 64 == 0);
    aligned_free(p);
}

TEST(arena_aligned_and_nonoverlapping) {
    Arena a(1u << 20, 64); // 1 MiB
    void* x = a.allocate(100);
    void* y = a.allocate(100);
    CHECK(reinterpret_cast<uintptr_t>(x) % 64 == 0);
    CHECK(reinterpret_cast<uintptr_t>(y) % 64 == 0);
    CHECK(x != y);
    // 100 bytes rounds up to 128 for the next aligned slot, so y is >= 64 past x
    CHECK(reinterpret_cast<char*>(y) - reinterpret_cast<char*>(x) >= 64);
}

TEST(arena_reset_reuses_memory_and_tracks_peak) {
    Arena a(1u << 20, 64);
    void* first = a.allocate(256);
    CHECK(a.used() == 256);
    a.reset();
    CHECK(a.used() == 0);
    void* again = a.allocate(256);
```



```

CHECK(first == again);           // same address after reset
CHECK(a.high_water() >= 256);    // peak remembered across the reset
}

```

REFERENCE SOLUTION

Add to `include/llm/memory.hpp`, inside namespace `llm`, after the aligned-allocation helpers:

```

// A bump / arena allocator for transient activations.
// Allocate once; carve out scratch with allocate(); reclaim it all with reset().
class Arena {
public:
    explicit Arena(size_t capacity_bytes, size_t alignment = 64)
        : alignment_(alignment), capacity_(capacity_bytes) {
        LLM_ASSERT(is_power_of_two(alignment), "alignment must be a power of two");
        ↪ base_ = static_cast<uint8_t*>(aligned_malloc(capacity_bytes, alignment));
        LLM_CHECK(base_ != nullptr, "Arena: backing allocation failed");
    }
    ~Arena() { aligned_free(base_); }
    Arena(const Arena&) = delete;
    Arena& operator=(const Arena&) = delete;

    void* allocate(size_t nbytes) {
        const size_t aligned_off = (offset_ + alignment_ - 1) & ~(alignment_ - 1);
        LLM_CHECK(aligned_off + nbytes <= capacity_, "Arena: out of memory");
        void* p = base_ + aligned_off;
        offset_ = aligned_off + nbytes;
        if (offset_ > high_water_) high_water_ = offset_;
        return p;
    }

    void reset() noexcept { offset_ = 0; } // keep high_water_
    size_t used() const noexcept { return offset_; }
    size_t high_water() const noexcept { return high_water_; }
    size_t capacity() const noexcept { return capacity_; }

private:
    uint8_t* base_ = nullptr;
    size_t alignment_ = 64;
    size_t capacity_ = 0;
    size_t offset_ = 0;
    size_t high_water_ = 0;
}

```

```
};
```

That is the whole allocator. Allocation is now a couple of arithmetic ops and a pointer add, and freeing a forward pass’s worth of activations is a single `reset()`.

Why it matters later. When we build the forward pass, we hand it an Arena and every intermediate tensor is carved from it instead of heap-allocated. The high-water mark tells us exactly how large to make that arena: run a forward pass at the maximum sequence length once, read `high_water()`, and size the arena to that. The result is an inference loop that, after warm-up, performs **zero heap allocations per token** — a property you can verify with the same `Storage::alloc_count()` meter from 1.1.

GOING FURTHER

- **Scoped sub-arenas.** Add a `Marker mark()` returning the current offset and a `void rewind(Marker)` restoring it. A function can then allocate temporaries and free *just its own* on return — a nested lifetime within the linear one.
- **Debug guard bytes.** In Debug builds, write a known pattern (`0xCD`) into freshly bumped regions, or place guard bytes between allocations and check them on reset, to catch buffer overruns in your kernels.
- **Over-alignment for specific tensors.** Let `allocate(nbytes, align)` take a per-call alignment $\geq$ the arena’s, so individual tensors can be more strongly aligned than the default.

1.3 Views and Reshape (Zero-Copy)

GOAL

Implement the operations that re-describe a tensor without moving data — reshape, permute, transpose, slice — as **views** that share storage with the original, plus a `contiguous()` that materializes a real copy only when one is genuinely needed.

BACKGROUND

This is the payoff for the storage/recipe separation. Each of these operations produces a new Tensor that points at the *same* Storage and differs only in shape, strides, and offset. No bytes are read or written. That is not just a memory optimization — it is a throughput optimization, because the data stays exactly where it is, hot in cache, while you change your mind about how to read it.

reshape reinterprets the same elements under a new shape with the same total count: a contiguous $[2, 6]$ becomes $[3, 4]$ or $[12]$ by recomputing contiguous strides. The catch: reshape-as-a-view is only valid when the tensor is **contiguous**, because it assumes the elements are in plain row-major order. If they are not (say you transposed first), there is no stride pattern that expresses the new shape over the old physical layout, so a copy is unavoidable. We make this explicit: reshape requires contiguity and tells you to call `contiguous()` first. (Many frameworks auto-copy; making it explicit teaches the invariant, and an engine generally *wants* to know when a copy is about to happen.)

transpose(d0, d1) swaps two dimensions by swapping their shape entries *and* their stride entries. The data does not move — you have changed which direction counts as “rows” and which as “columns.” The result is almost always non-contiguous, which is fine: strided kernels read it correctly via the indexing formula. This is the operation behind reading the same weight matrix as W or W^T for free.

permute(order) is the general case: an arbitrary reordering of all dimensions. Transpose is just permute restricted to swapping two axes. We use permute constantly in attention, where activations get reshaped to $[\text{batch}, \text{heads}, \text{seq}, \text{head_dim}]$ and the axes shuffled so each head’s data is laid out for a matmul.

slice(dim, start, stop) takes a sub-range along one dimension. The trick: advance the **offset** by $\text{start} * \text{stride}[\text{dim}]$ to point at the first element of the slice, and shrink $\text{shape}[\text{dim}]$ to $\text{stop} - \text{start}$. The strides are unchanged — you are looking at a window into the same buffer. This is how we will grab “the keys for positions 0 through t ” out of the kv cache without copying them.

contiguous() is the escape hatch. When you genuinely need elements laid out densely, it allocates a fresh buffer and copies the elements in row-major order, walking the source via its strides. If the tensor is *already* contiguous, it is a no-op that returns itself — no allocation. The art of fast tensor code is, in large part, calling `contiguous()` as rarely as possible: every call is a copy, and copies are what views exist to avoid.

The invariant tying it together: **a view shares storage; a copy allocates new storage**. You should be able to point at any line and say which it is. The boss will make you prove it.

THE TASK

Implement in `src/tensor.cpp` (declarations already in `tensor.hpp`): `reshape` (require equal element count and contiguity; share storage; recompute contiguous strides; keep the offset); `permute` (validate it is a true permutation; reorder shape and strides; share storage); `transpose` (swap the two shape and stride entries; share storage); `slice` (validate the range; advance offset by `start*strides[dim]`; set `shape[dim] = stop-start`; share storage); and `contiguous` (return `*this` if already contiguous; else allocate and copy by walking the source strides). Each view op leaves `storage_id()` unchanged and copies zero bytes; `contiguous()` on a non-contiguous tensor is the only one that allocates.

HINTS

Hint 1 — a view op is “copy `*this`, then edit the metadata”

For all four view ops, start with `Tensor out = *this;`. Copying a `Tensor` copies the `shared_ptr` (bumping the storage refcount — same bytes) and the small shape/stride vectors. Then mutate only `out.shape_`, `out.strides_`, and/or `out.offset_`. The storage pointer rides along untouched.

Hint 2 — slice changes the offset, not the strides

Slicing along `dim` means “skip the first `start` elements in that direction”: `offset_ += start * strides_[dim]`. The strides stay the same, and only `shape_[dim]` shrinks. Convince yourself with a 4x4: slicing rows `[1,3)` should land at `({0,0})` on the original at `({1,0})`.

Hint 3 — the contiguous() copy loop

The output is contiguous with offset 0, so its element `k` is at byte `k * element_size`. For each linear output index, find the matching source element: walk an `N`-dimensional coordinate counter (last dimension fastest), compute the source element index `offset_ + sum(coord_i * stride_i)`, and `memcpy` one element’s bytes. Increment the coordinate like an odometer.

ter.

Hint 4 — validating a permutation

A valid permutation of N dims uses each index in $[0, N)$ exactly once. Track a seen array; for each requested dim, check it is in range and not already seen, then mark it.

TESTS

```
// add to tests/test_tensor.cpp

TEST(reshape_zero_copy_preserves_data) {
    Tensor t = Tensor::empty({2, 6}, DType::F32);
    for (int i = 0; i < 12; ++i) t.data_ptr<float>()[i] = float(i);
    Tensor r = t.reshape({3, 4});
    CHECK(r.storage_id() == t.storage_id());           // SAME storage -> no copy
    CHECK(r.is_contiguous());
    CHECK_CLOSE(r.at<float>({2, 3}), 11.0f, 1e-6); // last element preserved
    r.at<float>({0, 0}) = 99.0f;                     // mutate the view...
    CHECK_CLOSE(t.at<float>({0, 0}), 99.0f, 1e-6); // ...base sees it (same bytes)
    ↪
}

TEST(transpose_swaps_strides_zero_copy) {
    Tensor t = Tensor::empty({2, 3}, DType::F32);
    for (int i = 0; i < 6; ++i) t.data_ptr<float>()[i] = float(i);
    Tensor tt = t.transpose(0, 1);                    // shape {3,2}
    CHECK(tt.storage_id() == t.storage_id());
    CHECK(tt.shape()[0] == 3 && tt.shape()[1] == 2);
    CHECK(!tt.is_contiguous());
    for (int i = 0; i < 2; ++i)                        // A[i][j] == A^T[j][i]
        for (int j = 0; j < 3; ++j)
            CHECK_CLOSE(t.at<float>({i, j}), tt.at<float>({j, i}), 1e-6);
}

TEST(slice_views_subregion) {
    Tensor t = Tensor::empty({4, 4}, DType::F32);
    for (int i = 0; i < 16; ++i) t.data_ptr<float>()[i] = float(i);
    Tensor rows = t.slice(0, 1, 3);                    // rows 1..2 -> shape {2,4}
    CHECK(rows.storage_id() == t.storage_id());
    CHECK(rows.shape()[0] == 2 && rows.shape()[1] == 4);
    CHECK_CLOSE(rows.at<float>({0, 0}), 4.0f, 1e-6); // == t[1][0]
```

```

    CHECK_CLOSE(rows.at<float>({1, 3}), 11.0f, 1e-6); // == t[2][3]
}

TEST(contiguous_materializes_a_transpose) {
    Tensor t = Tensor::empty({2, 3}, DType::F32);
    for (int i = 0; i < 6; ++i) t.data_ptr<float>()[i] = float(i);
    Tensor c = t.transpose(0, 1).contiguous(); // {3,2}, contiguous, real copy
    ↪
    CHECK(c.is_contiguous());
    CHECK(c.storage_id() != t.storage_id()); // different storage -> a copy
    const float* p = c.data_ptr<float>(); // memory order now
    ↪ 0,3,1,4,2,5
    CHECK_CLOSE(p[0], 0.0f, 1e-6);
    CHECK_CLOSE(p[1], 3.0f, 1e-6);
    CHECK_CLOSE(p[2], 1.0f, 1e-6);
}

TEST(reshape_requires_contiguous) {
    Tensor t = Tensor::empty({2, 3}, DType::F32);
    Tensor tt = t.transpose(0, 1); // non-contiguous
    Tensor ok = tt.contiguous().reshape({6}); // the supported path
    CHECK(ok.numel() == 6);
    CHECK(ok.is_contiguous());
}

```

REFERENCE SOLUTION

Add to `src/tensor.cpp` (inside namespace `llm`):

```

Tensor Tensor::reshape(Shape new_shape) const {
    LLM_CHECK(llm::numel(new_shape) == numel(),
        "reshape: element count must be preserved");
    LLM_CHECK(is_contiguous(),
        "reshape: tensor must be contiguous; call contiguous() first");
    Tensor out = *this; // shares storage_ (shared_ptr), same
    ↪ offset_
    out.shape_ = std::move(new_shape);
    out.strides_ = contiguous_strides(out.shape_);
    return out;
}

Tensor Tensor::permute(const std::vector<int>& dims) const {
    LLM_CHECK(dims.size() == ndim(), "permute: need one entry per dimension");
    Shape ns(ndim());

```

```

Strides nst(ndim());
std::vector<bool> seen(ndim(), false);
for (size_t i = 0; i < dims.size(); ++i) {
    const int d = dims[i];
    LLM_CHECK(d >= 0 && d < static_cast<int>(ndim()) && !seen[d],
              "permute: invalid permutation");
    seen[d] = true;
    ns[i] = shape_[d];
    nst[i] = strides_[d];
}
Tensor out = *this;
out.shape_ = std::move(ns);
out.strides_ = std::move(nst);
return out;
}

Tensor Tensor::transpose(int dim0, int dim1) const {
    LLM_CHECK(dim0 >= 0 && dim0 < static_cast<int>(ndim()) &&
              dim1 >= 0 && dim1 < static_cast<int>(ndim()),
              "transpose: dim out of range");
    Tensor out = *this;
    std::swap(out.shape_[dim0], out.shape_[dim1]);
    std::swap(out.strides_[dim0], out.strides_[dim1]);
    return out;
}

Tensor Tensor::slice(int dim, int64_t start, int64_t stop) const {
    LLM_CHECK(dim >= 0 && dim < static_cast<int>(ndim()), "slice: dim out of range
    ↪ ");
    LLM_CHECK(0 <= start && start <= stop && stop <= shape_[dim],
              "slice: range out of bounds");
    Tensor out = *this;
    out.offset_ = offset_ + start * strides_[dim]; // shift into the buffer
    out.shape_[dim] = stop - start; // shrink that dimension
    // strides unchanged
    return out;
}

Tensor Tensor::contiguous() const {
    if (is_contiguous()) return *this; // no copy needed
    Tensor out = Tensor::empty(shape_, dtype_);
    const size_t es = dtype_size(dtype_);
    const uint8_t* src = storage_->data();
    uint8_t* dst = out.storage_->data();
    const int64_t n = numel();
    std::vector<int64_t> coord(ndim(), 0);
    for (int64_t linear = 0; linear < n; ++linear) {

```

```

    int64_t src_elem = offset_;
    for (size_t d = 0; d < ndim(); ++d) src_elem += coord[d] * strides_[d];
    std::memcpy(dst + static_cast<size_t>(linear) * es,
                src + static_cast<size_t>(src_elem) * es, es);
    // odometer increment: last axis fastest
    for (int d = static_cast<int>(ndim()) - 1; d >= 0; --d) {
        if (++coord[d] < shape_[d]) break;
        coord[d] = 0;
    }
}
return out;
}

```

The mental model to keep. Reading these five functions back to back, notice that four of them are *pure metadata edits over shared storage* and exactly one touches bytes. That ratio — describe-don't-copy by default, materialize only on demand — is the entire design philosophy of fast tensor libraries, and it is why the next units can shuffle activations into whatever shape a kernel wants without drowning in memcpys.

GOING FURTHER

- **Broadcasting.** Implement `expand`, which sets a size-1 dimension's *stride* to 0 so iterating that axis re-reads the same element — the trick behind adding a bias vector to a matrix without copying. You will use it for bias adds and the attention mask.
- **as_strided / unfold.** A general “give me a view with these explicit shape and strides” primitive unlocks sliding-window views.
- **Negative-stride reversal.** A `flip(dim)` that points the offset at the last element and negates that dimension's stride reverses an axis with zero copies — a good check on whether your indexing is truly stride-driven.
- **Combine and stress.** Chain `slice` → `transpose` → `slice` and verify against a hand-computed layout.

1.4 Zero-Copy Discipline

BOSS CHALLENGE

A measured guarantee. Not “the views look right,” but a hard count proving they copy nothing: across a pipeline of slices, transposes, reshapes, and a thousand views, the number of data allocations stays constant — and a transpose-as-view is still element-for-element correct.

GOAL

Prove, with a meter rather than an inspection, that your view operations are genuinely zero-copy: across a pipeline of slices, transposes, reshapes, and a thousand successive views, the number of **data allocations stays constant** and the **storage stays shared** — while a transpose-as-view satisfies $A[i][j] = A^T[j][i]$ for every element.

BACKGROUND

Anyone can write a transpose that *looks* zero-copy. The discipline — the thing that separates an engine you can trust from one you hope is fast — is being able to *measure* that it copied nothing, so a future “optimization” that quietly introduces a copy gets caught immediately. This boss installs that habit.

The instrument is `Storage::alloc_count()` from Challenge 1.1: it ticks once per data-buffer allocation. A correct view operation never constructs a `Storage`, so across any sequence of views the count must not move. We anchor on a before reading and check *deltas*, so the test is robust no matter what other tests allocated first.

One honest caveat, stated because the expert reader will wonder about it: creating a view *does* still allocate the small shape and strides `std::vectors`. Those are *metadata* allocations, not *data* copies, and they are exactly what the inline-capacity small-vector extension from 1.1 removes. The boss deliberately measures `Storage` allocations — the megabyte-scale data buffers that actually matter — and not those few-dozen-byte vectors. Naming this precisely is itself part of the discipline.

THE TASK

Write a single test, `boss_zero_copy_discipline`, that: records `Storage::alloc_count()` as before; allocates one base tensor and asserts the count rose by exactly **1**; builds a pipeline of pure views (`slice` → `transpose` → `slice`) and asserts the count is *still* before + 1 and the result *shares storage*; reshapes the contiguous base and asserts no new allocation; takes **1000** successive views in a loop and asserts the count is unchanged; then builds a fresh tensor, transposes it as a view, asserts the transpose added **zero** allocations, and verifies $A[i][j] = A^T[j][i]$ across *all* elements.

The bar (the boss): every view step adds **zero** Storage allocations, storage identity is preserved through the whole pipeline, and the transpose-as-view is element-for-element correct. If the meter ever moves on a view op, you have a hidden copy — fix it.

HINTS

Hint 1 — anchor on deltas, not absolutes

Other tests run first and allocate, so never assert `alloc_count() == 1`. Capture `const uint64_t before = Storage::alloc_count();` at the top and assert `alloc_count() == before + k` for the small, known `k`.

Hint 2 — a guaranteed-view “no-op” slice for the 1000-loop

`t.slice(dim, 0, t.shape()[dim])` selects the whole dimension — logically a no-op, but it still goes through the view path and produces a new Tensor sharing storage. Reassign `t` to it 1000 times; the Storage count must not budge.

Hint 3 — what failure looks like

If `alloc_count()` jumps inside the pipeline, some “view” is calling `Tensor::empty` or `contiguous()` internally. The usual culprit is a reshape that silently materialized, or a slice/transpose that copied instead of editing metadata. The meter points you straight at the offending op.

TESTS

```

// add to tests/test_tensor.cpp

TEST(boss_zero_copy_discipline) {
    const uint64_t before = Storage::alloc_count();

    Tensor base = Tensor::empty({1024, 1024}, DType::F32); // (1 allocation)
    CHECK(Storage::alloc_count() == before + 1);

    // A pipeline of pure views: none may allocate data or copy bytes.
    Tensor v = base.slice(0, 0, 512) // {512,1024}
                .transpose(0, 1) // {1024,512}
                .slice(0, 0, 256); // {256,512}
    CHECK(Storage::alloc_count() == before + 1); // still just the base
    CHECK(v.storage_id() == base.storage_id()); // same bytes

    Tensor r = base.reshape({1024 * 1024}); // contiguous reshape: zero-
    ↪ copy
    CHECK(Storage::alloc_count() == before + 1);
    CHECK(r.storage_id() == base.storage_id());

    Tensor t = base; // 1000 views allocate no
    ↪ data
    for (int i = 0; i < 1000; ++i) t = t.slice(1, 0, t.shape()[1]);
    CHECK(Storage::alloc_count() == before + 1);
    CHECK(t.storage_id() == base.storage_id());

    Tensor a = Tensor::empty({64, 48}, DType::F32); // (+1 allocation)
    for (int i = 0; i < 64 * 48; ++i) a.data_ptr<float>()[i] = float(i);
    const uint64_t after_a = Storage::alloc_count();
    Tensor at = a.transpose(0, 1); // {48,64}
    CHECK(Storage::alloc_count() == after_a); // transpose copied nothing
    CHECK(at.storage_id() == a.storage_id());

    bool all_match = true;
    for (int i = 0; i < 64 && all_match; ++i)
        for (int j = 0; j < 48; ++j)
            if (a.at<float>({i, j}) != at.at<float>({j, i})) { all_match = false;
    ↪ break; }
    CHECK(all_match);
}

```

REFERENCE SOLUTION

There is no new production code to write — the boss is cleared by the Unit 1 implementations behaving correctly under the meter. The test above *is* the deliverable. Build and run:

```
./scripts/build.sh          # or: cmake --build build && ./build/llm_tests
```

Expected: `boss_zero_copy_discipline` reports [OK], and the full suite ends 0 failed. If it does not, the failing CHECK tells you precisely which view operation introduced a copy — that is the boss doing its job.

What you have actually proven. Not “I wrote views” but “across a realistic pipeline plus a thousand-iteration stress, my view operations performed exactly zero data allocations, kept storage shared throughout, and produced bit-correct transposed access.” That is a property you can now *defend with a number* — and re-check after any future change to the tensor code. This is the texture of the whole book: build the thing, then make a measurement assert that it is right.

GOING FURTHER

- **Forward-pass dry run.** Combine this with the Arena: allocate one arena, simulate carving a few activation tensors per “step,” `reset()` between steps, and assert via `Storage::alloc_count()` that after the first step no further data allocations occur — a preview of the zero-allocation inference loop you achieve for real in Unit 5/14.
- **Metadata-allocation count.** Add a second counter for shape/stride vector allocations and watch the inline-small-vector extension drive *it* to zero across the 1000-view loop.
- **Thread-safety note.** `s_alloc_count` is a plain `uint64_t`, fine for single-threaded tests. When Unit 3 introduces threads, make it `std::atomic` (or scope the meter to a single thread) so the instrument itself is not a data race.

Unit 1 — Milestone check

MILESTONE CHECK

You now have the foundation every kernel and layer will stand on:

- a **Tensor** type with shape, strides, dtype, and `shared_ptr<Storage>`, that knows its element count, byte size, and contiguity, and can read/write elements for testing;
- **aligned allocation** and an **arena allocator** that turns per-forward-pass scratch into a pointer bump and a single `reset()` — the basis for zero-allocation inference;
- **zero-copy views** — `reshape`, `permute`, `transpose`, `slice` — that re-describe data for free, with `contiguous()` as the explicit, measurable escape hatch when a real copy is required;
- and a **boss-grade guarantee**, backed by an allocation meter, that your views copy zero bytes of data.

The engine can now *hold and reshape* numbers. In **Unit 2** we make it *compute* with them: the elementwise math, the activation functions every Transformer needs (SiLU, GELU, and the gated SwiGLU), RMSNorm, and a numerically-stable softmax — the kernels that, together with matrix multiplication in Unit 3, do the actual arithmetic of a forward pass.

UNIT 2

Math Kernels

What you will build. The per-element and reduction kernels that — together with matrix multiplication in Unit 3 — perform the actual arithmetic of a forward pass: **elementwise operations** (add, multiply, scale, the residual add), the **activation functions** every Transformer needs (SiLU, GELU, and the gated SwiGLU), **RMSNorm**, and a **numerically-stable softmax**. The boss makes you prove the softmax survives adversarial inputs while matching a high-precision reference.

Where this fits. A Transformer block is: normalize, project, attend, project, add residual; then normalize, project, activate, project, add residual. Strip out the projections (matmul, Unit 3) and what is left — the norms, the activations, the softmax, the residual adds — is exactly this unit. These kernels are simple to write and easy to get *subtly* wrong; the errors that bite are numerical, not logical.

There is a temptation to wave these off as “just loops.” Two things make them worth a unit. First, they are **memory-bound**: an elementwise add reads $2N$ floats, writes N , and does N trivial additions, so the bottleneck is memory traffic, not arithmetic — which means how you *iterate* matters more than how you compute, and sets up the SIMD and cache work to come. Second, the activations and especially the softmax are where **floating-point numerics** stop being academic: e^{1000} is infinity, and a softmax that computes it naively returns NaN on inputs your model will absolutely produce. We build each kernel correct-and-stable, and the boss measures it.

We need one place for these to live. Create `include/llm/ops.hpp` and `src/ops.cpp`, add `src/ops.cpp` to the `llm_core` sources and `tests/test_ops.cpp` to `llm_tests`, and we will grow both files across the four challenges.

2.1 Elementwise Operations

GOAL

A small set of elementwise float kernels over contiguous tensors — `add`, `mul`, `scale`, the in-place residual `add_inplace` — plus a generic `apply_unary` that maps any scalar function over a tensor (the vehicle for the activations in 2.2).

BACKGROUND

An elementwise operation applies the same scalar computation independently to every element: $o_i = f(a_i)$ or $o_i = f(a_i, b_i)$. Because the per-element work is tiny and the data volume is large, these kernels are **memory-bandwidth-bound** — the processor spends its time waiting for memory, not calculating. The single most important consequence is that you want to walk memory *contiguously*, one cache line after the next, so the hardware prefetcher can stay ahead of you. That is why, for now, we require the inputs to be **contiguous** and simply iterate the flat buffer with `data_ptr<float>()` over `numel()` — one tight loop, sequential access. (The general strided case is a Going-Further exercise; in the engine we will arrange for these inputs to be contiguous.)

Two design choices recur. **In-place vs out-of-place:** `add(out, a, b)` writes a fresh buffer, while `add_inplace(a, b)` does `a += b`. The residual connections in a Transformer ($x \leftarrow x + \text{sublayer}(x)$) are naturally in-place, and doing them in place saves a buffer and a pass over memory. **The map pattern:** rather than hand-write a loop for every activation, we write one `apply_unary(out, in, f)` that takes a callable and applies it elementwise. Pass it `siluf` and you have a SiLU kernel; pass it `geluf` and you have GELU. Because `f` is a template parameter, the compiler inlines it into the loop — the abstraction costs nothing.

A note that pays off in Unit 3: write these loops in the simplest possible form — `for (i) o[i] = a[i] + b[i];` over a known count, no aliasing tricks, no early exits. That shape is exactly what a compiler can **auto-vectorize** into SIMD instructions. You often get much of the SIMD speedup for free, just by not getting in the optimizer's way.

THE TASK

In `ops.hpp` declare, and in `ops.cpp` define, float kernels over contiguous tensors: `add(out, a, b)` ($o = a + b$), `mul(out, a, b)` ($o = a \odot b$, the elementwise product we need for gating), `scale(out, in, float s)` ($o = s \cdot \text{in}$), and `add_inplace(a, b)` ($a += b$). Validate contiguity and matching element counts with `LLM_ASSERT`. In `ops.hpp` also provide a template `apply_unary(out, in, f)` that maps a scalar callable over a contiguous tensor.

HINTS

Hint 1 — one shape for all of them

Every kernel is the same skeleton: assert contiguous and equal `numel()`, fetch raw `float*` via `data_ptr<float>()`, then a single `for` over `numel()`. Factor the assertions into a small helper.

Hint 2 — `apply_unary` is a template in the header

Because the callable type is generic, `apply_unary` must be a template, so it lives in `ops.hpp`. The body is identical to the binary kernels but calls `f(x[i])`. Passing a stateless lambda or a function pointer both inline well.

TESTS

```
// tests/test_ops.cpp
#include "test_framework.hpp"
#include "llm/ops.hpp"
#include <cmath>
using namespace llm;

static Tensor vec(std::initializer_list<float> xs) {
    Tensor t = Tensor::empty({(int64_t)xs.size()}, DType::F32);
    float* p = t.data_ptr<float>();
    int i = 0; for (float x : xs) p[i++] = x;
    return t;
}

TEST(elementwise_add_mul_scale) {
    Tensor a = vec({0, 1, 2, 3});
    Tensor b = vec({0, 2, 4, 6});
```



```

Tensor o = Tensor::empty({4}, DType::F32);
add(o, a, b); CHECK_CLOSE(o.at<float>({3}), 9.0f, 1e-6);
mul(o, a, b); CHECK_CLOSE(o.at<float>({3}), 18.0f, 1e-6);
scale(o, a, 10.0f); CHECK_CLOSE(o.at<float>({2}), 20.0f, 1e-6);
}

TEST(add_inplace_is_residual) {
    Tensor x = vec({1, 1, 1});
    Tensor d = vec({2, 3, 4});
    add_inplace(x, d); // x <- x + d
    CHECK_CLOSE(x.at<float>({0}), 3.0f, 1e-6);
    CHECK_CLOSE(x.at<float>({2}), 5.0f, 1e-6);
}

TEST(apply_unary_maps_a_function) {
    Tensor in = vec({-2, 0, 3});
    Tensor out = Tensor::empty({3}, DType::F32);
    apply_unary(out, in, [](float v) { return v * v; });
    CHECK_CLOSE(out.at<float>({0}), 4.0f, 1e-6);
    CHECK_CLOSE(out.at<float>({2}), 9.0f, 1e-6);
}

```

REFERENCE SOLUTION

include/llm/ops.hpp (we grow this header through the unit):

```

// include/llm/ops.hpp
#pragma once
#include <stdint>
#include "llm/tensor.hpp"

namespace llm {

// ---- Elementwise (contiguous, float32) ----
void add(Tensor& out, const Tensor& a, const Tensor& b); // out = a + b
void mul(Tensor& out, const Tensor& a, const Tensor& b); // out = a * b
void scale(Tensor& out, const Tensor& in, float s); // out = s * in
void add_inplace(Tensor& a, const Tensor& b); // a += b (
    ↪ residual)

// Map a scalar callable over a contiguous tensor: out[i] = f(in[i]).
template <typename F>
inline void apply_unary(Tensor& out, const Tensor& in, F f) {
    LLM_ASSERT(out.is_contiguous() && in.is_contiguous(),

```

```

        "apply_unary: tensors must be contiguous");
    LLM_ASSERT(out.numel() == in.numel(), "apply_unary: size mismatch");
    float* o = out.data_ptr<float>();
    const float* x = in.data_ptr<float>();
    const int64_t n = in.numel();
    for (int64_t i = 0; i < n; ++i) o[i] = f(x[i]);
}

} // namespace llm

```

src/ops.cpp:

```

// src/ops.cpp
#include "llm/ops.hpp"

namespace llm {

static void require_same(const Tensor& a, const Tensor& b) {
    LLM_ASSERT(a.is_contiguous() && b.is_contiguous(), "elementwise: need
    ↪ contiguous");
    LLM_ASSERT(a.numel() == b.numel(), "elementwise: size mismatch");
}

void add(Tensor& out, const Tensor& a, const Tensor& b) {
    require_same(a, b); require_same(out, a);
    float* o = out.data_ptr<float>();
    const float* x = a.data_ptr<float>();
    const float* y = b.data_ptr<float>();
    const int64_t n = a.numel();
    for (int64_t i = 0; i < n; ++i) o[i] = x[i] + y[i];
}

void mul(Tensor& out, const Tensor& a, const Tensor& b) {
    require_same(a, b); require_same(out, a);
    float* o = out.data_ptr<float>();
    const float* x = a.data_ptr<float>();
    const float* y = b.data_ptr<float>();
    const int64_t n = a.numel();
    for (int64_t i = 0; i < n; ++i) o[i] = x[i] * y[i];
}

void scale(Tensor& out, const Tensor& in, float s) {
    require_same(out, in);
    float* o = out.data_ptr<float>();
    const float* x = in.data_ptr<float>();

```

```

    const int64_t n = in.numel();
    for (int64_t i = 0; i < n; ++i) o[i] = s * x[i];
}

void add_inplace(Tensor& a, const Tensor& b) {
    require_same(a, b);
    float* x = a.data_ptr<float>();
    const float* y = b.data_ptr<float>();
    const int64_t n = a.numel();
    for (int64_t i = 0; i < n; ++i) x[i] += y[i];
}

} // namespace llm

```

Why this shape. Keeping every kernel as a bare loop over a contiguous buffer is not laziness — it is the form the auto-vectorizer recognizes, and it is the form we will hand-vectorize in Unit 3 when we want to beat the compiler. The `apply_unary` template means the activations in the next challenge are three lines each, with no per-activation loop to maintain and no abstraction penalty at runtime.

GOING FURTHER

- **Strided support.** Generalize the kernels to non-contiguous inputs by walking coordinates and using strides (as `contiguous()` did in Unit 1). Slower, but occasionally necessary; the engine mostly avoids it.
- **Row broadcasting.** `add_bias(out, x, bias)` where `x` is `[..., D]` and `bias` is `[D]`: `o[r*D + c] = x[r*D + c] + bias[c]`. The first taste of broadcasting; useful even though Llama-style models use few biases.
- **Fusion.** A fused `scale_add(out, a, s, b)` computing $o = s \cdot a + b$ in one pass halves the memory traffic of doing them separately — a real win for bandwidth-bound code, and a preview of why operator fusion matters in Unit 14.

2.2 Activations: SiLU, GELU, SwiGLU

GOAL

The nonlinearities a Llama-family network runs on: a numerically-stable **sigmoid**, **SiLU** (the gate), **GELU** (exact and the common tanh approximation), and **SwiGLU** — the gated feed-forward combine $\text{SiLU}(g) \odot u$ that defines the modern FFN.

BACKGROUND

An activation is the per-element nonlinearity that lets a network represent more than a linear map. Three matter for us.

Sigmoid squashes any real number into $(0, 1)$:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

It looks harmless and hides a numerical trap. For very negative x , e^{-x} overflows to infinity; the naive formula then computes $1/\infty = 0$, which happens to be the right answer, but on some paths you get $\infty/\infty = \text{NaN}$. The fix is to pick the algebraically-equivalent form that never exponentiates a large positive number: for $x \geq 0$ use $1/(1 + e^{-x})$ (here $-x \leq 0$, so $e^{-x} \in (0, 1]$), and for $x < 0$ use $e^x/(1 + e^x)$ (here $x < 0$). Each branch only ever evaluates e^{negative} , which is bounded. This “choose the stable branch” move is the same idea that makes softmax safe, and it is worth getting into your hands now on the simplest case.

SiLU (also called *swish*) is sigmoid-gated identity:

$$\text{SiLU}(x) = x \sigma(x).$$

Unlike ReLU it is smooth and lets a little negative signal through, and it is the activation inside Llama’s feed-forward block. Because it is built on σ , it inherits the stable implementation for free.

GELU (Gaussian Error Linear Unit) weights the input by the probability that a standard normal is below it:

$$\text{GELU}(x) = x \Phi(x) = \frac{1}{2} x \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right).$$

This exact form needs `std::erf`. Many models ship the cheaper *tanh approximation*,

$$\text{GELU}(x) \approx \frac{1}{2} x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right] \right),$$

and which one a model expects is a metadata detail you must respect to match its outputs (we will read it from the model in Unit 10). We implement both. GPT-family models use GELU; Llama-family use SiLU/SwiGLU — we want both in the toolbox.

SwiGLU is not a pointwise function of one value; it is the *gated* combine at the heart of the modern feed-forward block. The `FFN` computes two separate linear projections of its input — a *gate* $g = xW_{\text{gate}}$ and an *up* $u = xW_{\text{up}}$ — and combines them elementwise:

$$\text{SwiGLU}(g, u) = \text{SiLU}(g) \odot u,$$

after which a third projection (W_{down}) maps back down. The two projections are matmuls (Unit 3); assembling the full `FFN` is Unit 5. The *kernel* this challenge owns is the elementwise step: given the two equal-length vectors g and u , produce $\text{SiLU}(g_i) u_i$. (This gating is exactly why we built an elementwise `mul` in 2.1 — though here we fuse the SiLU and the product into one pass.)

THE TASK

Add to `ops.hpp` inline scalar functions `sigmoidf`, `siluf`, `geluf` (tanh approximation), and `gelu_exactf` (erf-based) — `sigmoidf` must use the branch-stable form. Provide tensor wrappers `silu(out, in)` and `gelu(out, in)` (built on `apply_unary`), and declare `swiglu(out, gate, up)`; define `swiglu` in `ops.cpp` as a single fused pass computing $o_i = \text{SiLU}(g_i) u_i$.

HINTS

Hint 1 — the stable sigmoid branch

```
if (x >= 0) { float z = std::exp(-x); return 1 / (1 + z); } else {
float z = std::exp(x); return z / (1 + z); }. Test it at  $\pm 1000$ : both
must be finite, giving  $\approx 1$  and  $\approx 0$ .
```

Hint 2 — the GELU constants

$\sqrt{2/\pi} \approx 0.7978845608$ and the cubic coefficient is 0.044715 for the tanh form; the exact form uses $1/\sqrt{2} \approx 0.7071067812$ inside `std::erf`. Define them as `constexpr float`.

Hint 3 — SwiGLU is a fused binary kernel

It is shaped like mul from 2.1, but instead of $o[i] = g[i] * u[i]$ you write $o[i] = \text{siluf}(g[i]) * u[i]$. One pass, one buffer; do not compute SiLU into a temporary first.

TESTS

```
// add to tests/test_ops.cpp

TEST(sigmoid_is_stable_at_extremes) {
    CHECK(std::isfinite(sigmoidf(-1000.0f)));
    CHECK(std::isfinite(sigmoidf( 1000.0f)));
    CHECK_CLOSE(sigmoidf(-1000.0f), 0.0f, 1e-6);
    CHECK_CLOSE(sigmoidf( 1000.0f), 1.0f, 1e-6);
    CHECK_CLOSE(sigmoidf(0.0f),    0.5f, 1e-6);
}

TEST(silu_matches_definition) {
    CHECK_CLOSE(siluf(0.0f), 0.0f, 1e-6);           // 0 * sigma(0) = 0
    const float x = 1.5f;
    CHECK_CLOSE(siluf(x), x * (1.0f / (1.0f + std::exp(-x))), 1e-6);
}

TEST(gelu_tanh_close_to_exact) {
    for (float x : {-2.0f, -0.5f, 0.0f, 0.7f, 3.0f})
        CHECK_CLOSE(geluf(x), gelu_exactf(x), 2e-3); // approx within a few 1e-3
}

TEST(swiglu_is_silu_gate_times_up) {
    Tensor g = Tensor::empty({2}, DType::F32);
    Tensor u = Tensor::empty({2}, DType::F32);
    Tensor o = Tensor::empty({2}, DType::F32);
    g.data_ptr<float>()[0] = 1.0f; g.data_ptr<float>()[1] = -1.0f;
    u.data_ptr<float>()[0] = 3.0f; u.data_ptr<float>()[1] = 5.0f;
    swiglu(o, g, u);
    CHECK_CLOSE(o.at<float>({0}), siluf( 1.0f) * 3.0f, 1e-6);
    CHECK_CLOSE(o.at<float>({1}), siluf(-1.0f) * 5.0f, 1e-6);
}
```

REFERENCE SOLUTION

Add to ops.hpp (inside namespace `llm`, alongside the elementwise decls):

```
#include <cmath>

// ---- Scalar activations (inline; the compiler folds these into loops) ----
inline float sigmoidf(float x) {
    // Branch-stable: each path only ever exponentiates a non-positive number.
    if (x >= 0.0f) { const float z = std::exp(-x); return 1.0f / (1.0f + z); }
    else           { const float z = std::exp( x); return z / (1.0f + z); }
}

inline float siluf(float x) { return x * sigmoidf(x); }

inline float geluf(float x) {
    // tanh approximation
    constexpr float kSqrt20verPi = 0.7978845608028654f;
    const float x3 = x * x * x;
    return 0.5f * x * (1.0f + std::tanh(kSqrt20verPi * (x + 0.044715f * x3)));
}

inline float gelu_exactf(float x) {
    // exact, via erf
    constexpr float kInvSqrt2 = 0.7071067811865476f;
    return 0.5f * x * (1.0f + std::erf(x * kInvSqrt2));
}

// ---- Tensor activation wrappers ----
inline void silu(Tensor& out, const Tensor& in) {
    apply_unary(out, in, [](float v) { return siluf(v); });
}
inline void gelu(Tensor& out, const Tensor& in) {
    apply_unary(out, in, [](float v) { return geluf(v); });
}

// SwiGLU's elementwise step: out = SiLU(gate) * up (one fused pass).
void swiglu(Tensor& out, const Tensor& gate, const Tensor& up);
```

Add to ops.cpp:

```
void swiglu(Tensor& out, const Tensor& gate, const Tensor& up) {
    require_same(gate, up); require_same(out, gate);
    float* o = out.data_ptr<float>();
    const float* g = gate.data_ptr<float>();
    const float* u = up.data_ptr<float>();
```

```

const int64_t n = gate.numel();
for (int64_t i = 0; i < n; ++i) o[i] = siluf(g[i]) * u[i];
}

```

Why the stable branch matters in practice. It is tempting to think activations never see extreme inputs, but pre-activation values in a deep network — especially before training has settled, or in a quantized model where a few weights are large — routinely reach into the tens or hundreds. A naive `sigmoidf` that returns NaN once, anywhere, poisons everything downstream: one NaN in a logit vector and your sampler picks garbage. Writing the stable form here, on the easy case, builds the reflex you will need when the softmax forces the issue in the boss.

GOING FURTHER

- **Fast approximations.** A polynomial or bit-trick approximation to $\exp/\tanh$ can speed activations several-fold. Measure the accuracy trade-off against the exact form — and keep the exact one as your reference oracle.
- **Other gates.** GEGLU ($\text{GELU}(g) \odot u$) and ReGLU swap the gate function. Parameterizing `swiglu` over the gate function (a template, like `apply_unary`) gives you all of them with one kernel.
- **In-place SwiGLU.** If gate may be overwritten, write the result into it to save a buffer — mirroring the residual-add reasoning from 2.1.

2.3 RMSNorm

GOAL

Root-mean-square normalization: rescale each row vector by its own RMS and then by a learned per-dimension weight. This is the normalization Llama-family models use before attention and before the feed-forward block.

BACKGROUND

Normalization layers keep activations in a sane numeric range as signal flows through dozens of layers, which is what makes deep Transformers trainable and stable. **RMSNorm** is the variant modern LLMs adopted because it is simpler and cheaper than the classic LayerNorm while working just as well in practice.

For a row vector $x \in \mathbb{R}^D$ (one token's hidden state) and a learned weight $w \in \mathbb{R}^D$:

$$\text{RMSNorm}(x)_i = \frac{x_i}{\text{rms}(x)} w_i, \quad \text{rms}(x) = \sqrt{\frac{1}{D} \sum_{j=1}^D x_j^2 + \varepsilon}.$$

Read it in two steps. First, compute the root-mean-square of the vector — the square root of the mean of its squared entries — and divide every entry by it; this forces the vector to unit RMS regardless of its incoming scale. Then multiply elementwise by the learned weight w , which lets the model re-scale each dimension as it sees fit.

Two contrasts with LayerNorm are worth noting, because they explain why it is cheaper. RMSNorm does **not subtract the mean** (no centering) and has **no bias** term — it only rescales. Dropping the mean removes a reduction and a subtraction pass; in a model that runs this twice per layer across many layers, that adds up. The small constant ε (typically 10^{-5} or 10^{-6}) sits inside the square root to prevent division by zero on an all-zero vector and to bound the result when x is tiny.

One numerical detail that is easy to miss and important to get right: accumulate the sum of squares in **at least float32**, even if the activations themselves are stored in a lower precision. Squaring amplifies magnitude, and summing D (often thousands) of those squares loses precision quickly in half precision. Reference implementations compute this reduction in float32 (sometimes float64); we work in float32 throughout this unit, so we are already safe, but the principle — *reduce in higher precision than you store* — returns the moment we add quantization.

The operation is row-wise: each token's D -vector is normalized independently of the others. So for an input shaped $[N, D]$ (or any shape whose last dimension is the hidden size), we loop over the N rows and normalize each across its last dimension.

THE TASK

Declare `rmsnorm(out, in, weight, eps = 1e-5f)` in `ops.hpp` and define it in `ops.cpp`. Treat `in` as rows of length D = its last dimension; `weight` is a $[D]$ vector. For each row, compute the mean of squares plus `eps`, take the reciprocal square root, and write `out[j] = in[j] * inv_rms * weight[j]`. Accumulate the sum of squares in `float`. Assert contiguity and that `weight`'s length equals D .

HINTS

Hint 1 — rows times last-dim

Let $D = \text{in.shape().back}()$ and $\text{rows} = \text{in.numel}() / D$. Row r starts at `in.data_ptr<float>() + r*D`. Two inner loops over D : one to accumulate $\sum x_j^2$, one to write the normalized, weighted output.

Hint 2 — compute the reciprocal once

Form `inv_rms = 1.0f / std::sqrt(ss / D + eps)` once per row, then multiply — division is far more expensive than multiplication, and you would otherwise do D of them.

Hint 3 — a self-checking property

With `weight` all ones and `eps = 0`, the output of a row has rms exactly 1 by construction. That gives you a test that does not require hand-computing expected values.

TESTS

```
// add to tests/test_ops.cpp

TEST(rmsnorm_unit_weight_gives_unit_rms) {
    const int D = 4;
    Tensor x = Tensor::empty({1, D}, DType::F32);
    Tensor w = Tensor::empty({D}, DType::F32);
    Tensor o = Tensor::empty({1, D}, DType::F32);
    for (int j = 0; j < D; ++j) { x.data_ptr<float>()[j] = float(j + 1); // 1,2,3,4
        ↪                               w.data_ptr<float>()[j] = 1.0f; }
```

```

    rmsnorm(o, x, w, 0.0f);
    float ss = 0.0f;
    for (int j = 0; j < D; ++j) { float v = o.data_ptr<float>()[j]; ss += v * v; }
    CHECK_CLOSE(std::sqrt(ss / D), 1.0f, 1e-5);           // unit RMS by construction
}

TEST(rmsnorm_weight_scales_each_dim) {
    const int D = 3;
    Tensor x = Tensor::empty({1, D}, DType::F32);
    Tensor w = Tensor::empty({D}, DType::F32);
    Tensor o1 = Tensor::empty({1, D}, DType::F32);
    Tensor o2 = Tensor::empty({1, D}, DType::F32);
    for (int j = 0; j < D; ++j) x.data_ptr<float>()[j] = 1.0f; // rms(x) = 1
    for (int j = 0; j < D; ++j) w.data_ptr<float>()[j] = 1.0f;
    rmsnorm(o1, x, w, 0.0f);
    w.data_ptr<float>()[1] = 2.0f; // double dim 1
    rmsnorm(o2, x, w, 0.0f);
    CHECK_CLOSE(o2.at<float>({0, 1}), 2.0f * o1.at<float>({0, 1}), 1e-6);
    CHECK_CLOSE(o2.at<float>({0, 0}), o1.at<float>({0, 0}), 1e-6);
}

TEST(rmsnorm_rows_are_independent) {
    const int D = 2;
    Tensor x = Tensor::empty({2, D}, DType::F32);
    Tensor w = Tensor::empty({D}, DType::F32);
    Tensor o = Tensor::empty({2, D}, DType::F32);
    float v[4] = {3, 4, 30, 40}; // row1 = 10 *
    ↪ row0
    for (int i = 0; i < 4; ++i) x.data_ptr<float>()[i] = v[i];
    for (int j = 0; j < D; ++j) w.data_ptr<float>()[j] = 1.0f;
    rmsnorm(o, x, w, 0.0f);
    // scaling a row by a constant does not change its RMS-normalized result
    CHECK_CLOSE(o.at<float>({0, 0}), o.at<float>({1, 0}), 1e-5);
    CHECK_CLOSE(o.at<float>({0, 1}), o.at<float>({1, 1}), 1e-5);
}

```

REFERENCE SOLUTION

Declare in `ops.hpp`:

```
// Row-wise RMS normalization over the last dimension, scaled by `weight` [D].
void rmsnorm(Tensor& out, const Tensor& in, const Tensor& weight, float eps = 1e-5
    ↪ f);
```

Define in `ops.cpp`:

```
#include <cmath>

void rmsnorm(Tensor& out, const Tensor& in, const Tensor& weight, float eps) {
    LLM_ASSERT(in.is_contiguous() && out.is_contiguous() && weight.is_contiguous(),
    ↪
        "rmsnorm: tensors must be contiguous");
    LLM_ASSERT(in.numel() == out.numel(), "rmsnorm: in/out size mismatch");
    const int64_t D = in.shape().back();
    LLM_ASSERT(weight.numel() == D, "rmsnorm: weight length must equal last dim");
    const int64_t rows = in.numel() / D;

    const float* x = in.data_ptr<float>();
    const float* w = weight.data_ptr<float>();
    float* o = out.data_ptr<float>();

    for (int64_t r = 0; r < rows; ++r) {
        const float* xr = x + r * D;
        float* orow = o + r * D;

        float ss = 0.0f; // accumulate in float32
        for (int64_t j = 0; j < D; ++j) ss += xr[j] * xr[j];

        const float inv_rms = 1.0f / std::sqrt(ss / static_cast<float>(D) + eps);
        for (int64_t j = 0; j < D; ++j) orow[j] = xr[j] * inv_rms * w[j];
    }
}
```

Why RMSNorm, and why it is cheap. By dropping mean-centering and bias, RMSNorm needs one reduction (sum of squares) and two elementwise passes per row, versus LayerNorm's two reductions (mean, then variance) and an extra add. Run twice per layer over, say, 32 layers, that saving is real — and empirically the centering term was not buying much for these models. The learned weight preserves the model's ability to emphasize or suppress individual feature

dimensions, which is the part that actually mattered.

GOING FURTHER

- **Fuse the next matmul's scale.** RMSNorm is immediately followed by a linear projection; some engines fold the `inv_rms` scaling into the matmul to avoid a separate pass. A Unit 14 optimization, but worth filing away now.
- **Mixed precision.** Add an overload that reads half-precision input but still accumulates $\sum x_j^2$ in float32 (or float64). This is the realistic path once weights and activations are quantized.
- **LayerNorm for comparison.** Implement standard LayerNorm (subtract mean, divide by std, scale and shift) and confirm on a centered input that the two agree — a good way to feel exactly what RMSNorm drops.

2.4 Numerically-Stable Softmax

BOSS CHALLENGE

Implement row-wise softmax that survives adversarial inputs — values up to $\pm 10^4$ with no NaN or Inf — whose rows sum to 1 within 10^{-6} , and which matches a float64 reference within 10^{-5} . The naive formula fails all three. The bar is numerical, and it is measured.

GOAL

A softmax over the last dimension that is **overflow-free** on inputs your model will really produce, sums to exactly 1 per row within tolerance, and agrees with a double-precision reference. This is the function attention is built on, so it has to be bulletproof.

BACKGROUND

Softmax turns a vector of arbitrary real scores into a probability distribution:

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

Mathematically this is all you need. Numerically it is a landmine. The exponential overflows the float32 range at around $x \approx 88$ (since $e^{88} \approx 1.6 \times 10^{38}$, near

the float32 maximum). Attention logits — the inputs to this softmax — routinely exceed that, especially with long contexts or large head dimensions. Once any e^{x_j} becomes ∞ , the denominator is ∞ , every output is $\infty/\infty = \text{NaN}$, and the model is dead.

The fix exploits a beautiful invariance: subtracting any constant m from *every* score leaves the result unchanged, because the constant factors out of numerator and denominator,

$$\frac{e^{x_i - m}}{\sum_j e^{x_j - m}} = \frac{e^{-m} e^{x_i}}{e^{-m} \sum_j e^{x_j}} = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

Choose $m = \max_j x_j$. Then every exponent $x_i - m \leq 0$, so every $e^{x_i - m} \in (0, 1]$ — no overflow is possible, ever. The largest term becomes exactly $e^0 = 1$, and the denominator is a sum of values in $(0, 1]$, comfortably representable. This is the **max-subtraction trick**, and it is exact: it changes the intermediate numbers, not the answer.

The algorithm is therefore three passes over each row: find the max, exponentiate the shifted values while accumulating their sum, then divide each by the sum. (A famous *single-pass* reformulation — the online softmax — fuses the max and the sum, and it is the seed of Flash Attention; we derive it in Unit 8. The three-pass version is the correct, obvious oracle we build first and never throw away.)

Two edge behaviors are worth knowing. A row of **all-equal** scores produces a uniform distribution ($1/D$ each) — a good sanity check. And in causal attention, masked positions arrive as $-\infty$; with the max trick, $e^{-\infty - m} = 0$, so they contribute nothing and the unmasked entries still sum to 1 — exactly the behavior we want, and it falls out for free as long as at least one entry is finite.

This is the boss because numerical robustness is the kind of property you cannot eyeball. We will *measure* it: feed inputs that make the naive version explode, and assert the output is finite, normalized, and faithful to a high-precision reference.

THE TASK

Declare `softmax_lastdim(out, in)` in `ops.hpp` and define it in `ops.cpp`. Treat `in` as rows of length $D = \text{its last dimension}$. For each row: find the maximum, compute $e^{x_j - \max}$ into the output while summing them, then divide the row by the sum. Assert contiguity and matching sizes.

The bar (the boss): on a row containing values as large as $\pm 10^4$, every output is finite and non-negative; each row sums to 1 within 10^{-6} ; and every entry matches

a float64 reference softmax within 10^{-5} . Rows are normalized independently.

HINTS

Hint 1 — three passes, in this order

Pass 1: $m = \max$ over the row. Pass 2: for j : $e = \exp(x[j] - m)$; $out[j] = e$; $sum += e$; Pass 3: $inv = 1/sum$; for j : $out[j] *= inv$; Never exponentiate before subtracting the max.

Hint 2 — the reference is the same math in float64

Your oracle is not a magic library — it is this exact algorithm done in double. Write a tiny `std::vector<double>` helper that computes the stable softmax in double precision, and compare element-by-element. That is what “correct” means here.

Hint 3 — proving the naive version is wrong (optional but instructive)

Briefly compute $\sum_j e^{x_j}$ *without* subtracting the max on the adversarial input and watch it become `inf` (hence the outputs NaN). Seeing the failure makes the fix stick. Remove it afterward.

TESTS

```
// add to tests/test_ops.cpp
#include <algorithm>
#include <vector>

// Oracle: the same stable algorithm, in double precision.
static std::vector<double> softmax_ref(const float* x, int n) {
    double m = x[0];
    for (int i = 1; i < n; ++i) m = std::max(m, static_cast<double>(x[i]));
    std::vector<double> e(n);
    double s = 0.0;
    for (int i = 0; i < n; ++i) { e[i] = std::exp(static_cast<double>(x[i]) - m);
    ↪ s += e[i]; }
    for (int i = 0; i < n; ++i) e[i] /= s;
    return e;
}
```

```

TEST(boss_softmax_stable_finite_and_correct) {
    const int D = 6;
    float vals[D] = {1000.0f, 999.0f, -1000.0f, 0.0f, 500.0f, 1000.0f};
    Tensor x = Tensor::empty({1, D}, DType::F32);
    Tensor o = Tensor::empty({1, D}, DType::F32);
    for (int j = 0; j < D; ++j) x.data_ptr<float>()[j] = vals[j];

    softmax_lastdim(o, x);

    double sum = 0.0;
    for (int j = 0; j < D; ++j) {
        const float v = o.data_ptr<float>()[j];
        CHECK(std::isfinite(v)); // 1) no NaN / Inf
        CHECK(v >= 0.0f);
        sum += v;
    }
    CHECK_CLOSE(sum, 1.0, 1e-6); // 2) sums to one

    auto ref = softmax_ref(vals, D); // 3) matches float64 reference
    for (int j = 0; j < D; ++j)
        CHECK_CLOSE(o.data_ptr<float>()[j], ref[j], 1e-5);
}

TEST(boss_softmax_uniform_and_independent_rows) {
    const int D = 3;
    float v[6] = {0, 0, 0, 10000, 0, 0}; // row0 uniform; row1 extreme
    Tensor x = Tensor::empty({2, D}, DType::F32);
    Tensor o = Tensor::empty({2, D}, DType::F32);
    for (int i = 0; i < 6; ++i) x.data_ptr<float>()[i] = v[i];

    softmax_lastdim(o, x);

    double s0 = 0, s1 = 0;
    for (int j = 0; j < D; ++j) { s0 += o.data_ptr<float>()[j]; s1 += o.data_ptr<
    ↪ float>()[D + j]; }
    CHECK_CLOSE(s0, 1.0, 1e-6);
    CHECK_CLOSE(s1, 1.0, 1e-6);
    CHECK_CLOSE(o.data_ptr<float>()[0], 1.0 / 3.0, 1e-6); // uniform row
    CHECK_CLOSE(o.data_ptr<float>()[D + 0], 1.0, 1e-6); // extreme row -> ~
    ↪ one-hot
}

```

REFERENCE SOLUTION

Declare in ops.hpp:


```
// Numerically-stable softmax over the last dimension, row by row.
void softmax_lastdim(Tensor& out, const Tensor& in);
```

Define in ops.cpp:

```
#include <algorithm>
#include <cmath>

void softmax_lastdim(Tensor& out, const Tensor& in) {
    LLM_ASSERT(in.is_contiguous() && out.is_contiguous(),
               "softmax: tensors must be contiguous");
    LLM_ASSERT(in.numel() == out.numel(), "softmax: size mismatch");
    const int64_t D = in.shape().back();
    const int64_t rows = in.numel() / D;

    const float* x = in.data_ptr<float>();
    float* o = out.data_ptr<float>();

    for (int64_t r = 0; r < rows; ++r) {
        const float* xr = x + r * D;
        float* orow = o + r * D;

        float m = xr[0]; // 1) row max
        for (int64_t j = 1; j < D; ++j) m = std::max(m, xr[j]);

        float sum = 0.0f; // 2) shifted exp + sum
        for (int64_t j = 0; j < D; ++j) {
            const float e = std::exp(xr[j] - m); // exponent <= 0: no
            ↪ overflow
            orow[j] = e;
            sum += e;
        }

        const float inv = 1.0f / sum; // 3) normalize
        for (int64_t j = 0; j < D; ++j) orow[j] *= inv;
    }
}
```

Build and run. The adversarial-input test passes precisely because the exponent is never positive; drop the `- m` and watch all three assertions fail (NaN, a sum that is not 1, and a mismatch against the reference). **Boss cleared.**

What you have proven. Not “I wrote softmax” but “my softmax stays finite on

inputs that overflow the naive formula, produces a genuine probability distribution per row, and matches double precision to 10^{-5} .” That guarantee is what lets attention — which calls this on every row of its score matrix, with logits you do not control — be trusted in Unit 5, and it is the exact property the online-softmax reformulation in Unit 8 must preserve while computing it in a single pass.

GOING FURTHER

- **Softmax with temperature.** A scaling $\text{softmax}(x/T)$ sharpens ($T < 1$) or flattens ($T > 1$) the distribution — the temperature knob you will expose to the sampler in Unit 11. Fold $1/T$ into the shift.
- **Log-softmax.** $\log \text{softmax}(x)_i = (x_i - m) - \log \sum_j e^{x_j - m}$, computed without ever forming the probabilities — the stable way to get log-probabilities for perplexity and for speculative decoding’s accept test (Unit 12).
- **Online (single-pass) softmax.** Fuse the max and the sum by tracking a running max and a running sum and rescaling the sum whenever the max increases. It is the heart of Flash Attention; deriving it now is a strong warm-up for Unit 8.

Unit 2 — Milestone check

MILESTONE CHECK

The engine can now compute, not just store and reshape:

- **elementwise kernels** — add, mul, scale, the in-place residual add_inplace, and a generic apply_unary map — written in the tight, contiguous form the auto-vectorizer (and later, your hand-written SIMD) loves;
- the **activations** a Transformer runs on — a branch-stable sigmoidf, siluf, geluf (tanh) and gelu_exactf (erf), and the gated swiglu combine;
- **RMSNorm**, row-wise, with its reduction accumulated in float32;
- and a **boss-grade stable softmax**, proven overflow-free, normalized, and faithful to a double-precision reference on adversarial inputs.

Every arithmetic ingredient of a Transformer block is now in hand — except the one that dominates the compute and the one everything else orbits: the matrix

multiply. In **Unit 3** we build `GEMM`, the heart of the engine, starting from a naive triple loop with a verified reference and a `GFLOP/s` meter, then driving it through loop reordering, cache blocking, multi-threading, and `SIMD` — with a boss that demands a real, measured speedup.

UNIT 3

Matrix Multiplication

What you will build. The single most important kernel in the engine — general matrix multiplication (GEMM) — taken from a naive triple loop all the way to a multi-threaded, SIMD, cache-blocked implementation. Along the way you build a GFLOP/s **benchmark harness** and keep the naive version forever as a **correctness oracle**. The boss demands a real, measured speedup over your own best scalar code.

Where this fits. In a Transformer forward pass, the overwhelming majority of arithmetic is matrix multiplication: the four attention projections (Q , K , V , and the output), and the three feed-forward projections (gate, up, down). Everything from Unit 2 — the norms, activations, softmax — is a rounding error in the FLOP budget next to these. **Make GEMM fast and you have made the engine fast**; leave it naive and nothing else you do will matter.

This is the unit where the book earns its “systems” billing. The algorithm never changes — it is the same $O(MNK)$ triple loop from start to finish — yet over five challenges you will make it *an order of magnitude faster* by doing nothing but respecting the machine: how memory moves in cache lines, how data fits in cache, how many cores you have, and how wide your vector registers are. Each change is measured in GFLOP/s, and each is checked bit-for-bit (within floating-point tolerance) against the slow version, so “faster” never quietly becomes “wrong.”

We work in raw `float*` with explicit dimensions, not `Tensor`, for two reasons: the optimizations are about memory addresses and the raw form makes them legible, and the hot kernel should not pay for abstraction. A thin `Tensor` wrapper that validates shapes and dispatches to the fast kernel is a five-line exercise we leave for the end. Create `include/llm/gemm.hpp` and `src/gemm.cpp`, register them as usual, and let us begin with something correct and slow.

3.1 Naive GEMM, a Reference Oracle, and a GFLOP/s Harness

GOAL

The obvious, correct matrix multiply — the triple loop — plus the two tools every later challenge depends on: a **benchmark** that reports GFLOP/s, and the naive kernel itself serving as a **reference oracle** to verify every optimized version against.

BACKGROUND

General matrix multiplication computes $C = AB$ where A is $M \times K$, B is $K \times N$, and the result C is $M \times N$. Each output entry is a dot product of a row of A with a column of B :

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

We store all three row-major, so A_{ik} lives at $A[i*K + k]$, B_{kj} at $B[k*N + j]$, and C_{ij} at $C[i*N + j]$.

Why this kernel decides everything. Count the work. Producing each of the $M \cdot N$ entries takes K multiply-adds, i.e. $2K$ floating-point operations, so the whole multiply is

$$\text{FLOP} = 2MNK.$$

For a single Llama-style feed-forward projection with hidden size 4096 and intermediate size 11008 over a 512-token prompt, that is one matmul of roughly $2 \cdot 512 \cdot 11008 \cdot 4096 \approx 4.6 \times 10^{10}$ FLOP — and a layer has several, and a model has dozens of layers. This is why GEMM is the kernel: a $2\times$ improvement here is a $2\times$ improvement in the whole engine, and a naive implementation can be $20\text{--}50\times$ slower than a tuned one.

Measuring in GFLOP/s. Wall-clock time alone is meaningless across matrix sizes; the honest metric is *throughput* — billions of floating-point operations per second. Time the multiply, then compute $\text{GFLOP/s} = 2MNK / (\text{seconds} \times 10^9)$. A modern core peaks in the tens to low hundreds of GFLOP/s; seeing where your kernel lands relative to that ceiling tells you how much performance is left on the table. We warm up once (to page in memory and settle the caches), then time several iterations and average.

The reference oracle. Before optimizing anything, you need a version you *trust*. The naive triple loop is it: too slow to ship, but so simple it is obviously correct. Every faster kernel in this unit is validated by running it on the same random inputs as the naive kernel and checking the results match within tolerance. (Within *tolerance*, not bit-exactly: the optimized versions sum in a different order, and floating-point addition is not associative, so a relative difference around 10^{-3} is expected and fine. We met this same non-associativity in Unit 2's CHECK_CLOSE.) An optimization you cannot check against an oracle is a bug you have not noticed yet.

The naive loop order is **ijk**: for each row i , for each column j , sum over k . It is the most literal translation of the formula — and, as we will see in 3.2, nearly the worst possible order for the cache. That is fine. Correct first.

THE TASK

In `gemm.hpp/gemm.cpp`, implement `matmul_naive(C, A, B, M, N, K)` for row-major float matrices using the `ijk` triple loop, accumulating each dot product in a local float. Then build a benchmark helper that times a `matmul` callable and returns GFLOP/s (warm up once, time several iterations, $2MNK \cdot \text{iters}$ over elapsed seconds). Use 64-bit indexing inside the loops so large matrices do not overflow.

HINTS

Hint 1 — accumulate locally, write once

Keep the running dot product in a local `float acc` and store to `C[i*N + j]` only after the k -loop finishes. Writing to memory inside the inner loop is both slower and unnecessary.

Hint 2 — 64-bit indices in the address arithmetic

`A[i*K + k]` with 32-bit `int` multiplication can overflow for large matrices. Compute offsets as `(int64_t)i*K + k`. The loop counters can stay `int`; it is the *product* that overflows.

Hint 3 — a fair benchmark

Call the kernel once before timing (warm-up), use a steady clock (`std::chrono::steady_clock` or `high_resolution_clock`), and run enough iterations that the total is comfortably above a millisecond so timer granularity does not dominate.

TESTS

```
// tests/test_gemm.cpp
#include "test_framework.hpp"
#include "llm/gemm.hpp"
#include <vector>
using namespace llm;

TEST(matmul_naive_hand_computed) {
    // [2x3] * [3x2] = [2x2]
    float A[6] = {1, 2, 3, 4, 5, 6};           // 2x3
    float B[6] = {7, 8, 9, 10, 11, 12};        // 3x2
    float C[4];
    matmul_naive(C, A, B, /*M=*/2, /*N=*/2, /*K=*/3);
    CHECK_CLOSE(C[0], 58.0f, 1e-4); // 1*7 + 2*9 + 3*11
    CHECK_CLOSE(C[1], 64.0f, 1e-4); // 1*8 + 2*10 + 3*12
    CHECK_CLOSE(C[2], 139.0f, 1e-4); // 4*7 + 5*9 + 6*11
    CHECK_CLOSE(C[3], 154.0f, 1e-4); // 4*8 + 5*10 + 6*12
}

TEST(matmul_naive_identity) {
    const int n = 4;
    std::vector<float> A(n * n), I(n * n, 0.0f), C(n * n);
    for (int i = 0; i < n * n; ++i) A[i] = float(i + 1);
    for (int i = 0; i < n; ++i) I[i * n + i] = 1.0f; // identity
    matmul_naive(C.data(), A.data(), I.data(), n, n, n); // A * I == A
    for (int i = 0; i < n * n; ++i) CHECK_CLOSE(C[i], A[i], 1e-4);
}
```

REFERENCE SOLUTION

```
include/llm/gemm.hpp:
```

```

// include/llm/gemm.hpp
#pragma once
#include <chrono>
#include <cstdint>
#include "llm/common.hpp"

namespace llm {

//  $C[M,N] = A[M,K] * B[K,N]$ , all row-major float32. C is overwritten.
void matmul_naive(float* C, const float* A, const float* B, int M, int N, int K);

// Throughput of a matmul callable, in GFLOP/s. fn() must perform one M-N-K
  ↳ multiply.
template <typename F>
double bench_gflops(F&& fn, int M, int N, int K, int iters = 10) {
    fn(); // warm up: page in memory, settle caches
    const auto t0 = std::chrono::steady_clock::now();
    for (int it = 0; it < iters; ++it) fn();
    const auto t1 = std::chrono::steady_clock::now();
    const double secs = std::chrono::duration<double>(t1 - t0).count();
    const double flop = 2.0 * M * N * K * iters;
    return flop / (secs * 1e9);
}

} // namespace llm

```

src/gemm.cpp:

```

// src/gemm.cpp
#include "llm/gemm.hpp"

namespace llm {

void matmul_naive(float* C, const float* A, const float* B, int M, int N, int K) {
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            float acc = 0.0f;
            for (int k = 0; k < K; ++k)
                acc += A[(int64_t)i * K + k] * B[(int64_t)k * N + j];
            C[(int64_t)i * N + j] = acc;
        }
    }
}

}

```



```
} // namespace llm
```

A throwaway benchmark you can run from the CLI or a scratch test:

```
// e.g. in a benchmark main or a (non-asserting) test
#include <vector>
#include <cstdio>
using namespace llm;

int bench_demo() {
    const int M = 512, N = 512, K = 512;
    std::vector<float> A(M * K, 1.0f), B(K * N, 1.0f), C(M * N);
    const double gf = bench_gflops(
        [&] { matmul_naive(C.data(), A.data(), B.data(), M, N, K); }, M, N, K);
    std::printf("naive: %.2f GFLOP/s\n", gf);
    return 0;
}
```

Run it and write the number down — it is your baseline. On a typical core the naive kernel lands in the low single-digit GFLOP/s, a small fraction of what the hardware can do. Every challenge that follows is about closing that gap, and the only reason we can chase speed recklessly is that `matmul_naive` stands behind us as the arbiter of truth.

GOING FURTHER

- **A Tensor wrapper.** `matmul(Tensor& C, const Tensor& A, const Tensor& B)` that checks `A.shape()[1] == B.shape()[0]`, asserts contiguity, and calls the raw kernel. This is what the rest of the engine will call.
- **Peak-GFLOP/s estimate.** Compute your core's theoretical peak ($\text{cores} \times \text{clock} \times \text{SIMD width} \times 2$ for FMA) and express each kernel's result as a percentage of it. That ratio is the number tuned BLAS libraries obsess over.
- **A reusable correctness harness.** A `check_against_naive(fn, M, N, K)` helper that fills random inputs, runs both, and reports the worst mismatch — you will reuse it in every remaining challenge.

3.2 Loop Reordering: ijk to ikj

GOAL

Reorder the three loops from **ijk** to **ikj** so the innermost loop walks memory contiguously. This single change — no new math, no blocking, no threads — typically delivers a several-fold speedup, and it is the highest return-on-effort move in the unit.

BACKGROUND

The naive ijk order is correct but fights the memory system. Look at what its inner loop (over k) touches: $A[i \cdot K + k]$ advances by 1 element each step (good, contiguous), but $B[k \cdot N + j]$ advances by N elements each step — it strides *down a column* of a row-major matrix. Memory does not move one float at a time; it moves in **cache lines** of 64 bytes (sixteen floats). A contiguous walk uses all sixteen floats of each line it loads; a column walk uses *one* float per line and throws the other fifteen away, then needs a new line for the next step. For a large B that does not fit in cache, the inner loop becomes a stream of cache misses.

Now swap the inner two loops to get **ikj**: for each row i , for each k , for each j . Watch the inner loop (over j) this time. The value $A[i \cdot K + k]$ is a single scalar — loop-invariant in j , so load it once. $B[k \cdot N + j]$ and $C[i \cdot N + j]$ both advance by 1 element per step — they stream contiguously along a row. So the inner loop reads a cache-line-friendly run of B , multiplies it by a broadcast scalar, and accumulates into a cache-line-friendly run of C . Every cache line that arrives is fully used. The reformulation is:

$$\text{for each } i, k : \quad C_{i,:} += A_{ik} \cdot B_{k,:},$$

read as “scale row k of B by the scalar A_{ik} and add it into row i of C .” This is sometimes called the *SAXPY* or *rank-1 update* formulation, and it is exactly the shape that vectorizes well later (Unit 3.5).

One consequence to handle: ikj *accumulates* into C across the k -loop rather than computing each entry in one shot, so C must be **zeroed first**. That is a cheap pass, and the cache win dwarfs it.

The lesson generalizes far beyond GEMM: the same arithmetic, executed in an order that respects cache lines, can run several times faster. The instruction count

barely changes; the *memory traffic* collapses. This is the first and most important idea in making numerical code fast, and you get it here for the price of swapping two for lines.

THE TASK

Implement `matmul_ikj(C, A, B, M, N, K)`: zero `C`, then loop i , then k , then j , with the inner loop doing `C[i*N + j] += A[i*K + k] * B[k*N + j]` — hoisting `A[i*K + k]` and the row base pointers out of the inner loop. Verify it against `matmul_naive` on random inputs, and benchmark both to see the speedup.

HINTS

Hint 1 — hoist everything loop-invariant

Before the j -loop, compute `const float a = A[(int64_t)i*K + k];`, `const float* brow = B + (int64_t)k*N;`, and `float* crow = C + (int64_t)i*N;`. The inner loop is then just `crow[j] += a * brow[j];` — pure contiguous streaming.

Hint 2 — do not forget to zero C

Because ikj adds partial products across k , initialize `C` to zero first (a single `std::memset` or a flat loop over $M*N$). Skipping this gives garbage plus the right answer.

Hint 3 — benchmark at a size that exceeds cache

The win shows up clearly only when B is too big for L2. Benchmark at, say, $M = N = K = 512$ or 1024 ; at tiny sizes both versions fit in cache and look similar.

TESTS

```
// add to tests/test_gemm.cpp
#include "llm/gemm.hpp"
#include <vector>
#include <cstdint>
```

```

// A tiny deterministic RNG so tests are reproducible.
static void fill_pseudo(std::vector<float>& v, uint32_t seed) {
    uint32_t s = seed;
    for (float& x : v) { s = s * 1664525u + 1013904223u; x = (s >> 8) * (1.0f /
    ↪ 16777216.0f) - 0.5f; }
}

// Reusable: does `fn` match the naive oracle on random M-N-K inputs?
template <typename F>
static void check_against_naive(F&& fn, int M, int N, int K) {
    std::vector<float> A(M * K), B(K * N), C_ref(M * N), C_got(M * N);
    fill_pseudo(A, 1); fill_pseudo(B, 2);
    matmul_naive(C_ref.data(), A.data(), B.data(), M, N, K);
    fn(C_got.data(), A.data(), B.data(), M, N, K);
    float worst = 0.0f;
    for (int i = 0; i < M * N; ++i) worst = std::max(worst, std::fabs(C_got[i] -
    ↪ C_ref[i]));
    CHECK(worst < 1e-2f);    // reordered summation: small fp differences
    ↪ expected
}

TEST(matmul_ikj_matches_naive) {
    check_against_naive([](float* C, const float* A, const float* B, int M, int N,
    ↪ int K)
        { matmul_ikj(C, A, B, M, N, K); }, 64, 48, 80);
    check_against_naive([](float* C, const float* A, const float* B, int M, int N,
    ↪ int K)
        { matmul_ikj(C, A, B, M, N, K); }, 128, 128, 128);
}

```

REFERENCE SOLUTION

Declare in `gemm.hpp`, define in `gemm.cpp`:

```

// gemm.hpp
void matmul_ikj(float* C, const float* A, const float* B, int M, int N, int K);

```

```

// gemm.cpp
#include <cstring>

void matmul_ikj(float* C, const float* A, const float* B, int M, int N, int K) {
    std::memset(C, 0, sizeof(float) * (size_t)M * N);    // ikj accumulates

```

```

for (int i = 0; i < M; ++i) {
    float* crow = C + (int64_t)i * N;
    for (int k = 0; k < K; ++k) {
        const float a = A[(int64_t)i * K + k];    // scalar, hoisted
        const float* brow = B + (int64_t)k * N;    // contiguous row of B
        for (int j = 0; j < N; ++j)
            crow[j] += a * brow[j];              // contiguous stream
    }
}

```

Benchmark `matmul_ikj` next to `matmul_naive` at 512^3 and compare the GFLOP/s. The speedup — often $3\text{--}8\times$ — comes entirely from the inner loop now consuming whole cache lines instead of one float per line. You changed the order of operations, not the operations. That is the whole trick, and it recurs everywhere in this book.

GOING FURTHER

- **Try all six orders.** Benchmark `ijk`, `ikj`, `jik`, `jki`, `kij`, `kji`. Explain the ranking from the access patterns of the inner loop alone. (`ikj` and `kij` win; `jki` and `kji` lose badly.)
- **Compiler auto-vectorization.** Inspect the assembly (or compile with a vectorization report). The `ikj` inner loop is a textbook candidate; the compiler may already emit `SIMD` for it. Unit 3.5 is about beating what it does on its own.
- **Accumulator caveat.** With `-ffast-math` the compiler may reassociate the sum; without it, it cannot. Note how that interacts with matching the naive oracle.

3.3 Cache Blocking (Tiling)

GOAL

Partition the multiply into **tiles** small enough to live in cache, so each chunk of *A* and *B* is loaded once and reused many times before eviction. This buys another speedup on top of `ikj`, and the gap widens as the matrices grow past cache size.

BACKGROUND

The *ikj* kernel walks memory contiguously, but for large matrices it still has a reuse problem. Consider its full sweep: for one row i , it touches *every* row of B (all $K \cdot N$ elements) to produce one row of C . When B is larger than the cache, those rows are evicted before the next i needs them, so B is re-streamed from main memory M times. The arithmetic is bounded by memory bandwidth, and bandwidth is the scarce resource.

Blocking fixes this by changing *which* part of the problem you finish first. Instead of completing an entire row of C before moving on, you carve A , B , and C into rectangular tiles and compute one tile of C to completion using small tiles of A and B — tiles chosen to fit in cache. While those tiles are resident, every element you loaded gets reused across the whole tile before you move on and let it be evicted. The reuse factor rises from “once” toward “the tile dimension,” which is exactly the bandwidth reduction you want.

Concretely, choose block sizes M_c , N_c , K_c and loop over tiles:

- an outer loop selects a K_c -wide slab and an M_c -tall band of A , and a K_c -tall, N_c -wide tile of B ;
- these are sized so that the active tiles of B (and a strip of A) sit in L2/L1 cache together;
- an inner *ikj* kernel multiplies the tiles, accumulating into the corresponding $M_c \times N_c$ tile of C .

Because a given C -tile receives contributions from several K_c -slabs, C is again accumulated (zero it first), and the k tiling must be the loop that sweeps those contributions into the same C -tile.

How big should the tiles be? Big enough to amortize loop overhead, small enough that the working set — roughly a $K_c \times N_c$ tile of B plus an $M_c \times K_c$ strip of A — fits comfortably in the target cache level. Values like $M_c = N_c = K_c = 64$ (a 64×64 float tile is 16 KiB) are a reasonable starting point for L1/L2; the true optimum is hardware-specific and found by experiment. We expose the block sizes as constants so you can tune them and *measure*.

The deeper idea — the one that carries into Flash Attention in Unit 8 — is that when the data does not fit in fast memory, you restructure the computation into tiles that *do*, trading a more complex loop nest for dramatically less traffic to slow memory. GEMM is where you learn the pattern; attention is where it becomes spectacular.

THE TASK

Implement `matmul_blocked(C, A, B, M, N, K)` with compile-time block sizes `MC`, `NC`, `KC`. Zero `C`; loop over C -tiles and K_c -slabs; for each tile run the `ikj` kernel restricted to the tile's index ranges (clamping at the matrix edges). Verify against `matmul_naive`; benchmark against `matmul_ikj` at a large size and tune the block sizes.

HINTS

Hint 1 — the loop nest

Outer tile loops over `ic` (step `MC`), `kc` (step `KC`), `jc` (step `NC`); inside, clamp `iend = min(ic+MC, M)` etc.; then the familiar `ikj` over `i` in `[ic, iend)`, `k` in `[kc, kend)`, `j` in `[jc, jend)`. Keeping `kc` outside `jc` lets a C -tile accumulate all its k -contributions with B 's slab hot.

Hint 2 — clamp at the edges

M, N, K need not be multiples of the block sizes. Use `std::min` for each tile's end index so the last (ragged) tile is handled correctly.

Hint 3 — tune by measuring, not guessing

Try `MC=NC=KC` of 32, 64, 128, 256 and benchmark each at 1024^3 . The best value depends on your cache sizes; there is no universal answer, which is the point.

TESTS

```
// add to tests/test_gemm.cpp (reuses check_against_naive from 3.2)

TEST(matmul_blocked_matches_naive) {
    auto fn = [](float* C, const float* A, const float* B, int M, int N, int K)
        { matmul_blocked(C, A, B, M, N, K); };
    check_against_naive(fn, 128, 128, 128);
    check_against_naive(fn, 100, 130, 70);    // not multiples of the block size
}
```

REFERENCE SOLUTION

```
// gemm.hpp
void matmul_blocked(float* C, const float* A, const float* B, int M, int N, int K);
    ↪
```

```
// gemm.cpp
#include <algorithm>
#include <cstring>

void matmul_blocked(float* C, const float* A, const float* B, int M, int N, int K)
    ↪ {
    constexpr int MC = 64, NC = 64, KC = 64;           // tile sizes (tune these)
    std::memset(C, 0, sizeof(float) * (size_t)M * N);

    for (int ic = 0; ic < M; ic += MC) {
        const int iend = std::min(ic + MC, M);
        for (int kc = 0; kc < K; kc += KC) {
            const int kend = std::min(kc + KC, K);
            for (int jc = 0; jc < N; jc += NC) {
                const int jend = std::min(jc + NC, N);
                // ikj over the tile; the B slab [kc,kend)x[jc,jend) stays hot
                for (int i = ic; i < iend; ++i) {
                    float* crow = C + (int64_t)i * N;
                    for (int k = kc; k < kend; ++k) {
                        const float a = A[(int64_t)i * K + k];
                        const float* brow = B + (int64_t)k * N;
                        for (int j = jc; j < jend; ++j)
                            crow[j] += a * brow[j];
                    }
                }
            }
        }
    }
}
```

Benchmark `matmul_blocked` against `matmul_ikj` at 1024^3 or larger. At small sizes the two are nearly identical — everything already fits in cache, so blocking adds only loop overhead — but as the matrices outgrow L2, blocking pulls ahead because *B* is re-read from main memory far fewer times. Sweep MC/NC/KC and watch the throughput change; that sensitivity *is* the cache hierarchy made visible.

GOING FURTHER

- **Two-level blocking.** Add an inner tier of tiles sized for L1 inside the L2 tiles. Real BLAS blocks for multiple cache levels plus registers (the register tier is Unit 3.5).
- **Why kc placement matters.** Reorder the tile loops and measure. Putting the N -tile innermost keeps a B slab resident across the k -accumulation, which is usually best; confirm it empirically.
- **Cache-size introspection.** Query the L1/L2 sizes (platform-specific) and pick block sizes from them at runtime instead of hard-coding 64.

3.4 Multi-threading Across Cores

GOAL

Spread the multiply across all the machine's cores by giving each thread a disjoint band of output rows to compute with the blocked kernel. Because the output regions never overlap, no locks or synchronization are needed — and the speedup scales with core count until memory bandwidth becomes the ceiling.

BACKGROUND

GEMM is **embarrassingly parallel**, and it is worth seeing exactly why. Every output element C_{ij} is a function of inputs only — a row of A and a column of B , both read-only — and depends on *no other output element*. So if you carve C into disjoint regions and assign each to a different thread, the threads share the inputs (reading is always safe to do concurrently) and write to *separate* parts of the output. There is nothing to coordinate: no locks, no atomics, no barriers. This is the easiest and most profitable kind of parallelism there is, and GEMM hands it to you for free.

The natural partition is by **rows of C** . With T threads, give each thread a contiguous band of roughly M/T rows; thread t computes rows $[t \cdot \frac{M}{T}, (t+1) \cdot \frac{M}{T})$ using exactly the blocked kernel from 3.3, restricted to its row range. Each thread reads its own rows of A and all of B (shared, read-only) and writes only its own rows of C . Correct by construction.

A few realities temper the ideal:

Memory bandwidth is shared. All cores pull from the same main memory through a shared bus. A kernel that is *memory-bound* — as scalar GEMM largely is — will scale nicely until the cores collectively saturate that bandwidth, then flatten out: adding more threads cannot move data faster than the bus allows. This is a major reason the next challenge matters. SIMD raises each core's arithmetic throughput so the kernel becomes *compute-bound*, at which point more cores help again. Threading and vectorization are complementary, not alternatives.

Thread creation has a cost. Spawning an OS thread takes on the order of microseconds. For one large multiply that is negligible, but an engine performs *thousands* of matmuls per generated token, and paying that startup repeatedly would dominate. Production engines use a **thread pool**: spawn the worker threads once at startup and feed them tasks, so the per-matmul cost is just handing out work. We use `raw std::thread` here for clarity — the pattern is identical — and leave the pool as a (strongly recommended) extension you will want before the engine ships.

False sharing is the one subtlety. Two threads writing to the same 64-byte cache line — even to different floats in it — force that line to ping-pong between their caches, quietly destroying performance. Partitioning by whole rows almost entirely avoids it: each thread writes long contiguous runs of C , and only the single cache line straddling a partition boundary could ever be shared. With realistic row widths that boundary effect is negligible, but it is the thing to know about the moment you parallelize.

THE TASK

Implement `matmul_parallel(C, A, B, M, N, K, num_threads = 0)`. With `num_threads <= 0`, default to `std::thread::hardware_concurrency()`. Partition the M rows into that many contiguous bands; launch one `std::thread` per band, each running the blocked kernel (3.3) over its rows (and zeroing only its own rows of C); then join. Verify against `matmul_naive`; benchmark against `matmul_blocked` and watch the speedup track your core count. Link a threads library in CMake (`find_package(Threads REQUIRED)`) and `target_link_libraries(... Threads::Threads)`.

HINTS

Hint 1 — factor the per-band kernel into a free function

Pull the blocked kernel into static void `gemm_rows(C, A, B, M, N, K, row0, row1)` that computes rows `[row0, row1)`. `matmul_parallel` then just slices M and hands each slice to a thread running `gemm_rows`. `matmul_blocked` can share the same body with `row0=0, row1=M`.

Hint 2 — each thread zeroes only its own rows

Inside `gemm_rows`, `memset` the block `C + (int64_t)row0*N` of length `(row1-row0)*N`. Threads never touch each other's rows, so no coordination is needed for the zeroing either.

Hint 3 — do not launch more threads than rows

Clamp `num_threads` to at most M , and compute `rows_per = (M + T - 1) / T` so the bands cover all rows with the last one possibly short. Break out of the launch loop once a band would be empty.

TESTS

```
// add to tests/test_gemm.cpp (reuses check_against_naive from 3.2)

TEST(matmul_parallel_matches_naive) {
    auto fn = [](float* C, const float* A, const float* B, int M, int N, int K)
        { matmul_parallel(C, A, B, M, N, K, /*threads=*/4); };
    check_against_naive(fn, 200, 256, 128);
    check_against_naive(fn, 257, 129, 64);    // ragged sizes + uneven row split
}

TEST(matmul_parallel_single_thread_equals_blocked) {
    const int M = 96, N = 80, K = 64;
    std::vector<float> A(M * K), B(K * N), C1(M * N), C2(M * N);
    fill_pseudo(A, 3); fill_pseudo(B, 4);
    matmul_blocked(C1.data(), A.data(), B.data(), M, N, K);
    matmul_parallel(C2.data(), A.data(), B.data(), M, N, K, 1);
    for (int i = 0; i < M * N; ++i) CHECK_CLOSE(C2[i], C1[i], 1e-4); // same work,
    ↪ 1 thread
}
```

REFERENCE SOLUTION

```
// gemm.hpp
void matmul_parallel(float* C, const float* A, const float* B,
                    int M, int N, int K, int num_threads = 0);
```

```
// gemm.cpp
#include <thread>
#include <vector>
#include <algorithm>
#include <cstring>

// Blocked ikj over output rows [row0, row1). Reused by blocked + parallel.
static void gemm_rows(float* C, const float* A, const float* B,
                     int M, int N, int K, int row0, int row1) {
    constexpr int NC = 64, KC = 64;
    std::memset(C + (int64_t)row0 * N, 0,
               sizeof(float) * (size_t)(row1 - row0) * N); // this band only
    for (int kc = 0; kc < K; kc += KC) {
        const int kend = std::min(kc + KC, K);
        for (int jc = 0; jc < N; jc += NC) {
            const int jend = std::min(jc + NC, N);
            for (int i = row0; i < row1; ++i) {
                float* crow = C + (int64_t)i * N;
                for (int k = kc; k < kend; ++k) {
                    const float a = A[(int64_t)i * K + k];
                    const float* brow = B + (int64_t)k * N;
                    for (int j = jc; j < jend; ++j) crow[j] += a * brow[j];
                }
            }
        }
    }
}

void matmul_parallel(float* C, const float* A, const float* B,
                    int M, int N, int K, int num_threads) {
    if (num_threads <= 0) {
        num_threads = (int)std::thread::hardware_concurrency();
        if (num_threads <= 0) num_threads = 1;
    }
    num_threads = std::min(num_threads, M); // at most one band per
    ↪ row
    const int rows_per = (M + num_threads - 1) / num_threads;
```

```

std::vector<std::thread> pool;
pool.reserve(num_threads);
for (int t = 0; t < num_threads; ++t) {
    const int r0 = t * rows_per;
    const int r1 = std::min(r0 + rows_per, M);
    if (r0 >= r1) break;
    pool.emplace_back(gemm_rows, C, A, B, M, N, K, r0, r1);
}
for (std::thread& th : pool) th.join();
}

```

(You can now reimplement `matmul_blocked` as a one-liner — `gemm_rows(C, A, B, M, N, K, 0, M)` — so both share exactly one tuned kernel body.)

Benchmark `matmul_parallel` against `matmul_blocked` at 1024^3 or larger. On an 8-core machine you should see something approaching — though rarely equal to — an $8\times$ speedup, the shortfall being the shared-bandwidth ceiling discussed above. Try a few thread counts and plot throughput against threads; the curve that rises then flattens is the memory wall, and it is precisely what `SIMD` in the boss challenge helps you push past.

GOING FURTHER

- **A real thread pool.** Spawn T workers once, feed them row-band tasks through a queue, and reuse them across every `matmul`. This removes per-call thread-creation overhead — essential when the engine issues thousands of `matmuls` per token.
- **Block-level (2D) tasks.** Partition C into a grid of tiles rather than horizontal bands and hand tiles to threads. Finer-grained tasks balance load better when M is small but N is large.
- **Pin threads / NUMA awareness.** On multi-socket machines, where a thread runs and where its memory lives matters enormously. Affinity and NUMA-local allocation are the next frontier once a single socket is saturated.

3.5 SIMD and Packing

BOSS CHALLENGE

Hand-vectorize the inner kernel with an AVX2/FMA register-blocked micro-kernel fed by packed operands, and measure at least a $1.5\times$ speedup over your scalar blocked kernel from 3.3 — still matching the naive reference within tolerance. A portable scalar fallback keeps it building everywhere. The bar is a measured ratio.

GOAL

Reach the per-core performance tier real BLAS libraries use: a small tile of C held entirely in **vector registers**, updated with **fused multiply-adds** over **packed**, contiguous, aligned operands. Measured bar: $\geq 1.5\times$ **faster than `matmul_blocked`** (single-threaded, isolating the vectorization win), correct within tolerance. Combined with the threading from 3.4, this is the engine's production `matmul`.

BACKGROUND

Everything so far improved *memory* behavior. This challenge improves *arithmetic* behavior, using two ideas that always travel together in high-performance GEMM: register-blocked SIMD micro-kernels and operand packing.

SIMD and FMA. A modern core's vector unit operates on several floats per instruction: AVX2 registers (`__m256`) hold 8 floats, and the **fused multiply-add** (`_mm256_fmadd_ps`) computes $a \cdot b + c$ across all 8 lanes in one instruction with a single rounding. One FMA on `__m256` is 16 FLOP. Scalar code leaves $\frac{7}{8}$ of that throughput unused. The scalar `ijk` inner loop *can* be auto-vectorized by the compiler, but you can usually beat it by controlling the register schedule yourself.

The register-blocked micro-kernel. The trick is to compute a small tile of C — say $M_R \times N_R = 8 \times 8$ — keeping the *entire tile in registers* across the whole k -loop. With $N_R = 8$, each output row is one `__m256`, so 8 rows need 8 accumulator registers (AVX2 has 16, leaving room for operands). For each k you load one 8-wide vector of B (the tile's 8 columns at depth k) and, for each of the 8 rows, broadcast the scalar A_{ik} and FMA it into that row's accumulator. The C tile never leaves registers until the k -loop ends — no repeated loads and stores of partial sums, which is where the scalar kernel bleeds time. This is *register* blocking: a

third tier below the L2 and L1 tiles of 3.3.

Packing, and why it is necessary. For the micro-kernel to issue back-to-back aligned vector loads, its inputs must be laid out exactly the way it reads them — contiguously and in order. In the original matrices they are not: the micro-kernel wants, for each k , the 8 A -values of its row tile together and the 8 B -values of its column tile together, but those are scattered by the matrices' row strides. So before running the micro-kernel we **pack**: copy the relevant strip of A into a small buffer ordered as the kernel reads it (the M_R values for each k contiguous), and likewise pack the B panel (N_R values for each k contiguous). The pack itself costs a copy, but that copy is amortized over the many FMAs of the micro-kernel, and in return every load inside the hot loop is sequential and the prefetcher is perfectly happy. This packing step is exactly what libraries like OpenBLAS and BLIS do; it is the difference between “vectorized” and “fast.”

The roofline view. Threading (3.4) raised the bandwidth you can consume; SIMD raises the arithmetic you can perform per byte read. Together they move the kernel from memory-bound toward compute-bound and let you approach the hardware's peak FLOP rate — which is why the engine's real matmul uses *all five* techniques at once.

Two honest caveats. First, FMA and the reordered summation mean the result is *not* bit-identical to the naive kernel — it is in fact usually *more* accurate (one rounding instead of two per multiply-add), but it differs, so we check against the oracle within a tolerance, not exactly. Second, the 8×8 micro-kernel as written assumes M and N are multiples of 8; ragged edges are handled with masked loads or a scalar tail (Going Further). We keep the demonstration clean and benchmark at multiple-of-8 sizes. And because not every machine has AVX2, the kernel sits behind `##if defined(__AVX2__)` with a scalar fallback, so the engine still builds and runs anywhere — just slower where the vector unit is absent.

THE TASK

Implement, in `gemm.cpp`: `pack_A` (copy an $8 \times K_c$ strip of A into `Apack` ordered as `[k*8 + i]`), `pack_B` (copy a $K_c \times 8$ tile of B into `Bpack` ordered as `[k*8 + j]`), an 8×8 `micro_kernel` using `_mm256` accumulators and `_mm256_fmadd_ps` under `##if defined(__AVX2__)` with a scalar fallback, and `matmul_simd(C, A, B, M, N, K)` that tiles K by K_c , packs, and drives the micro-kernel over all 8×8 output tiles (assume M, N multiples of 8). Build with `-mavx2 -mfma`. Verify against `matmul_naive` within tolerance; benchmark against `matmul_blocked` and confirm $\geq 1.5\times$.

HINTS

Hint 1 — eight accumulators, held across k

Declare `__m256 c0..c7 = _mm256_setzero_ps();` before the k -loop. Inside: `__m256 b = _mm256_loadu_ps(Bpack + k*8);` then `c0 = _mm256_fmadd_ps(_mm256_broadcast_ss(Apack + k*8 + 0), b, c0);` for each row $0..7$. Store to C only after the loop, adding into the existing C (it accumulates across K_c -tiles).

Hint 2 — the packing layouts must match the kernel's reads

The kernel reads `Bpack[k*8 + j]` and `Apack[k*8 + i]`. So `pack_B` writes `Bpack[k*8 + j] = B[(kc+k)*N + (jc+j)]` and `pack_A` writes `Apack[k*8 + i] = A[(ic+i)*K + (kc+k)]`. Get this correspondence wrong and the oracle check will tell you immediately.

Hint 3 — pack B once per K -slab, A once per row-tile

Structure the driver so the B panel for the current K_c -slab is packed once and reused across all row tiles, and each 8-row strip of A is packed once and reused across all column tiles. Re-packing inside the innermost loop throws away the benefit.

Hint 4 — the fallback must compute the same thing

The `##else` branch should produce results matching the AVX2 path (within fp tolerance): a plain triple loop over the 8×8 tile reading the same `Apack/Bpack`. Test both branches if you can (compile once with and once without `-mavx2`).

TESTS

```
// add to tests/test_gemm.cpp (reuses check_against_naive, fill_pseudo)

TEST(boss_matmul_simd_matches_naive) {
    auto fn = [](float* C, const float* A, const float* B, int M, int N, int K)
        { matmul_simd(C, A, B, M, N, K); };
    check_against_naive(fn, 64, 64, 64);
}
```



```

    check_against_naive(fn, 128, 256, 96);    // M, N multiples of 8; K arbitrary
    check_against_naive(fn, 512, 512, 512);
}

// The measured bar: run as a benchmark (timing asserts are machine-dependent,
// so we print the ratio and assert only a conservative lower bound).
TEST(boss_matmul_simd_is_faster) {
    const int M = 512, N = 512, K = 512;
    std::vector<float> A(M * K), B(K * N), C(M * N);
    fill_pseudo(A, 5); fill_pseudo(B, 6);
    const double gf_blocked = bench_gflops(
        [&]{ matmul_blocked(C.data(), A.data(), B.data(), M, N, K); }, M, N, K);
    const double gf_simd = bench_gflops(
        [&]{ matmul_simd(C.data(), A.data(), B.data(), M, N, K); }, M, N, K);
    std::printf("blocked: %.1f GFLOP/s   simd: %.1f GFLOP/s   speedup: %.2fx\n",
                gf_blocked, gf_simd, gf_simd / gf_blocked);
    CHECK(gf_simd >= 1.5 * gf_blocked);      // the boss bar
}

```

REFERENCE SOLUTION

Add to `gemm.cpp` (declare `matmul_simd` in `gemm.hpp`):

```

// gemm.cpp
#include <vector>
#include <algorithm>
#include <cstring>
#if defined(__AVX2__)
    #include <immintrin.h>
#endif

static constexpr int MR = 8, NR = 8;

// Pack an MR x kl strip of A (rows ic.., depth kc..) as Apack[k*MR + i].
static void pack_A(const float* A, int K, int ic, int kc, int kl, float* Apack) {
    for (int k = 0; k < kl; ++k)
        for (int i = 0; i < MR; ++i)
            Apack[k * MR + i] = A[(int64_t)(ic + i) * K + (kc + k)];
}

// Pack a kl x NR tile of B (depth kc.., cols jc..) as Bpack[k*NR + j].
static void pack_B(const float* B, int N, int kc, int jc, int kl, float* Bpack) {
    for (int k = 0; k < kl; ++k)
        for (int j = 0; j < NR; ++j)

```

```

        Bpack[k * NR + j] = B[(int64_t)(kc + k) * N + (jc + j)];
    }

    // C_tile[8][8] += sum_k Apack[k*8 + i] * Bpack[k*8 + j]. Accumulates into C.
    static void micro_kernel(const float* Apack, const float* Bpack, int kl,
                             float* C, int N) {
    #if defined(__AVX2__)
        __m256 c0 = _mm256_setzero_ps(), c1 = _mm256_setzero_ps();
        __m256 c2 = _mm256_setzero_ps(), c3 = _mm256_setzero_ps();
        __m256 c4 = _mm256_setzero_ps(), c5 = _mm256_setzero_ps();
        __m256 c6 = _mm256_setzero_ps(), c7 = _mm256_setzero_ps();
        for (int k = 0; k < kl; ++k) {
            const __m256 b = _mm256_loadu_ps(Bpack + k * NR);
            const float* a = Apack + k * MR;
            c0 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 0), b, c0);
            c1 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 1), b, c1);
            c2 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 2), b, c2);
            c3 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 3), b, c3);
            c4 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 4), b, c4);
            c5 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 5), b, c5);
            c6 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 6), b, c6);
            c7 = _mm256_fmadd_ps(_mm256_broadcast_ss(a + 7), b, c7);
        }
        #define ADD_ROW(r, cc) \
            _mm256_storeu_ps(C + (r) * N, _mm256_add_ps(_mm256_loadu_ps(C + (r) * N),
        ↪ cc))
        ADD_ROW(0, c0); ADD_ROW(1, c1); ADD_ROW(2, c2); ADD_ROW(3, c3);
        ADD_ROW(4, c4); ADD_ROW(5, c5); ADD_ROW(6, c6); ADD_ROW(7, c7);
        #undef ADD_ROW
    #else
        for (int i = 0; i < MR; ++i)
            for (int k = 0; k < kl; ++k) {
                const float a = Apack[k * MR + i];
                for (int j = 0; j < NR; ++j)
                    C[(int64_t)i * N + j] += a * Bpack[k * NR + j];
            }
    #endif
    }

    void matmul_simd(float* C, const float* A, const float* B, int M, int N, int K) {
        LLM_ASSERT(M % MR == 0 && N % NR == 0,
            "matmul_simd demo requires M, N multiples of 8 (see Going Further)")
        ↪ ;
        constexpr int KC = 256; // K-slab for packing/
        ↪ reuse
        std::memset(C, 0, sizeof(float) * (size_t)M * N);

```

```

std::vector<float> Bpack((size_t)KC * N);           // whole B panel for a K-
↪ slab
std::vector<float> Apack((size_t)MR * KC);           // one row-strip of A

for (int kc = 0; kc < K; kc += KC) {
    const int kl = std::min(KC, K - kc);
    for (int j = 0; j < N; j += NR)                 // pack B panel once per
↪ slab
        pack_B(B, N, kc, j, kl, Bpack.data() + (size_t)j * kl);
    for (int i = 0; i < M; i += MR) {
        pack_A(A, K, i, kc, kl, Apack.data());       // pack A strip once per
↪ row-tile
        for (int j = 0; j < N; j += NR)
            micro_kernel(Apack.data(), Bpack.data() + (size_t)j * kl, kl,
                          C + (int64_t)i * N + j, N);
    }
}
}

```

Build with AVX2 enabled — in CMakeLists.txt, for GCC/Clang:

```

target_compile_options(llm_core PRIVATE
    $<${CXX_COMPILER_ID:GNU,Clang}: -mavx2 -mfma>)

```

Run `boss_matmul_simd_is_faster` and read the printed ratio. On an AVX2 core you should comfortably clear $1.5\times$ over the scalar blocked kernel — often $2\text{--}4\times$ — with the oracle check confirming the result is still correct. Now layer the row-band threading from 3.4 on top (partition the i -tile loop across threads, each with its own `Apack`), and the per-core SIMD gain multiplies by the core count: that combined kernel is what the engine actually calls.

What you have proven. Not “I used some intrinsics,” but “my vectorized, packed micro-kernel runs at least $1.5\times$ the throughput of my best scalar kernel and still matches the reference to tolerance” — a measured, defensible speedup on the kernel that dominates the entire engine’s runtime. Every later unit’s performance rests on this.

GOING FURTHER

- **Ragged edges.** Handle M, N not divisible by 8 with masked loads (`_mm256_maskload_ps`) or a scalar tail for the boundary tiles, so the kernel works on any size.
- **Wider micro-kernels and AVX-512.** Try $M_R \times N_R$ of 6×16 or 8×16 (two `_mm256` per row), or AVX-512 (`_m512`, 16 floats). More accumulators hide FMA latency better — measure the sweet spot for your core.
- **Aligned packing.** Allocate `Apack/Bpack` 64-byte aligned (your `aligned_malloc` from Unit 1) and use aligned loads (`_mm256_load_ps`) for a small extra gain.
- **Prefetching.** Issue `_mm_prefetch` on the next B panel while computing the current tile to hide memory latency — the last few percent toward peak.

Unit 3 — Milestone check

MILESTONE CHECK

You built the engine's most important kernel and made it genuinely fast — the same $O(MNK)$ arithmetic, accelerated by an order of magnitude through respect for the hardware:

- a naive `matmul_naive` that is your permanent **correctness oracle**, and a **GFLOP/s harness** to measure every change;
- **ikj reordering**, turning the inner loop into a contiguous cache-line-friendly stream;
- **cache blocking**, so large operands are reused while hot instead of re-streamed from memory;
- **multi-threading**, splitting disjoint output rows across cores with no synchronization;
- and a **boss-grade AVX2/FMA micro-kernel with packing**, proven by measurement to beat the scalar blocked kernel by at least $1.5\times$ while still matching the oracle — with a portable scalar fallback.

The engine can now hold and reshape tensors (Unit 1), compute the element-wise math and softmax of a Transformer (Unit 2), and multiply matrices at high throughput (Unit 3) — the full arithmetic substrate of inference. Next we turn to **text**: **Unit 4** builds the **tokenizer** — byte-level BPE, the vocabulary and merge rules, encoding and decoding — so the engine can turn the strings a user types

into the token IDs the model consumes, and back again. From there, Unit 5 assembles the first real Transformer.

UNIT 4

The Tokenizer

What you will build. The bridge between human text and the integer IDs a model consumes: a **byte-level BPE tokenizer**. You will build the **vocabulary** (token bytes $\leftrightarrow$ ID), the **BPE merge** algorithm that compresses bytes into tokens, **pre-tokenization** and the full **encoder** (text $\rightarrow$ IDs), and the **decoder** (IDs $\rightarrow$ text). The boss proves the whole pipeline is a perfect round-trip: **byte-for-byte**, on emoji, scripts, and arbitrary bytes alike.

Where this fits. A language model does not see characters; it sees token IDs — integers indexing an embedding table. The tokenizer is the boundary at *both* ends of the engine: it turns the user’s prompt into IDs the model can read, and turns the IDs the model emits back into text. Get it wrong and nothing else can be right — the model would receive garbage and produce garbage. It also has to be *exactly* the tokenizer the model was trained with, which is why we build it to match the byte-level BPE scheme Llama-family models use.

Tokenization sits at an awkward seam: text is messy (Unicode, whitespace, emoji, scripts the author never anticipated) and models want a clean, fixed-size vocabulary of integers. The scheme that squares this circle is **byte-level Byte-Pair Encoding**. Two ideas combine. *Byte-level* means the irreducible alphabet is the 256 possible byte values, so **any** input whatsoever — any language, any symbol, even invalid Unicode — is representable; there is no “unknown token,” ever. *BPE* then learns to glue common byte sequences into single tokens (" the", "ing", a whole common word), so frequent text costs few tokens while rare text gracefully falls back toward individual bytes. The result is a compact vocabulary that never chokes on input.

We build it in the order data flows: first the vocabulary and its byte foundation, then the merge algorithm at the heart of encoding, then pre-tokenization and the

full encoder, then the decoder — and a boss that holds the round-trip to a byte-exact standard. Create `include/llm/tokenizer.hpp` and `src/tokenizer.cpp` and register them as usual.

4.1 Bytes and the Vocabulary

GOAL

The foundation: a `Tokenizer` holding a **vocabulary** that maps token byte strings to integer IDs and back, seeded with the 256 base byte tokens so that every possible byte is already encodable before a single merge is learned.

BACKGROUND

A **token** is a sequence of bytes with an integer ID. Crucially, a token's bytes need *not* be valid UTF-8 on their own — a token can be a fragment of a multi-byte character (the first two bytes of a three-byte emoji, say). The vocabulary only ever deals in raw bytes; meaning (“is this a complete character?”) is reconstructed later, when bytes are concatenated and interpreted. Keeping the tokenizer byte-oriented rather than character-oriented is what makes it robust.

The **base of the vocabulary is the 256 single bytes**, tokens 0 through 255, each the one-byte string for that value. This is the safety net: because every byte is its own token, any byte sequence can *always* be encoded, even if no merge applies — worst case, a string tokenizes to its raw bytes. On top of this base sit the learned merge tokens (“th”, “ the”, ...) at higher IDs, and a handful of **special tokens** (beginning-of-sequence, end-of-sequence, padding) at reserved IDs; those are not produced from text but inserted by the engine, and we wire them in during encoding.

The data structure is a **bidirectional map**. Encoding needs token-bytes $\rightarrow$ ID (“what is the ID of “ the”?”), so an `unordered_map<string,int>`. Decoding needs ID $\rightarrow$ token-bytes (“what bytes does token 1234 stand for?”), so a `vector<string>` indexed by ID. Adding a token appends to the vector and inserts into the map, keeping them in lock-step. That is the whole vocabulary; the intelligence is in the merges, which come next.

A note on text in C++: we use `std::string` purely as a container of bytes (a `std::string` can hold embedded null bytes and arbitrary byte values just fine).

We are *not* treating it as human-readable text, so be careful with signedness when converting — a char may be signed, so cast through `unsigned char` before widening to `int` to avoid bytes above 127 becoming negative.

THE TASK

In `tokenizer.hpp/tokenizer.cpp`, define a `Tokenizer` class holding `std::vector<std::string> id_to_token_` and `std::unordered_map<std::string,int> token_to_id_`. Provide `add_token(bytes)` (returns the existing or newly-assigned ID), `id_of(bytes)` (-1 if absent), `token_of(id)` (the bytes), and `vocab_size()`. Provide a factory with `byte_vocab()` that adds the 256 single-byte tokens in order, so byte value *b* is token ID *b*.

HINTS

Hint 1 — a one-byte string from a byte value

`std::string(1, static_cast<char>(static_cast<unsigned char>(b)))` makes the one-byte token for value *b* (0–255). Build them in a loop for (`int b = 0; b < 256; ++b`) so the IDs line up with the byte values.

Hint 2 — keep the two maps in lock-step

`add_token` should: look up in `token_to_id_`; if present return that ID; otherwise `id = id_to_token_.size()`, push the bytes onto `id_to_token_`, insert `{bytes, id}` into `token_to_id_`, and return `id`. Never modify one map without the other.

Hint 3 — signedness

When you turn input bytes into IDs later, write `static_cast<unsigned char>(c)` first. Forgetting this is the classic byte-level bug: `0x80–0xFF` come out negative and index out of range.

TESTS

```
// tests/test_tokenizer.cpp
#include "test_framework.hpp"
```



```

#include "llm/tokenizer.hpp"
using namespace llm;

TEST(byte_vocab_has_256_tokens) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    CHECK(t.vocab_size() == 256);
    CHECK(t.id_of(std::string(1, 'A')) == 65);    // 'A' == 0x41
    CHECK(t.token_of(65) == "A");
}

TEST(every_byte_value_is_representable) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    for (int b = 0; b < 256; ++b) {
        const std::string one(1, static_cast<char>(static_cast<unsigned char>(b)));
        CHECK(t.id_of(one) == b);                // byte b -> id b
    }
}

TEST(add_token_is_idempotent_and_extends) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    const int id1 = t.add_token("the");
    const int id2 = t.add_token("the");          // same token -> same id
    CHECK(id1 == id2);
    CHECK(id1 == 256);                          // first id past the byte base
    CHECK(t.vocab_size() == 257);
}

```

REFERENCE SOLUTION

include/llm/tokenizer.hpp:

```

// include/llm/tokenizer.hpp
#pragma once
#include <cstdint>
#include <string>
#include <unordered_map>
#include <vector>
#include "llm/common.hpp"

namespace llm {

class Tokenizer {
public:

```

```

// Seed with the 256 single-byte tokens (real vocab + merges arrive in Unit 10)
→ .
static Tokenizer with_byte_vocab();

int vocab_size() const { return static_cast<int>(id_to_token_.size()); }

int add_token(const std::string& token);           // existing-or-new id
int id_of(const std::string& token) const;         // -1 if absent
const std::string& token_of(int id) const;         // raw bytes for an id

protected:
    std::vector<std::string>          id_to_token_; // id -> bytes
    std::unordered_map<std::string,int> token_to_id_; // bytes -> id
};

} // namespace llm

```

src/tokenizer.cpp:

```

// src/tokenizer.cpp
#include "llm/tokenizer.hpp"

namespace llm {

int Tokenizer::add_token(const std::string& token) {
    auto it = token_to_id_.find(token);
    if (it != token_to_id_.end()) return it->second;
    const int id = static_cast<int>(id_to_token_.size());
    id_to_token_.push_back(token);
    token_to_id_.emplace(token, id);
    return id;
}

int Tokenizer::id_of(const std::string& token) const {
    auto it = token_to_id_.find(token);
    return it == token_to_id_.end() ? -1 : it->second;
}

const std::string& Tokenizer::token_of(int id) const {
    LLM_ASSERT(id >= 0 && id < static_cast<int>(id_to_token_.size()),
               "token_of: id out of range");
    return id_to_token_[id];
}

Tokenizer Tokenizer::with_byte_vocab() {

```

```

    Tokenizer t;
    for (int b = 0; b < 256; ++b)
        t.add_token(std::string(1, static_cast<char>(static_cast<unsigned char>(b))
        ↪ ));
    return t;
}

} // namespace llm

```

The vocabulary looks almost trivial, and that is the point: the byte base is a tiny amount of code that buys an enormous guarantee — **nothing is unencodable**. Every clever thing the tokenizer does is layered on a foundation that cannot fail to represent its input.

GOING FURTHER

- **Special tokens.** Reserve IDs for <bos>, <eos>, <pad> and store their bytes (or sentinel strings). We use them in 4.3; adding the slots now makes the vocab layout match real models.
- **The GPT-2 byte-to-unicode map.** Some tokenizers remap the 256 bytes to 256 printable Unicode code points so the vocabulary file is human-readable. It changes the on-disk representation, not the algorithm; Unit 10 handles whichever format a model ships.
- **Reserve capacity.** A real vocabulary is 32k–256k tokens; `reserve()` both containers when you know the size to avoid rehashing during load.

4.2 BPE Merges and Encoding

GOAL

The algorithm at the core of the tokenizer: given an ordered list of **merge rules**, encode a run of bytes into tokens by repeatedly applying the **highest-priority** applicable merge until none remain. This is how `[h][e][l][l][o]` becomes `[he][ll][o]`.

BACKGROUND

Byte-Pair Encoding compresses a sequence by iteratively gluing the most valuable adjacent pair into a single unit. The “most valuable” pairs were chosen once, at training time, by counting which adjacent pairs were most frequent across a corpus; the result is a **ranked list of merge rules**, each saying “token *A* followed by token *B* becomes token *AB*,” ordered so that rank 0 is the merge to apply first, rank 1 next, and so on. We are not learning these rules — the model ships with them — we are *applying* them, which is the encoding half of BPE.

The encoding algorithm is greedy and rank-driven. Start with the sequence of individual bytes. Then loop: among all adjacent pairs currently in the sequence, find the one whose merge rule has the **lowest rank** (highest priority); if there is one, replace that pair with its merged token and repeat; if no adjacent pair has a rule, stop. Each iteration shortens the sequence by one, gluing the single best pair, so frequent multi-byte sequences collapse into single tokens while anything unrecognized is simply left as bytes.

```
encode "hello"  (bytes:  h e l l o)

merge rules:   rank 0:  l + l  -> ll          rank 1:  h + e  -> he

step 0:  [h] [e] [l] [l] [o]      pairs: (h,e)=1 (e,l)=- (l,l)=0 (l,o)=-
                                           lowest rank is (l,l)=0 -> merge it
step 1:  [h] [e] [ll] [o]         pairs: (h,e)=1 (e,ll)=- (ll,o)=-
                                           lowest rank is (h,e)=1 -> merge it
step 2:  [he] [ll] [o]            no pair has a rule -> done

result:  3 tokens:  "he"  "ll"  "o"
```

Notice that **rank order matters**: (l,l) merged before (h,e) only because it had a lower rank, and a different ranking would yield different tokens. This is why an engine must apply the model’s *exact* merge list in the *exact* priority order — the same text under a different order tokenizes differently, and a mismatch would feed the model a sequence it was never trained on.

Two implementation notes. First, a **merged token’s ID** is found by looking up the concatenation of the two operands’ bytes in the vocabulary — the merge rule guarantees that combined token exists. Second, to test a pair quickly you want a **pair** → **rank** lookup; rather than hash a `std::pair`, pack the two IDs into a single 64-bit key (a in the high 32 bits, b in the low 32) and use an `unordered_map<int64_t,int>`. The naive “scan for the best pair each iteration” is

$O(n^2)$ in the run length, but pre-tokenization (4.3) keeps runs short — typically a word — so it is perfectly fast in practice; a priority-queue version for long runs is a Going Further.

THE TASK

Extend `Tokenizer` with `add_merge(a, b, rank)` — which ensures the merged token `a+b` is in the vocab and records rank for the pair — and `bpe_encode_bytes(piece)`, which converts the bytes of `piece` to initial IDs (0–255) and runs the greedy lowest-rank merge loop until no rule applies, returning the token IDs. Store ranks in an `unordered_map<int64_t, int>` keyed by the packed ID pair.

HINTS

Hint 1 — packing a pair of IDs into a key

`static int64_t pair_key(int a, int b) { return (int64_t(a) << 32) | uint32_t(b); }`. Use it both when recording a merge and when probing for one — the two must agree.

Hint 2 — the merge loop

Each pass: scan `i` from 0 to `size-2`, look up `pair_key(ids[i], ids[i+1])`, track the minimum rank and its position. If found, set `ids[best] = id_of(token_of(ids[best]) + token_of(ids[best+1]))` and erase the element after it. Repeat until no pair is found.

Hint 3 — initial IDs are just the byte values

Because byte `b` is token ID `b`, the initial sequence is simply each `static_cast<unsigned char>(c)` of the input pushed as an `int`. No lookup needed for the starting point.

TESTS

```
// add to tests/test_tokenizer.cpp
```

```

TEST(bpe_merges_in_rank_order) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    t.add_merge("l", "l", 0);           // "ll" has priority over "he"
    t.add_merge("h", "e", 1);           // "he"
    auto ids = t.bpe_encode_bytes("hello");
    REQUIRE(ids.size() == 3);
    CHECK(t.token_of(ids[0]) == "he");
    CHECK(t.token_of(ids[1]) == "ll");
    CHECK(t.token_of(ids[2]) == "o");
}

TEST(bpe_applies_chained_merges) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    t.add_merge("a", "a", 0);           // "aa"
    t.add_merge("aa", "a", 1);          // "aa" + "a" -> "aaa"
    auto ids = t.bpe_encode_bytes("aaa");
    REQUIRE(ids.size() == 1);
    CHECK(t.token_of(ids[0]) == "aaa");
}

TEST(bpe_without_rules_is_raw_bytes) {
    Tokenizer t = Tokenizer::with_byte_vocab(); // no merges
    auto ids = t.bpe_encode_bytes("hi");
    REQUIRE(ids.size() == 2);
    CHECK(ids[0] == static_cast<int>('h'));
    CHECK(ids[1] == static_cast<int>('i'));
}

```

REFERENCE SOLUTION

Add to tokenizer.hpp (inside the class):

```

public:
    // Record "a" + "b" -> "ab" with the given rank (lower rank merges earlier).
    void add_merge(const std::string& a, const std::string& b, int rank);

    // Greedy BPE over a run of raw bytes -> token ids.
    std::vector<int> bpe_encode_bytes(const std::string& piece) const;

protected:
    std::unordered_map<int64_t, int> merge_rank_; // pair_key(a_id, b_id) -> rank
    static int64_t pair_key(int a, int b) {
        return (static_cast<int64_t>(a) << 32) | static_cast<uint32_t>(b);
    }

```

Add to `tokenizer.cpp`:

```
#include <climits>

void Tokenizer::add_merge(const std::string& a, const std::string& b, int rank) {
    const int ia = id_of(a), ib = id_of(b);
    LLM_ASSERT(ia >= 0 && ib >= 0, "add_merge: operands must already be in the
    ↪ vocab");
    add_token(a + b);
    merge_rank_[pair_key(ia, ib)] = rank;
}

std::vector<int> Tokenizer::bpe_encode_bytes(const std::string& piece) const {
    std::vector<int> ids;
    ids.reserve(piece.size());
    for (unsigned char c : piece) ids.push_back(static_cast<int>(c)); // bytes
    ↪ 0..255

    while (ids.size() >= 2) {
        int best_rank = INT_MAX, best_pos = -1;
        for (size_t i = 0; i + 1 < ids.size(); ++i) {
            auto it = merge_rank_.find(pair_key(ids[i], ids[i + 1]));
            if (it != merge_rank_.end() && it->second < best_rank) {
                best_rank = it->second;
                best_pos = static_cast<int>(i);
            }
        }
        if (best_pos < 0) break; // no applicable rule -> done
        const std::string merged =
            token_of(ids[best_pos]) + token_of(ids[best_pos + 1]);
        ids[best_pos] = id_of(merged);
        ids.erase(ids.begin() + best_pos + 1);
    }
    return ids;
}
```

This loop is the whole of BPE decoding-into-tokens, and its behavior is entirely determined by the rank table. Feed it the model's real merge list (Unit 10) and the same "hello" will split exactly as that model expects; feed it nothing and it degrades gracefully to raw bytes. Either way it terminates — every iteration removes an element — and either way every input is encodable.

GOING FURTHER

- **Priority-queue BPE.** Replace the per-pass scan with a heap of candidate merges keyed by rank, updating neighbors after each merge. This takes a long run from $O(n^2)$ to roughly $O(n \log n)$ — worth it if you ever tokenize without pre-tokenization.
- **Caching by piece.** The same word appears constantly; memoize `bpe_encode_bytes` on the piece string and most calls become a hash lookup. Real tokenizers cache aggressively.
- **Dropout / regularization.** Some training schemes randomly skip merges (“BPE-dropout”). Not needed for inference, but it explains why a model might be robust to slightly different tokenizations.

4.3 Pre-tokenization and the Full Encoder

GOAL

Assemble the complete `encode(text)`: first **pre-tokenize** — split the text into pieces so merges never cross natural boundaries — then run BPE within each piece, concatenating the results, with optional **special tokens** (a BOS marker) around them.

BACKGROUND

Running BPE on a whole sentence at once would be a mistake. Nothing stops a merge from gluing the last byte of one word to the first byte of the next, producing tokens that straddle the space between words. That both pollutes the vocabulary with nonsensical cross-word fragments and makes a word’s tokenization depend on its neighbors. The fix, universal in modern tokenizers, is **pre-tokenization**: before any merging, chop the text into pieces — runs of letters, runs of digits, punctuation, whitespace — and run BPE *independently within each piece*. Merges then respect word boundaries by construction, because a merge can only ever combine tokens inside the same piece.

Production tokenizers express the split as a carefully tuned regular expression. The GPT-2/Llama family pattern isolates contractions, then runs of letters, runs of numbers, runs of other symbols, and runs of whitespace — and, importantly,

attaches a *leading space* to the word that follows it, so " the" (with its space) is a single token, distinct from "the" at the start of a line. That space-prefix convention is why tokenized text can be reassembled with spacing intact.

That exact regex relies on Unicode character-class support that C++'s `std::regex` does not handle well, so production code uses a UTF-8-aware splitter (or a library like PCRE2). For this book we implement a **simplified pre-tokenizer** — split into maximal runs of whitespace versus non-whitespace — which captures the essential idea (BPE runs per piece, never across a space) and, as we will prove in the boss, round-trips perfectly. The precise pattern is a property of the specific model and is loaded alongside its vocabulary in Unit 10; swapping our splitter for the model's is a localized change, because everything downstream operates on whatever pieces it is handed.

Finally, **special tokens**. Models are trained to expect certain markers: a **beginning-of-sequence** (BOS) token at the very start of the input, and an **end-of-sequence** (EOS) token the model emits (and we watch for) to signal it is done. These are not produced by BPE from text — they live at reserved IDs and are inserted by the engine. Our encoder optionally prepends BOS; the generation loop in Unit 11 will handle EOS on the way out.

The full pipeline, then: **text** → pre-tokenize into pieces → for each piece, bytes → BPE → IDs → concatenate, optionally with BOS in front.

THE TASK

Extend `Tokenizer` with `set_special_tokens(bos_id, eos_id)` (and `bos_id()`, `eos_id()` accessors), a static `pretokenize(text)` that splits into maximal whitespace / non-whitespace runs, and `encode(text, add_bos = false)` that optionally prepends BOS, then for each piece appends `bpe_encode_bytes(piece)`.

HINTS

Hint 1 — the run-splitter

Walk the string; at each position note whether the current byte is whitespace, then extend while the next byte has the *same* whitespace-ness. Emit that run as a piece and continue. "a b" becomes ["a", " ", "b"] — spaces are their own pieces, so BPE never merges across them.

Hint 2 — assemble, do not re-implement

encode is glue: push bos if asked, then for each piece call the `bpe_encode_bytes` you already wrote and insert its IDs onto the output. No new algorithm here.

Hint 3 — guard the special tokens

If `add_bos` is requested, assert a bos ID was actually set (`bos_id_ >= 0`); silently emitting nothing for a missing special token hides bugs.

TESTS

```
// add to tests/test_tokenizer.cpp

TEST(encode_runs_bpe_per_piece) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    t.add_merge("l", "l", 0); t.add_merge("e", "ll", 1); // "ell"
    t.add_merge("h", "ell", 2); t.add_merge("hell", "o", 3); // -> "hello"
    auto ids = t.encode("hello world");
    // "hello" -> 1 token, " " -> 1 token, "world" -> 5 raw bytes
    REQUIRE(ids.size() == 7);
    CHECK(t.token_of(ids[0]) == "hello");
    CHECK(t.token_of(ids[1]) == " ");
    CHECK(t.token_of(ids[2]) == "w");
}

TEST(encode_prepends_bos_when_asked) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    const int bos = t.add_token("<bos>");
    t.set_special_tokens(bos, /*eos=*/-1);
    auto ids = t.encode("hi", /*add_bos=*/true);
    REQUIRE(ids.size() == 3); // <bos> h i
    CHECK(ids[0] == bos);
    CHECK(t.token_of(ids[1]) == "h");
}
```

REFERENCE SOLUTION

Add to `tokenizer.hpp`:

```

public:
    void set_special_tokens(int bos_id, int eos_id) { bos_id_ = bos_id; eos_id_ =
        ↪ eos_id; }
    int bos_id() const { return bos_id_; }
    int eos_id() const { return eos_id_; }

    std::vector<int> encode(const std::string& text, bool add_bos = false) const;

protected:
    int bos_id_ = -1;
    int eos_id_ = -1;
    static std::vector<std::string> pretokenize(const std::string& text);

```

Add to tokenizer.cpp:

```

static bool is_space_byte(unsigned char c) {
    return c == ' ' || c == '\t' || c == '\n' || c == '\r' || c == '\f' || c == '\v'
        ↪ ;
}

std::vector<std::string> Tokenizer::pretokenize(const std::string& text) {
    std::vector<std::string> pieces;
    size_t i = 0, n = text.size();
    while (i < n) {
        const size_t start = i;
        const bool space = is_space_byte(static_cast<unsigned char>(text[i]));
        while (i < n &&
            is_space_byte(static_cast<unsigned char>(text[i])) == space) ++i;
        pieces.push_back(text.substr(start, i - start));    // a maximal same-
        ↪ class run
    }
    return pieces;
}

std::vector<int> Tokenizer::encode(const std::string& text, bool add_bos) const {
    std::vector<int> out;
    if (add_bos) {
        LLM_ASSERT(bos_id_ >= 0, "encode: add_bos set but no BOS token configured")
        ↪ ;
        out.push_back(bos_id_);
    }
    for (const std::string& piece : pretokenize(text)) {
        const std::vector<int> ids = bpe_encode_bytes(piece);
        out.insert(out.end(), ids.begin(), ids.end());
    }
}

```

```
    }  
    return out;  
}
```

With this, the engine can turn a prompt into the integer sequence the model reads. The pieces keep merges honest, the per-piece BPE does the compression, and the special tokens frame the sequence the way the model expects. Swap in a real vocabulary, merge list, and split pattern in Unit 10 and this same encode produces exactly the IDs that model was trained on.

GOING FURTHER

- **The real regex.** Implement the GPT-2/Llama pattern with a UTF-8-aware scanner (letters, digits, punctuation, the leading-space-with-word rule). This is what makes your IDs bit-match a reference tokenizer on real prose.
- **Special-token splitting.** Let users put literal `<bos>/<eos>` in a string and have the encoder recognize and emit the reserved IDs instead of byte-encoding the literal text — needed for chat templates.
- **Add-space-prefix.** Adopt the convention of prefixing a leading space to the first piece (or each word), matching how many models treat the start of a sequence.

4.4 Decoding

GOAL

The reverse map: turn a sequence of token IDs back into the original text by looking up each token's bytes and concatenating them — skipping special tokens — so that the bytes the model emits become the characters a user reads.

BACKGROUND

Decoding is the easy direction, and understanding *why* it is easy cements the whole design. Each token is just a string of bytes; the original text was nothing but the concatenation of all its tokens' bytes; so to decode you look up each ID's

bytes and append them in order. That is the entire algorithm. There is no inverse BPE to run, no ambiguity to resolve — concatenation is exact.

The one conceptual trap is to decode **token by token into text**. You must not. Because a token may be a *fragment* of a multi-byte UTF-8 character — the first two bytes of a three-byte glyph might be one token and the third byte the next — an individual token's bytes are not necessarily a valid character. Only after you concatenate *all* the tokens' bytes do you have the complete, valid UTF-8 stream. So the rule is: **concatenate bytes first, interpret as text second**. Our decode returns the raw byte string (which *is* the original text when the inputs were valid UTF-8); turning that into, say, a displayed string is the caller's business and is trivially correct once the bytes are whole.

This has a real consequence for **streaming** generation, where the engine emits text one token at a time as the model produces it (Unit 11). A freshly generated token may end in the middle of a character, so naively printing each token's bytes can momentarily show a broken glyph. The fix is to buffer trailing bytes that do not yet form a complete character and flush them once the next token completes it. We note the technique here and implement it where it belongs, in the generation loop; for batch decoding of a finished sequence the plain concatenation is all you need.

Finally, decoding must **skip special tokens** by default: the BOS/EOS markers are control signals, not text, so emitting their sentinel bytes into the output would be wrong. We keep an option to render them (useful for debugging chat templates), but the default drops them.

THE TASK

Add `decode(ids, skip_special = true)` to `Tokenizer`: walk the IDs, skip any equal to `bos_id_` or `eos_id_` when `skip_special` is set, and append `token_of(id)` (raw bytes) to an output string. Return the bytes.

HINTS

Hint 1 — it really is just concatenation

Loop over the IDs; for each, skip it when `skip_special` is set and it equals the BOS or EOS ID, otherwise append `token_of(id)` to the output with `+=`. The append copies raw bytes; do not attempt any per-token UTF-8 validation.

Hint 2 — the round-trip you are about to prove

For any text, `decode(encode(text))` should equal text byte-for-byte. If it does not, the bug is almost always a signedness slip in encoding (bytes ≥ 128) — which is why the boss tests multi-byte and raw-byte inputs explicitly.

TESTS

```
// add to tests/test_tokenizer.cpp

TEST(decode_concatenates_token_bytes) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    t.add_merge("l", "l", 0);
    const auto ids = t.bpe_encode_bytes("hello");
    CHECK(t.decode(ids) == "hello");
}

TEST(decode_skips_specials_by_default) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    const int bos = t.add_token("<bos>"), eos = t.add_token("<eos>");
    t.set_special_tokens(bos, eos);
    const std::vector<int> ids = { bos, int('h'), int('i'), eos };
    CHECK(t.decode(ids) == "hi"); // specials dropped
    CHECK(t.decode(ids, /*skip_special=*/false) == "<bos>hi<eos>");
}
```

REFERENCE SOLUTION

Add to `tokenizer.hpp`:

```
public:
    std::string decode(const std::vector<int>& ids, bool skip_special = true)
        ↪ const;
```

Add to `tokenizer.cpp`:

```
std::string Tokenizer::decode(const std::vector<int>& ids, bool skip_special)
    ↪ const {
```

```

std::string out;
for (int id : ids) {
    if (skip_special && (id == bos_id_ || id == eos_id_)) continue;
    out += token_of(id);           // append raw bytes; UTF-8 emerges from the
    ↪ whole
}
return out;
}

```

The asymmetry between the two directions is the signature of a good design: encoding does the clever, rank-driven work, while decoding is a trivial, exact concatenation. That triviality is not an accident — it is what guarantees the round-trip the boss is about to measure.

GOING FURTHER

- **Streaming UTF-8 buffer.** Implement an incremental decoder that holds back trailing bytes until they complete a character, so token-by-token output never shows a broken glyph. You will wire it into the generation loop in Unit 11.
- **Rendering special tokens.** Add a mode that maps special IDs to readable strings ("`<bos>`") for debugging, distinct from emitting their raw sentinel bytes.
- **Lossy inputs.** Decide what to do if asked to decode an ID out of range (clamp, throw, or emit the UTF-8 replacement character) — robustness for corrupt or adversarial token streams.

4.5 Round-trip Fidelity

BOSS CHALLENGE

Prove the tokenizer is lossless: for every string in a deliberately hostile corpus — empty, whitespace, accents, CJK, four-byte emoji, ZWJ sequences, and arbitrary non-UTF-8 bytes — `decode(encode(s))` equals `s` **byte-for-byte**. Plus a fuzz test over hundreds of random byte strings. The bar is 100% exact, no exceptions.

GOAL

Establish the one property a tokenizer must never violate: it is a **lossless round-trip**. Whatever bytes go in must come back out exactly — not approximately, not “close enough,” but identical — across the full range of nasty real-world input.

BACKGROUND

A tokenizer is the engine’s contract with reality: it promises that translating text into tokens and back changes nothing. If that promise breaks — if a single emoji or accented letter comes back mangled — then the model is being fed something subtly different from what the user wrote, and every output is suspect. This is exactly the kind of invariant that should be *proven by measurement* rather than assumed, because it is so easy to break in ways that pass casual testing (plain ASCII round-trips fine; the bug only shows on byte `0xC3`).

The byte-level design *guarantees* the round-trip in principle: every byte is its own token, BPE only ever groups bytes (never invents or drops them), and decoding concatenates bytes back in order. So `decode(encode(s))` must reproduce `s`’s exact byte stream. The boss verifies the *implementation* honors that guarantee on inputs chosen to break it:

- **The empty string** and lone whitespace — boundary cases that off-by-one bugs love.
- **Multi-byte utf-8**: accented Latin (café, two-byte sequences), CJK (three-byte), and **four-byte emoji** including **ZWJ sequences** (a single visible glyph made of several joined code points). These stress the byte handling hardest.
- **Arbitrary bytes** `0x00–0xFF`, including sequences that are *not* valid utf-8 at all — because the tokenizer must round-trip raw bytes, not just well-formed text.

The single most common way this fails is a **signedness bug**: a char is signed on most platforms, so a byte ≥ 128 sign-extends to a negative int, which then indexes out of range or maps to the wrong token. Testing four-byte emoji and high bytes explicitly is what flushes that bug out — which is the entire reason the corpus looks the way it does. A second failure mode is letting a special token leak into or out of the byte stream; the fuzz test (with no special tokens configured) and the explicit corpus together close both gaps.

THE TASK

Write `boss_roundtrip_byte_for_byte`: build a byte-vocab tokenizer with a few merges (the property must hold *regardless* of which merges are active), and assert `decode(encode(s)) == s` for a corpus spanning empty/whitespace/accented/CJK/emoji/raw-byte strings. Add `boss_roundtrip_fuzz`: generate hundreds of random byte strings (any byte value, any length) and assert each round-trips byte-for-byte.

The bar (the boss): every string — hand-picked and fuzzed — satisfies `decode(encode(s)) == s` exactly. One mismatch anywhere fails the boss.

HINTS

Hint 1 — raw bytes in a test literal

Construct byte strings with an explicit length so an embedded NUL byte is not treated as a terminator — pass the byte count as the second `std::string` constructor argument (the corpus test's hex-escaped entries show the exact form). Emoji and CJK can be ordinary UTF-8 string literals, provided the source file is saved as UTF-8.

Hint 2 — a tiny fuzzer

Reuse the LCG from Unit 3 ($s = s * 1664525 + 1013904223$). For each trial pick a length in `[0, 64)` and push that many low bytes of the generator (mask with `0xFF`). Compare `decode(encode(str))` to `str`.

Hint 3 — if it fails, suspect signedness first

A mismatch only on inputs with high bytes is the signed-char tell. Audit every place a `char` becomes an `int` for a missing `static_cast<unsigned char>`.

TESTS

```
// add to tests/test_tokenizer.cpp
#include <stdint>
```

```

TEST(boss_roundtrip_byte_for_byte) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    t.add_merge("l", "l", 0);           // a few merges; round-trip must
    ↪ hold anyway
    t.add_merge("e", "ll", 1);
    t.add_merge("s", "s", 2);

    const std::vector<std::string> corpus = {
        "Hello, world!",
        "",                               // empty
        " ",                             // lone space
        " leading and internal spaces ",
        "tabs\tand\nnewlines\r\n",
        "caf\xC3\xA9 d\xC3\xA9j\xC3\xA0 na\xC3\xAFve", // 2-byte UTF-8 (café
    ↪ déjà naïve)
        "\xE6\x97\xA5xE6\x9C\xACxE8\xAA\x9E",         // 3-byte UTF-8   𐄀𐄁𐄂()
        "emoji \xF0\x9F\x98\x80 \xF0\x9F\x9A\x80",     // 4-byte UTF-8 (emoji)
        "mixed ABC 123 !@#$ ",
        std::string("\x00\x01\x02\xFE\xFF", 5),        // arbitrary non-UTF-8
    ↪ bytes
    };
    for (const std::string& s : corpus) {
        const std::vector<int> ids = t.encode(s);
        const std::string back = t.decode(ids);
        CHECK(back == s);                             // byte-for-byte
    }
}

TEST(boss_roundtrip_fuzz) {
    Tokenizer t = Tokenizer::with_byte_vocab();
    t.add_merge("a", "a", 0); t.add_merge("aa", "a", 1);
    uint32_t s = 0x12345;
    auto rnd = [&] { s = s * 1664525u + 1013904223u; return s; };
    for (int trial = 0; trial < 500; ++trial) {
        const int len = static_cast<int>(rnd() % 64);
        std::string str;
        for (int i = 0; i < len; ++i)
            str.push_back(static_cast<char>(rnd() & 0xFFu)); // any byte
    ↪ 0..255
        CHECK(t.decode(t.encode(str)) == str);             // exact round-
    ↪ trip
    }
}

```

REFERENCE SOLUTION

There is no new production code: the boss is cleared by the Unit 4 implementation behaving correctly under adversarial input. The tests above *are* the deliverable. Build and run; both must report [OK] and the suite must end 0 failed. If fuzz or the emoji line fails, the implementation has a byte-handling bug — almost certainly a missing unsigned char cast — and the failing case points right at it.

What you have proven. Not “the tokenizer seems to work,” but “across empty input, every UTF-8 width, four-byte emoji and joined-glyph sequences, and hundreds of random raw-byte strings, encoding then decoding reproduces the input byte-for-byte, every time.” That is the contract the rest of the engine relies on: whatever the user types reaches the model intact, and whatever the model emits reaches the user intact. With it secured, the tokenizer is done as an algorithm — only the model-specific data (vocabulary, merges, split pattern) remains to be loaded, in Unit 10.

GOING FURTHER

- **Differential test against a reference.** Once you can load a real model’s tokenizer (Unit 10), encode a large text corpus with both yours and the reference implementation and assert identical ID sequences — the stronger “matches the real tokenizer” bar on top of round-trip fidelity.
- **Round-trip the IDs, too.** Verify `encode(decode(ids)) == ids` for ID sequences produced by `encode` (it should hold for canonically-tokenized input), and understand why it can fail for arbitrary hand-built ID sequences.
- **Performance.** Tokenize a multi-megabyte file and measure throughput; profile where the time goes (usually the per-piece BPE scan) and apply the caching from 4.2’s Going Further.

Unit 4 — Milestone check

MILESTONE CHECK

The engine can now cross the boundary between text and tokens:

- a **byte-level vocabulary** whose 256-byte base makes every possible input encodable — no unknown tokens, ever;
- the **BPE merge algorithm**, applying a ranked rule list to compress bytes

into tokens exactly as the model expects;

- **pre-tokenization** and a full encode that keeps merges within natural pieces and frames the sequence with special tokens;
- a trivially-exact decode; and
- a **boss-grade guarantee**, proven on emoji, scripts, and random bytes, that the whole pipeline is **byte-for-byte lossless**.

We now have every piece the model needs around it — tensors and memory (Unit 1), the math kernels and softmax (Unit 2), fast matrix multiplication (Unit 3), and text $\leftrightarrow$ tokens (Unit 4) — but not yet the model itself. **Unit 5** assembles the first real **Transformer**: token embeddings, the attention mechanism with rotary position encoding (ROPE), grouped-query attention, the RMSNorm-and-SwiGLU block from Unit 2, and the residual stream — wired into a forward pass that takes token IDs and produces next-token logits. This is where the engine starts to think.

UNIT 5

The Transformer

What you will build. The model itself. You assemble the infrastructure of Units 1–3 into a working **Transformer: token embeddings, rotary position encoding** (ROPE), **grouped-query attention** with a causal mask, the **RMSNorm-and-SwiGLU block** from Unit 2, and the **residual stream** that ties it together — wired into a **forward pass** that turns token IDs into next-token logits. The boss proves the whole thing is a correct *causal* language model.

Where this fits. Everything before now was scaffolding: tensors, kernels, matmul, the tokenizer. This is the structure that actually computes. After this unit, the engine can take a sequence of token IDs and produce a probability distribution over the next token — which, run in a loop (Unit 11), is text generation. The architecture we build is the Llama-family decoder-only Transformer, the same shape as Llama 2/3, Mistral, Qwen, and most modern open models.

A decoder-only Transformer is, at heart, a stack of identical layers operating on a **residual stream**: a running hidden state, one vector per token, that each layer reads, refines, and writes back. Two kinds of refinement alternate. **Attention** lets each token gather information from earlier tokens — this is where context happens. The **feed-forward network** then processes each token’s vector independently — this is where most of the model’s learned knowledge lives. Wrap each in a normalization and a residual add, stack a few dozen, cap with a projection to vocabulary scores, and you have a model that predicts the next token. We build it bottom-up: embeddings and the skeleton, then ROPE, then attention, then the block, then the full forward pass.

Create `include/llm/model.hpp` and `src/model.cpp`. The weights and activations are Tensors (Unit 1); the projections are matrix multiplies (Unit 3); the norms,

activations, and softmax are the kernels from Unit 2. This unit is largely the act of *connecting* what you already built — which is exactly why building those pieces well mattered.

5.1 Token Embeddings and the Model Skeleton

GOAL

The entry point and the overall shape: a **model configuration** (the architecture’s hyperparameters), the **embedding lookup** that turns token IDs into the initial residual-stream vectors, and the skeleton of `forward()` — `embed`, (layers to come), `final norm`, `project to logits`.

BACKGROUND

A model does not operate on token IDs directly; it operates on **vectors**. The **token embedding** is a learned table of shape $[\text{vocab_size} \times \text{hidden_size}]$: row t is the hidden vector for token ID t . “Embedding” a sequence is therefore pure lookup — gather the rows named by the token IDs into a $[\text{seq_len} \times \text{hidden_size}]$ tensor. That tensor *is* the initial **residual stream**, the state every layer will read and update.

The residual stream is the spine of the architecture. Picture a $[\text{seq} \times \text{hidden}]$ tensor flowing downward through the layers. Each layer computes an update of the same shape and *adds* it back — $x \leftarrow x + \text{sublayer}(x)$ — rather than replacing x . These residual connections are what let gradients flow through dozens of layers during training, and at inference they mean each layer contributes a refinement on top of everything the earlier layers established. Nothing is overwritten; information accumulates.

```
token IDs: [ t0  t1  t2  ...  t(S-1) ]
|   embed: gather rows of token_embedding [vocab x hidden]
v
x : [S, hidden]                                <-- the residual stream
|
|   for each of n_layers layers:
|       x = x + attention( rmsnorm(x) )    (attention sub-layer)
|       x = x + ffn(      rmsnorm(x) )    (feed-forward sub-layer)
```

```

      v
x : [S, hidden]
  |   rmsnorm (final)
  |   lm_head: project [S, hidden] -> [S, vocab]
      v
logits : [S, vocab]      (next-token scores at every position)

```

After the last layer, a **final RMSNorm** stabilizes the stream, and the **language model head** (`lm_head`) — a matrix of shape $[\text{vocab} \times \text{hidden}]$ — projects each token’s hidden vector to a **logit** per vocabulary entry: an unnormalized score for “what token comes next.” For position i , `logits[i]` ranks every possible next token given tokens $0..i$. (Many models *tie* weights, reusing the token-embedding matrix as the `lm_head`; we support both.)

The **configuration** names the architecture: `hidden_size`, `n_layers`, `n_heads` and `n_kv_heads` (grouped-query attention, 5.3), `head_dim`, `intermediate_size` (the FFN’s inner width), `vocab_size`, `rope_theta` (5.2), and `rms_norm_eps`. These are exactly the numbers a model file will hand us in Unit 10; here we define the struct they populate. A concrete example: Llama-3.2-1B uses `hidden_size = 2048`, `n_layers = 16`, `n_heads = 32`, `n_kv_heads = 8`, `head_dim = 64`, `intermediate_size = 8192`.

THE TASK

In `model.hpp`, define `ModelConfig` (the fields above) and weight structs `LayerWeights` (per-layer norms and projection matrices — we fill these in 5.3–5.4) and `ModelWeights` (`token_embedding`, a `vector<LayerWeights>`, `final_norm`, `lm_head`). Implement `embed_tokens(out, ids, embedding)` — copy embedding row `ids[i]` into out row `i` — and a helper `linear(out, x, w)` computing `out[i,o] = sum_k x[i,k]*w[o,k]` (the `[out,in]` weight convention; it dispatches to Unit 3’s GEMM in production). Sketch `forward(model, cfg, ids)` as: `embed` $\rightarrow$ (layer loop, added in 5.4) $\rightarrow$ `rmsnorm` with `final_norm` $\rightarrow$ `linear` with `lm_head` $\rightarrow$ return `[seq, vocab]` logits.

HINTS

Hint 1 — embedding is a row gather

Each row is `hidden_size` contiguous floats. `embed_tokens` is a loop:

```
std::memcpy(out_row_i, embedding.data_ptr<float>() +
(int64_t)ids[i]*hidden, hidden*sizeof(float)).
```

No arithmetic — just copy the right row.

Hint 2 — the linear/weight convention

Transformer weights are stored `[out_features, in_features]`, and a linear layer computes $y = xW^T$, i.e. `y[i,o] = dot(x_row_i, w_row_o)`. Write the obvious triple loop now; swapping it for the packed SIMD GEMM of Unit 3 (with `B` transposed) is a drop-in later.

Hint 3 — keep weights as Tensors, activations in an arena

Store all weight matrices as Tensors. Allocate the per-forward activations (`[seq, hidden]` and friends) from the Arena of Unit 1 so a forward pass does no heap allocation after warm-up.

TESTS

```
// tests/test_model.cpp
#include "test_framework.hpp"
#include "llm/model.hpp"
#include <cstring>
using namespace llm;

TEST(embed_gathers_rows) {
    const int vocab = 4, hidden = 3;
    Tensor emb = Tensor::empty({vocab, hidden}, DType::F32);
    for (int i = 0; i < vocab * hidden; ++i) emb.data_ptr<float>()[i] = float(i);
    // row r is [3r, 3r+1, 3r+2]
    std::vector<int> ids = {2, 0, 3};
    Tensor out = Tensor::empty(({(int64_t)ids.size(), hidden}, DType::F32);
    embed_tokens(out, ids, emb);
    CHECK_CLOSE(out.at<float>({0, 0}), 6.0f, 1e-6);    // token 2 -> row 2 ->
    ↪ [6,7,8]
    CHECK_CLOSE(out.at<float>({0, 2}), 8.0f, 1e-6);
    CHECK_CLOSE(out.at<float>({1, 0}), 0.0f, 1e-6);    // token 0 -> [0,1,2]
    CHECK_CLOSE(out.at<float>({2, 1}), 10.0f, 1e-6);    // token 3 -> [9,10,11]
```



```

}

TEST(linear_matches_dot_products) {
    const int S = 2, K = 3, N = 2;
    Tensor x = Tensor::empty({S, K}, DType::F32);
    Tensor w = Tensor::empty({N, K}, DType::F32);    // [out, in]
    float xs[6] = {1, 2, 3, 4, 5, 6};
    float ws[6] = {1, 0, -1, 2, 2, 2};              // w row0=(1,0,-1), row1
    ↪ =(2,2,2)
    for (int i = 0; i < 6; ++i) { x.data_ptr<float>()[i] = xs[i]; w.data_ptr<float>
    ↪ >()[i] = ws[i]; }
    Tensor y = Tensor::empty({S, N}, DType::F32);
    linear(y, x, w);
    CHECK_CLOSE(y.at<float>({0, 0}), 1*1 + 2*0 + 3*-1, 1e-5);    // -2
    CHECK_CLOSE(y.at<float>({0, 1}), 1*2 + 2*2 + 3*2, 1e-5);    // 12
    CHECK_CLOSE(y.at<float>({1, 0}), 4*1 + 5*0 + 6*-1, 1e-5);    // -2
}

```

REFERENCE SOLUTION

include/llm/model.hpp:

```

// include/llm/model.hpp
#pragma once
#include <vector>
#include "llm/tensor.hpp"

namespace llm {

struct ModelConfig {
    int vocab_size      = 0;
    int hidden_size    = 0;
    int n_layers       = 0;
    int n_heads        = 0;
    int n_kv_heads     = 0;    // <= n_heads (grouped-query attention)
    int head_dim       = 0;    // usually hidden_size / n_heads
    int intermediate_size= 0;   // FFN inner width
    float rope_theta    = 10000.0f;
    float rms_norm_eps  = 1e-5f;
};

struct LayerWeights {
    Tensor attn_norm;          // [hidden]
    Tensor wq, wk, ww, wo;    // wq:[n_heads*hd, hidden] wk,ww:[n_kv*hd,

```

```

    ↪ hidden]

    Tensor ffn_norm;           // wo:[hidden, n_heads*hd]
    Tensor w_gate, w_up;      // [hidden]
    Tensor w_down;            // [intermediate, hidden]
                                // [hidden, intermediate]
};

struct ModelWeights {
    Tensor token_embedding;    // [vocab, hidden]
    std::vector<LayerWeights> layers;
    Tensor final_norm;        // [hidden]
    Tensor lm_head;           // [vocab, hidden] (may alias token_embedding)
};

// Gather embedding rows named by ids into out [seq, hidden].
void embed_tokens(Tensor& out, const std::vector<int>& ids, const Tensor&
    ↪ embedding);

// Linear layer with [out, in] weight: out[i,o] = sum_k x[i,k]*w[o,k].
void linear(Tensor& out, const Tensor& x, const Tensor& w);

// Full forward pass: token ids -> logits [seq, vocab]. (Completed in 5.5.)
Tensor forward(const ModelWeights& model, const ModelConfig& cfg,
    const std::vector<int>& ids);

} // namespace llm

```

src/model.cpp (the parts for this challenge):

```

// src/model.cpp
#include "llm/model.hpp"
#include "llm/ops.hpp"
#include <cstring>

namespace llm {

void embed_tokens(Tensor& out, const std::vector<int>& ids, const Tensor&
    ↪ embedding) {
    const int hidden = static_cast<int>(embedding.shape()[1]);
    const float* table = embedding.data_ptr<float>();
    float* o = out.data_ptr<float>();
    for (size_t i = 0; i < ids.size(); ++i)
        std::memcpy(o + i * hidden,
            table + static_cast<int64_t>(ids[i]) * hidden,
            sizeof(float) * hidden);
}

```

```

}

void linear(Tensor& out, const Tensor& x, const Tensor& w) {
    const int S = static_cast<int>(x.shape()[0]);
    const int K = static_cast<int>(x.shape()[1]);
    const int N = static_cast<int>(w.shape()[0]);    // [out, in]
    const float* X = x.data_ptr<float>();
    const float* W = w.data_ptr<float>();
    float* Y = out.data_ptr<float>();
    for (int i = 0; i < S; ++i)
        for (int o = 0; o < N; ++o) {
            float acc = 0.0f;
            for (int k = 0; k < K; ++k)
                acc += X[(int64_t)i * K + k] * W[(int64_t)o * K + k];
            Y[(int64_t)i * N + o] = acc;           // production: dispatch to Unit
        }
    }

    ↪ -3 GEMM

}

} // namespace llm

```

The two operations here are deliberately humble — a row copy and a dot-product loop — but they fix the conventions the rest of the unit depends on: the residual stream is [seq, hidden], weights are [out, in], and a “linear layer” means $xW^\top$. With the skeleton in place, we can start filling in what happens *inside* a layer, beginning with how the model learns where each token sits.

GOING FURTHER

- **Tied embeddings.** When `lm_head` aliases `token_embedding`, store one tensor and point both at it (shared Storage from Unit 1) — a real memory saving for large vocabularies.
- **Embedding scale.** Some architectures multiply embeddings by $\sqrt{\text{hidden}}$. Note where such a scale would go; the model file says whether it applies.
- **Dispatch linear to GEMM.** Wire `linear` to call `matmul_simd` with the weight treated as the transposed operand, so every projection runs at the throughput you built in Unit 3.

5.2 Rotary Position Encoding (RoPE)

GOAL

Give the model a sense of *where* each token is. Implement **RoPE**: rotate pairs of dimensions of the query and key vectors by an angle proportional to position, so that the attention score between two tokens depends on their **relative** position — the encoding modern LLMs use.

BACKGROUND

Self-attention has a blind spot: it is **permutation-equivariant**. On its own, attention treats a sequence as a *set* — shuffle the tokens and the math is unchanged, so “dog bites man” and “man bites dog” would be indistinguishable. The model must be told the order, and how we tell it is **position encoding**.

Older models *added* a learned or sinusoidal position vector to each embedding. RoPE (Rotary Position Embedding) does something cleverer that has become standard: it **rotates** the query and key vectors by a position-dependent angle. Pair up the `head_dim` coordinates of a head’s vector into `head_dim/2` two-dimensional planes; in plane i , rotate by an angle $\text{pos} \cdot \theta_i$ where the per-plane frequency is

$$\theta_i = \text{base}^{-2i/\text{head_dim}}, \quad i = 0, 1, \dots, \frac{\text{head_dim}}{2} - 1,$$

with `base` the `rope_theta` hyperparameter (typically 10^4 ; Llama 3 uses 5×10^5 to reach longer contexts). The frequencies decrease geometrically, so low-index planes spin quickly and high-index planes slowly — a spectrum of wavelengths, much like the classic sinusoidal scheme, but applied as rotations.

The reason rotation is the right move is a small, beautiful identity. Rotating a 2-D vector by angle α is multiplication by a rotation matrix $R(\alpha)$, and rotations compose by adding angles, with $R(\alpha)^\top R(\beta) = R(\beta - \alpha)$. So if a query at position m and a key at position n are each rotated by their position’s angle, their dot product becomes

$$(R(m\theta_i)q) \cdot (R(n\theta_i)k) = q^\top R(m\theta_i)^\top R(n\theta_i)k = q^\top R((n-m)\theta_i)k,$$

which depends on m and n *only through their difference* $n - m$. Attention thus becomes **relative-position-aware** for free: the score between two tokens is governed by how far apart they are, not where they sit absolutely — which is exactly

what you want for language, and why RoPE extrapolates to longer sequences gracefully.

Two practical points. RoPE is applied to the **query and key** vectors only (not values), **per head**, just before computing attention scores — values carry content, not position. And there are two layout **conventions** for which coordinates form a pair: the “interleaved” convention pairs adjacent dims $(2i, 2i+1)$, while the “half” convention (which we implement) pairs dim i with dim $i + \text{head_dim}/2$. They are equivalent up to a permutation of the weight columns; which one a model uses is a metadata detail resolved when we load it in Unit 10. Every property we test below holds for either.

Concretely, for a head vector x with $\text{half} = \text{head_dim}/2$, at position pos , with $c = \cos(\text{pos} \cdot \theta_i)$ and $s = \sin(\text{pos} \cdot \theta_i)$:

$$x'_i = x_i c - x_{i+\text{half}} s, \quad x'_{i+\text{half}} = x_{i+\text{half}} c + x_i s.$$

THE TASK

Implement `apply_rope(x, seq, n_heads, head_dim, theta, pos0 = 0)` that rotates, in place, each head’s vector in x of shape $[\text{seq}, \text{n_heads}, \text{head_dim}]$. Token t uses position $\text{pos0} + t$ (the offset lets a cached continuation start mid-sequence, Unit 7). Precompute the $\text{head_dim}/2$ frequencies once; for each token, head, and plane, apply the rotation above.

HINTS

Hint 1 — precompute the frequencies

Compute `inv_freq[i] = std::pow(theta, -2.0f*i/head_dim)` once into a small vector before the loops — $\text{head_dim}/2$ values, reused for every token and head. Avoid calling `pow` in the inner loop.

Hint 2 — index the head vector

With x contiguous as $[\text{seq}, \text{n_heads}, \text{head_dim}]$, head h of token t starts at $((\text{int64_t})t * \text{n_heads} + h) * \text{head_dim}$. Rotate plane i using elements $v[i]$ and $v[i+\text{half}]$; write both back.

Hint 3 — read both before writing either

The rotation mixes $v[i]$ and $v[i+\text{half}]$, so capture $x0 = v[i]$ and $x1 = v[i+\text{half}]$ into locals *first*, then overwrite. Updating in place without the temporaries corrupts the second component.

TESTS

```
// add to tests/test_model.cpp
#include <cmath>

TEST(rope_at_position_zero_is_identity) {
    const int hd = 4;
    Tensor x = Tensor::empty({1, 1, hd}, DType::F32);
    const float orig[hd] = {0.3f, -0.7f, 1.1f, 0.5f};
    for (int i = 0; i < hd; ++i) x.data_ptr<float>()[i] = orig[i];
    apply_rope(x, /*seq=*/1, /*heads=*/1, hd, /*theta=*/10000.0f, /*pos0=*/0);
    for (int i = 0; i < hd; ++i) CHECK_CLOSE(x.data_ptr<float>()[i], orig[i], 1e-6)
    ↪ ;
}

TEST(rope_preserves_norm) {
    const int hd = 8;
    Tensor x = Tensor::empty({1, 1, hd}, DType::F32);
    float before = 0.0f;
    for (int i = 0; i < hd; ++i) { float v = 0.1f * i - 0.3f; x.data_ptr<float>()[
    ↪ i] = v; before += v * v; }
    apply_rope(x, 1, 1, hd, 10000.0f, /*pos0=*/5);
    float after = 0.0f;
    for (int i = 0; i < hd; ++i) { float v = x.data_ptr<float>()[i]; after += v * v
    ↪ ; }
    CHECK_CLOSE(std::sqrt(after), std::sqrt(before), 1e-5); // rotation is norm-
    ↪ preserving
}

TEST(rope_dot_depends_on_relative_position) {
    // One plane (hd=2): inv_freq[0] = theta^0 = 1, so angle = pos. q=k=(1,0).
    // After RoPE, q_m . k_n = cos(m)cos(n)+sin(m)sin(n) = cos(m-n).
    const int hd = 2;
    auto roped = [&](int pos) {
        Tensor x = Tensor::empty({1, 1, hd}, DType::F32);
        x.data_ptr<float>()[0] = 1.0f; x.data_ptr<float>()[1] = 0.0f;
        apply_rope(x, 1, 1, hd, 10000.0f, pos);
        return std::pair<float, float>(x.data_ptr<float>()[0], x.data_ptr<float>())
    }
}
```

```

    ↪ [1]);
};
const auto qm = roped(5), kn = roped(2);           // m - n = 3
const float dot = qm.first * kn.first + qm.second * kn.second;
CHECK_CLOSE(dot, std::cos(3.0f), 1e-5);
}

```

REFERENCE SOLUTION

Declare in `model.hpp`, define in `model.cpp`:

```

// model.hpp
void apply_rope(Tensor& x, int seq, int n_heads, int head_dim, float theta, int
    ↪ pos0 = 0);

```

```

// model.cpp
#include <cmath>
#include <vector>

void apply_rope(Tensor& x, int seq, int n_heads, int head_dim, float theta, int
    ↪ pos0) {
    const int half = head_dim / 2;
    std::vector<float> inv_freq(half);
    for (int i = 0; i < half; ++i)
        inv_freq[i] = std::pow(theta, -2.0f * static_cast<float>(i) / head_dim);

    float* X = x.data_ptr<float>();           // [seq, n_heads,
    ↪ head_dim]
    for (int t = 0; t < seq; ++t) {
        const float pos = static_cast<float>(pos0 + t);
        for (int h = 0; h < n_heads; ++h) {
            float* v = X + ((int64_t)t * n_heads + h) * head_dim;
            for (int i = 0; i < half; ++i) {
                const float ang = pos * inv_freq[i];
                const float c = std::cos(ang), s = std::sin(ang);
                const float x0 = v[i], x1 = v[i + half]; // read both first
                v[i]           = x0 * c - x1 * s;
                v[i + half] = x1 * c + x0 * s;
            }
        }
    }
}

```

RoPE is a small function with an outsized role: it is the only place the model learns about order, and it does so without any added parameters — pure trigonometry on the query and key vectors. The relative-position test is the one to internalize: it shows, concretely, that two identical tokens three apart produce an attention contribution of $\cos(3)$ regardless of where they sit, which is the entire point of the rotary scheme. Next we put these rotated queries and keys to work.

GOING FURTHER

- **Precomputed cos/sin tables.** For a fixed maximum sequence length, precompute a `[max_seq, head_dim/2]` table of cos and sin once at load time and look them up, removing the trig from the hot path entirely.
- **Context extension.** Implement RoPE scaling (linear/NTK/YARN) — adjustments to the frequencies that let a model trained on short contexts run on longer ones. These are increasingly common and are described by model metadata.
- **The interleaved convention.** Add the adjacent-pair variant and a flag to select it, so you can match models that store weights in that layout without permuting them.

5.3 Attention with Grouped-Query Heads

GOAL

The mechanism that lets tokens see each other. Implement a full attention sub-layer: project to **queries, keys, and values**, apply RoPE, compute **scaled dot-product attention** with a **causal mask**, support **grouped-query attention** (GQA, fewer key/value heads than query heads), and project the result back.

BACKGROUND

Attention is how a token pulls in information from other tokens. Each token forms three vectors by linear projection of its hidden state: a **query** (what am I looking for?), a **key** (what do I offer?), and a **value** (what do I contribute if

attended to?). Token i scores every token j by the dot product of i 's query with j 's key, turns those scores into weights with a softmax, and takes the weighted sum of the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{\text{head_dim}}}\right) V.$$

The $\sqrt{\text{head_dim}}$ **scaling** is not cosmetic: without it, larger head dimensions produce dot products with larger variance, pushing the softmax toward a near one-hot spike where almost all weight lands on a single token. Dividing by $\sqrt{\text{head_dim}}$ keeps the scores in a sane range and the softmax informative. (And this is precisely the numerically-stable softmax from Unit 2 — on logits we do not control, which is why we made it bulletproof.)

Multiple heads. Rather than one attention over the whole hidden vector, the model splits it into `n_heads` independent **heads** of `head_dim` each, runs the attention above separately per head, and concatenates the results. Different heads learn to attend to different relationships — syntax, coreference, position — in parallel.

The causal mask. This is a decoder predicting the next token, so token i must only attend to tokens $0..i$: peeking at future tokens would leak the answer. We enforce this by restricting each query's softmax to keys at positions $\leq i$ (equivalently, setting future scores to $-\infty$ before the softmax). This single restriction is what makes the model *causal*, and the boss will verify it end-to-end.

Grouped-query attention. Keys and values are the bulk of what must be *stored* during generation (the kv cache, Unit 7), so models economize by using *fewer* key/value heads than query heads: $n_{\text{kv_heads}} \leq n_{\text{heads}}$. Query heads are divided into $n_{\text{heads}} / n_{\text{kv_heads}}$ groups, and all queries in a group share one key/value head. The extremes are familiar — $n_{\text{kv_heads}} = n_{\text{heads}}$ is ordinary multi-head attention, $n_{\text{kv_heads}} = 1$ is multi-query attention — and the middle ground, GQA, is what Llama 2 70B and the Llama 3 family use to shrink the cache with negligible quality cost. The query head h maps to key/value head $\lfloor h / (n_{\text{heads}} / n_{\text{kv_heads}}) \rfloor$.

Putting it together, the sub-layer is: project x to Q [seq, $n_{\text{heads}} * \text{head_dim}$] and K, V [seq, $n_{\text{kv_heads}} * \text{head_dim}$]; view as heads; RoPE the queries and keys; for each head and each query position i , score against keys $0..i$, scale, softmax, and take the weighted sum of values; concatenate the heads; and project through w_o back to [seq, hidden]. (The caller normalizes x first and adds the residual after — that wrapping is the block, in 5.4.)

THE TASK

Implement `attention(out, x, w, cfg, pos0)`: linear projections to `q`, `k`, `v`; apply `rope` on `q` (`n_heads`) and `k` (`n_kv_heads`); then, for each head `h` (mapping to `kv_h = h / (n_heads/n_kv_heads)`) and each query `i`, compute scaled scores against keys `0..i`, a stable softmax over that range, and the weighted sum of values into the head's output; finally linear through `wo` into `out [seq, hidden]`.

HINTS

Hint 1 — indexing heads in a flat buffer

With `q` contiguous as `[seq, n_heads, head_dim]`, head `h` of token `i` is at `((int64_t)i*n_heads + h)*head_dim`; for `k/v` use `n_kv_heads` in place of `n_heads` and `kv_h` in place of `h`. Get these strides right and the rest is arithmetic.

Hint 2 — causal mask = restrict the loop

You do not need an explicit $-\infty$ buffer: simply loop keys `j` from `0` to `i` (inclusive) when scoring and when summing values. Positions `j > i` are never touched, which is exactly the mask.

Hint 3 — stable softmax over the live range

Reuse the Unit 2 recipe inline over `j = 0..i`: find the max score, exponentiate `score - max`, sum, then normalize. The range length varies per `i`, so an inline pass is cleaner here than calling the fixed-width `softmax_lastdim`.

Hint 4 — GQA mapping

`group = n_heads / n_kv_heads`; `kv_h = h / group`; . Several query heads will share each `kv_h`; that sharing is the whole point of GQA. With `n_kv_heads == n_heads`, `group == 1` and every head has its own keys/values (plain multi-head).

TESTS

```

// add to tests/test_model.cpp
#include <cstdlib>
#include <cmath>

static Tensor randt(std::initializer_list<int64_t> shape, uint32_t seed) {
    Tensor t = Tensor::empty(shape, DType::F32);
    uint32_t s = seed; float* p = t.data_ptr<float>();
    for (int64_t i = 0; i < t.numel(); ++i) {
        s = s * 1664525u + 1013904223u;
        p[i] = (s >> 9) * (1.0f / 8388608.0f) - 0.5f;    // ~ [-0.5, 0.5)
    }
    return t;
}

static LayerWeights attn_weights(const ModelConfig& c, uint32_t seed) {
    LayerWeights w;
    const int qd = c.n_heads * c.head_dim, kvd = c.n_kv_heads * c.head_dim;
    w.wq = randt({qd, c.hidden_size}, seed + 1);
    w.wk = randt({kvd, c.hidden_size}, seed + 2);
    w.wv = randt({kvd, c.hidden_size}, seed + 3);
    w.wo = randt({c.hidden_size, qd}, seed + 4);
    return w;
}

TEST(attention_output_shape) {
    ModelConfig c; c.hidden_size = 8; c.n_heads = 2; c.n_kv_heads = 1; c.head_dim
    ↪ = 4;
    const int S = 3;
    Tensor x = randt({S, c.hidden_size}, 10);
    Tensor o = Tensor::empty({S, c.hidden_size}, DType::F32);
    attention(o, x, attn_weights(c, 1), c, /*pos0=*/0);
    CHECK(o.shape()[0] == S && o.shape()[1] == c.hidden_size);
}

TEST(attention_is_causal) {
    ModelConfig c; c.hidden_size = 8; c.n_heads = 2; c.n_kv_heads = 1; c.head_dim
    ↪ = 4;
    const LayerWeights w = attn_weights(c, 7);
    const int S = 5, hidden = c.hidden_size;
    Tensor x = randt({S, hidden}, 3);

    Tensor o1 = Tensor::empty({S, hidden}, DType::F32);
    attention(o1, x, w, c, 0);

    Tensor x2 = randt({S, hidden}, 3);    // identical copy...
    for (int d = 0; d < hidden; ++d) x2.data_ptr<float>()[3 * hidden + d] += 1.0f;
    ↪ // ...perturb pos 3

```

```

Tensor o2 = Tensor::empty({S, hidden}, DType::F32);
attention(o2, x2, w, c, 0);

for (int i = 0; i < 3; ++i)                                // positions < 3
↳ unchanged
    for (int d = 0; d < hidden; ++d)
        CHECK_CLOSE(o1.at<float>({i, d}), o2.at<float>({i, d}), 1e-5);

bool changed = false;                                       // position 4 (attends
↳ to 3) changes
    for (int d = 0; d < hidden; ++d)
        if (std::fabs(o1.at<float>({4, d}) - o2.at<float>({4, d})) > 1e-4f)
↳ changed = true;
        CHECK(changed);
}

```

REFERENCE SOLUTION

Declare in `model.hpp`, define in `model.cpp`:

```

// model.hpp
void attention(Tensor& out, const Tensor& x, const LayerWeights& w,
               const ModelConfig& cfg, int pos0 = 0);

```

```

// model.cpp
#include <cmath>
#include <vector>

void attention(Tensor& out, const Tensor& x, const LayerWeights& w,
               const ModelConfig& cfg, int pos0) {
    const int S = static_cast<int>(x.shape()[0]);
    const int H = cfg.n_heads, KVH = cfg.n_kv_heads, D = cfg.head_dim;
    const int qd = H * D, kvd = KVH * D, group = H / KVH;
    const float scale = 1.0f / std::sqrt(static_cast<float>(D));

    Tensor q = Tensor::empty({S, qd}, DType::F32);
    Tensor k = Tensor::empty({S, kvd}, DType::F32);
    Tensor v = Tensor::empty({S, kvd}, DType::F32);
    linear(q, x, w.wq); linear(k, x, w.wk); linear(v, x, w.wv);

    apply_rope(q, S, H, D, cfg.rope_theta, pos0);           // queries
    apply_rope(k, S, KVH, D, cfg.rope_theta, pos0);         // keys (values are NOT

```

```

    ↪ rotated)

    const float* Q = q.data_ptr<float>();
    const float* K = k.data_ptr<float>();
    const float* V = v.data_ptr<float>();

    Tensor concat = Tensor::empty({S, qd}, DType::F32);    // [seq, n_heads*
    ↪ head_dim]
    float* O = concat.data_ptr<float>();
    std::vector<float> scores(S);

    for (int h = 0; h < H; ++h) {
        const int kvh = h / group;
        for (int i = 0; i < S; ++i) {
            const float* qi = Q + ((int64_t)i * H + h) * D;
            float m = -INFINITY;    // scores over keys 0..
            ↪ i
            for (int j = 0; j <= i; ++j) {
                const float* kj = K + ((int64_t)j * KVH + kvh) * D;
                float dot = 0.0f;
                for (int d = 0; d < D; ++d) dot += qi[d] * kj[d];
                dot *= scale;
                scores[j] = dot;
                if (dot > m) m = dot;
            }
            float sum = 0.0f;    // stable softmax over
            ↪ 0..i
            for (int j = 0; j <= i; ++j) { float e = std::exp(scores[j] - m);
            ↪ scores[j] = e; sum += e; }
            const float inv = 1.0f / sum;

            float* oi = O + ((int64_t)i * H + h) * D;    // weighted sum of
            ↪ values
            for (int d = 0; d < D; ++d) oi[d] = 0.0f;
            for (int j = 0; j <= i; ++j) {
                const float wij = scores[j] * inv;
                const float* vj = V + ((int64_t)j * KVH + kvh) * D;
                for (int d = 0; d < D; ++d) oi[d] += wij * vj[d];
            }
        }
        linear(out, concat, w.wo);    // project heads back
        ↪ to hidden
    }
}

```

This is the conceptual center of the whole architecture, and the causality test is

the one that matters: perturbing token 3 changes the outputs at positions 3 and 4 but leaves 0, 1, 2 *bit-identical*, proving the mask actually prevents the past from seeing the future. Note how much you reused — `linear` (Unit 3), `apply_rope` (5.2), the stable-softmax recipe (Unit 2) — which is the point: attention is an assembly of parts you already trust. (It is also $O(\text{seq}^2)$ in time and the obvious target for Flash Attention in Unit 8; the clear version comes first.)

GOING FURTHER

- **Batch the score matrix.** Replace the per-position inner loops with a $[\text{seq}, \text{seq}]$ score matrix computed by GEMM (Unit 3), masked and softmaxed row-wise, then multiplied by V — much faster, and the structure Flash Attention then fuses.
- **Attention sinks / sliding windows.** Some models restrict attention to a recent window or always keep the first token. Add a configurable mask to support them.
- **Numerical check vs a reference.** For a tiny config, compute attention in float64 (or NumPy) and assert your float32 output matches within tolerance — a golden test alongside the causal property.

5.4 The Transformer Block and Feed-Forward Network

GOAL

Assemble a complete **Transformer block**: pre-normalize, run attention, add the residual; pre-normalize again, run the **SwiGLU feed-forward network**, add the residual. Stacking these blocks is the body of the model.

BACKGROUND

A Llama-style layer is two sub-layers, each wrapped identically: **normalize, transform, add back**. Written out, with x the residual stream:

$$h = x + \text{Attention}(\text{RMSNorm}(x)), \quad y = h + \text{FFN}(\text{RMSNorm}(h)).$$

Two design choices in that formula are worth naming. First, **pre-normalization**: the RMSNorm sits *before* each sub-layer, not after. Pre-norm keeps the residual

path itself un-normalized — a clean highway that information travels with minimal interference — while each sub-layer reads a normalized copy. This is what makes very deep Transformers trainable, and it is universal in modern models. Second, the **residual add**: each sub-layer produces a *delta* that is added to the stream rather than replacing it, so the layer refines rather than overwrites — the accumulation principle from 5.1, applied twice per layer.

The attention sub-layer you already built (5.3). The other is the **feed-forward network**, where most of a model’s parameters — and, the evidence suggests, most of its stored knowledge — reside. In Llama-family models it is the gated **SwiGLU** FFN from Unit 2:

$$\text{FFN}(y) = W_{\text{down}} \left(\text{SiLU} (y W_{\text{gate}}^{\top}) \odot (y W_{\text{up}}^{\top}) \right).$$

Read it left to right: two separate linear projections lift y from `hidden_size` up to the wider `intermediate_size` — a *gate* and an *up* — the gate is passed through SiLU and multiplied elementwise into the up (this is the gating that makes FFNs expressive), and a third projection brings the result back down to `hidden_size`. The intermediate width is typically several times the hidden size (Llama-3.2-1B: 8192 versus 2048), which is why the FFN dominates the parameter count. Every piece is already in hand: three linear calls (Unit 3) around one `swiglu` (Unit 2).

A full model is just `n_layers` of these blocks applied in sequence to the residual stream, each with its own weights. Because each block reads and writes the same `[seq, hidden]` tensor in place, stacking them is a plain loop — which is the last piece we wire up in the boss.

THE TASK

Implement `ffn(out, x, w, cfg)`: linear to gate and up (`[seq, intermediate]`), `swiglu` them, linear through `w_down` into `out [seq, hidden]`. Then `transformer_block(x, w, cfg, pos0)` that updates `x` in place: `rmsnorm` with `attn_norm` → attention → `add_inplace`; `rmsnorm` with `ffn_norm` → `ffn` → `add_inplace`.

HINTS

Hint 1 — the FFN is three linears and a gate

```
linear(gate, x, w.w_gate); linear(up, x, w.w_up); swiglu(act, gate,
up); linear(out, act, w.w_down);. gate and up are [seq, intermediate];
```

act too; out is [seq, hidden].

Hint 2 — normalize a copy, add to the original

`transformer_block` mutates `x` (the stream). Norm into a *separate* buffer, run the sub-layer on that buffer, then `add_inplace(x, sublayer_output)`. Never normalize `x` in place — the residual path must stay un-normalized.

Hint 3 — reuse one scratch buffer

The same [seq, hidden] normed buffer serves both sub-layers, and the same applies to the FFN's intermediates. From an Arena (Unit 1), this is a handful of bump allocations reset after the forward pass.

TESTS

```
// add to tests/test_model.cpp (reuses randt, attn_weights)

static LayerWeights full_weights(const ModelConfig& c, uint32_t seed) {
    LayerWeights w = attn_weights(c, seed);
    w.attn_norm = randt({c.hidden_size}, seed + 10);
    w.ffn_norm = randt({c.hidden_size}, seed + 11);
    w.w_gate = randt({c.intermediate_size, c.hidden_size}, seed + 12);
    w.w_up = randt({c.intermediate_size, c.hidden_size}, seed + 13);
    w.w_down = randt({c.hidden_size, c.intermediate_size}, seed + 14);
    return w;
}

TEST(block_preserves_shape) {
    ModelConfig c; c.hidden_size = 8; c.n_heads = 2; c.n_kv_heads = 1;
    c.head_dim = 4; c.intermediate_size = 16;
    const int S = 4;
    Tensor x = randt({S, c.hidden_size}, 21);
    transformer_block(x, full_weights(c, 1), c, /*pos0=*/0);
    CHECK(x.shape()[0] == S && x.shape()[1] == c.hidden_size);
}

TEST(block_is_identity_when_outputs_zeroed) {
    ModelConfig c; c.hidden_size = 8; c.n_heads = 2; c.n_kv_heads = 2;
    c.head_dim = 4; c.intermediate_size = 16;
    LayerWeights w = full_weights(c, 5);
    // Zero the two output projections -> each sub-layer contributes exactly 0.
```



```

    for (int i = 0; i < w.wo.numel(); ++i) w.wo.data_ptr<float>()[i] = 0.0
    ↪ f;
    for (int i = 0; i < w.w_down.numel(); ++i) w.w_down.data_ptr<float>()[i] = 0.0
    ↪ f;

    const int S = 4, hidden = c.hidden_size;
    Tensor x = randt({S, hidden}, 9);
    Tensor x0 = Tensor::empty({S, hidden}, DType::F32);
    std::memcpy(x0.data_ptr<float>(), x.data_ptr<float>(), sizeof(float) * S *
    ↪ hidden);

    transformer_block(x, w, c, 0);
    for (int i = 0; i < S * hidden; ++i)
        CHECK_CLOSE(x.data_ptr<float>()[i], x0.data_ptr<float>()[i], 1e-5); //
    ↪ residual identity
}

```

REFERENCE SOLUTION

Declare in `model.hpp`, define in `model.cpp`:

```

// model.hpp
void ffn(Tensor& out, const Tensor& x, const LayerWeights& w, const ModelConfig&
    ↪ cfg);
void transformer_block(Tensor& x, const LayerWeights& w, const ModelConfig& cfg,
    ↪ int pos0 = 0);

```

```

// model.cpp
void ffn(Tensor& out, const Tensor& x, const LayerWeights& w, const ModelConfig&
    ↪ cfg) {
    const int S = static_cast<int>(x.shape()[0]);
    const int I = cfg.intermediate_size;
    Tensor gate = Tensor::empty({S, I}, DType::F32);
    Tensor up = Tensor::empty({S, I}, DType::F32);
    linear(gate, x, w.w_gate);
    linear(up, x, w.w_up);
    Tensor act = Tensor::empty({S, I}, DType::F32);
    swiglu(act, gate, up); // SiLU(gate) * up (Unit 2)
    linear(out, act, w.w_down); // back to [seq, hidden]
}

void transformer_block(Tensor& x, const LayerWeights& w,

```

```

        const ModelConfig& cfg, int pos0) {
    const int S = static_cast<int>(x.shape()[0]);
    const int hidden = cfg.hidden_size;
    Tensor normed = Tensor::empty({S, hidden}, DType::F32);
    Tensor delta = Tensor::empty({S, hidden}, DType::F32);

    // attention sub-layer: x += Attention(RMSNorm(x))
    rmsnorm(normed, x, w.attn_norm, cfg.rms_norm_eps);
    attention(delta, normed, w, cfg, pos0);
    add_inplace(x, delta);

    // feed-forward sub-layer: x += FFN(RMSNorm(x))
    rmsnorm(normed, x, w.ffn_norm, cfg.rms_norm_eps);
    ffn(delta, normed, w, cfg);
    add_inplace(x, delta);
}

```

A block is almost entirely composition: two norms, an attention, an FFN, two adds, every one of them a function you built and tested earlier. That is the reward for the bottom-up discipline — the most important structure in the model reads as a short, obvious sequence of trusted calls. The identity-when-zeroed test confirms the residual wiring: with both sub-layer outputs forced to zero, the stream passes through untouched, which means the additions are genuinely *residual* and not accidentally overwriting state. One loop over these blocks, a final norm, and a projection to logits, and the model is complete — the boss.

GOING FURTHER

- **Fuse the FFN gate.** Compute gate and up in one matmul against a stacked $[2 \times \text{intermediate}, \text{hidden}]$ weight, then split — one GEMM call instead of two, better cache use. Many runtimes pack the weights this way.
- **In-place norm scratch.** Confirm a single normed/delta pair (or arena markers) suffices across both sub-layers, so a block adds no per-call heap allocation.
- **Post-norm / other blocks.** Implement the original post-norm ordering and compare stability intuition; some non-Llama models differ here, and the loader will tell you which.

5.5 The Full Forward Pass

BOSS CHALLENGE

Wire embeddings, the stacked blocks, the final norm, and the output projection into one `forward()` that turns token IDs into logits — and prove it is a correct *causal* language model: perturbing the token at position t changes the logits at positions $\geq t$ but leaves every position $< t$ **bit-identical**, with finite, deterministic output throughout.

GOAL

Complete the model: `embed` $\rightarrow$ `n_layers transformer blocks` $\rightarrow$ `final rmsnorm` $\rightarrow$ `lm_head projection` $\rightarrow$ `logits [seq, vocab]`. Then verify the whole pipeline is genuinely causal, numerically sound, and deterministic — the end-to-end correctness certificate for everything in this unit.

BACKGROUND

The forward pass is the assembly of the entire unit, and it is gratifyingly short: embed the token IDs into the residual stream, run the stream through every layer in turn, apply the final RMSNorm, and project to vocabulary logits with the `lm_head`. The output `logits[i]` is the model's scored prediction for the token following position i , given tokens $0..i$ — run a sampler over `logits[seq-1]` and you have the next token, which (Unit 11) is generation.

The right way to certify all of this at once is to test the property that *defines* a decoder language model: **causality**. The prediction at each position must depend only on that position and the ones before it; the future must not leak backward. This is not a small claim — it threads through every component. If the attention mask is off by one, causality breaks. If a stride is wrong and a head reads the next token's key, causality breaks. If RoPE is misapplied, or the residual stream is mis-indexed, or the `lm_head` mixes positions, causality breaks. So a single experiment exercises the embedding, every layer's masked attention, the residual wiring, the norms, and the projection simultaneously:

Take two token sequences identical except at one position t . Run both forward. The logits at every position *before* t must be **bit-identical** (within floating-point tolerance), because nothing before t can see to-

ken t . The logits at position t *must* differ (it now sees a different token). Positions after t may differ too.

If that holds, the model is causal, and — because causality is so sensitive to wiring mistakes — the whole forward pass is almost certainly correct. We pair it with two cheaper checks: **finiteness** (no NaN/Inf survives the dozens of matmuls, softmaxes, and norms a multi-layer pass performs) and **determinism** (the same input yields bit-identical output, since a forward pass is pure). Together these are the boss: not “the model runs,” but “the model is a correct, causal, numerically-sound function from tokens to logits.”

THE TASK

Complete `forward(model, cfg, ids)`: embed_tokens into `x [seq, hidden]`; loop `transformer_block` over all `n_layers`; `rmsnorm` with `final_norm`; `linear` with `lm_head` into logits `[seq, vocab]`; return them. Then write the boss tests against a tiny random model: (1) shape `[seq, vocab]` and all-finite; (2) the causality experiment above; (3) determinism (two identical runs are bit-identical).

The bar (the boss): perturbing token t leaves all logits at positions $< t$ bit-identical and changes the logits at position t ; the output is finite and shaped `[seq, vocab]`; repeated runs match exactly.

HINTS

Hint 1 — forward is a short assembly

```
embed_tokens(x, ids, model.token_embedding);      then      for (l)
transformer_block(x, model.layers[l], cfg, 0); then rmsnorm(normed,
x, model.final_norm, eps);      then      linear(logits, normed,
model.lm_head);. Each token's position is its index, so pos0 = 0 for a
fresh sequence.
```

Hint 2 — build the two sequences to differ at exactly one spot

Copy the ID vector and change a single element at position t . Everything hinges on the two inputs being identical everywhere else, so the only thing that can move the earlier logits is a causality bug.

Hint 3 — if causality fails

The earlier-position logits moving means future information is leaking backward. Suspect, in order: the attention key/value loop bound (must be $j \leq i$), the head strides, then RoPE's position offset. The position where the divergence *starts* often points at the culprit.

TESTS

```
// add to tests/test_model.cpp (reuses randt, full_weights)
#include <cmath>

static ModelWeights tiny_model(const ModelConfig& c, uint32_t seed) {
    ModelWeights m;
    m.token_embedding = randt({c.vocab_size, c.hidden_size}, seed);
    m.final_norm      = randt({c.hidden_size}, seed + 100);
    m.lm_head         = randt({c.vocab_size, c.hidden_size}, seed + 200);
    for (int l = 0; l < c.n_layers; ++l)
        m.layers.push_back(full_weights(c, seed + 1000u * (l + 1)));
    return m;
}

static ModelConfig tiny_cfg() {
    ModelConfig c; c.vocab_size = 32; c.hidden_size = 8; c.n_layers = 2;
    c.n_heads = 2; c.n_kv_heads = 1; c.head_dim = 4; c.intermediate_size = 16;
    return c;
}

TEST(boss_forward_shape_and_finite) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 1);
    const std::vector<int> ids = {3, 7, 1, 9, 2};
    Tensor logits = forward(m, c, ids);
    REQUIRE(logits.shape()[0] == static_cast<int>(ids.size()));
    REQUIRE(logits.shape()[1] == c.vocab_size);
    for (int i = 0; i < logits.numel(); ++i)
        CHECK(std::isfinite(logits.data_ptr<float>()[i]));
}

TEST(boss_forward_is_causal_end_to_end) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    const std::vector<int> a = {5, 2, 8, 1, 6};
    const std::vector<int> b = {5, 2, 8, 4, 6};    // differs ONLY at position
    ↪ 3
}
```

```

Tensor L1 = forward(m, c, a);
Tensor L2 = forward(m, c, b);
const int V = c.vocab_size;

for (int i = 0; i < 3; ++i)                                // positions < 3: bit-
↳ identical
    for (int t = 0; t < V; ++t)
        CHECK_CLOSE(L1.at<float>({i, t}), L2.at<float>({i, t}), 1e-4);

bool changed = false;                                       // position 3: must differ
for (int t = 0; t < V; ++t)
    if (std::fabs(L1.at<float>({3, t}) - L2.at<float>({3, t})) > 1e-4f)
↳ changed = true;
    CHECK(changed);
}

TEST(boss_forward_is_deterministic) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 2);
    const std::vector<int> ids = {1, 2, 3};
    Tensor x = forward(m, c, ids);
    Tensor y = forward(m, c, ids);
    for (int i = 0; i < x.numel(); ++i)
        CHECK_CLOSE(x.data_ptr<float>()[i], y.data_ptr<float>()[i], 0.0f); //
↳ exact
}

```

REFERENCE SOLUTION

Define forward in model.cpp (declared in 5.1):

```

// model.cpp
Tensor forward(const ModelWeights& model, const ModelConfig& cfg,
    const std::vector<int>& ids) {
    const int S = static_cast<int>(ids.size());
    const int hidden = cfg.hidden_size;

    Tensor x = Tensor::empty({S, hidden}, DType::F32);
    embed_tokens(x, ids, model.token_embedding);           // residual stream,
↳ position 0..S-1

    for (int l = 0; l < cfg.n_layers; ++l)
        transformer_block(x, model.layers[l], cfg, /*pos0=*/0);
}

```

```

Tensor normed = Tensor::empty({S, hidden}, DType::F32);
rmsnorm(normed, x, model.final_norm, cfg.rms_norm_eps);

Tensor logits = Tensor::empty({S, cfg.vocab_size}, DType::F32);
linear(logits, normed, model.lm_head);           // [seq, vocab]
return logits;
}

```

The boss is cleared by this function and everything beneath it behaving correctly under the causal test — there is no extra production code, only the proof. Run it: forward returns finite [seq, vocab] logits, perturbing token 3 leaves positions 0, 1, 2 untouched while changing position 3, and repeated runs agree to the bit.

What you have proven. Not “a Transformer that runs,” but “a Transformer that is a *correct causal language model*”: information never flows backward in time, the numerics survive a full multi-layer pass without blowing up, and the mapping from tokens to logits is deterministic. That is the certificate that the embedding, RoPE, masked grouped-query attention, the SwiGLU blocks, the residual stream, and the output projection are all wired correctly together — the hardest thing in the book to get right, now verified by a single sharp property.

GOING FURTHER

- **Golden logits.** Reimplement the tiny model’s forward pass in float64 (or NumPy) and assert your float32 logits match within tolerance — an absolute-correctness check on top of the structural one.
- **Only the last position.** During generation you need logits for just the final token. Add a path that skips computing `lm_head` for positions you will not sample, saving a large matmul against the vocabulary.
- **Arena-allocate the pass.** Route every activation through an Arena and confirm, with the `Storage::alloc_count()` meter from Unit 1, that a warm forward pass performs zero heap allocations.

Unit 5 — Milestone check

MILESTONE CHECK

The engine now contains a working model — a real, causal Transformer:

- **token embeddings** that seed the residual stream, and an `lm_head` that reads it out as vocabulary logits;
- **RoPE**, giving the model relative-position awareness through pure rotation;
- **grouped-query attention** with a causal mask — the mechanism by which tokens attend to their past;
- the **RMSNorm + SwiGLU block** with pre-normalization and residual connections, stacked `n_layers` deep;
- and a **boss-grade certificate** that the full forward pass is causal end-to-end, finite, and deterministic.

This is the architectural heart of the engine, and it works. What remains is to make it *practical and fast*, and to feed it a *real* model. **Unit 6** (★) generalizes the feed-forward block into a **Mixture of Experts** — many FFNS plus a router that sends each token to only a few — the architecture behind Mixtral, DeepSeek, and the largest open models, and a natural extension now that the dense block is in hand. After that, Unit 7 introduces the **kv cache** so generation does not recompute the whole sequence at every step, and Unit 8 (★) fuses attention into **Flash Attention**.

UNIT 6

Mixture of Experts

What you will build. The architecture that lets models grow enormous without growing per-token cost: a **Mixture of Experts** (MOE) feed-forward block. You build the **router** (a gating network that scores experts and picks the top few per token), the **expert FFNs** (each a SwiGLU block from Unit 5), and the **weighted combine** — then an **efficient grouped dispatch**. The boss proves the routing is genuinely *sparse*: only the chosen experts ever touch a token.

Where this fits. This is a drop-in generalization of the dense feed-forward block you just built. The dense FFN runs the same weights on every token; MOE keeps many FFNs and routes each token to a handful, so a model can hold hundreds of billions of parameters while *activating* only a small fraction per token. It is the design behind Mixtral, DeepSeek-MoE, Qwen-MoE, and the largest open models — and because we have the dense block already, adding it now is the natural next step.

The feed-forward network is where most of a Transformer’s parameters live, which makes it the natural place to add capacity. But adding capacity densely — a wider FFN — makes *every* token more expensive. Mixture of Experts breaks that coupling with a simple, powerful idea: keep **many** feed-forward networks (the *experts*), and for each token use a small **router** to pick only the few most relevant ones. A token’s output is the weighted sum of just its chosen experts. The model’s total parameter count is the sum of all experts — potentially huge — but the **compute per token** is fixed at the top- k experts it actually visits. Capacity scales with the number of experts; cost scales with k . That decoupling is why the frontier of open models is increasingly sparse.

The intuition for *why* it learns well is specialization: different experts come to handle different kinds of token — code versus prose, one language versus another,

syntax versus facts — and the router learns to send each token where it will be best processed. We build it in three steps: the router that makes the choice, the experts and the combine that act on it, and an efficient dispatch that exploits the sparsity — with a boss that proves the sparsity is real. Add an `MOE` config and weights to `model.hpp`; the experts reuse the `SwiGLU_FFN` machinery from Unit 5.

6.1 The Router

GOAL

The gating network that decides where each token goes: a linear projection from the hidden vector to one score per expert, **top- k** selection of the highest-scoring experts, and a **softmax over those k** to produce normalized combination weights.

BACKGROUND

The router is a small but consequential piece. For each token it produces a score for every expert, then commits to the best few. Concretely, a router weight matrix W_r of shape $[\text{num_experts} \times \text{hidden}]$ gives per-expert logits

$$\text{gate_logits} = x W_r^\top \in \mathbb{R}^{\text{num_experts}},$$

one score per expert for this token. From these we take the **top- k** : the indices of the k largest logits (k is small — often 1 or 2; Mixtral uses 2 of 8). Then we apply a **softmax over just those k selected logits** to get weights that sum to 1:

$$w_j = \frac{e^{\ell_{e_j}}}{\sum_{j'=1}^k e^{\ell_{e_{j'}}}}, \quad j = 1, \dots, k,$$

where $e_1, \dots, e_k$ are the chosen expert indices and ℓ the gate logits. The token's `FFN` output will be $\sum_j w_j \cdot \text{expert}_{e_j}(x)$ — a convex combination of its top experts. Each token thus carries away k (expert, weight) pairs.

A few points of nuance. The softmax is taken over the *selected* logits, so the weights of the experts a token actually uses always sum to one regardless of how the unselected experts scored. (A common variant softmaxes over *all* experts first and then renormalizes the top- k ; the two differ slightly and the choice is a model-metadata detail, like so many others, resolved at load time. We implement top- k -then-softmax, the Mixtral convention.) The router is **tiny** relative to the experts —

`num_experts` is at most a few hundred and the matrix is $[\text{num_experts} \times \text{hidden}]$, dwarfed by even one expert's FFN — so the gating cost is negligible; the experts are where the compute goes.

This top- k choice is also where the **sparsity** that defines MOE originates: by committing to k experts and ignoring the rest, the router guarantees that only k FFNs run per token. The boss will hold us to that guarantee; here we make the choice correctly.

THE TASK

Add to `model.hpp`: `num_experts` and `top_k` fields (extend `ModelConfig` or a small `MoEConfig`), an `ExpertWeights` struct (`w_gate`, `w_up`, `w_down`, as in Unit 5's FFN), and an `MoELayerWeights` struct (a router tensor $[\text{num_experts}, \text{hidden}]$ and a vector<`ExpertWeights`>). Implement `route(x, router_w, num_experts, top_k)` returning, per token, the top- k expert indices and their softmax-normalized weights (e.g. in a struct with flat $[\text{seq} \times \text{top_k}]$ arrays).

HINTS

Hint 1 — logits then selection

Compute `gate_logits = linear(x, router_w)` into $[\text{seq}, \text{num_experts}]$. For each token, select top- k by repeatedly taking the current max logit, recording its index, and masking it to $-\infty$ — k passes over `num_experts`. With small k and `num_experts` this is plenty fast.

Hint 2 — softmax over the k chosen logits

After collecting the k selected logits, run the stable softmax over *those k values only* (subtract their max, exponentiate, normalize). The resulting weights sum to 1 and pair with the k expert indices.

Hint 3 — a flat routing result

Return two flat arrays of length `seq*top_k`: `experts[i*top_k + j]` and `weights[i*top_k + j]` for token i 's j -th choice. This layout makes the dispatch in 6.2–6.3 a simple loop.

TESTS

```
// add to tests/test_moe.cpp
#include "test_framework.hpp"
#include "llm/model.hpp"
#include <cmath>
using namespace llm;

TEST(router_selects_top_k) {
    const int E = 4, hidden = 1, K = 2;
    // router_w is [E, hidden]; with hidden=1 the logit for expert e is x[0]*
    ↪ router_w[e].
    Tensor rw = Tensor::empty({E, hidden}, DType::F32);
    float w[E] = {0.1f, 0.9f, 0.4f, 0.2f}; // expert 1 highest, then 2
    for (int e = 0; e < E; ++e) rw.data_ptr<float>()[e] = w[e];
    Tensor x = Tensor::empty({1, hidden}, DType::F32);
    x.data_ptr<float>()[0] = 1.0f; // logits == w
    Routing r = route(x, rw, E, K);
    REQUIRE((int)r.experts.size() == K);
    CHECK(r.experts[0] == 1); // top is expert 1
    CHECK(r.experts[1] == 2); // then expert 2
}

TEST(router_weights_sum_to_one) {
    const int E = 8, hidden = 4, K = 2;
    Tensor rw = Tensor::empty({E, hidden}, DType::F32);
    uint32_t s = 5; for (int i = 0; i < E * hidden; ++i) { s = s*1664525u
    ↪ +1013904223u; rw.data_ptr<float>()[i] = (s>>9)*(1.0f/8388608.0f)-0.5f; }
    Tensor x = Tensor::empty({1, hidden}, DType::F32);
    for (int i = 0; i < hidden; ++i) x.data_ptr<float>()[i] = 0.3f * i - 0.2f;
    Routing r = route(x, rw, E, K);
    float sum = 0.0f; for (int j = 0; j < K; ++j) sum += r.weights[j];
    CHECK_CLOSE(sum, 1.0f, 1e-6); // softmax over the chosen k
}
```

REFERENCE SOLUTION

Add to model.hpp:

```
// model.hpp (MoE additions)
struct ExpertWeights {
    Tensor w_gate, w_up; // [intermediate, hidden]
    Tensor w_down; // [hidden, intermediate]
```

```

};
struct MoELayerWeights {
    Tensor router;                // [num_experts, hidden]
    std::vector<ExpertWeights> experts;
};

struct Routing {                // per token, flattened [seq*top_k]
    std::vector<int> experts;
    std::vector<float> weights;
};

Routing route(const Tensor& x, const Tensor& router_w, int num_experts, int top_k);
    ↪

```

Add to model.cpp:

```

// model.cpp
#include <cmath>
#include <limits>

Routing route(const Tensor& x, const Tensor& router_w, int num_experts, int top_k)
    ↪ {
    const int S = static_cast<int>(x.shape()[0]);
    Tensor logits = Tensor::empty({S, num_experts}, DType::F32);
    linear(logits, x, router_w);                // [seq, num_experts]

    Routing r;
    r.experts.resize((size_t)S * top_k);
    r.weights.resize((size_t)S * top_k);

    std::vector<float> row(num_experts);
    for (int i = 0; i < S; ++i) {
        const float* L = logits.data_ptr<float>() + (int64_t)i * num_experts;
        for (int e = 0; e < num_experts; ++e) row[e] = L[e];

        // top-k by repeated max-and-mask
        float chosen_logit[64];                // top_k is small
        for (int j = 0; j < top_k; ++j) {
            int best = 0; float bestv = -std::numeric_limits<float>::infinity();
            for (int e = 0; e < num_experts; ++e)
                if (row[e] > bestv) { bestv = row[e]; best = e; }
            r.experts[(size_t)i * top_k + j] = best;
            chosen_logit[j] = bestv;
            row[best] = -std::numeric_limits<float>::infinity(); // mask it out
        }
    }
}

```

```

// softmax over the k chosen logits
float m = chosen_logit[0];
for (int j = 1; j < top_k; ++j) m = std::max(m, chosen_logit[j]);
float sum = 0.0f;
for (int j = 0; j < top_k; ++j) { float e = std::exp(chosen_logit[j] - m);
    ↪ chosen_logit[j] = e; sum += e; }
for (int j = 0; j < top_k; ++j)
    r.weights[(size_t)i * top_k + j] = chosen_logit[j] / sum;
}
return r;
}

```

The router is the brain of the moE layer: a cheap linear map and a top- k pick that together decide, per token, which slice of the model’s vast parameter set actually fires. Its output — k experts and k weights for every token — is the routing plan the rest of the layer executes. Note that the experts it does *not* select cost nothing, which is the entire economic argument for moE and the property the boss will verify.

GOING FURTHER

- **Softmax-then-top- k .** Implement the alternative convention (softmax over all experts, then keep and renormalize the top- k) behind a flag, to match models that use it.
- **Load-balancing telemetry.** Count how many tokens each expert receives across a batch. In training a loss term keeps this balanced; at inference, lopsided counts explain throughput variance and motivate the grouped dispatch of 6.3.
- **Noisy / temperature routing.** Some routers add noise or a temperature to the gate logits. Not needed for inference, but instructive for understanding how the router was trained.

6.2 The Mixture-of-Experts FFN

GOAL

Act on the router’s plan: run each token’s top- k **expert FFNs** (each a SwiGLU block) and **combine** their outputs, weighted by the gate weights, into the token’s result. This is the complete MOE feed-forward layer — correct first, fast in 6.3.

BACKGROUND

With routing decided, the MOE layer’s computation is straightforward. An **expert** is nothing exotic: it is exactly the SwiGLU feed-forward network from Unit 5 — a gate projection, an up projection, the SiLU-gated product, and a down projection — just with its own private weights. The layer holds `num_experts` of them. For a given token, the router has named k experts $e_1, \dots, e_k$ and weights $w_1, \dots, w_k$; the token’s output is the weighted sum

$$y = \sum_{j=1}^k w_j \cdot \text{Expert}_{e_j}(x).$$

A token that the router sends strongly to one expert gives that expert most of the weight; a token split between two gets a blend. Because the weights sum to one, the combine is a convex mixture of the chosen experts’ outputs — the layer’s name made literal.

The **naive** way to compute this, and the one we write first, is a loop over tokens: for each token, look up its k (expert, weight) pairs, evaluate those k expert FFNs on the token’s vector, and accumulate the weighted results. It is obviously correct and a fine reference. It is also inefficient — it processes one token at a time, reloading an expert’s weights for every token that happens to use it, and it cannot use the batched, cache-friendly matmuls of Unit 3 because each token’s set of experts differs. That inefficiency is precisely what the grouped dispatch in 6.3 repairs; here we establish the correct behavior the fast version must reproduce.

Two architectural notes. First, the experts are *independent* — there is no interaction between them within a layer, which is exactly why the computation parallelizes and why grouping by expert (6.3) is sound. Second, some models (notably the DeepSeek family) add a **shared expert** that is applied to *every* token in addition to the routed ones, capturing common computation that would otherwise be duplicated across experts; it is a small addition — one always-on FFN added to the routed sum — that we leave to Going Further. The pure routed form below is the core that every MOE variant builds on.

THE TASK

Implement a single-token expert `ffn` helper (SwiGLU: gate and up projections, SiLU-gated product, down projection) and `moe_ffn(out, x, w, cfg)`: call `route`, then for each token loop its `top-k` (expert, weight) pairs, evaluate that expert on the token, and accumulate `weight * expert_output` into the token's row of `out`. (Add `num_experts` and `top_k` to `ModelConfig`.)

HINTS

Hint 1 — the expert is just SwiGLU

For one token vector `x` and expert `e`: `gate[o] = dot(x, w_gate row o)`, `up[o] = dot(x, w_up row o)` for `o` in `[0, intermediate)`; fuse `act[o] = siluf(gate[o]) * up[o]` (the `siluf` from Unit 2); then `out[h] = dot(act, w_down row h)` for `h` in `[0, hidden)`.

Hint 2 — accumulate the weighted mixture

Zero the token's output row, then for each of its `top_k` slots: evaluate the expert into a scratch `[hidden]` buffer and add `weight * scratch` into the row. After `top_k` slots the row holds the convex combination.

Hint 3 — index the routing arrays

Token `i`'s `j`-th choice is `r.experts[i*top_k + j]` with weight `r.weights[i*top_k + j]` — the flat layout from 6.1. The expert index selects which `w.experts[e]` to run.

TESTS

```
// add to tests/test_moe.cpp
#include <cstdlib>

static Tensor randt(std::initializer_list<int64_t> shape, uint32_t seed) {
    Tensor t = Tensor::empty(shape, DType::F32);
    uint32_t s = seed; float* p = t.data_ptr<float>();
    for (int64_t i = 0; i < t.numel(); ++i) { s = s*1664525u+1013904223u; p[i] = (
        ↪ s>>9)*(1.0f/8388608.0f)-0.5f; }
    return t;
}
```



```

}

TEST(moe_output_shape) {
    ModelConfig c; c.hidden_size = 8; c.intermediate_size = 16; c.num_experts = 4;
    ↪ c.top_k = 2;
    MoELayerWeights w;
    w.router = randt({c.num_experts, c.hidden_size}, 1);
    for (int e = 0; e < c.num_experts; ++e)
        w.experts.push_back({ randt({c.intermediate_size, c.hidden_size}, 10+e),
                               randt({c.intermediate_size, c.hidden_size}, 20+e),
                               randt({c.hidden_size, c.intermediate_size}, 30+e) });
    ↪
    const int S = 5;
    Tensor x = randt({S, c.hidden_size}, 7);
    Tensor o = Tensor::empty({S, c.hidden_size}, DType::F32);
    moe_ffn(o, x, w, c);
    CHECK(o.shape()[0] == S && o.shape()[1] == c.hidden_size);
}

TEST(moe_one_expert_equals_dense_ffn) {
    ModelConfig c; c.hidden_size = 8; c.intermediate_size = 16; c.num_experts = 1;
    ↪ c.top_k = 1;
    ExpertWeights e{ randt({c.intermediate_size, c.hidden_size}, 2),
                    randt({c.intermediate_size, c.hidden_size}, 3),
                    randt({c.hidden_size, c.intermediate_size}, 4) };
    MoELayerWeights mw; mw.router = randt({1, c.hidden_size}, 1); mw.experts.
    ↪ push_back(e);
    LayerWeights dw; dw.w_gate = e.w_gate; dw.w_up = e.w_up; dw.w_down = e.w_down;

    const int S = 3;
    Tensor x = randt({S, c.hidden_size}, 9);
    Tensor o_moe = Tensor::empty({S, c.hidden_size}, DType::F32);
    Tensor o_dense = Tensor::empty({S, c.hidden_size}, DType::F32);
    moe_ffn(o_moe, x, mw, c);           // 1 expert, weight softmax[l] = 1
    ffn(o_dense, x, dw, c);             // dense SwiGLU FFN from Unit 5
    for (int i = 0; i < S * c.hidden_size; ++i)
        CHECK_CLOSE(o_moe.data_ptr<float>()[i], o_dense.data_ptr<float>()[i], 1e-4)
    ↪ ;
}

```

REFERENCE SOLUTION

Add to `model.hpp`: `int num_experts = 0; int top_k = 0;` in `ModelConfig`, and

```
// model.hpp
void moe_ffn(Tensor& out, const Tensor& x, const MoELayerWeights& w, const
    ↪ ModelConfig& cfg);
```

Add to model.cpp:

```
// model.cpp
#include "llm/ops.hpp"      // siluf
#include <vector>

// One expert (SwiGLU) on a single token vector x[hidden] -> out[hidden].
static void expert_ffn_token(float* out, const float* x, const ExpertWeights& e,
    int hidden, int inter) {
    std::vector<float> act(inter);
    const float* Wg = e.w_gate.data_ptr<float>();
    const float* Wu = e.w_up.data_ptr<float>();
    for (int o = 0; o < inter; ++o) {
        float a = 0.0f, b = 0.0f;
        const float* wg = Wg + (int64_t)o * hidden;
        const float* wu = Wu + (int64_t)o * hidden;
        for (int k = 0; k < hidden; ++k) { a += x[k] * wg[k]; b += x[k] * wu[k]; }
        act[o] = siluf(a) * b;           // SiLU(gate) * up
    }
    const float* Wd = e.w_down.data_ptr<float>();
    for (int h = 0; h < hidden; ++h) {
        float a = 0.0f;
        const float* wd = Wd + (int64_t)h * inter;
        for (int o = 0; o < inter; ++o) a += act[o] * wd[o];
        out[h] = a;
    }
}

void moe_ffn(Tensor& out, const Tensor& x, const MoELayerWeights& w, const
    ↪ ModelConfig& cfg) {
    const int S = static_cast<int>(x.shape()[0]);
    const int hidden = cfg.hidden_size, inter = cfg.intermediate_size, K = cfg.
    ↪ top_k;
    const Routing r = route(x, w.router, cfg.num_experts, K);

    const float* X = x.data_ptr<float>();
    float*      O = out.data_ptr<float>();
    std::vector<float> tmp(hidden);

    for (int i = 0; i < S; ++i) {
```

```

float* oi = 0 + (int64_t)i * hidden;
for (int h = 0; h < hidden; ++h) oi[h] = 0.0f;
for (int j = 0; j < K; ++j) {
    const int e = r.experts[(size_t)i * K + j];
    const float g = r.weights[(size_t)i * K + j];
    expert_ffn_token(tmp.data(), X + (int64_t)i * hidden, w.experts[e],
    ↪ hidden, inter);
    for (int h = 0; h < hidden; ++h) oi[h] += g * tmp[h];    // weighted
    ↪ combine
    }
}
}

```

The MOE layer now works: route, run the chosen experts, mix. The one-expert test pins it to the dense FFN you already trust — with a single expert and $k = 1$, the softmax weight is exactly 1 and MOE reduces to the dense block, byte-comparable to Unit 5’s ffn. That equivalence is the sanity anchor; what it does *not* yet show is that the layer is *sparse* and *fast*. The per-token loop evaluates experts one token at a time, which is both slow and wasteful when many tokens share an expert. The next challenge groups tokens by expert to fix that — and proves, with a hard measurement, that only the routed experts ever run.

GOING FURTHER

- **Shared expert.** Add an always-on expert applied to every token and summed with the routed result (the DeepSeek design). It is one extra FFN call per token, outside the routing.
- **Capacity factor.** Real serving caps how many tokens an expert will accept per batch (a “capacity”); overflow tokens are dropped or passed through. Add a cap and a policy — it bounds worst-case cost when routing is unbalanced.
- **Reuse ffn for batched experts.** Notice `expert_ffn_token` is the single-token form of Unit 5’s ffn; in 6.3 you will call the *batched* ffn-style matmuls on a whole group of tokens at once.

6.3 Grouped Dispatch and Proven Sparsity

BOSS CHALLENGE

Invert the loop: group tokens by expert, run each expert once over its whole group with batched matmuls, and scatter the results back. Prove it matches the naive layer within tolerance — and prove the routing is genuinely **sparse**: zeroing a non-routed expert leaves a token bit-identical, while zeroing a routed one changes it. Sparsity, measured.

GOAL

Make the MOE layer fast and confirm it is sparse. Replace the per-token loop with a **grouped dispatch** — bucket tokens by expert, evaluate each expert once over its batch of tokens, scatter the weighted outputs back — and verify both that it reproduces the naive result and that only the routed experts ever affect a token.

BACKGROUND

The naive layer loops over tokens, and that ordering is the problem: every token's expert set is different, so the work cannot be batched, an expert's weights are re-fetched for each token that uses it, and none of Unit 3's fast matmul applies. The fix is to **invert the loop**. Instead of "for each token, run its experts," do "**for each expert, run all the tokens routed to it.**" Since experts are independent, this reordering changes nothing about the result — but it changes everything about performance.

Concretely: first **bucket** the routing — build, for each expert, the list of (token, weight) pairs assigned to it. Then for each expert that received at least one token, **gather** those tokens' hidden vectors into a contiguous $[n_e \times \text{hidden}]$ buffer, run the expert's FFN as **batched matmuls** over all n_e tokens at once (the gate, up, and down projections become three GEMM calls, exactly the fast path of Unit 3), and finally **scatter** each output row back to its token, scaled by that token's gate weight. Gather, batched compute, scatter — the standard shape of efficient MOE.

This grouping makes the **sparsity concrete**. An expert that no token routed to has an empty bucket and is **skipped entirely** — its weights are never read, its matmuls never run. The cost of the layer is therefore proportional to the number of (token, expert) assignments, which is $\text{seq} \times \text{top_k}$, *not* $\text{seq} \times \text{num_experts}$. That

is the whole economic promise of moe realized in code: you pay for the experts you use, not the experts you have.

Because sparsity is the entire point, it is what the boss **measures** — and the test is elegant. Pick a token and note which experts it routed to. Now **zero the weights of an expert it did *not* route to**: the token’s output must be **bit-identical**, because that expert never ran for it. Then **zero the weights of an expert it *did* route to**: the token’s output must *change*, because that expert’s contribution vanished. If both hold, the routing genuinely gates — the model’s parameters outside a token’s top- k have provably zero influence on it. Paired with a check that the grouped dispatch reproduces the naive layer, this certifies the moe block is both correct and sparse.

THE TASK

Implement `moe_ffn_grouped(out, x, w, cfg)`: route; bucket tokens by expert; for each non-empty bucket, gather its tokens into a contiguous buffer, run the expert via batched `linear/swiglu/linear` (Units 3 and 2), and scatter weight * output back to each token; skip empty buckets. Then the boss tests: (1) `moe_ffn_grouped` matches `moe_ffn` within tolerance; (2) for some token, zeroing a non-routed expert leaves its output unchanged while zeroing a routed expert changes it.

The bar (the boss): grouped output matches the naive layer to tolerance, unused experts are skipped, and the perturbation test confirms only routed experts influence a token.

HINTS

Hint 1 — bucket, then gather/compute/scatter

```
Build vector<vector<int>> toks(E) and vector<vector<float>> wgts(E)
from the routing. For each expert e with toks[e] non-empty: gather
rows into xe [n, hidden], run linear(g, xe, w_gate); linear(u, xe,
w_up); swiglu(a, g, u); linear(ye, a, w_down);, then add wgts[e][t] *
ye_row_t into out row toks[e][t].
```

Hint 2 — skipping is the sparsity

`if (toks[e].empty()) continue;` is not just an optimization — it is the observable sparsity. An unused expert's weights are never touched, which is exactly what the perturbation test detects.

Hint 3 — deep-copy to perturb one expert

A copied `MoELayerWeights` shares tensor storage with the original (Unit 1), so to zero one expert without disturbing the rest, *reassign* that expert's three tensors to freshly-allocated zero tensors. Then the original is untouched and only the copy's expert is dead.

TESTS

```
// add to tests/test_moe.cpp (reuses randt)
#include <set>
#include <cstring>

static Tensor zeros_like(const Tensor& t) {
    Tensor z = Tensor::empty(t.shape(), t.dtype());
    std::memset(z.data_ptr<float>(), 0, sizeof(float) * t.numel());
    return z;
}

static MoELayerWeights moe_weights(const ModelConfig& c, uint32_t seed) {
    MoELayerWeights w;
    w.router = randt({c.num_experts, c.hidden_size}, seed);
    for (int e = 0; e < c.num_experts; ++e)
        w.experts.push_back({ randt({c.intermediate_size, c.hidden_size}, seed +
    ↪ 10 + e),
                                randt({c.intermediate_size, c.hidden_size}, seed +
    ↪ 50 + e),
                                randt({c.hidden_size, c.intermediate_size}, seed +
    ↪ 90 + e) });
    return w;
}

TEST(boss_moe_grouped_matches_naive) {
    ModelConfig c; c.hidden_size = 8; c.intermediate_size = 16; c.num_experts = 6;
    ↪ c.top_k = 2;
    const MoELayerWeights w = moe_weights(c, 3);
    const int S = 7;
    Tensor x = randt({S, c.hidden_size}, 5);
```

```

Tensor o1 = Tensor::empty({S, c.hidden_size}, DType::F32);
Tensor o2 = Tensor::empty({S, c.hidden_size}, DType::F32);
moe_ffn(o1, x, w, c);
moe_ffn_grouped(o2, x, w, c);
for (int i = 0; i < S * c.hidden_size; ++i)
    CHECK_CLOSE(o1.data_ptr<float>()[i], o2.data_ptr<float>()[i], 1e-4);
}

TEST(boss_moe_routing_is_sparse) {
    ModelConfig c; c.hidden_size = 8; c.intermediate_size = 16; c.num_experts = 6;
    ↪ c.top_k = 2;
    const MoELayerWeights w = moe_weights(c, 8);
    const int S = 4, hidden = c.hidden_size;
    Tensor x = randt({S, hidden}, 2);

    Tensor base = Tensor::empty({S, hidden}, DType::F32);
    moe_ffn(base, x, w, c);

    const Routing r = route(x, w.router, c.num_experts, c.top_k);
    std::set<int> routed0;
    for (int j = 0; j < c.top_k; ++j) routed0.insert(r.experts[j]); // token 0's
    ↪ experts
    int non = -1; for (int e = 0; e < c.num_experts; ++e) if (!routed0.count(e)) {
    ↪ non = e; break; }
    const int rt = *routed0.begin();
    REQUIRE(non >= 0);

    auto kill = [&](int e) { // copy weights, zero expert
    ↪ e
        MoELayerWeights w2 = w;
        w2.experts[e].w_gate = zeros_like(w.experts[e].w_gate);
        w2.experts[e].w_up = zeros_like(w.experts[e].w_up);
        w2.experts[e].w_down = zeros_like(w.experts[e].w_down);
        Tensor o = Tensor::empty({S, hidden}, DType::F32);
        moe_ffn(o, x, w2, c);
        return o;
    };

    Tensor o_non = kill(non); // non-routed: token 0
    ↪ unchanged
    for (int h = 0; h < hidden; ++h)
        CHECK_CLOSE(base.at<float>({0, h}), o_non.at<float>({0, h}), 1e-5);

    Tensor o_rt = kill(rt); // routed: token 0 changes
    bool changed = false;
    for (int h = 0; h < hidden; ++h)
        if (std::fabs(base.at<float>({0, h}) - o_rt.at<float>({0, h})) > 1e-5f)

```

```

    ↪ changed = true;
    CHECK(changed);
}

```

REFERENCE SOLUTION

Declare in `model.hpp`, define in `model.cpp`:

```

// model.hpp
void moe_ffn_grouped(Tensor& out, const Tensor& x,
                    const MoELayerWeights& w, const ModelConfig& cfg);

```

```

// model.cpp
#include <vector>
#include <cstring>

void moe_ffn_grouped(Tensor& out, const Tensor& x,
                    const MoELayerWeights& w, const ModelConfig& cfg) {
    const int S = static_cast<int>(x.shape()[0]);
    const int hidden = cfg.hidden_size, inter = cfg.intermediate_size;
    const int E = cfg.num_experts, K = cfg.top_k;
    const Routing r = route(x, w.router, E, K);

    std::vector<std::vector<int>>> toks(E);           // tokens routed to each
    ↪ expert
    std::vector<std::vector<float>>> wgts(E);
    for (int i = 0; i < S; ++i)
        for (int j = 0; j < K; ++j) {
            const int e = r.experts[(size_t)i * K + j];
            toks[e].push_back(i);
            wgts[e].push_back(r.weights[(size_t)i * K + j]);
        }

    const float* X = x.data_ptr<float>();
    float*      O = out.data_ptr<float>();
    std::memset(O, 0, sizeof(float) * (size_t)S * hidden);

    for (int e = 0; e < E; ++e) {
        const int n = static_cast<int>(toks[e].size());
        if (n == 0) continue;           // unused expert: never
    ↪ evaluated
    }
}

```



```

Tensor xe = Tensor::empty({n, hidden}, DType::F32); // gather
for (int t = 0; t < n; ++t)
    std::memcpy(xe.data_ptr<float>() + (int64_t)t * hidden,
                X + (int64_t)toks[e][t] * hidden, sizeof(float) * hidden);

Tensor g = Tensor::empty({n, inter}, DType::F32); // batched expert
↪ FFN
Tensor u = Tensor::empty({n, inter}, DType::F32);
Tensor a = Tensor::empty({n, inter}, DType::F32);
Tensor ye = Tensor::empty({n, hidden}, DType::F32);
linear(g, xe, w.experts[e].w_gate);
linear(u, xe, w.experts[e].w_up);
swiglu(a, g, u);
linear(ye, a, w.experts[e].w_down);

for (int t = 0; t < n; ++t) { // scatter with gate weight
    const float gw = wgts[e][t];
    float* dst = 0 + (int64_t)toks[e][t] * hidden;
    const float* src = ye.data_ptr<float>() + (int64_t)t * hidden;
    for (int h = 0; h < hidden; ++h) dst[h] += gw * src[h];
}
}
}

```

The grouped layer is faster for the same reason the dense FFN was fast — it batches into the matmuls of Unit 3 — and it makes the sparsity visible: skipped experts cost nothing. The perturbation test is the certificate worth dwelling on. It demonstrates, by direct measurement, that the parameters *outside* a token’s top- k have exactly zero effect on its output, which is the precise sense in which MOE “has” billions of parameters it does not “use” for any given token. **Boss cleared.**

What you have proven. Not “an MOE that runs,” but “an MOE that is both correct and genuinely sparse”: the efficient grouped dispatch reproduces the reference exactly, and only a token’s routed experts influence its result — the rest are provably inert. That is the property the entire sparse-model paradigm rests on, now measured rather than assumed.

GOING FURTHER

- **Expert parallelism.** Place different experts on different devices and route with an all-to-all exchange of token vectors. This is how trillion-parameter MOEs are served; the gather/scatter you wrote is its single-device shadow.
- **Capacity and overflow.** Cap each expert's batch size and decide what happens to overflow tokens (drop, or route to the next-best expert). This bounds worst-case latency when routing is skewed.
- **Fused gate+up and quantized experts.** Apply Unit 5's fused gate/up trick per expert, and (after Unit 9) store expert weights quantized — experts are the bulk of an MOE's memory, so they are the prime target.

Unit 6 — Milestone check

MILESTONE CHECK

The engine's feed-forward block can now be a Mixture of Experts:

- a **router** that scores experts per token and commits to the top- k with normalized weights;
- **expert FFNS** (SwiGLU, from Unit 5) and a **weighted combine** into each token's output;
- an **efficient grouped dispatch** — gather by expert, batched matmuls, scatter — that skips unused experts; and
- a **boss-grade proof of sparsity**: only a token's routed experts affect it, so compute scales with `top_k`, not `num_experts`.

The model is now both complete (Unit 5) and scalable (Unit 6). What it is not yet is *efficient to generate with*: every forward pass still reprocesses the entire sequence from scratch, which is ruinous when producing tokens one at a time. **Unit 7** fixes this with the **kv cache** — storing each layer's keys and values so that generating the next token costs one new position's work instead of recomputing all of history — the single most important optimization for generation speed, and the reason grouped-query attention (Unit 5) mattered.

UNIT 7

The KV Cache

What you will build. The optimization that makes generation tractable: a **key/value cache**. You build the per-layer **cache structure** (preallocated K and V buffers with a length cursor), **cached attention** that appends the new token's keys and values and attends over all of history, and a **prefill-then-decode** forward pass. The boss proves the cache is an *exact* optimization: incremental generation yields the same logits as recomputing the whole sequence.

Where this fits. Generation is autoregressive — one token at a time, each conditioned on all the tokens before it. Done naively, every new token triggers a full forward pass over the entire sequence so far, so producing n tokens costs work proportional to n^2 . The kv cache collapses that to roughly n : store what does not change, and each step does the work of a single new position. It is the single most important optimization for generation speed — and the payoff for grouped-query attention (Unit 5), which shrank exactly the thing we now have to store.

Here is the observation the whole unit rests on. In causal attention, a token attends only to itself and the tokens before it; the future never influences the past. So when you append a new token to a sequence, the **keys and values of all the previous tokens do not change** — they were computed from positions the new token cannot affect. Recomputing them on every generation step, as a naive full forward pass does, is pure waste. The fix is to **cache** them: keep every layer's keys and values for the tokens seen so far, and at each new step compute K and V for only the *new* token, append them, and attend the new query over the whole cached history.

This reshapes generation into two phases. **Prefill** processes the prompt — all its tokens at once — filling the cache and producing the first prediction. **Decode**

then generates one token at a time: embed the single new token, compute its K/V , append to the cache, attend over everything cached, and read out the next logits. Prefill is a big parallel pass; decode is a long sequence of cheap incremental steps. Real engines spend most of their time in decode, which is exactly where the cache pays off.

We build it in three steps: the cache structure, the cached attention that reads and writes it, and the prefill/decode forward pass — with a boss that proves the result is identical to the slow, full-recompute path. Add a `KVCache` to `model.hpp`; the cached attention is a close cousin of Unit 5's.

7.1 The Cache Structure

GOAL

The storage: a `KVCache` holding, per layer, preallocated key and value buffers sized to the maximum sequence length, with a **length cursor** tracking how many positions are live and an **append** that writes new positions. Preallocating means generation never reallocates.

BACKGROUND

What exactly do we cache? For *every layer*, the keys and values of every token processed so far — because each layer computes its own K and V , and each layer's attention needs the full history of *its* keys and values. So the cache is, per layer, two tensors of shape $[\text{max_seq} \times \text{n_kv_heads} \times \text{head_dim}]$: room for the keys and the values of up to `max_seq` positions. We store `n_kv_heads` (not `n_heads`) because grouped-query attention shares key/value heads — and this is where that sharing pays its dividend, shrinking the cache by the group factor.

We **preallocate** to the maximum sequence length up front and never grow the buffers during generation. This is the same discipline as the arena in Unit 1: decode runs thousands of steps, and reallocating (and copying) the cache on each would be ruinous, so we pay once for the largest size we will need and then only write into it. A **length cursor** records how many positions are currently valid; appending n new positions writes their keys and values into rows $[\text{length}, \text{length} + n)$ and advances the cursor by n . Reset sets the cursor to zero, reusing the same memory for a new sequence.

It is worth feeling the **size** of this. The cache holds

$$2 \times n_{\text{layers}} \times \text{max_seq} \times n_{\text{kv_heads}} \times \text{head_dim}$$

floats (the 2 for keys and values). For a model with 32 layers, 8 key/value heads, head dimension 128, at a context of 8192 tokens, that is over 4 billion floats — gigabytes, often exceeding the size of the model’s weights at long context. This is why the kv cache is a central concern of real serving systems, and why so much effort goes into shrinking it: grouped-query attention (fewer heads, Unit 5), quantizing the cache (Unit 9’s techniques applied to K/V), and paged allocation (Unit 13). Here we build the straightforward dense version; knowing its cost is the motivation for everything that optimizes it later.

THE TASK

Add a KVCache to `model.hpp`: per-layer `k` and `v` tensors of shape `[max_seq, n_kv_heads*head_dim]`, an `int` `length` cursor, the dimensions it needs, a factory `allocate(cfg, max_seq)`, an `append_layer(layer, k_new, v_new, n)` that copies `n` new rows into the cache at the current `length` offset, an `advance(n)` that moves the cursor, and `reset()`.

HINTS

Hint 1 — one buffer pair per layer

`std::vector<Tensor> k, v;` each sized `[max_seq, n_kv_heads*head_dim]`. `allocate` fills both vectors with `n_layers` freshly-empty’d tensors and records the dimensions.

Hint 2 — append writes at the cursor, advance moves it

Keep them separate. During a forward pass, every layer appends its new keys/values at the *same* offset length (they all process the same new tokens), and the cursor advances *once* after all layers — so `append_layer` writes at length without touching the cursor, and the forward pass calls `advance(n)` at the end.

Hint 3 — a row is one position's K (or V) across kv-heads

Row p of the layer's k tensor is $n_kv_heads * head_dim$ contiguous floats: position p 's keys for all key/value heads. Appending n rows is one memcpy of $n * n_kv_heads * head_dim$ floats into the buffer at offset `length`.

TESTS

```
// add to tests/test_kvcache.cpp
#include "test_framework.hpp"
#include "llm/model.hpp"
using namespace llm;

TEST(kvcache_allocates_per_layer) {
    ModelConfig c; c.n_layers = 3; c.n_kv_heads = 2; c.head_dim = 4;
    KVCache cache = KVCache::allocate(c, /*max_seq=*/16);
    CHECK((int)cache.k.size() == c.n_layers);
    CHECK((int)cache.v.size() == c.n_layers);
    CHECK(cache.k[0].shape()[0] == 16); // max_seq rows
    CHECK(cache.k[0].shape()[1] == c.n_kv_heads * c.head_dim);
    CHECK(cache.length == 0);
}

TEST(kvcache_append_and_advance) {
    ModelConfig c; c.n_layers = 1; c.n_kv_heads = 1; c.head_dim = 2;
    KVCache cache = KVCache::allocate(c, 8);
    const int kvd = c.n_kv_heads * c.head_dim; // 2
    Tensor knew = Tensor::empty({2, kvd}, DType::F32); // 2 new positions
    Tensor vnew = Tensor::empty({2, kvd}, DType::F32);
    float kk[4] = {1, 2, 3, 4}, vv[4] = {5, 6, 7, 8};
    for (int i = 0; i < 4; ++i) { knew.data_ptr<float>()[i] = kk[i]; vnew.data_ptr<
    ↪ float>()[i] = vv[i]; }

    cache.append_layer(0, knew, vnew, 2);
    cache.advance(2);
    CHECK(cache.length == 2);
    CHECK_CLOSE(cache.k[0].at<float>({0, 0}), 1.0f, 1e-6); // first appended row
    CHECK_CLOSE(cache.k[0].at<float>({1, 1}), 4.0f, 1e-6);
    CHECK_CLOSE(cache.v[0].at<float>({1, 0}), 7.0f, 1e-6);

    cache.reset();
    CHECK(cache.length == 0); // memory reused for
    ↪ next sequence
}
```

REFERENCE SOLUTION

Add to `model.hpp`:

```
// model.hpp
struct KVCache {
    int n_layers = 0, n_kv_heads = 0, head_dim = 0, max_seq = 0;
    int length = 0; // live positions
    std::vector<Tensor> k; // per layer: [max_seq, n_kv_heads*
    ↪ head_dim]
    std::vector<Tensor> v;

    static KVCache allocate(const ModelConfig& cfg, int max_seq);
    void append_layer(int layer, const Tensor& k_new, const Tensor& v_new, int n);
    void advance(int n) { length += n; }
    void reset() { length = 0; }
};
```

Add to `model.cpp`:

```
// model.cpp
#include <cstring>

KVCache KVCache::allocate(const ModelConfig& cfg, int max_seq) {
    KVCache c;
    c.n_layers = cfg.n_layers; c.n_kv_heads = cfg.n_kv_heads;
    c.head_dim = cfg.head_dim; c.max_seq = max_seq; c.length = 0;
    const int kvd = cfg.n_kv_heads * cfg.head_dim;
    c.k.reserve(cfg.n_layers); c.v.reserve(cfg.n_layers);
    for (int l = 0; l < cfg.n_layers; ++l) {
        c.k.push_back(Tensor::empty({max_seq, kvd}, DType::F32));
        c.v.push_back(Tensor::empty({max_seq, kvd}, DType::F32));
    }
    return c;
}

void KVCache::append_layer(int layer, const Tensor& k_new, const Tensor& v_new,
    ↪ int n) {
    const int kvd = n_kv_heads * head_dim;
    LLM_CHECK(length + n <= max_seq, "KVCache: capacity exceeded");
    std::memcpy(k[layer].data_ptr<float>() + (int64_t)length * kvd,
        k_new.data_ptr<float>(), sizeof(float) * (size_t)n * kvd);
    std::memcpy(v[layer].data_ptr<float>() + (int64_t)length * kvd,
        v_new.data_ptr<float>(), sizeof(float) * (size_t)n * kvd);
}
```

The cache is just memory plus a cursor — but it is the memory that turns a quadratic generation loop into a linear one. The append-at-cursor / advance-once split matters: all layers write the same new positions during a forward pass, so the cursor moves a single time at the end, keeping every layer’s view of “how many positions exist” consistent. With somewhere to put the keys and values, we can now write the attention that fills and reads it.

GOING FURTHER

- **Quantized kv cache.** Store K/V in int8 (or lower) instead of float32 to cut cache memory by $4\times$ — the techniques of Unit 9 applied to the cache. At long context the cache dwarfs the weights, so this is high-leverage.
- **Ring buffer / sliding window.** For windowed-attention models, make the cache a ring buffer of the window size so memory stays bounded regardless of sequence length.
- **Per-sequence caches for batching.** Generating for many sequences at once needs a cache per sequence; the paged scheme of Unit 13 manages these without preallocating max-length per sequence.

7.2 Cached Attention

GOAL

Attention that reads and writes the cache: project the *new* tokens’ queries, keys, and values; apply RoPE at the correct absolute positions; **append** the new keys/values to the cache; and attend each new query over **all of history** — cached plus new. For a single decode token this attends the entire past in one cheap step.

BACKGROUND

Cached attention is Unit 5’s attention with one change in where keys and values come from. Instead of computing K and V for the whole sequence and attending within it, we compute K and V for only the **new** tokens, append them to the layer’s cache, and attend the new queries over the *entire* cached history. The values and most of the keys are already sitting in the cache from earlier steps; we

add only what is new.

The one detail that must be exactly right is **position**. The new tokens occupy absolute positions $[\text{length}, \text{length} + n)$ where `length` is the count already cached, so their queries and keys are RoPE-rotated with **that offset** — `pos0 = cache.length`. This matters because the cached keys were rotated by *their* absolute positions when they were written, in earlier steps; for the dot products to encode relative position correctly (the whole point of RoPE, Unit 5), every key in the cache must carry the rotation of its true absolute position, and the new query must carry its own. Get the offset right and a cached attention is *indistinguishable* from recomputing the full sequence — which is exactly what the boss will check.

The causal structure is unchanged, just expressed against the cache: the query at absolute position p attends cached keys $0..p$. In **decode**, the new token is a single position at the end, so it attends *all* `length + 1` keys — one query against the whole past, which is cheap. In **prefill**, several new tokens arrive together and each attends the keys up to its own position (a triangular pattern within the new block, plus nothing before since the cache starts empty). The grouped-query mapping (`kv_h = h / group`) carries over verbatim, and the values read for the weighted sum come straight from the cache.

So the sub-layer becomes: project the new tokens to Q, K, V ; RoPE Q and K at offset `cache.length`; append the new K, V to the cache; for each head and each new query, score against the cached keys up to that query's position, stable-softmax, and take the weighted sum of the cached values; concatenate the heads and project through w_o . Everything reused, one offset added.

THE TASK

Implement `attention_cached(out, x, w, cfg, cache, layer)`: linear the new tokens to $q, k_{\text{new}}, v_{\text{new}}$; apply_rope on q and k_{new} with `pos0 = cache.length`; `cache.append_layer(layer, k_new, v_new, n)`; then for each head h (`kv_h = h/group`) and each new query i at absolute position `cache.length + i`, score against cached keys $0..(\text{cache.length} + i)$, stable-softmax, weighted-sum the cached values, and finally linear through w_o . (The forward pass advances the cursor after all layers.)

HINTS

Hint 1 — the position offset is everything

`const int pos0 = cache.length;` then `apply_rope(q, n, n_heads, head_dim, theta, pos0);` and the same for `k_new` with `n_kv_heads`. Query `i`'s absolute position is `pos0 + i`; it attends cached keys `j = 0 .. pos0 + i`.

Hint 2 — append before you read

Call `append_layer` first, so the cache's rows `0 .. pos0+n-1` are all valid; then read keys and values directly from `cache.k[layer]` / `cache.v[layer]`. The new query can thus attend the keys you just wrote.

Hint 3 — it is Unit 5's inner loop over the cache

The per-head, per-query score/softmax/weighted-sum is identical to Unit 5; only the key/value source (the cache buffer, length `n_kv_heads*head_dim` per row) and the loop bound (`j <= pos0+i`) differ. Reuse the structure.

TESTS

```
// add to tests/test_kvcache.cpp (reuses randt, attn_weights from the model tests)
↪
#include <cstring>

static Tensor rows(const Tensor& x, int r0, int r1) {
    const int hidden = static_cast<int>(x.shape()[1]);
    Tensor s = Tensor::empty({r1 - r0, hidden}, DType::F32);
    std::memcpy(s.data_ptr<float>(), x.data_ptr<float>() + (int64_t)r0 * hidden,
                sizeof(float) * (size_t)(r1 - r0) * hidden);
    return s;
}

TEST(cached_attention_equals_full_when_prefilled_whole) {
    ModelConfig c; c.hidden_size = 8; c.n_heads = 2; c.n_kv_heads = 1; c.head_dim
    ↪ = 4; c.n_layers = 1;
    const LayerWeights w = attn_weights(c, 7);
    const int S = 5;
    Tensor x = randt({S, c.hidden_size}, 3);

    Tensor o_full = Tensor::empty({S, c.hidden_size}, DType::F32);
```

```

attention(o_full, x, w, c, /*pos0=*/0); // Unit 5 reference

KVCache cache = KVCache::allocate(c, 32);
Tensor o_cached = Tensor::empty({S, c.hidden_size}, DType::F32);
attention_cached(o_cached, x, w, c, cache, /*layer=*/0);
cache.advance(S);
for (int i = 0; i < S * c.hidden_size; ++i)
    CHECK_CLOSE(o_full.data_ptr<float>()[i], o_cached.data_ptr<float>()[i], 1e
    ↪ -4);
}

TEST(cached_attention_incremental_equals_full) {
    ModelConfig c; c.hidden_size = 8; c.n_heads = 2; c.n_kv_heads = 1; c.head_dim
    ↪ = 4; c.n_layers = 1;
    const LayerWeights w = attn_weights(c, 11);
    const int S = 5, hidden = c.hidden_size;
    Tensor x = randt({S, hidden}, 4);

    Tensor o_full = Tensor::empty({S, hidden}, DType::F32);
    attention(o_full, x, w, c, 0);

    KVCache cache = KVCache::allocate(c, 32);
    Tensor pre = rows(x, 0, 3); // prefill first 3
    ↪ positions
    Tensor o_pre = Tensor::empty({3, hidden}, DType::F32);
    attention_cached(o_pre, pre, w, c, cache, 0); cache.advance(3);
    for (int i = 0; i < 3; ++i)
        for (int d = 0; d < hidden; ++d)
            CHECK_CLOSE(o_pre.at<float>({i, d}), o_full.at<float>({i, d}), 1e-4);

    for (int i = 3; i < S; ++i) { // decode the rest
    ↪ one at a time
        Tensor xi = rows(x, i, i + 1);
        Tensor oi = Tensor::empty({1, hidden}, DType::F32);
        attention_cached(oi, xi, w, c, cache, 0); cache.advance(1);
        for (int d = 0; d < hidden; ++d)
            CHECK_CLOSE(oi.at<float>({0, d}), o_full.at<float>({i, d}), 1e-4);
    }
}

```

REFERENCE SOLUTION

Declare in `model.hpp`, define in `model.cpp`:

```
// model.hpp
void attention_cached(Tensor& out, const Tensor& x, const LayerWeights& w,
                     const ModelConfig& cfg, KVCache& cache, int layer);
```

```
// model.cpp
void attention_cached(Tensor& out, const Tensor& x, const LayerWeights& w,
                     const ModelConfig& cfg, KVCache& cache, int layer) {
    const int n = static_cast<int>(x.shape()[0]); // new tokens this
    ↪ call
    const int H = cfg.n_heads, KVH = cfg.n_kv_heads, D = cfg.head_dim;
    const int qd = H * D, kvd = KVH * D, group = H / KVH;
    const float scale = 1.0f / std::sqrt(static_cast<float>(D));
    const int pos0 = cache.length; // new tokens at pos0
    ↪ ..pos0+n-1

    Tensor q = Tensor::empty({n, qd}, DType::F32);
    Tensor k_new = Tensor::empty({n, kvd}, DType::F32);
    Tensor v_new = Tensor::empty({n, kvd}, DType::F32);
    linear(q, x, w.wq); linear(k_new, x, w.wk); linear(v_new, x, w.wv);

    apply_rope(q, n, H, D, cfg.rope_theta, pos0);
    apply_rope(k_new, n, KVH, D, cfg.rope_theta, pos0);
    cache.append_layer(layer, k_new, v_new, n); // history now 0..
    ↪ pos0+n-1

    const float* Q = q.data_ptr<float>();
    const float* Kc = cache.k[layer].data_ptr<float>(); // [max_seq, kvd]
    const float* Vc = cache.v[layer].data_ptr<float>();

    Tensor concat = Tensor::empty({n, qd}, DType::F32);
    float* O = concat.data_ptr<float>();
    std::vector<float> scores(pos0 + n);

    for (int h = 0; h < H; ++h) {
        const int kvh = h / group;
        for (int i = 0; i < n; ++i) {
            const int qpos = pos0 + i; // absolute position
            const float* qi = Q + ((int64_t)i * H + h) * D;
            float m = -INFINITY;
            for (int j = 0; j <= qpos; ++j) { // attend cached
                ↪ keys 0..qpos
                const float* kj = Kc + ((int64_t)j * KVH + kvh) * D;
                float dot = 0.0f;
                for (int d = 0; d < D; ++d) dot += qi[d] * kj[d];
```

```

        dot *= scale; scores[j] = dot; if (dot > m) m = dot;
    }
    float sum = 0.0f;
    for (int j = 0; j <= qpos; ++j) { float e = std::exp(scores[j] - m);
    ↪ scores[j] = e; sum += e; }
    const float inv = 1.0f / sum;
    float* oi = 0 + ((int64_t)i * H + h) * D;
    for (int d = 0; d < D; ++d) oi[d] = 0.0f;
    for (int j = 0; j <= qpos; ++j) {
        const float wij = scores[j] * inv;
        const float* vj = Vc + ((int64_t)j * KVH + kvh) * D;
        for (int d = 0; d < D; ++d) oi[d] += wij * vj[d];
    }
}
}
linear(out, concat, w.wo);
}

```

The two tests are the crux: a cached attention that processes the whole sequence at once matches Unit 5’s attention, and — more importantly — processing it *incrementally* (prefill a few, then one token at a time) produces the same per-position outputs. That second property is the entire justification for the cache: the decode step, attending one new query over a growing history, computes exactly what the full recompute would, at a fraction of the cost. Wire this into the block and the forward pass and generation becomes linear — the boss.

GOING FURTHER

- **Batch the cached scores.** The single-query decode dot products against all cached keys are a matrix–vector product; express them as a GEMV (Unit 3) for speed. Flash Attention (Unit 8) fuses this further.
- **Position via a per-token array.** Instead of a single `pos0`, accept an explicit array of positions, supporting non-contiguous position schemes and prompt-caching where positions are reused.
- **Write keys/values pre-rotated vs raw.** We cache RoPE’d keys. Some designs cache raw keys and rotate on read; compare the trade-offs for context-extension schemes that change the rotation.

7.3 Prefill, Decode, and Exact Equivalence

BOSS CHALLENGE

Wire the cache into the forward pass — a parallel **prefill** over the prompt, then incremental **decode** one token at a time — and prove it is exact: the logits produced step-by-step with the cache match a single full recompute of the whole sequence, position for position, within tolerance. A pure optimization, verified.

GOAL

Complete cached generation: a `forward_cached` that processes new tokens through cached blocks, advancing the cursor once per call — used as one big **prefill** over the prompt and then a series of single-token **decode** steps. Then prove the cache changes *speed but not results*: incremental logits equal full-recompute logits everywhere.

BACKGROUND

The cached forward pass mirrors Unit 5's, swapping each block's attention for the cached version and advancing the cache cursor once after all layers (every layer processed the same new tokens, so the position count moves a single time). Embed the new tokens, run them through the cached blocks, advance the cursor by the number of new tokens, apply the final norm, and project to logits.

This one function serves both phases of generation. **Prefill**: call it with the entire prompt — a parallel pass that fills the cache for all prompt positions and returns logits for each (the last of which predicts the first generated token). **Decode**: call it repeatedly with a single new token — each call embeds one token, computes its keys and values, appends them, attends over the whole cached history, and returns one position's logits. Prefill is one large matmul-heavy pass; decode is a long run of cheap single-token steps. This split is the shape of every real inference engine.

Why this is the right boss is a matter of trust. The kv cache is supposed to be a *pure* optimization — it must compute byte-for-byte what the naive full recompute computes, or generation silently drifts onto a different trajectory than the model intends. So the property to verify is **exact equivalence**: run a sequence two ways and demand identical logits.

- (A) One full forward pass over the entire sequence (Unit 5’s forward).
- (B) Prefill a prompt, then decode the remaining tokens one at a time with the cache. The logits at every position must match between (A) and (B), to tolerance.

If they match, then the cache structure, the RoPE position offsets, the append/advance bookkeeping, and the prefill/decode split are *all* correct — because any one of them being wrong would divert the logits. (A misplaced cursor advance, a stale cache row, an off-by-one position, a forgotten offset: every such bug shows up as a divergence.) And the equivalence also makes the **complexity win** concrete. Generating token k by full recompute reprocesses all k tokens, so the expensive projections and FFNS cost $O(k)$ per step and $O(n^2)$ to generate n tokens; with the cache, each decode step processes a *single* new token, so those same projections and FFNS cost $O(1)$ per step and $O(n)$ overall. The cache moves the dominant cost from quadratic to linear in the number of generated tokens — the difference between feasible and infeasible.

THE TASK

Implement `transformer_block_cached` (Unit 5’s block, but calling `attention_cached`) and `forward_cached(model, cfg, new_ids, cache)` (`embed` → `cached blocks` → `cache.advance(n)` → `final rmsnorm` → `lm_head`). Then the boss test: build a tiny model and a sequence; compute full-recompute logits with `forward`; compute incremental logits by `forward_cached` on a prompt then one token at a time; assert they match at every position.

The bar (the boss): for every position, the cached/incremental logits equal the full-recompute logits within tolerance, and the cache length ends equal to the sequence length.

HINTS

Hint 1 — the cached block is a one-line edit

Identical to `transformer_block`, but call `attention_cached(delta, normed, w, cfg, cache, layer)` instead of `attention`. The FFN, norms, and residual adds are unchanged.

Hint 2 — advance once, after the layer loop

`forward_cached` loops the cached blocks over all layers (each appending at the same cursor), then calls `cache.advance(n)` exactly once. Advancing inside the layer loop would corrupt every later layer's notion of history length.

Hint 3 — build the two paths to be comparable

For the test, the same `ModelWeights` drives both. Prefill the first `P` tokens in one call, then loop the remaining tokens as single-element vectors. Compare each returned row to the matching row of the full-recompute logits.

TESTS

```
// add to tests/test_kvcache.cpp
// (reuses randt, full_weights, tiny_model, tiny_cfg from the model tests)
#include <cmath>

TEST(boss_kv_cache_matches_full_recompute) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    const std::vector<int> seq = {3, 7, 1, 9, 2, 5, 8};
    const int S = static_cast<int>(seq.size()), V = c.vocab_size;

    Tensor full = forward(m, c, seq); // (A) full recompute

    KVCache cache = KVCache::allocate(c, /*max_seq=*/64); // (B) prefill +
    ↪ decode
    const int P = 3;
    const std::vector<int> prompt(seq.begin(), seq.begin() + P);
    Tensor pre = forward_cached(m, c, prompt, cache); // prefill -> [P, V]
    for (int i = 0; i < P; ++i)
        for (int t = 0; t < V; ++t)
            CHECK_CLOSE(pre.at<float>({i, t}), full.at<float>({i, t}), 1e-3);

    for (int i = P; i < S; ++i) { // decode one token
    ↪ at a time
        Tensor step = forward_cached(m, c, {seq[i]}, cache); // -> [1, V]
        for (int t = 0; t < V; ++t)
            CHECK_CLOSE(step.at<float>({0, t}), full.at<float>({i, t}), 1e-3);
    }
    CHECK(cache.length == S); // whole sequence now
}
```



```

    ↪ cached
}

```

REFERENCE SOLUTION

Declare in `model.hpp`, define in `model.cpp`:

```

// model.hpp
void transformer_block_cached(Tensor& x, const LayerWeights& w,
                             const ModelConfig& cfg, KVCache& cache, int layer);

    ↪
Tensor forward_cached(const ModelWeights& model, const ModelConfig& cfg,
                     const std::vector<int>& new_ids, KVCache& cache);

```

```

// model.cpp
void transformer_block_cached(Tensor& x, const LayerWeights& w,
                             const ModelConfig& cfg, KVCache& cache, int layer) {
    const int S = static_cast<int>(x.shape()[0]);
    const int hidden = cfg.hidden_size;
    Tensor normed = Tensor::empty({S, hidden}, DType::F32);
    Tensor delta = Tensor::empty({S, hidden}, DType::F32);

    rmsnorm(normed, x, w.attn_norm, cfg.rms_norm_eps);
    attention_cached(delta, normed, w, cfg, cache, layer); // <- the only change
    add_inplace(x, delta);

    rmsnorm(normed, x, w.ffn_norm, cfg.rms_norm_eps);
    ffn(delta, normed, w, cfg); // (or moe_ffn, Unit 6)
    ↪
    add_inplace(x, delta);
}

Tensor forward_cached(const ModelWeights& model, const ModelConfig& cfg,
                     const std::vector<int>& new_ids, KVCache& cache) {
    const int n = static_cast<int>(new_ids.size());
    const int hidden = cfg.hidden_size;

    Tensor x = Tensor::empty({n, hidden}, DType::F32);
    embed_tokens(x, new_ids, model.token_embedding);

    for (int l = 0; l < cfg.n_layers; ++l)
        transformer_block_cached(x, model.layers[l], cfg, cache, l);
}

```

```

cache.advance(n);                                     // advance once, after
↳ all layers

Tensor normed = Tensor::empty({n, hidden}, DType::F32);
rmsnorm(normed, x, model.final_norm, cfg.rms_norm_eps);
Tensor logits = Tensor::empty({n, cfg.vocab_size}, DType::F32);
linear(logits, normed, model.lm_head);
return logits;
}

```

Run the boss. The prefilled prompt’s logits match the full pass, and then *every* decoded token’s logits match the corresponding position of the full recompute — the cache reproduces the slow path exactly while doing a fraction of the work. **Boss cleared.**

What you have proven. Not “a cache that speeds things up,” but “a cache that is *exact*”: incremental, cached generation computes precisely the same logits as recomputing the entire sequence at every step, position for position. That equivalence certifies the whole mechanism — storage, position offsets, append/advance, the prefill/decode split — and it is what lets you trust that the fast path and the slow path are the same model. With it, generation has gone from quadratic to linear in the expensive operations, which is the difference between a toy and an engine.

GOING FURTHER

- **The generation loop.** Combine `forward_cached` with a sampler (Unit 11): prefill the prompt, then repeatedly sample from the last logits and feed the chosen token back as the next single-token decode step until eos.
- **Last-position-only logits.** During decode you only need logits for the final token; skip the `lm_head` matmul for earlier positions in prefill if you will not sample them, saving a large vocabulary projection.
- **Zero-allocation decode.** Route every per-step activation through an arena (Unit 1) and use the `Storage::alloc_count()` meter to confirm a warm decode step performs no heap allocation — essential for steady, low-latency generation.

Unit 7 — Milestone check

MILESTONE CHECK

The engine can now generate efficiently rather than wastefully:

- a preallocated **kv cache**, per layer, holding the keys and values of all tokens seen so far (shrunk by grouped-query attention);
- **cached attention** that appends the new token's keys/values and attends over the full history with the correct RoPE offset;
- a **prefill-then-decode** forward pass — one parallel pass over the prompt, then cheap single-token steps; and
- a **boss-grade proof** that cached generation is *exactly* equivalent to full recompute, turning the dominant cost from quadratic to linear in the generated length.

Generation is now efficient, but the attention itself still has two weaknesses: it materializes a full score row per query and re-reads keys and values from memory in a way that wastes bandwidth, and its memory traffic grows with sequence length. **Unit 8** (★) addresses both with **Flash Attention** — a reformulation that computes exact attention in a single fused pass using the online softmax, never materializing the full score matrix, tiling the computation to stay in fast memory. It is the most celebrated systems result in modern LLM inference, and the online-softmax warm-up from Unit 2 was its seed.

UNIT 8

Flash Attention

What you will build. The reformulation that makes attention fast and memory-light: **Flash Attention**. You build the **online softmax** — a way to compute a softmax-weighted sum incrementally, never storing the full vector — then **tiled attention** that streams key/value blocks through it, computing exact attention in one fused pass without ever materializing the $[\text{seq} \times \text{seq}]$ score matrix. The boss proves it matches naive attention exactly while using only block-sized scratch.

Where this fits. Naive attention (Unit 5) builds the entire score matrix, softmaxes it, and multiplies by the values — three passes over an $O(\text{seq}^2)$ array in slow memory. For long sequences that memory traffic, not the arithmetic, is the bottleneck. Flash Attention computes the *same* result in a single fused, tiled pass whose working set is independent of sequence length. It is the most celebrated systems result in modern LLM inference, and its numerical heart — the online softmax — is the warm-up you met in Unit 2.

Attention's cost is not where intuition first puts it. The arithmetic — QK^T , the softmax, the multiply by V — is only part of the story; the other part is **memory traffic**. Naive attention writes the full $[\text{seq} \times \text{seq}]$ score matrix to slow memory, reads it back to softmax it, and reads it again to multiply by the values. At long context that matrix is enormous and the hardware spends its time shuttling it to and from memory rather than computing — attention becomes **memory-bound**. Worse, the matrix itself must be *stored*, so memory footprint grows quadratically with sequence length.

Flash Attention's insight is that the score matrix never needs to exist all at once. If you process the keys and values in **blocks** and maintain a **running softmax** as you go — updating an accumulated output block by block, rescaling when

needed — you can produce *exactly* the same result while holding only one block in fast memory at a time. The $O(\text{seq})$ score row collapses to an $O(\text{block})$ working set; the round-trips to slow memory collapse to a single fused pass that stays in cache (or, on a GPU, in SRAM). The FLOP count is unchanged; the memory traffic — the thing that was actually slow — falls dramatically. This is the same “tile to stay in fast memory” idea you used for GEMM in Unit 3, now applied to attention, and powered by the same numerically-stable softmax from Unit 2.

We build it in three steps: the online softmax that makes incremental accumulation possible, the tiled flash attention that applies it, and a boss that proves exactness at scale with bounded memory. The online softmax is pure numerics — no model context needed — so we start there.

8.1 Online Softmax

GOAL

The algebraic core of Flash Attention: compute a softmax-weighted sum of value vectors **incrementally**, processing scores in blocks while maintaining a running maximum, a running denominator, and a running output — rescaled whenever the maximum grows — so the full score vector is never stored.

BACKGROUND

Recall the stable softmax-weighted sum from Unit 5’s attention: for scores $s_0, \dots, s_{N-1}$ and value vectors $v_0, \dots, v_{N-1}$,

$$\text{out} = \sum_j \frac{e^{s_j - m}}{\sum_k e^{s_k - m}} v_j, \quad m = \max_j s_j.$$

Computed directly, this needs the *whole* score vector twice — once to find m and the denominator, once to weight the values. The **online softmax** computes the identical result in a single streaming pass, by carrying just three running quantities and updating them as each new block of scores arrives:

- m — the maximum score seen so far (start at $-\infty$);
- ℓ — the running denominator $\sum e^{s-m}$ over scores seen so far (start at 0);

- `acc` — the running weighted output $\sum e^{s-m} v$ (start at 0).

When a new block arrives, its scores may contain a value larger than the current m , which would change the basis of every exponential computed so far. The trick is to **rescale**. Let $m' = \max(m, \max(\text{block}))$ be the updated maximum. The old ℓ and `acc` were normalized relative to m ; multiplying them by $e^{m-m'}$ re-bases them to m' . Then we add the new block's contributions, exponentiated relative to m' :

$$\ell' = \ell e^{m-m'} + \sum_{j \in \text{block}} e^{s_j-m'}, \quad \text{acc}' = \text{acc} e^{m-m'} + \sum_{j \in \text{block}} e^{s_j-m'} v_j,$$

and set $m \leftarrow m'$. After the last block, the answer is `out = acc /  $\ell$` .

Two things make this work. First, the rescaling keeps ℓ and `acc` *exactly* consistent with the running maximum at every step, so the streamed result is algebraically identical to the all-at-once computation — no approximation. Second, because every exponent is taken relative to the current maximum, no $e^{(\cdot)}$ ever overflows, so it inherits the numerical stability of Unit 2's softmax. (The initial state $m = -\infty$ handles itself: the first block's rescale factor $e^{-\infty-m'} = 0$ wipes the empty accumulators before adding real contributions.)

This is the entire mathematical content of Flash Attention. Everything else is applying it: the “scores” will be $q \cdot k$ over blocks of keys, and the “values” the corresponding V rows, so attention becomes a single pass that accumulates the output without ever forming the score row.

THE TASK

Implement an `OnlineSoftmax` state (`m`, `l`, and an `acc` vector of length `head_dim`) and `online_update(state, scores, block, values, head_dim)` that updates it with a block of block scores and their `[block, head_dim]` value vectors, applying the rescale-then-add recurrence; plus `online_finalize(state, out, head_dim)` writing `acc / l`.

HINTS

Hint 1 — compute the block max, then the new running max

```
float bmax = max over the block's scores; float new_m =
std::max(state.m, bmax);. The correction factor for the old state is
std::exp(state.m - new_m) (which is 0 on the very first block, since
```

state.m starts at $-\infty$).

Hint 2 — rescale old, then add new

Multiply state.l and every entry of state.acc by the correction factor. Then loop the block: `p = std::exp(scores[j] - new_m); state.l += p;` and `state.acc[d] += p * values[j*head_dim + d]`. Finally `state.m = new_m`.

Hint 3 — finalize divides once

`online_finalize` computes `inv = 1 / state.l` and writes `out[d] = state.acc[d] * inv`. Do not normalize inside `online_update` — the denominator is only final after the last block.

TESTS

```
// tests/test_flash.cpp
#include "test_framework.hpp"
#include "llm/flash.hpp"
#include <vector>
#include <cmath>
using namespace llm;

// Reference: full stable softmax over scores, then weighted sum of values.
static std::vector<float> ref_attend(const std::vector<float>& s,
                                     const std::vector<float>& v, int N, int D) {
    float m = s[0]; for (int j = 1; j < N; ++j) m = std::max(m, s[j]);
    std::vector<float> w(N); float sum = 0.0f;
    for (int j = 0; j < N; ++j) { w[j] = std::exp(s[j] - m); sum += w[j]; }
    std::vector<float> out(D, 0.0f);
    for (int j = 0; j < N; ++j) for (int d = 0; d < D; ++d) out[d] += (w[j] / sum)
    ↪ * v[j * D + d];
    return out;
}

TEST(online_softmax_matches_full_one_block) {
    const int N = 6, D = 4;
    std::vector<float> s = {0.5f, -1.0f, 2.0f, 0.0f, 1.5f, -0.5f}, v(N * D);
    for (int i = 0; i < N * D; ++i) v[i] = 0.1f * i - 0.3f;
    OnlineSoftmax st(D);
```

```

    online_update(st, s.data(), N, v.data(), D);
    std::vector<float> out(D); online_finalize(st, out.data(), D);
    auto ref = ref_attend(s, v, N, D);
    for (int d = 0; d < D; ++d) CHECK_CLOSE(out[d], ref[d], 1e-5);
}

TEST(online_softmax_block_invariant) {
    const int N = 7, D = 3;
    std::vector<float> s = {1, -2, 3, 0, 2, -1, 4}, v(N * D);
    for (int i = 0; i < N * D; ++i) v[i] = 0.2f * i;
    OnlineSoftmax st(D);
    online_update(st, s.data(), 4, v.data(), D); // block 1
    online_update(st, s.data() + 4, 3, v.data() + 4 * D, D); // block 2
    std::vector<float> out(D); online_finalize(st, out.data(), D);
    auto ref = ref_attend(s, v, N, D);
    for (int d = 0; d < D; ++d) CHECK_CLOSE(out[d], ref[d], 1e-5); // same as
    ↪ one block
}

TEST(online_softmax_stable_on_large_scores) {
    const int N = 4, D = 2;
    std::vector<float> s = {1000.0f, 999.0f, -1000.0f, 1000.0f}, v(N * D);
    for (int i = 0; i < N * D; ++i) v[i] = float(i + 1);
    OnlineSoftmax st(D);
    online_update(st, s.data(), 2, v.data(), D);
    online_update(st, s.data() + 2, 2, v.data() + 2 * D, D);
    std::vector<float> out(D); online_finalize(st, out.data(), D);
    for (int d = 0; d < D; ++d) CHECK(std::isfinite(out[d])); // no overflow
    ↪
}

```

REFERENCE SOLUTION

include/llm/flash.hpp:

```

// include/llm/flash.hpp
#pragma once
#include <cmath>
#include <vector>
#include "llm/common.hpp"
#include "llm/model.hpp"

namespace llm {

```



```

struct OnlineSoftmax {
    float m;                // running max
    float l;                // running denominator
    std::vector<float> acc;   // running weighted output [head_dim]
    explicit OnlineSoftmax(int head_dim)
        : m(-INFINITY), l(0.0f), acc(head_dim, 0.0f) {}
};

// Fold a block of `block` scores (and their [block, head_dim] values) into the
// ↪ state.
inline void online_update(OnlineSoftmax& st, const float* scores, int block,
                        const float* values, int head_dim) {
    float bmax = -INFINITY;
    for (int j = 0; j < block; ++j) bmax = std::max(bmax, scores[j]);
    const float new_m = std::max(st.m, bmax);
    const float corr = std::exp(st.m - new_m);    // 0 on the first block (st.m
    ↪ = -inf)

    st.l *= corr;                                // rescale old state to new_m
    for (int d = 0; d < head_dim; ++d) st.acc[d] *= corr;

    for (int j = 0; j < block; ++j) {            // add this block
        const float p = std::exp(scores[j] - new_m);
        st.l += p;
        const float* vj = values + (int64_t)j * head_dim;
        for (int d = 0; d < head_dim; ++d) st.acc[d] += p * vj[d];
    }
    st.m = new_m;
}

inline void online_finalize(const OnlineSoftmax& st, float* out, int head_dim) {
    const float inv = 1.0f / st.l;
    for (int d = 0; d < head_dim; ++d) out[d] = st.acc[d] * inv;
}

} // namespace llm

```

The block-invariance test is the one that captures the magic: feeding the scores in two chunks produces the same answer as feeding them all at once, which is precisely what licenses tiling. The rescale-and-add recurrence is doing the book-keeping that a single global max and sum would otherwise require — only it does it on the fly, with a constant-size state. That is the whole trick; the next challenge points it at attention.

GOING FURTHER

- **Two-pass vs one-pass equivalence.** Prove on paper that the recurrence equals the two-pass softmax for any block schedule. The invariant “ (ℓ, acc) are correct relative to the current m ” is the key.
- **Fewer rescales.** If a later block’s max never exceeds the running max, corr is 1 and the rescale is a no-op — skip it. In practice the first block often sets a max the rest rarely beat.
- **Log-sum-exp output.** Track $m + \log \ell$ alongside the output to get the row’s normalizer cheaply — useful for speculative decoding (Unit 12) and for stitching attention across devices.

8.2 Tiled Flash Attention

GOAL

Point the online softmax at attention: for each query, stream the key/value blocks through the running softmax — computing block scores, folding them in, discarding the block — so the full score row never exists. The result is exact attention in a single fused, causal, grouped-query pass with block-sized memory.

BACKGROUND

Naive attention (Unit 5) does three things per query: form the whole score row $q \cdot K^\top$, softmax it, and weight the values. Flash attention does all three at once, block by block, using the online softmax. For a query q , walk the keys and values in **blocks**; for each block, compute that block’s scores ($q \cdot k$ for each key in the block, scaled by $1/\sqrt{\text{head_dim}}$), and `online_update` the running state with those scores and the block’s value rows. After the last block, `online_finalize` divides the accumulator by the denominator to give the query’s attention output. At no point does more than one block of scores exist — the $O(\text{seq})$ score row has become an $O(\text{block})$ working set.

This is exactly the tiling idea from Unit 3’s GEMM, now over attention. The full Flash Attention kernel tiles *both* dimensions — an outer loop over blocks of queries, an inner loop over blocks of keys — so that a query block and a key

block sit together in fast memory while their interaction is computed. On a CPU we can keep it simple: loop queries one at a time (each is independent and carries its own small online-softmax state) and tile only the keys, which is what bounds the memory. The structure generalizes directly to the two-level tiling a GPU wants.

The **causal mask** lives naturally in the tiling. A query at absolute position $qpos$ attends keys $0..qpos$, so the key-block loop runs from block 0 up to the block containing $qpos$, and the final (diagonal) block is **partially** included — only its keys up to $qpos$. Capping each block's end at $qpos + 1$ handles full blocks, the partial diagonal block, and the decode case (a single query attending all of history) uniformly. Key blocks entirely in the future are simply never visited. Grouped-query attention carries over unchanged: query head h reads key/value head h/group .

So the kernel is: project to Q, K, V and apply RoPE (as in Unit 5); then for each head and each query, run an online softmax over the causal key blocks, sourcing scores from $q \cdot k$ and values from the corresponding V rows; finalize; and project the concatenated heads through w_o . The output is bit-for-bit what naive attention computes (modulo floating-point summation order), which is what the boss will verify — but it is computed in one pass with a constant-size scratch.

THE TASK

Implement `flash_attention(0, Q, K, V, seq, n_heads, n_kv_heads, head_dim, pos0, scale, block)`: for each head ($k_v_h = h/\text{group}$) and each query i (absolute position $pos0+i$), run an `OnlineSoftmax`; loop key blocks kb from 0 to $pos0+i$ in steps of `block`, capping the block end at $pos0+i+1$; per block compute scaled scores and gather the block's value rows, then `online_update`; `online_finalize` into 0. Then a wrapper `attention_flash(out, x, w, cfg, pos0, block)` that projects, RoPEs, calls `flash_attention`, and projects through w_o — mirroring Unit 5's attention.

HINTS

Hint 1 — the key-block loop with causal cap

```
for (int kb = 0; kb <= qpos; kb += block) { int end =
    std::min(kb+block, qpos+1); int bn = end-kb; ... }. bn is the block's
live length — full for interior blocks, short for the diagonal one.
```

Hint 2 — gather the block's values for the online update

K and V are laid out [pos, kv_heads, head_dim], so a fixed kv_h's rows are strided. Compute each bscore[j] = scale * dot(qi, K row) and memcpy each value row into a contiguous bval[j*head_dim ..] buffer, then call online_update(st, bscore, bn, bval, head_dim).

Hint 3 — the wrapper is Unit 5's setup minus the score matrix

attention_flash projects q,k,v, RoPEs q (n_heads) and k (n_kv_heads) at pos0, calls flash_attention into a [seq, n_heads*head_dim] buffer, then linear(out, that, w.wo). The scratch (bscore, bval) is block-sized, independent of seq.

TESTS

```
// add to tests/test_flash.cpp (reuses randt, attn_weights from the model tests)
#include "llm/model.hpp"

TEST(flash_attention_matches_naive) {
    ModelConfig c; c.hidden_size = 16; c.n_heads = 4; c.n_kv_heads = 2; c.head_dim
    ↪ = 4;
    const LayerWeights w = attn_weights(c, 7);
    const int S = 9, hidden = c.hidden_size;
    Tensor x = randt({S, hidden}, 3);

    Tensor o_naive = Tensor::empty({S, hidden}, DType::F32);
    attention(o_naive, x, w, c, /*pos0=*/0); // Unit 5 reference

    Tensor o_flash = Tensor::empty({S, hidden}, DType::F32);
    attention_flash(o_flash, x, w, c, /*pos0=*/0, /*block=*/4);
    for (int i = 0; i < S * hidden; ++i)
        CHECK_CLOSE(o_naive.data_ptr<float>()[i], o_flash.data_ptr<float>()[i], 1e
    ↪ -4);
}

TEST(flash_attention_block_size_invariant) {
    ModelConfig c; c.hidden_size = 16; c.n_heads = 4; c.n_kv_heads = 4; c.head_dim
    ↪ = 4;
    const LayerWeights w = attn_weights(c, 12);
    const int S = 10, hidden = c.hidden_size;
    Tensor x = randt({S, hidden}, 6);
```

```

Tensor a = Tensor::empty({S, hidden}, DType::F32);
Tensor b = Tensor::empty({S, hidden}, DType::F32);
attention_flash(a, x, w, c, 0, /*block=*/2);
attention_flash(b, x, w, c, 0, /*block=*/8);
for (int i = 0; i < S * hidden; ++i)
    CHECK_CLOSE(a.data_ptr<float>()[i], b.data_ptr<float>()[i], 1e-5); //
    ↪ tiling is exact
}

```

REFERENCE SOLUTION

Declare in `model.hpp` (or `flash.hpp`), define in `model.cpp`:

```

// flash.hpp / model.hpp
void flash_attention(float* O, const float* Q, const float* K, const float* V,
                    int seq, int n_heads, int n_kv_heads, int head_dim,
                    int pos0, float scale, int block);
void attention_flash(Tensor& out, const Tensor& x, const LayerWeights& w,
                    const ModelConfig& cfg, int pos0 = 0, int block = 64);

```

```

// model.cpp
#include "llm/flash.hpp"
#include <algorithm>
#include <cstring>

void flash_attention(float* O, const float* Q, const float* K, const float* V,
                    int seq, int n_heads, int n_kv_heads, int head_dim,
                    int pos0, float scale, int block) {
    const int group = n_heads / n_kv_heads;
    std::vector<float> bscore(block);
    std::vector<float> bval((size_t)block * head_dim);

    for (int h = 0; h < n_heads; ++h) {
        const int kvh = h / group;
        for (int i = 0; i < seq; ++i) {
            const int qpos = pos0 + i;
            // attend keys 0..
            ↪ qpos

            const float* qi = Q + ((int64_t)i * n_heads + h) * head_dim;
            OnlineSoftmax st(head_dim);

            for (int kb = 0; kb <= qpos; kb += block) {

```

```

        const int end = std::min(kb + block, qpos + 1); // causal cap
        const int bn = end - kb;
        for (int j = 0; j < bn; ++j) {
            const float* kj = K + ((int64_t)(kb + j) * n_kv_heads + kvh) *
    ↪ head_dim;

            float dot = 0.0f;
            for (int d = 0; d < head_dim; ++d) dot += qi[d] * kj[d];
            bscore[j] = dot * scale;
            const float* vj = V + ((int64_t)(kb + j) * n_kv_heads + kvh) *
    ↪ head_dim;

            std::memcpy(bval.data() + (int64_t)j * head_dim, vj, sizeof(
    ↪ float) * head_dim);
        }
        online_update(st, bscore.data(), bn, bval.data(), head_dim);
    }
    online_finalize(st, 0 + ((int64_t)i * n_heads + h) * head_dim,
    ↪ head_dim);
}
}

void attention_flash(Tensor& out, const Tensor& x, const LayerWeights& w,
                    const ModelConfig& cfg, int pos0, int block) {
    const int S = static_cast<int>(x.shape()[0]);
    const int H = cfg.n_heads, KVH = cfg.n_kv_heads, D = cfg.head_dim;
    const int qd = H * D, kvd = KVH * D;
    const float scale = 1.0f / std::sqrt(static_cast<float>(D));

    Tensor q = Tensor::empty({S, qd}, DType::F32);
    Tensor k = Tensor::empty({S, kvd}, DType::F32);
    Tensor v = Tensor::empty({S, kvd}, DType::F32);
    linear(q, x, w.wq); linear(k, x, w.wk); linear(v, x, w.wv);
    apply_rope(q, S, H, D, cfg.rope_theta, pos0);
    apply_rope(k, S, KVH, D, cfg.rope_theta, pos0);

    Tensor concat = Tensor::empty({S, qd}, DType::F32);
    flash_attention(concat.data_ptr<float>(), q.data_ptr<float>(),
                   k.data_ptr<float>(), v.data_ptr<float>(),
                   S, H, KVH, D, pos0, scale, block);
    linear(out, concat, w.wo);
}

```

Flash attention now produces the same outputs as Unit 5’s attention — the equivalence test confirms it — but it never builds the score matrix, and its scratch (bscore and bval) is fixed at block rows no matter how long the sequence. The block-size-invariance test is the proof that the tiling is sound: the answer does

not depend on how finely you chop the keys, exactly as the online softmax guaranteed. The same `flash_attention` core works for cached decode — pass the cache’s K/V and `pos0 = cache.length` and a single query streams over all of history. The boss now pins down exactness at scale and the bounded memory that is the whole reason this exists.

GOING FURTHER

- **Two-level tiling.** Add an outer loop over query blocks so a query tile and a key tile are resident together — the structure of the real GPU kernel, and a cache win on CPU too.
- **SIMD the block.** The per-block score dot products and the accumulate are the hot loop; vectorize them with the `avx` kernels from Unit 3 for a large speedup.
- **Skip-on-no-rescale and partial blocks.** Combine the “`corr == 1`” shortcut from 8.1 with the causal diagonal block to minimize wasted work near the boundary.

8.3 Exactness at Scale

BOSS CHALLENGE

Prove the reformulation paid off without changing the answer: flash attention matches naive attention within floating-point tolerance across sequence lengths up to 256, the result is invariant to block size, and it *never* materializes the $O(\text{seq}^2)$ score matrix — its attention scratch is $O(\text{block} \cdot \text{head_dim})$, bounded independent of sequence length.

GOAL

Certify Flash Attention on the two axes that matter: **exactness** — it reproduces naive attention at every scale, since it is a reformulation and not an approximation — and **bounded memory** — its working set does not grow with sequence length, which is the entire reason it exists.

BACKGROUND

Flash Attention earns its place only if two claims hold together. The first is **exactness**. The online softmax is algebraically identical to the two-pass softmax, so flash attention should compute the same result as Unit 5’s naive attention — not approximately, but to within floating-point rounding. The summation *order* differs (flash accumulates block by block, with rescaling; naive accumulates left to right), so the two will not be bit-identical, but the difference must stay at the level of floating-point noise even as the sequence grows. The boss checks this at $\text{seq} = 1, 16, 64, 256$ — the decode case, short, medium, and long — and demands the maximum per-element difference stay below a tight tolerance throughout. If it does, flash attention is a true drop-in for the naive version anywhere in the engine.

The second claim is **bounded memory**, and it is the whole point. Naive attention’s cost at long context is dominated by the $[\text{seq} \times \text{seq}]$ score matrix it must store and stream through slow memory; at $\text{seq} = 256$ that is a 65,536-entry matrix *per head*, and it grows quadratically. Flash attention never forms it. Its attention-specific scratch — the block of scores and the block of gathered values — is sized by block and head_dim only, a constant that does not depend on seq. (The $O(\text{seq})$ buffers for the query, key, value, and output projections remain, of course; those are the inputs and outputs. What flash eliminates is the quadratic *intermediate*.) Running correctly at $\text{seq} = 256$ with the same small scratch as at $\text{seq} = 16$ is the observable form of this property.

A third check ties the two together: **block-size invariance**. Because the online softmax is exact for any block schedule, the result must not depend on how finely the keys are tiled — block 1, block 16, and block 128 must all agree. This is the proof that the tiling itself is sound: the performance knob (block size) is decoupled from the numerical answer, so an implementation can tune block for cache behavior without touching correctness. Together — exact at every scale, invariant to the tiling, bounded in memory — these are the certificate that Flash Attention delivers the same attention, cheaper.

THE TASK

Write the boss tests against the kernel from 8.2. (1) **Exactness at scale**: for $\text{seq} \in \{1, 16, 64, 256\}$, compute naive and flash attention on the same input and assert the maximum per-element difference is below tolerance. (2) **Block-size invariance**: at a fixed sequence length, assert flash attention agrees across several block sizes (including the extremes 1 and seq). Both run through the causal,

grouped-query path, so they also exercise the mask and GQA.

The bar (the boss): flash matches naive within tolerance at every tested length up to 256; the output is invariant to block size; and — by construction — no $O(\text{seq}^2)$ score matrix is ever allocated.

HINTS

Hint 1 — range over sequence lengths

for (int S : {1, 16, 64, 256}) { ... }. Build a fresh input per length, run both attentions, and track `max_abs = std::max(max_abs, fabs(naive[i] - flash[i]))` over all elements; assert it is below $1e-3$.

Hint 2 — a block-1 reference

For invariance, take `block = 1` as the reference (the finest possible tiling) and compare `block = 4, 16, 64, seq` against it. They should match to $1e-4$ or tighter.

Hint 3 — the memory claim is structural

You do not need to instrument the allocator: inspect `flash_attention` and confirm its only scratch is `bscore` (block) and `bval` (`block*head_dim`) — no `seq*seq` buffer anywhere. The `seq=256` run exercises exactly the regime where the naive matrix would be largest.

TESTS

```
// add to tests/test_flash.cpp (reuses randt, attn_weights)
#include <cmath>

TEST(boss_flash_exact_at_scale) {
    ModelConfig c; c.hidden_size = 16; c.n_heads = 4; c.n_kv_heads = 2; c.head_dim
    ↪ = 4;
    const LayerWeights w = attn_weights(c, 21);
    const int hidden = c.hidden_size;
    for (int S : {1, 16, 64, 256}) {
        Tensor x = randt({S, hidden}, 100u + (uint32_t)S);
        Tensor o_naive = Tensor::empty({S, hidden}, DType::F32);
```

```

    Tensor o_flash = Tensor::empty({S, hidden}, DType::F32);
    attention(o_naive, x, w, c, /*pos0=*/0);
    attention_flash(o_flash, x, w, c, /*pos0=*/0, /*block=*/32);
    double max_abs = 0.0;
    for (int i = 0; i < S * hidden; ++i)
        max_abs = std::max(max_abs,
                           (double)std::fabs(o_naive.data_ptr<float>()[i] - o_flash.
    ↪ data_ptr<float>()[i]));
        CHECK(max_abs < 1e-3); // exact within fp noise at
    ↪ every scale
    }
}

TEST(boss_flash_block_size_invariant) {
    ModelConfig c; c.hidden_size = 16; c.n_heads = 4; c.n_kv_heads = 4; c.head_dim
    ↪ = 4;
    const LayerWeights w = attn_weights(c, 31);
    const int S = 128, hidden = c.hidden_size;
    Tensor x = randt({S, hidden}, 7);

    Tensor ref = Tensor::empty({S, hidden}, DType::F32);
    attention_flash(ref, x, w, c, 0, /*block=*/1); // finest tiling as reference
    for (int blk : {4, 16, 64, 128}) {
        Tensor o = Tensor::empty({S, hidden}, DType::F32);
        attention_flash(o, x, w, c, 0, blk);
        for (int i = 0; i < S * hidden; ++i)
            CHECK_CLOSE(o.data_ptr<float>()[i], ref.data_ptr<float>()[i], 1e-4);
    }
}

```

REFERENCE SOLUTION

There is no new production code: the boss is cleared by the kernel of 8.2 behaving correctly at scale. The tests above are the deliverable — build and run; `boss_flash_exact_at_scale` must pass at every length including 256, and `boss_flash_block_size_invariant` must show the answer is independent of the tile size. If the scale test fails only at the largest length, suspect a numerical issue in the rescale (a missing block, or a tolerance set unrealistically tight); if the invariance test fails, the tiling or the causal cap is wrong.

What you have proven. Not “a faster attention,” but “the *same* attention, computed without the quadratic matrix”: flash attention reproduces naive attention to floating-point tolerance at sequence lengths up to 256, gives an answer that does not depend on the tiling, and does so with a working set fixed by the block

size rather than the sequence. That is the formal content of the most important systems result in modern LLM inference — and you reached it from the online-softmax exercise in Unit 2.

GOING FURTHER

- **Benchmark the traffic.** Count bytes moved (or wall-clock) for naive vs flash at growing seq; the gap widens with length as the quadratic matrix dominates the naive path. This is the measurement that motivated the whole unit.
- **Flash decode.** Wire flash_attention into attention_cached (pass the cache's K/V and `pos0 = cache.length`); a single decode query then streams over all of history with constant scratch — flash attention's biggest practical win.
- **Toward the GPU kernel.** Add the outer query-block loop and map blocks to SRAM; the CUDA version in Unit 15 is this same algorithm with the tiles placed by hand in fast memory.

Unit 8 — Milestone check

MILESTONE CHECK

The engine now computes attention the way real systems do:

- the **online softmax** — a softmax-weighted sum computed incrementally, with a constant-size state, exact for any block schedule;
- **tiled flash attention** that streams key/value blocks through it, computing exact causal grouped-query attention in one fused pass *without* the $O(\text{seq}^2)$ score matrix; and
- a **boss-grade certificate** of exactness at scale (matching naive up to $\text{seq} = 256$), block-size invariance, and bounded memory.

The model is now complete (Unit 5), scalable (Unit 6), efficient to generate with (Unit 7), and memory-light in its attention (Unit 8). One enormous lever remains untouched: **size**. Every weight is still a 32-bit float, so a model's memory footprint — and the bandwidth to stream it on every forward pass — is as large as it can be. **Unit 9** introduces **quantization**: storing weights in 4 or 8 bits with block-wise scales (the `q4_0` and `q8_0` formats), cutting memory and bandwidth several-fold with negligible quality loss — the optimization that lets a large model fit, and run, on ordinary hardware.

UNIT 9

Quantization

What you will build. The optimization that shrinks the model: **block-wise quantization**. You build the **Q8_0** format (8-bit weights with a per-block scale) and the **Q4_0** format (4-bit packed weights), plus **quantized linear** layers that multiply float activations by quantized weights. The boss measures the trade-off directly: memory drops by the format's factor while the quantized forward pass stays faithful to the float one.

Where this fits. Every weight so far has been a 32-bit float — four bytes each. A model with billions of parameters is therefore tens of gigabytes, too large for most hardware, and since a forward pass must *stream every weight from memory*, inference is bandwidth-bound: smaller weights are not just easier to store but faster to run. Quantization stores weights in 4 or 8 bits with block-local scales, cutting memory and bandwidth several fold at negligible quality cost. It is the single change that lets a large model fit, and run, on a laptop — and it is exactly what the GGUF files we load in Unit 10 contain.

The numbers make the case. At four bytes per weight, a seven-billion-parameter model occupies about 28 GB — more than the memory of most consumer GPUs, and a great deal to move across the memory bus on every single forward pass. Because inference reads the entire weight set per pass, it is **memory-bandwidth-bound**: the arithmetic units sit idle waiting for weights to arrive. Storing each weight in one byte (Q8_0) or half a byte (Q4_0) shrinks the footprint roughly four- or eightfold and cuts the bytes streamed by the same factor — a direct speedup on top of the memory saving.

The difficulty is that low-bit integers have almost no range — a 4-bit value holds sixteen levels — so a single scale across a whole matrix would crush precision. The fix, universal in practical formats, is **block-wise quantization**: split the

weights into small blocks (we use 32), and give *each block its own scale*. A block of tiny weights gets a fine scale; a block containing an outlier gets a coarse one — but that outlier no longer degrades the precision of every other block, as a global scale would. This local adaptivity is what makes aggressive quantization keep its quality. We build the two foundational symmetric formats from the `GGUF/llama.cpp` family — `q8_0` (near-lossless) and `q4_0` (the popular “small and good enough” default) — and the quantized matmul that puts them to work, with a boss that measures both halves of the bargain. Add `include/llm/quant.hpp`.

9.1 Block Quantization (Q8_0)

GOAL

The foundational format: quantize a run of float weights into 8-bit blocks, each a single **scale** plus 32 `int8` values, and dequantize back. Symmetric, block-local, and near-lossless — the template every other format refines.

BACKGROUND

`q8_0` is the simplest useful quantization, and understanding it gives you the pattern for all the rest. Take the weights in **blocks of 32**. Within a block, find the largest magnitude, $\text{absmax} = \max_i |w_i|$, and choose a **scale** that maps that magnitude to the top of the 8-bit signed range:

$$d = \frac{\text{absmax}}{127}.$$

Each weight is then the nearest integer multiple of d , stored as an `int8`:

$$q_i = \text{round}\left(\frac{w_i}{d}\right) \in [-127, 127], \quad w_i \approx q_i \cdot d.$$

This is **symmetric** quantization — the levels are centered at zero with no separate offset, which suits the roughly zero-mean, symmetric distribution of neural-network weights. Dequantization is the trivial inverse: multiply each stored integer by its block’s scale.

Two properties matter. The **error** is bounded by half a step: because each weight is rounded to the nearest multiple of d , $|w_i - q_i d| \leq d/2 = \text{absmax}/254$, tiny relative to the block’s range — which is why `q8_0` is effectively lossless in practice.

And the **block-local scale** is the whole point: a block of small weights gets a correspondingly small d and fine resolution, independent of what other blocks contain. Note also that the block's largest-magnitude weight round-trips *exactly* — it quantizes to ± 127 , which dequantizes back to $\pm \text{absmax}$.

The **storage** is one scale plus 32 bytes per block: $4 + 32 = 36$ bytes for 32 weights, or 1.125 bytes per weight — about $3.6\times$ smaller than float32. (Real GGUF stores the scale as a 16-bit float, giving 34 bytes and $\approx 3.8\times$; we use a 32-bit scale for clarity and note the difference.) The 4-bit format in 9.2 pushes this further; the structure is the same.

THE TASK

In `quant.hpp`, define `QK = 32`, a `Q8Block` (a float scale and `int8_t q[QK]`), `quantize_q8_0(w, n)` returning a vector of blocks (per block: compute `absmax`, `d = absmax/127`, and the clamped rounded `int8` quants), and `dequantize_q8_0(blocks, out)` writing `q[i]*scale`. (n is a multiple of `QK`.)

HINTS

Hint 1 — absmax then scale

Per block: `float amax = 0; for i: amax = std::max(amax, std::fabs(w[i]));` then `float d = amax / 127.0f;` and `float id = d != 0 ? 1.0f/d : 0.0f;` (guard the all-zero block).

Hint 2 — round, then clamp

`int q = (int)std::lround(w[i] * id); q = std::max(-127, std::min(127, q));` and store as `int8_t`. The clamp guards rounding that lands at ± 128 .

Hint 3 — dequant is one multiply

`out[b*QK + i] = blocks[b].q[i] * blocks[b].scale;` That is the entire decode path — the cheapness of dequantization is why quantized inference is fast.

TESTS

```
// tests/test_quant.cpp
#include "test_framework.hpp"
#include "llm/quant.hpp"
#include <vector>
#include <cmath>
using namespace llm;

TEST(q8_roundtrip_error_within_half_step) {
    const int64_t n = 64; // 2 blocks
    std::vector<float> w(n);
    uint32_t s = 1;
    for (auto& x : w) { s = s*1664525u+1013904223u; x = ((s>>9)*(1.0f/8388608.0f)
    ↪ -0.5f) * 3.0f; }
    auto blocks = quantize_q8_0(w.data(), n);
    std::vector<float> dq(n); dequantize_q8_0(blocks, dq.data());
    for (size_t b = 0; b < blocks.size(); ++b) {
        const float d = blocks[b].scale;
        for (int i = 0; i < QK; ++i)
            CHECK(std::fabs(w[b*QK + i] - dq[b*QK + i]) <= d * 0.5f + 1e-6f);
    }
}

TEST(q8_preserves_block_absmax) {
    std::vector<float> w(QK, 0.1f);
    w[5] = -2.0f; w[20] = 1.5f; // absmax = 2.0 at index 5
    auto blocks = quantize_q8_0(w.data(), QK);
    std::vector<float> dq(QK); dequantize_q8_0(blocks, dq.data());
    CHECK_CLOSE(blocks[0].scale, 2.0f / 127.0f, 1e-7);
    CHECK_CLOSE(dq[5], -2.0f, 1e-4); // largest magnitude is
    ↪ exact (q = -127)
}
```

REFERENCE SOLUTION

include/llm/quant.hpp:

```
// include/llm/quant.hpp
#pragma once
#include <cstdint>
#include <vector>
#include <cmath>
```

```

#include <algorithm>

namespace llm {

constexpr int QK = 32;                // block size

struct Q8Block {
    float scale;                       // d (GGUF stores this as f16)
    int8_t q[QK];
};

inline std::vector<Q8Block> quantize_q8_0(const float* w, int64_t n) {
    const int64_t nb = n / QK;
    std::vector<Q8Block> out(nb);
    for (int64_t b = 0; b < nb; ++b) {
        const float* blk = w + b * QK;
        float amax = 0.0f;
        for (int i = 0; i < QK; ++i) amax = std::max(amax, std::fabs(blk[i]));
        const float d = amax / 127.0f;
        const float id = (d != 0.0f) ? 1.0f / d : 0.0f;
        out[b].scale = d;
        for (int i = 0; i < QK; ++i) {
            int q = static_cast<int>(std::lround(blk[i] * id));
            q = std::max(-127, std::min(127, q));
            out[b].q[i] = static_cast<int8_t>(q);
        }
    }
    return out;
}

inline void dequantize_q8_0(const std::vector<Q8Block>& blocks, float* out) {
    for (size_t b = 0; b < blocks.size(); ++b) {
        const float d = blocks[b].scale;
        for (int i = 0; i < QK; ++i) out[b * QK + i] = blocks[b].q[i] * d;
    }
}

} // namespace llm

```

q8_0 is quantization stripped to its essentials: per-block absmax, one scale, rounded integers, and a one-multiply decode. The half-step error bound and the exact round-trip of the block maximum are the two facts to carry forward — they explain both why the format is nearly lossless and how the block-local scale protects precision. With the 8-bit case understood, the 4-bit format is the same idea with tighter packing, and then we make quantized weights actually

compute.

GOING FURTHER

- **f16 scales.** Store the scale as a 16-bit float (a small conversion) to match GGUF byte-for-byte and trim the per-block overhead — meaningful across millions of blocks.
- **Activation quantization.** Quantize activations to int8 as well, so a matmul can be done with integer dot products and a single float rescale at the end — the basis of the fast path in 9.2’s Going Further.
- **Asymmetric / zero-point.** For tensors that are not zero-centered, add an offset (zero-point) alongside the scale. Weights rarely need it; some activations do.

9.2 4-bit Packing and Quantized Linear

GOAL

Halve the bits again — **q4_0**, four bits per weight, two packed into each byte — and then make quantized weights actually compute: a **quantized linear** layer that multiplies float activations by quantized weights, dequantizing block by block so the big float matrix never exists.

BACKGROUND

q4_0 is q8_0’s idea at four bits. Same blocks of 32, same symmetric scale, but each weight is now one of sixteen levels. We use the signed range $[-8, 7]$, so the block’s scale maps its largest magnitude to 8:

$$d = \frac{\text{absmax}}{8}, \quad q_i = \text{round}\left(\frac{w_i}{d}\right) \in [-8, 7], \quad w_i \approx q_i d.$$

A 4-bit value cannot be stored directly, so we **pack** two per byte. Shift each quant into $[0, 15]$ by adding 8 (a nibble), and place two nibbles in one byte. Following the GGUF/llama.cpp convention, byte t holds weight t in its low nibble and weight $t + 16$ in its high nibble — the two halves of the 32-block interleaved — so 16 bytes hold all 32 nibbles. Dequantization unpacks each nibble, subtracts the 8 offset, and multiplies by d .

The trade-off is explicit. The error bound is now $d/2 = \text{absmax}/16$ — coarser than `q8_0`'s $\text{absmax}/254$ — so `q4_0` carries a small but real quality cost. In return, storage falls to $4 + 16 = 20$ bytes per 32 weights, 0.625 bytes each, about $6.4\times$ smaller than `float32` (and $\approx 7.1\times$ with an `f16` scale). That is why `q4_0` is the default people reach for: a large model that would not otherwise fit suddenly does, at a quality cost most users never notice.

But a format is only useful if you can compute with it, and the operation that matters is the **matmul** — it is where the weights are read, so it is where the bandwidth saving is realized. A **quantized linear** layer computes the same $y_{io} = \sum_k x_{ik} w_{ok}$ as before, except the weight rows are stored quantized. The clear implementation **dequantizes on the fly**: for each output, walk that row's blocks, and for each block multiply its dequantized weights by the corresponding slice of the activation, accumulating into the dot product. Crucially, the weights are dequantized in small blocks *inside* the loop and immediately consumed — the full float weight matrix is never materialized, so what streams from memory is the compact quantized data. (The genuinely fast path also quantizes the *activations* to `int8` and uses integer dot products; we note it in *Going Further* and keep the readable float version here.)

THE TASK

Add to `quant.hpp`: a `Q4Block (float scale, uint8_t q[QK/2]);` quantize_q4_0/dequantize_q4_0 with the nibble packing above; row-wise matrix quantizers `quantize_matrix_q4_0(w, N, K)` and `quantize_matrix_q8_0(w, N, K)` (each weight row $\rightarrow K/QK$ blocks, row-major); and quantized linears `linear_q4_0(Y, X, qw, S, N, K)` and `linear_q8_0(Y, X, qw, S, N, K)` computing XW^T with on-the-fly dequantization.

HINTS

Hint 1 — pack two nibbles

```
int q0 = lround(blk[i]*id)+8; int q1 = lround(blk[i+16]*id)+8;
clamp each to [0,15], then q[i] = (uint8_t)(q0 | (q1 << 4));. Unpack
with (byte & 0x0F)-8 and (byte >> 4)-8.
```

Hint 2 — a quantized matrix is rows of blocks

`quantize_matrix_*` quantizes each of the N rows (length K) independently and concatenates, so row o occupies blocks $[o*(K/QK), (o+1)*(K/QK))$. The linear indexes `qw.data() + o*(K/QK)` to find a row.

Hint 3 — dequantize inside the dot product

For output o , loop its blocks; for block b with scale d , take the matching QK -slice of x and accumulate $x[t]*(quant*d)$ for the 32 weights. No separate dequantized buffer — multiply as you unpack.

TESTS

```
// add to tests/test_quant.cpp
#include <stdint>

static float rms_rel(const float* a, const float* b, int n) {
    double num = 0, den = 0;
    for (int i = 0; i < n; ++i) { double e = a[i]-b[i]; num += e*e; den += (double)
    ↪ b[i]*b[i]; }
    return (float)std::sqrt(num / (den + 1e-12));
}

static void float_linear(float* Y, const float* X, const float* W, int S, int N,
    ↪ int K) {
    for (int i = 0; i < S; ++i) for (int o = 0; o < N; ++o) {
        float a = 0; for (int k = 0; k < K; ++k) a += X[i*K+k] * W[o*K+k];
        Y[i*N+o] = a;
    }
}

TEST(q4_roundtrip_error_within_one_step) {
    const int64_t n = 64;
    std::vector<float> w(n);
    uint32_t s = 9; for (auto& x : w) { s = s*1664525u+1013904223u; x = ((s>>9)
    ↪ *(1.0f/8388608.0f)-0.5f)*2.0f; }
    auto blocks = quantize_q4_0(w.data(), n);
    std::vector<float> dq(n); dequantize_q4_0(blocks, dq.data());
    for (size_t b = 0; b < blocks.size(); ++b) {
        const float d = blocks[b].scale;
        for (int i = 0; i < QK; ++i)
            CHECK(std::fabs(w[b*QK + i] - dq[b*QK + i]) <= d + 1e-6f); // 4-bit
    ↪ on [-8,7]: worst case one step
    }
```

```

    }
}

TEST(quantized_linear_q8_close_to_float) {
    const int S = 2, N = 4, K = 64; // K multiple of QK
    std::vector<float> w(N*K), x(S*K);
    uint32_t s = 3;
    for (auto& z : w) { s = s*1664525u+1013904223u; z = (s>>9)*(1.0f/8388608.0f)
    ↪ -0.5f; }
    for (auto& z : x) { s = s*1664525u+1013904223u; z = ((s>>9)*(1.0f/8388608.0f)
    ↪ -0.5f)*0.8f; }
    std::vector<float> yf(S*N); float_linear(yf.data(), x.data(), w.data(), S, N, K
    ↪ );
    auto qw = quantize_matrix_q8_0(w.data(), N, K);
    std::vector<float> yq(S*N); linear_q8_0(yq.data(), x.data(), qw, S, N, K);
    CHECK(rms_rel(yq.data(), yf.data(), S*N) < 0.02f); // Q8 near-lossless
}

TEST(quantized_linear_q4_close_to_float) {
    const int S = 2, N = 4, K = 64;
    std::vector<float> w(N*K), x(S*K);
    uint32_t s = 5;
    for (auto& z : w) { s = s*1664525u+1013904223u; z = (s>>9)*(1.0f/8388608.0f)
    ↪ -0.5f; }
    for (auto& z : x) { s = s*1664525u+1013904223u; z = ((s>>9)*(1.0f/8388608.0f)
    ↪ -0.5f)*0.8f; }
    std::vector<float> yf(S*N); float_linear(yf.data(), x.data(), w.data(), S, N, K
    ↪ );
    auto qw = quantize_matrix_q4_0(w.data(), N, K);
    std::vector<float> yq(S*N); linear_q4_0(yq.data(), x.data(), qw, S, N, K);
    CHECK(rms_rel(yq.data(), yf.data(), S*N) < 0.12f); // Q4: small but real
    ↪ error
}

```

REFERENCE SOLUTION

Add to quant.hpp:

```

// quant.hpp
struct Q4Block {
    float scale; // d
    uint8_t q[QK / 2]; // byte t: weight t (low nibble), weight t
    ↪ +16 (high nibble)
};

```

```

inline std::vector<Q4Block> quantize_q4_0(const float* w, int64_t n) {
    const int64_t nb = n / QK;
    std::vector<Q4Block> out(nb);
    for (int64_t b = 0; b < nb; ++b) {
        const float* blk = w + b * QK;
        float amax = 0.0f;
        for (int i = 0; i < QK; ++i) amax = std::max(amax, std::fabs(blk[i]));
        const float d = amax / 8.0f;
        const float id = (d != 0.0f) ? 1.0f / d : 0.0f;
        out[b].scale = d;
        for (int i = 0; i < QK / 2; ++i) {
            int q0 = static_cast<int>(std::lround(blk[i] * id)) + 8;
            int q1 = static_cast<int>(std::lround(blk[i + QK / 2] * id)) + 8;
            q0 = std::max(0, std::min(15, q0));
            q1 = std::max(0, std::min(15, q1));
            out[b].q[i] = static_cast<uint8_t>(q0 | (q1 << 4));
        }
    }
    return out;
}

inline void dequantize_q4_0(const std::vector<Q4Block>& blocks, float* out) {
    for (size_t b = 0; b < blocks.size(); ++b) {
        const float d = blocks[b].scale;
        for (int i = 0; i < QK / 2; ++i) {
            const uint8_t byte = blocks[b].q[i];
            out[b * QK + i] = (static_cast<int>(byte & 0x0F) - 8) * d;
            out[b * QK + i + QK / 2] = (static_cast<int>(byte >> 4) - 8) * d;
        }
    }
}

```

```

// quant.hpp -- row-wise matrix quantizers and quantized linears
template <class QuantFn>
inline auto quantize_matrix(const float* w, int N, int K, QuantFn qfn) {
    using Block = typename decltype(qfn(w, (int64_t)K))::value_type;
    std::vector<Block> out; out.reserve((size_t)N * (K / QK));
    for (int o = 0; o < N; ++o) {
        auto row = qfn(w + (int64_t)o * K, K);
        out.insert(out.end(), row.begin(), row.end());
    }
    return out;
}

```

```

inline std::vector<Q4Block> quantize_matrix_q4_0(const float* w, int N, int K) {
    return quantize_matrix(w, N, K, [](const float* p, int64_t n){ return
        ↪ quantize_q4_0(p, n); });
}

inline std::vector<Q8Block> quantize_matrix_q8_0(const float* w, int N, int K) {
    return quantize_matrix(w, N, K, [](const float* p, int64_t n){ return
        ↪ quantize_q8_0(p, n); });
}

inline void linear_q8_0(float* Y, const float* X, const std::vector<Q8Block>& qw,
    int S, int N, int K) {
    const int bpr = K / QK;
    for (int i = 0; i < S; ++i) {
        const float* xi = X + (int64_t)i * K;
        for (int o = 0; o < N; ++o) {
            const Q8Block* row = qw.data() + (int64_t)o * bpr;
            float acc = 0.0f;
            for (int b = 0; b < bpr; ++b) {
                const float d = row[b].scale; const float* xk = xi + b * QK;
                for (int t = 0; t < QK; ++t) acc += xk[t] * (row[b].q[t] * d);
            }
            Y[(int64_t)i * N + o] = acc;
        }
    }
}

inline void linear_q4_0(float* Y, const float* X, const std::vector<Q4Block>& qw,
    int S, int N, int K) {
    const int bpr = K / QK;
    for (int i = 0; i < S; ++i) {
        const float* xi = X + (int64_t)i * K;
        for (int o = 0; o < N; ++o) {
            const Q4Block* row = qw.data() + (int64_t)o * bpr;
            float acc = 0.0f;
            for (int b = 0; b < bpr; ++b) {
                const float d = row[b].scale; const float* xk = xi + b * QK;
                for (int t = 0; t < QK / 2; ++t) {
                    const uint8_t byte = row[b].q[t];
                    acc += xk[t] * ((static_cast<int>(byte & 0x0F) - 8) *
        ↪ d);
                    acc += xk[t + QK / 2] * ((static_cast<int>(byte >> 4) - 8) *
        ↪ d);
                }
            }
            Y[(int64_t)i * N + o] = acc;
        }
    }
}

```

```
}
```

Quantized weights now compute. The two fidelity tests draw the line between the formats: q8_0 reproduces the float matmul to a fraction of a percent, while q4_0's coarser steps cost a few percent — exactly the near-lossless-versus-good-enough distinction, now measured. And note what the linears do *not* do: they never build the float weight matrix, so the only weight bytes touched are the compact quantized ones, which is where the speed comes from. The boss puts both properties together on a realistic projection and certifies the trade.

GOING FURTHER

- **Integer dot products.** Quantize the activation slice to int8 per block, accumulate int32 dot products of quants, and rescale once by $d_x d_w$. With AVX-VNNI this is a multiple-times speedup over the float path — the real reason quantized inference is fast.
- **k-quants.** Implement a q4_k-style format: a super-block with a coarse scale and per-sub-block fine scales, squeezing more quality from the same bit budget. This is what most modern GGUF files actually use.
- **Mixed precision.** Keep sensitive tensors (embeddings, the output projection, attention norms) at higher precision while quantizing the bulk FFN weights hardest — the standard recipe for minimizing quality loss.

9.3 The Compression–Fidelity Trade

BOSS CHALLENGE

Measure both sides of the bargain on a realistic projection: memory drops by the format's factor ($\geq 3.5\times$ for q8_0, $\geq 6\times$ for q4_0), the quantized matmul stays faithful to float (q8_0 within a fraction of a percent, q4_0 within a few), and — the decisive test for a language model — the quantized output **predicts the same token**.

GOAL

Certify quantization the way it is actually judged: how much memory it saves, and whether the model still behaves. Measure the compression ratio exactly, bound the quantized-versus-float error for both formats, and prove that the quantized output projection picks the **same top token** as the float one.

BACKGROUND

Quantization is a trade, and a fair verdict measures both sides. The **compression** side is exact and computable: a float matrix is 4 bytes per weight, while q8_0 packs 32 weights into a 36-byte block (1.125 bytes each, $\approx 3.6\times$) and q4_0 into a 20-byte block (0.625 bytes each, $\approx 6.4\times$). Computing these ratios from the actual block sizes confirms the format delivers the footprint it promises — the reason a model fits at all.

The **fidelity** side is where judgement is needed, and the right metric is not generic numerical error but *model behavior*. A language model's forward pass ends in a projection to vocabulary logits, and generation acts on the **ranking** of those logits — greedy decoding takes the argmax, sampling draws from the induced distribution. So the question that matters is not “how far did the logits move?” but “did the prediction change?” A few percent of logit error that never reorders the top token is, behaviorally, no error at all — which is precisely why q4_0 is so widely used despite its coarser steps. The boss therefore measures the q8_0 and q4_0 error against float (expecting near-lossless and few-percent respectively) *and* checks that both formats, applied to the output projection, select the same top token as float.

To make the prediction test meaningful, we set it up with a clear winner — one output whose weight is aligned with the activation, so float ranks it first by a comfortable margin — and confirm that quantizing the weights, in either format, preserves that ranking. This is the formal content of the claim that has driven quantization into ubiquity: **several-fold less memory, and the model still says the same thing**. Pass this, and quantization is not a lossy compromise to apologize for but a free-seeming win to take.

THE TASK

Write the boss tests. (1) **Compression**: quantize an $[N, K]$ matrix to both formats and assert `float_bytes / q8_bytes >= 3.5` and `float_bytes / q4_bytes >= 6.0` (using `sizeof(Q8Block) / sizeof(Q4Block)` per block). (2) **Fidelity + predic-**

tion: build a [vocab, hidden] projection with one output row aligned to a chosen activation (a clear winner); compute float, q8_0, and q4_0 logits; assert the float argmax is the winner, both quantized argmaxes match it, and the q8/q4 relative errors are within 0.01/0.12.

The bar (the boss): compression ratios met; q8_0 within 1% and q4_0 within 12% of float; and both quantized projections predict the same top token as float.

HINTS

Hint 1 — exact byte accounting

```
float_bytes = (size_t)N*K*sizeof(float);          q8_bytes =
q8.size()*sizeof(Q8Block);    q4_bytes = q4.size()*sizeof(Q4Block);.
The block structs have no padding (36 and 20 bytes), so the ratios come
out near 3.56 and 6.4.
```

Hint 2 — engineer a clear winner

Pick winner and set that weight row to $x[k] * 2$ so its dot with x is $2\|x\|^2$ — comfortably the largest. Quantizing scales the row block-wise but preserves its direction, so the winner survives in both formats.

Hint 3 — argmax over the logits

```
int arg = 0; for (o = 1..vocab) if (l[o] > l[arg]) arg = o;.    Com-
pute it for the float, q8, and q4 logit vectors and compare to winner.
```

TESTS

```
// add to tests/test_quant.cpp (reuses rms_rel, float_linear)

TEST(boss_quantization_compression_ratio) {
    const int N = 64, K = 128;
    std::vector<float> w(N * K);
    uint32_t s = 1; for (auto& z : w) { s = s*1664525u+1013904223u; z = (s>>9)
    ↪ *(1.0f/8388608.0f)-0.5f; }
    auto q8 = quantize_matrix_q8_0(w.data(), N, K);
    auto q4 = quantize_matrix_q4_0(w.data(), N, K);
    const double fb = (double)N * K * sizeof(float);
```

```

    const double q8b = (double)q8.size() * sizeof(Q8Block);
    const double q4b = (double)q4.size() * sizeof(Q4Block);
    CHECK(fb / q8b >= 3.5); // ~3.56x
    CHECK(fb / q4b >= 6.0); // ~6.4x
}

TEST(boss_quantized_projection_preserves_prediction) {
    const int vocab = 48, hidden = 64; // hidden is a multiple of QK
    std::vector<float> w(vocab * hidden), x(hidden);
    uint32_t s = 11;
    for (auto& z : w) { s = s*1664525u+1013904223u; z = (s>>9)*(1.0f/8388608.0f)
    ↪ -0.5f; }
    for (auto& z : x) { s = s*1664525u+1013904223u; z = (s>>9)*(1.0f/8388608.0f)
    ↪ -0.5f; }
    const int winner = 7; // make row 7 the clear best
    ↪ match
    for (int k = 0; k < hidden; ++k) w[winner * hidden + k] = x[k] * 2.0f;

    std::vector<float> lf(vocab); float_linear(lf.data(), x.data(), w.data(), 1,
    ↪ vocab, hidden);
    auto q8 = quantize_matrix_q8_0(w.data(), vocab, hidden);
    auto q4 = quantize_matrix_q4_0(w.data(), vocab, hidden);
    std::vector<float> l8(vocab), l4(vocab);
    linear_q8_0(l8.data(), x.data(), q8, 1, vocab, hidden);
    linear_q4_0(l4.data(), x.data(), q4, 1, vocab, hidden);

    auto argmax = [&](const std::vector<float>& l) {
        int a = 0; for (int o = 1; o < vocab; ++o) if (l[o] > l[a]) a = o; return
    ↪ a;
    };
    CHECK(argmax(lf) == winner); // float ranks the winner first
    CHECK(argmax(l8) == winner); // Q8 preserves the prediction
    CHECK(argmax(l4) == winner); // Q4 preserves it too
    CHECK(rms_rel(l8.data(), lf.data(), vocab) < 0.01f);
    CHECK(rms_rel(l4.data(), lf.data(), vocab) < 0.12f);
}

```

REFERENCE SOLUTION

There is no new production code: the boss is cleared by the formats and quantized linears of 9.1–9.2. The tests are the deliverable. Build and run — the compression ratios land near $3.56\times$ and $6.4\times$, `q8_0` tracks float to well under a percent, `q4_0` to a few percent, and *both* pick the same top token as the float projection. If a prediction test fails, the winner’s margin was too thin or the packing/unpacking is mismatched; if a ratio falls short, check for struct padding inflating the block

size.

What you have proven. Not “smaller weights,” but the full bargain measured: storing weights in 8 or 4 bits shrinks them by roughly $3.6\times$ or $6.4\times$, the quantized matmul stays close to float (near-lossless at 8 bits, a controlled few percent at 4), and — the property that actually matters — the model **predicts the same token**. That is why quantization is not an optional nicety but the default way large models are shipped and run: it makes them fit, makes them faster to stream, and changes what they say not at all.

GOING FURTHER

- **Perplexity, not just argmax.** Measure the KL divergence between the float and quantized output distributions over many inputs — the finer-grained quality metric the format designers optimize.
- **Quantize the whole model.** Replace every linear in the forward pass with its quantized counterpart and compare end-to-end logits and generated text; budget which tensors stay high-precision (Unit 9b’s mixed precision).
- **Dequant throughput.** Time the quantized matmul against float at realistic sizes; the win appears once the data no longer fits in cache and bandwidth dominates — the regime real models live in.

Unit 9 — Milestone check

MILESTONE CHECK

The engine can now store and compute with quantized weights:

- **q8_0** — 8-bit, per-block symmetric, near-lossless, $\approx 3.6\times$ smaller;
- **q4_0** — 4-bit packed, two nibbles per byte, $\approx 6.4\times$ smaller at a few percent error;
- **quantized linear** layers that multiply float activations by quantized weights with on-the-fly dequantization, never materializing the float matrix; and
- a **boss-grade measurement** of the trade: the compression is real, the fidelity is high, and the model *predicts the same token*.

The engine is now complete (Unit 5), scalable (Unit 6), efficient (Units 7–8), and

compact (Unit 9) — yet it has never read a real model. Every test so far has run on random weights. **Unit 10** changes that: it parses **GGUF**, the `llama.cpp` model format, reading the configuration, the tokenizer’s vocabulary and merges, and the quantized tensors we just learned to decode — and loads an actual Llama or Qwen model into the structures we built. It is the moment the engine stops running on noise and starts running a real, trained network — the threshold of a living inference engine.

UNIT 10

Model Loading

What you will build. The bridge from a file on disk to a running network: a **GGUF parser**. You read the **header** and **metadata** (the architecture’s hyperparameters and the tokenizer’s vocabulary and merges), the **tensor descriptors** (each weight’s name, shape, type, and offset), **memory-map** the file so weights page in on demand, and **assemble** a real model into the `ModelConfig`, `Tokenizer`, and `ModelWeights` you built — then run it. The boss writes a model to GGUF, loads it back, and proves the round-trip is faithful.

Where this fits. Every test so far has run on random weights. This unit is where the engine stops running on noise and starts running a *trained* network. GGUF is the format of the `llama.cpp` ecosystem — the way Llama, Qwen, Mistral, and thousands of other models are distributed — and parsing it connects everything you have built (tensors, the tokenizer, the Transformer, quantization) to actual model files.

A GGUF file is a self-contained description of a model: in one binary blob it carries the **architecture configuration** (hidden size, layer count, head counts, and the rest), the **tokenizer** (the vocabulary and the BPE merges), and all the **weight tensors** — usually quantized in the formats from Unit 9. The layout is four sections in order: a small **header** (a magic number, a version, and how many tensors and metadata entries follow); a **metadata** section of typed key–value pairs holding the config and tokenizer; a list of **tensor descriptors** (each tensor’s name, dimensions, type, and byte offset); and finally the **tensor data** itself, aligned and packed.

The loader’s task is to walk those sections and reconstitute the model in the structures we already have. Read the metadata into a typed map and pull out the hyperparameters and tokenizer; read the tensor descriptors into a table keyed by

name; **memory-map** the file so the multi-gigabyte weight data is paged in by the operating system on demand rather than copied wholesale into RAM; and for each weight the model needs, find its bytes by name and decode them (dequantizing per Unit 9) into the right slot. Then the forward pass from Unit 5, the cache from Unit 7, and the quantized math from Unit 9 run a real network. We build it in three steps — header and metadata, tensors and mapping, then the full assemble-and-run — with a boss that certifies the parse by round-tripping a model through it. Add `include/llm/gguf.hpp`.

10.1 Reading GGUF: Header and Metadata

GOAL

Parse the front of the file: verify the **header** (magic, version, counts) and read the **metadata** key-value section — typed values including integers, floats, strings, and arrays — into a map the rest of the loader can query.

BACKGROUND

GGUF begins with a fixed **header**: a 4-byte magic number spelling GGUF (the little-endian value `0x46554747`), a 32-bit **version** (currently 3), and two 64-bit counts — the number of **tensors** and the number of **metadata** key-value pairs that follow. Verifying the magic is how you reject a file that is not GGUF before reading further; the counts tell you how many entries each subsequent section holds.

Then comes the **metadata**: `metadata_kv_count` entries, each a **key** followed by a typed **value**. A key is a length-prefixed string — a 64-bit byte length, then that many UTF-8 bytes (no terminator). A value is a 32-bit **type tag** followed by the value's bytes in a format determined by the tag. The scalar types are the obvious ones — unsigned and signed 8/16/32/64-bit integers, 32- and 64-bit floats, a bool, and a string (itself length-prefixed). The one compound type is **array**: its payload is an element type tag, a 64-bit count, and then that many elements of that type. Arrays are how the tokenizer ships — the vocabulary is a string array, the token scores a float array — so handling them is essential.

Everything is **little-endian** (the format's convention, and a match for x86 and ARM); a big-endian host would need byte swaps, which we note and skip. The cleanest way to read this is a **byte cursor** over the file: a pointer that a templated

`read<T>()` advances by `sizeof(T)` after copying out a value, with a `read_string()` that reads a length then that many bytes. With those two primitives, header and metadata parsing is a short, direct transcription of the layout — read the header fields, then loop the key–value pairs, reading each key string and then its typed value into a `MetaValue` stored under that key.

THE TASK

In `gguf.hpp`, define the type tags (`GGUF_U32`, `GGUF_F32`, `GGUF_STRING`, `GGUF_ARRAY`, etc.), a `MetaValue` (a tag plus storage for an integer, a float, a string, and the array variants), a `ByteCursor` with `read<T>()` and `read_string()`, and a `read_value(cursor)` that reads one typed value (handling arrays by reading element type, count, then elements). Then a `GGUFReader` that, given the file bytes, checks the magic and version, reads the two counts, and loads all metadata pairs into a `map<string, MetaValue>`, with typed accessors (`meta_u32`, `meta_f32`, `meta_str`, `meta_str_array`).

HINTS

Hint 1 — the byte cursor

```
template<class T> T read() { T v; std::memcpy(&v, p, sizeof(T)); p
+= sizeof(T); return v; } and std::string read_string() { uint64_t
n = read<uint64_t>(); std::string s((const char*)p, n); p += n;
return s; }. Copy, then advance — never dereference past end.
```

Hint 2 — `read_value` dispatches on the tag

Read the `uint32_t` tag; for scalars, `read<T>()` the matching width into the `MetaValue`; for `GGUF_STRING`, `read_string()`; for `GGUF_ARRAY`, read an element tag and a `uint64_t` count, then loop reading that many elements of the element type into the right vector.

Hint 3 — store integers uniformly

Keep all integer scalars in one `uint64_t` field (sign-extending signed types into it) so accessors are simple; keep floats in a `double`. The accessors cast on the way out (`meta_u32` returns `(uint32_t)value.u`).

TESTS

```

// tests/test_gguf.cpp
#include "test_framework.hpp"
#include "llm/gguf.hpp"
#include <vector>
#include <cstring>
using namespace llm;

// Helpers to build a little-endian GGUF blob by hand.
static void put_u32(std::vector<uint8_t>& b, uint32_t v) { for (int i=0;i<4;++i) b.
    ↪ push_back(v>>(8*i)); }
static void put_u64(std::vector<uint8_t>& b, uint64_t v) { for (int i=0;i<8;++i) b.
    ↪ push_back(v>>(8*i)); }
static void put_f32(std::vector<uint8_t>& b, float f) { uint32_t u; std::memcpy
    ↪ (&u,&f,4); put_u32(b,u); }
static void put_str(std::vector<uint8_t>& b, const std::string& s) {
    put_u64(b, s.size()); b.insert(b.end(), s.begin(), s.end());
}

TEST(gguf_pares_header_and_scalars) {
    std::vector<uint8_t> b;
    put_u32(b, 0x46554747); // magic "GGUF"
    put_u32(b, 3); // version
    put_u64(b, 0); // tensor_count
    put_u64(b, 2); // metadata_kv_count
    put_str(b, "llama.block_count"); put_u32(b, GGUF_U32); put_u32(b, 24);
    put_str(b, "llama.rope.freq_base"); put_u32(b, GGUF_F32); put_f32(b, 10000.0f);
    ↪

    GGUFReader r;
    REQUIRE(r.parse(b.data(), b.size()));
    CHECK(r.meta_u32("llama.block_count") == 24);
    CHECK_CLOSE(r.meta_f32("llama.rope.freq_base"), 10000.0f, 1e-3);
}

TEST(gguf_pares_string_array) {
    std::vector<uint8_t> b;
    put_u32(b, 0x46554747); put_u32(b, 3); put_u64(b, 0); put_u64(b, 1);
    put_str(b, "tokenizer.ggml.tokens");
    put_u32(b, GGUF_ARRAY); put_u32(b, GGUF_STRING); put_u64(b, 3);
    put_str(b, "<bos>"); put_str(b, "he"); put_str(b, "llo");

    GGUFReader r;
    REQUIRE(r.parse(b.data(), b.size()));
    const auto& toks = r.meta_str_array("tokenizer.ggml.tokens");

```



```

    REQUIRE(toks.size() == 3);
    CHECK(toks[0] == "<bos>");
    CHECK(toks[2] == "llo");
}

```

REFERENCE SOLUTION

include/llm/gguf.hpp (header and metadata parts):

```

// include/llm/gguf.hpp
#pragma once
#include <stdint>
#include <cstring>
#include <string>
#include <vector>
#include <unordered_map>
#include "llm/common.hpp"

namespace llm {

enum GGUFType : uint32_t {
    GGUF_U8=0, GGUF_I8=1, GGUF_U16=2, GGUF_I16=3, GGUF_U32=4, GGUF_I32=5,
    GGUF_F32=6, GGUF_BOOL=7, GGUF_STRING=8, GGUF_ARRAY=9, GGUF_U64=10,
    GGUF_I64=11, GGUF_F64=12
};

struct MetaValue {
    GGUFType type = GGUF_U32;
    uint64_t u = 0;           // any integer/bool (signed sign-extended)
    double f = 0.0;           // any float
    std::string str;
    GGUFType arr_type = GGUF_U32;
    std::vector<uint64_t> arr_u;
    std::vector<double> arr_f;
    std::vector<std::string> arr_s;
};

struct ByteCursor {
    const uint8_t* p;
    const uint8_t* end;
    template <class T> T read() { T v; std::memcpy(&v, p, sizeof(T)); p += sizeof(T); return v; }
    std::string read_string() {
        const uint64_t n = read<uint64_t>();

```

```

        std::string s(reinterpret_cast<const char*>(p), reinterpret_cast<const char
        ↪ *>(p) + n);
        p += n; return s;
    }
};

inline void read_scalar(ByteCursor& c, GGUFType t, MetaValue& v) {
    switch (t) {
        case GGUF_U8: v.u = c.read<uint8_t>(); break;
        case GGUF_I8: v.u = (uint64_t)(int64_t)c.read<int8_t>(); break;
        case GGUF_U16: v.u = c.read<uint16_t>(); break;
        case GGUF_I16: v.u = (uint64_t)(int64_t)c.read<int16_t>(); break;
        case GGUF_U32: v.u = c.read<uint32_t>(); break;
        case GGUF_I32: v.u = (uint64_t)(int64_t)c.read<int32_t>(); break;
        case GGUF_U64: v.u = c.read<uint64_t>(); break;
        case GGUF_I64: v.u = (uint64_t)c.read<int64_t>(); break;
        case GGUF_F32: v.f = c.read<float>(); break;
        case GGUF_F64: v.f = c.read<double>(); break;
        case GGUF_BOOL: v.u = c.read<uint8_t>(); break;
        case GGUF_STRING: v.str = c.read_string(); break;
        default: break;
    }
}

inline MetaValue read_value(ByteCursor& c) {
    MetaValue v; v.type = static_cast<GGUFType>(c.read<uint32_t>());
    if (v.type == GGUF_ARRAY) {
        v.arr_type = static_cast<GGUFType>(c.read<uint32_t>());
        const uint64_t n = c.read<uint64_t>();
        for (uint64_t i = 0; i < n; ++i) {
            MetaValue e; read_scalar(c, v.arr_type, e);
            if (v.arr_type == GGUF_STRING) v.arr_s.push_back
            ↪ (e.str);
            else if (v.arr_type == GGUF_F32 || v.arr_type == GGUF_F64) v.arr_f.
            ↪ push_back(e.f);
            else v.arr_u.push_back
            ↪ (e.u);
        }
    } else {
        read_scalar(c, v.type, v);
    }
    return v;
}

class GGUFReader {
public:
    bool parse(const uint8_t* data, size_t size) { // parse an in-memory

```

```

↪ blob
    ByteCursor c{data, data + size};
    if (c.read<uint32_t>() != 0x46554747u) return false;    // magic
    version_ = c.read<uint32_t>();
    n_tensors_ = c.read<uint64_t>();
    const uint64_t n_kv = c.read<uint64_t>();
    for (uint64_t i = 0; i < n_kv; ++i) {
        std::string key = c.read_string();
        meta_[key] = read_value(c);
    }
    cursor_after_meta_ = c.p;                                // tensors continue
↪ here (Unit 10.2)
    base_ = data;
    return true;
}

uint32_t meta_u32(const std::string& k, uint32_t def = 0) const {
    auto it = meta_.find(k); return it == meta_.end() ? def : (uint32_t)it->
↪ second.u;
}
float meta_f32(const std::string& k, float def = 0.0f) const {
    auto it = meta_.find(k); return it == meta_.end() ? def : (float)it->
↪ second.f;
}
std::string meta_str(const std::string& k) const {
    auto it = meta_.find(k); return it == meta_.end() ? std::string() : it->
↪ second.str;
}
const std::vector<std::string>& meta_str_array(const std::string& k) const {
    static const std::vector<std::string> empty;
    auto it = meta_.find(k); return it == meta_.end() ? empty : it->second.
↪ arr_s;
}

protected:
    std::unordered_map<std::string, MetaValue> meta_;
    const uint8_t* base_ = nullptr;
    const uint8_t* cursor_after_meta_ = nullptr;
    uint32_t version_ = 0;
    uint64_t n_tensors_ = 0;
};

} // namespace llm

```

The header is a sanity gate and the metadata is the model's specification sheet — read into a typed map, it is everything the engine needs to *configure* itself, short

of the weights. The byte cursor keeps the parsing honest: every field is read at its exact width and the cursor advances in lockstep with the file layout, which is the whole discipline of binary parsing. Next we read where the weights live and map the file into memory.

GOING FURTHER

- **Bounds checking.** Validate every read against end and reject truncated files with a clear error — real loaders must survive corrupt or malicious inputs without reading out of bounds.
- **Big-endian hosts.** Add byte-swapping in `read<T>()` when the host is big-endian, so the same parser works everywhere.
- **Dump tool.** Write a tiny utility that prints every metadata key and value — invaluable for inspecting an unfamiliar model file and discovering its exact key names.

10.2 Tensor Descriptors and Memory Mapping

GOAL

Find where the weights live and make them accessible: read the **tensor descriptors** (name, dimensions, type, offset), compute the aligned **data section**, **memory-map** the file, and decode any tensor by name into a float Tensor — dequantizing q4_0/q8_0 per Unit 9.

BACKGROUND

After the metadata come `tensor_count` **descriptors**, one per weight. Each is a name (a length-prefixed string like `"blk.0.attn_q.weight"`), a 32-bit dimension count, that many 64-bit **dimensions**, a 32-bit **type** (the ggml type — `f32`, `f16`, `q4_0`, `q8_0`, and others), and a 64-bit **offset**. The offset is relative to the start of the **data section**, which begins after the last descriptor, padded up to the file's alignment (the `general.alignment` metadata value, default 32). So a tensor's bytes are at `data_section + offset`.

One subtlety repays attention: **dimension order**. GGUF stores dimensions fastest-varying first. A weight we treat as `[out, in]` in row-major (each output's in

weights contiguous, the convention from Unit 5) is stored with dims {in, out}. So our row-major shape is the *reverse* of the GGUF dims — and, because the byte layout already has the in dimension contiguous, the data maps directly onto our [out, in] tensor with no transpose. Get this reversal right and weights drop into place; get it wrong and every matmul is silently transposed.

Decoding a tensor into float means dispatching on its type. f32 is a copy. f16 requires converting each 16-bit half to a 32-bit float (an exponent-rebias and mantissa-shift). q8_0 and q4_0 are dequantized exactly as in Unit 9 — with one format detail: GGUF stores the per-block scale as an f16, so a q8_0 block is $2 + 32 = 34$ bytes (f16 scale, 32 int8) and a q4_0 block is $2 + 16 = 18$ bytes (f16 scale, 16 packed nibbles). The dequantization arithmetic is identical to what you wrote; only the scale's width differs, which is why we noted the f16 scale back in Unit 9.

Finally, **memory mapping**. Models are gigabytes, and copying all of that into RAM on load would be slow and wasteful — especially since a quantized model may be mostly read once per token. So we mmap the file read-only: the operating system maps the file's bytes into the address space and pages them in on demand, sharing them across processes and never materializing more than is touched. The posix call is open/fstat/mmap (Windows uses CreateFileMapping / MapViewOfFile — the same idea); the mapped pointer is then exactly the “file bytes” the parser already operates on.

THE TASK

Extend GGUFReader: after the metadata, read the tensor descriptors into a map<string, TensorInfo> (dims, type, offset); compute the aligned data-section pointer; add tensor(name) and tensor_data(name). Add a load(path) that mmaps the file and calls parse. Add f16_to_f32, dequant_gguf_q8_0/dequant_gguf_q4_0 (f16-scale block layouts), and load_tensor_f32(reader, name) returning a float Tensor of the reverse-of-GGUF-dims shape, dispatching on type.

HINTS

Hint 1 — the data section is aligned

After reading all descriptors, `pos = cursor - base; align = meta_u32("general.alignment", 32); data = base + (pos + align - 1) / align * align;` Each tensor is then `data + info.offset`.

Hint 2 — reverse the dims

`std::vector<int64_t> shape(info.dims.rbegin(), info.dims.rend());`
 gives the row-major [out, in] shape; the bytes copy straight in because GGUF's fastest dimension is your contiguous one.

Hint 3 — f16-scale blocks

`dequant_gguf_q8_0`: per 34-byte block, read a `uint16_t`, f16 to f32 it to the scale `d`, then `out[i] = int8[i] * d`. `dequant_gguf_q4_0`: per 18-byte block, f16 scale then the nibble unpack from Unit 9 ((lo) - 8, (hi) - 8, times `d`).

TESTS

```
// add to tests/test_gguf.cpp (reuses put_u32/put_u64/put_f32/put_str)

TEST(gguf_dequant_q8_0_block) {
    std::vector<uint8_t> blk;
    blk.push_back(0x00); blk.push_back(0x3C);           // f16 1.0 (0x3C00), little-
    ↪ endian
    for (int i = 0; i < 32; ++i) blk.push_back((uint8_t)(int8_t)(i - 16)); //
    ↪ -16..15
    std::vector<float> out(32);
    dequant_gguf_q8_0(blk.data(), 32, out.data());
    for (int i = 0; i < 32; ++i) CHECK_CLOSE(out[i], (float)(i - 16), 1e-3); //
    ↪ scale 1.0
}

TEST(gguf_pares_tensor_and_loads_f32) {
    std::vector<uint8_t> b;
    put_u32(b, 0x46554747); put_u32(b, 3);
    put_u64(b, 1); put_u64(b, 0);                       // 1 tensor, 0 metadata
    put_str(b, "token_embd.weight");
    put_u32(b, 2);                                       // n_dims
    put_u64(b, 4); put_u64(b, 3);                       // dims {in=4, out=3} ->
    ↪ shape {3,4}
    put_u32(b, GGM_F32); put_u64(b, 0);                 // type, offset
    while (b.size() % 32 != 0) b.push_back(0);           // align data section
    for (int i = 0; i < 12; ++i) put_f32(b, (float)i); // 3x4 data

    GGUFReader r;
    REQUIRE(r.parse(b.data(), b.size()));
    const auto* ti = r.tensor("token_embd.weight");
```

```

    REQUIRE(ti != nullptr);
    CHECK(ti->dims[0] == 4 && ti->dims[1] == 3 && ti->type == GGML_F32);
    Tensor t = load_tensor_f32(r, "token_embd.weight");
    CHECK(t.shape()[0] == 3 && t.shape()[1] == 4);    // reversed dims
    CHECK_CLOSE(t.at<float>({0, 0}), 0.0f, 1e-6);
    CHECK_CLOSE(t.at<float>({2, 3}), 11.0f, 1e-6);
}

```

REFERENCE SOLUTION

Add to `gguf.hpp` (extending `GGUFReader` and adding decode helpers). First, the tensor parsing inside `parse` — after the metadata loop, before returning:

```

// gguf.hpp -- ggml types and the tensor section
enum GGMLType : uint32_t { GGML_F32 = 0, GGML_F16 = 1, GGML_Q4_0 = 2, GGML_Q8_0 =
    ↪ 8 };

struct TensorInfo { std::vector<int64_t> dims; GGMLType type; uint64_t offset; };

// ... inside class GGUFReader, add members:
//     std::unordered_map<std::string, TensorInfo> tensors_;
//     const uint8_t* data_ = nullptr;
// ... and at the end of parse(), continue the same cursor `c`:
void parse_tensors(ByteCursor& c) {
    for (uint64_t i = 0; i < n_tensors_; ++i) {
        std::string name = c.read_string();
        TensorInfo ti;
        const uint32_t nd = c.read<uint32_t>();
        for (uint32_t d = 0; d < nd; ++d) ti.dims.push_back((int64_t)c.read<
            ↪ uint64_t>());
        ti.type = static_cast<GGMLType>(c.read<uint32_t>());
        ti.offset = c.read<uint64_t>();
        tensors_[name] = ti;
    }
    const uint64_t align = meta_u32("general.alignment", 32);
    const uint64_t pos = (uint64_t)(c.p - base_);
    data_ = base_ + (pos + align - 1) / align * align;    // aligned data
    ↪ section
}

const TensorInfo* tensor(const std::string& n) const {
    auto it = tensors_.find(n); return it == tensors_.end() ? nullptr : &it->
    ↪ second;
}

```

```

const uint8_t* tensor_data(const std::string& n) const {
    auto it = tensors_.find(n); return it == tensors_.end() ? nullptr : data_ + it
    ↪ ->second.offset;
}

```

(parse now ends with `parse_tensors(c); return true;`;) The memory-mapped loader:

```

// gguf.hpp
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

// ... inside GGUFReader:
bool load(const std::string& path) {
    int fd = ::open(path.c_str(), O_RDONLY);
    if (fd < 0) return false;
    struct stat st;
    if (::fstat(fd, &st) != 0) { ::close(fd); return false; }
    mapped_size_ = (size_t)st.st_size;
    void* m = ::mmap(nullptr, mapped_size_, PROT_READ, MAP_PRIVATE, fd, 0);
    ::close(fd);
    if (m == MAP_FAILED) return false;
    mapped_ = static_cast<const uint8_t*>(m);
    return parse(mapped_, mapped_size_);           // weights now page in on
    ↪ demand
}
~GGUFReader() { if (mapped_) ::munmap(const_cast<uint8_t*>(mapped_), mapped_size_);
    ↪ }
// members: const uint8_t* mapped_ = nullptr; size_t mapped_size_ = 0;

```

The decode helpers:

```

// gguf.hpp
#include "llm/tensor.hpp"

inline float f16_to_f32(uint16_t h) {
    const uint32_t sign = (uint32_t)(h & 0x8000) << 16;
    uint32_t exp = (h >> 10) & 0x1F, mant = h & 0x3FF, bits;
    if (exp == 0) {
        if (mant == 0) bits = sign;           // +/- 0
        else { exp = 1; while ((mant & 0x400) == 0) { mant <<= 1; --exp; }

```



```

        mant &= 0x3FF; bits = sign | ((exp + 112) << 23) | (mant << 13); }
    } else if (exp == 0x1F) {
        bits = sign | 0x7F800000u | (mant << 13);           // inf / nan
    } else {
        bits = sign | ((exp + 112) << 23) | (mant << 13);    // 112 = 127 - 15
    }
    float f; std::memcpy(&f, &bits, sizeof(f)); return f;
}

inline void dequant_gguf_q8_0(const uint8_t* p, int64_t n, float* out) {
    for (int64_t b = 0; b < n / 32; ++b) {
        uint16_t dh; std::memcpy(&dh, p, 2); p += 2;
        const float d = f16_to_f32(dh);
        const int8_t* q = reinterpret_cast<const int8_t*>(p); p += 32;
        for (int i = 0; i < 32; ++i) out[b * 32 + i] = q[i] * d;
    }
}

inline void dequant_gguf_q4_0(const uint8_t* p, int64_t n, float* out) {
    for (int64_t b = 0; b < n / 32; ++b) {
        uint16_t dh; std::memcpy(&dh, p, 2); p += 2;
        const float d = f16_to_f32(dh);
        const uint8_t* q = p; p += 16;
        for (int i = 0; i < 16; ++i) {
            out[b * 32 + i] = ((int)(q[i] & 0x0F) - 8) * d;
            out[b * 32 + i + 16] = ((int)(q[i] >> 4) - 8) * d;
        }
    }
}

inline Tensor load_tensor_f32(const GGUFReader& r, const std::string& name) {
    const TensorInfo* ti = r.tensor(name);
    LLM_CHECK(ti != nullptr, "GGUF tensor not found: " + name);
    int64_t n = 1; for (int64_t d : ti->dims) n *= d;
    std::vector<int64_t> shape(ti->dims.rbegin(), ti->dims.rend()); // reverse
    ↪ to row-major
    Tensor t = Tensor::empty(shape, DType::F32);
    const uint8_t* data = r.tensor_data(name);
    float* out = t.data_ptr<float>();
    switch (ti->type) {
        case GGML_F32: std::memcpy(out, data, sizeof(float) * n); break;
        case GGML_F16: { auto* h = reinterpret_cast<const uint16_t*>(data);
            for (int64_t i = 0; i < n; ++i) out[i] = f16_to_f32(h[i])
            ↪ ; } break;
        case GGML_Q8_0: dequant_gguf_q8_0(data, n, out); break;
        case GGML_Q4_0: dequant_gguf_q4_0(data, n, out); break;
        default: LLM_CHECK(false, "unsupported ggml type");
    }
}

```

```
    }  
    return t;  
}
```

With descriptors parsed and the file mapped, any weight is one `load_tensor_f32` away — decoded from whatever format the file stores into the plain float tensors the rest of the engine expects. The dimension reversal is the detail that makes the matmuls line up, and the f16-scale dequant is where Unit 9’s formats meet a real file. All that remains is to call this for every tensor a model needs, read the config and tokenizer from the metadata, and press “go” — the boss.

GOING FURTHER

- **Keep weights quantized.** Instead of dequantizing to float on load, store the quantized bytes and use the quantized matmul of Unit 9 directly — preserving the memory saving at run time, not just on disk. This is what a production engine does.
- **advise hints.** Tell the kernel your access pattern (`MADV_SEQUENTIAL` for a one-shot load, `MADV_WILLNEED` to prefetch) to speed the first forward pass.
- **Lazy per-tensor decode.** Decode each tensor only when first used, so loading a model and running a short prompt need not touch every byte — useful for very large MOES.

10.3 Assemble and Run a Model

BOSS CHALLENGE

Wire the parser into a real loader: extract the `ModelConfig` from metadata, load every named weight into `ModelWeights`, and run a forward pass. Prove it by round-trip: write a model to `GGUF`, load it back, and show the loaded model’s logits **match the original’s** — certifying that every weight landed in the right place, correctly shaped.

GOAL

Close the loop from file to running network. Build `load_config` (metadata → `ModelConfig`) and `load_model` (named tensors → `ModelWeights`), then certify the loader by writing a known model to GGUF, loading it, and asserting the loaded forward pass reproduces the original's logits exactly.

BACKGROUND

The loader's final job is mapping: turn the parsed metadata and tensors into the engine's structures. **Config** comes from architecture-prefixed metadata keys — with `general.architecture` giving the prefix ("llama", "qwen2", ...), the hyperparameters live under keys like "llama.embedding_length" (hidden size), "llama.block_count" (layers), "llama.attention.head_count" and ".head_count_kv" (the head counts that drive grouped-query attention), "llama.feed_forward_length" (the FFN width), ".layer_norm_rms_epsilon", and "llama.rope.freq_base". The vocabulary size is the length of the tokenizer's token array. **Weights** come from the tensor table by name, following `llama.cpp`'s convention: "token_embd.weight", "output_norm.weight", "output.weight" (the `lm_head`, sometimes absent and tied to the embedding), and per layer "blk.{i}.attn_q.weight", ".attn_k/v/output.weight", ".attn_norm.weight", ".ffn_gate/up/down.weight", ".ffn_norm.weight". Each is decoded with the `load_tensor_f32` of 10.2.

The hard part is not writing this mapping — it is *trusting* it. A loader has many ways to be subtly wrong: a misread head count corrupts attention, a misspelled tensor name silently leaves a weight zero, the dimension reversal from 10.2 transposes a matrix, a dtype mishandled turns weights to noise. None of these necessarily crash; they just make the model produce garbage. So we need a check that exercises *all* of it at once, and the cleanest is a **round-trip**: write a known model to a GGUF blob, load it back through the real loader, and demand that the reloaded model behaves identically.

The decisive measurement is **forward equivalence**. Run the original in-memory model and the loaded model on the same tokens; their logits must match. This single comparison certifies everything: if any weight were missing, misnamed, transposed, or mis-decoded, the outputs would diverge. Because we write the tensors as `f32`, the round-trip is lossless and the match is essentially exact — and the very same `load_tensor_f32` would dequantize a `q4_0/q8_0` real model through the identical path, so passing this proves the mechanism, not just the float case. (Writing GGUF is not part of an inference engine, but a small writer

is the natural way to manufacture a test fixture, and doubles as the basis of a model-conversion tool.)

THE TASK

Implement `load_config(reader)` (read the arch prefix, then the hyperparameter keys; derive `head_dim` and `vocab_size`) and `load_model(reader, cfg)` (load every tensor by its llama.cpp name into `ModelWeights`, tying `lm_head` to the embedding if "output.weight" is absent). Then the boss: a minimal GGUF writer for the fixture; write a tiny random model; load it back; assert the config matches and `forward(loader) equals forward(original)` within tolerance.

The bar (the boss): the loaded `ModelConfig` matches the written one, and the loaded model's logits match the original model's at every position — a faithful round-trip from memory to file and back.

HINTS

Hint 1 — arch-prefixed keys

```
std::string arch = r.meta_str("general.architecture"); auto k =
[&](std::string s){ return arch + "." + s; }; then cfg.hidden_size =
r.meta_u32(k("embedding_length")); and so on. head_dim = hidden_size
/ n_heads; unless a key overrides it.
```

Hint 2 — per-layer names

```
std::string pre = "blk." + std::to_string(i) + "."; then load pre +
"attn_q.weight" etc. into m.layers[i]. A helper that builds the name and
calls load_tensor_f32 keeps it tidy.
```

Hint 3 — the writer mirrors the reader

To write a tensor: emit its name, the *reversed* shape as dims, GGML_F32, and an offset into the data section; align each tensor's offset to 32; pad to 32 before the data section. The reader's dimension-reversal then restores the original shape.

TESTS

```

// add to tests/test_gguf.cpp (reuses put_*, tiny_model/tiny_cfg from model tests)
↪
#include "llm/model.hpp"

// Minimal F32 GGUF writer for fixtures.
struct GgufWriter {
    std::vector<uint8_t> meta; int n_meta = 0;
    struct T { std::string name; std::vector<int64_t> dims; const float* data;
    ↪ int64_t n; };
    std::vector<T> ts;
    void u32(const std::string& k, uint32_t v){ put_str(meta,k); put_u32(meta,
    ↪ GGUF_U32); put_u32(meta,v); ++n_meta; }
    void f32(const std::string& k, float v){ put_str(meta,k); put_u32(meta,
    ↪ GGUF_F32); put_f32(meta,v); ++n_meta; }
    void str(const std::string& k, const std::string& s){ put_str(meta,k); put_u32
    ↪ (meta,GGUF_STRING); put_str(meta,s); ++n_meta; }
    void strs(const std::string& k, const std::vector<std::string>& a){
        put_str(meta,k); put_u32(meta,GGUF_ARRAY); put_u32(meta,GGUF_STRING);
    ↪ put_u64(meta,a.size());
        for (auto& s : a) put_str(meta,s); ++n_meta; }
    void tensor(const std::string& name, const Tensor& t){
        ts.push_back({name, std::vector<int64_t>(t.shape().rbegin(), t.shape().
    ↪ rend()),
            t.data_ptr<float>(), t.numel())}); }
    std::vector<uint8_t> build(){
        std::vector<uint8_t> b;
        put_u32(b,0x46554747); put_u32(b,3); put_u64(b,ts.size()); put_u64(b,
    ↪ n_meta);
        b.insert(b.end(), meta.begin(), meta.end());
        std::vector<int64_t> off(ts.size()); int64_t cur = 0;
        for (size_t i=0;i<ts.size();++i){ off[i]=cur; cur += ts[i].n*4; cur=(cur
    ↪ +31)/32*32; }
        for (size_t i=0;i<ts.size();++i){
            put_str(b, ts[i].name); put_u32(b,(uint32_t)ts[i].dims.size());
            for (int64_t d : ts[i].dims) put_u64(b,(uint64_t)d);
            put_u32(b, GGML_F32); put_u64(b,(uint64_t)off[i]);
        }
        while (b.size()%32) b.push_back(0);
        size_t ds = b.size(); b.resize(ds + cur);
        for (size_t i=0;i<ts.size();++i) std::memcpy(b.data()+ds+off[i], ts[i].
    ↪ data, ts[i].n*4);
        return b;
    }
};

```

```

TEST(boss_gguf_roundtrip_forward) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);

    GgufWriter w;
    w.str("general.architecture", "llama");
    w.u32("llama.embedding_length", c.hidden_size);
    w.u32("llama.block_count", c.n_layers);
    w.u32("llama.attention.head_count", c.n_heads);
    w.u32("llama.attention.head_count_kv", c.n_kv_heads);
    w.u32("llama.feed_forward_length", c.intermediate_size);
    w.f32("llama.attention.layer_norm_rms_epsilon", c.rms_norm_eps);
    w.f32("llama.rope.freq_base", c.rope_theta);
    std::vector<std::string> toks(c.vocab_size);
    for (int i = 0; i < c.vocab_size; ++i) toks[i] = "t" + std::to_string(i);
    w.strs("tokenizer.ggml.tokens", toks);

    w.tensor("token_embd.weight", m.token_embedding);
    w.tensor("output_norm.weight", m.final_norm);
    w.tensor("output.weight", m.lm_head);
    for (int i = 0; i < c.n_layers; ++i) {
        const std::string p = "blk." + std::to_string(i) + ".";
        w.tensor(p+"attn_norm.weight", m.layers[i].attn_norm);
        w.tensor(p+"attn_q.weight", m.layers[i].wq);
        w.tensor(p+"attn_k.weight", m.layers[i].wk);
        w.tensor(p+"attn_v.weight", m.layers[i].wv);
        w.tensor(p+"attn_output.weight", m.layers[i].wo);
        w.tensor(p+"ffn_norm.weight", m.layers[i].ffn_norm);
        w.tensor(p+"ffn_gate.weight", m.layers[i].w_gate);
        w.tensor(p+"ffn_up.weight", m.layers[i].w_up);
        w.tensor(p+"ffn_down.weight", m.layers[i].w_down);
    }
    const std::vector<uint8_t> blob = w.build();

    GGUFReader r; REQUIRE(r.parse(blob.data(), blob.size()));
    const ModelConfig lc = load_config(r);
    CHECK(lc.hidden_size == c.hidden_size && lc.n_layers == c.n_layers);
    CHECK(lc.n_heads == c.n_heads && lc.n_kv_heads == c.n_kv_heads);
    CHECK(lc.intermediate_size == c.intermediate_size && lc.vocab_size == c.
    ↪ vocab_size);

    const ModelWeights lm = load_model(r, lc);
    const std::vector<int> ids = {3, 7, 1, 9, 2};
    Tensor a = forward(m, c, ids);
    Tensor b = forward(lm, lc, ids);
    for (int i = 0; i < a.numel(); ++i)

```

```

        CHECK_CLOSE(a.data_ptr<float>()[i], b.data_ptr<float>()[i], 1e-4);    //
        ↪ faithful round-trip
    }

```

REFERENCE SOLUTION

Add to a loader (model.cpp or gguf.hpp):

```

// loader
ModelConfig load_config(const GGUFReader& r) {
    const std::string arch = r.meta_str("general.architecture");
    auto k = [&](const std::string& s){ return arch + "." + s; };
    ModelConfig c;
    c.hidden_size      = r.meta_u32(k("embedding_length"));
    c.n_layers         = r.meta_u32(k("block_count"));
    c.n_heads          = r.meta_u32(k("attention.head_count"));
    c.n_kv_heads       = r.meta_u32(k("attention.head_count_kv"), c.n_heads);
    c.intermediate_size = r.meta_u32(k("feed_forward_length"));
    c.rms_norm_eps     = r.meta_f32(k("attention.layer_norm_rms_epsilon"), 1e-5f);
    ↪
    c.rope_theta       = r.meta_f32(k("rope.freq_base"), 10000.0f);
    c.head_dim         = (c.n_heads > 0) ? c.hidden_size / c.n_heads : 0;
    c.vocab_size       = (int)r.meta_str_array("tokenizer.ggml.tokens").size();
    return c;
}

ModelWeights load_model(const GGUFReader& r, const ModelConfig& cfg) {
    ModelWeights m;
    m.token_embedding = load_tensor_f32(r, "token_embd.weight");
    m.final_norm      = load_tensor_f32(r, "output_norm.weight");
    m.lm_head = r.tensor("output.weight") ? load_tensor_f32(r, "output.weight")
                                           : m.token_embedding;    // tied if
    ↪ absent
    m.layers.resize(cfg.n_layers);
    for (int i = 0; i < cfg.n_layers; ++i) {
        const std::string p = "blk." + std::to_string(i) + ".";
        LayerWeights& w = m.layers[i];
        w.attn_norm = load_tensor_f32(r, p + "attn_norm.weight");
        w.wq        = load_tensor_f32(r, p + "attn_q.weight");
        w.wk        = load_tensor_f32(r, p + "attn_k.weight");
        w.wv        = load_tensor_f32(r, p + "attn_v.weight");
        w.wo        = load_tensor_f32(r, p + "attn_output.weight");
        w.ffn_norm  = load_tensor_f32(r, p + "ffn_norm.weight");
        w.w_gate    = load_tensor_f32(r, p + "ffn_gate.weight");
    }
}

```

```
        w.w_up      = load_tensor_f32(r, p + "ffn_up.weight");
        w.w_down    = load_tensor_f32(r, p + "ffn_down.weight");
    }
    return m;
}
```

Run the boss. The reloaded config matches field for field, and the reloaded model's logits match the original's to floating-point exactness — which can only happen if every one of the dozens of weight tensors was found by name, shaped by the reversed dims, and decoded correctly. **Boss cleared.**

What you have proven. Not “a parser that runs,” but “a loader that faithfully re-constitutes a model”: a network written to GGUF and read back through the real `load_config` and `load_model` produces an identical forward pass. The round-trip exercises the entire path at once — header, metadata, tensor descriptors, alignment, dimension order, and dtype decode — so its success is a strong certificate that the engine can read actual model files. Swap the fixture for a downloaded quantized GGUF and the same code path, dequantizing through Unit 9, loads a real trained Llama or Qwen.

GOING FURTHER

- **Load a real file.** Point load at a downloaded TinyLlama or Qwen2.5-0.5B GGUF, build the tokenizer from `tokenizer.ggml.tokens/.merges`, and run a forward pass — the first time the engine touches a real model.
- **Tie-weight and naming variants.** Handle architectures that name tensors differently or that always tie the output to the embedding; a robust loader tolerates the variation across model families.
- **Validate shapes against config.** Cross-check each loaded tensor's shape against what the config implies (e.g. `wq` is `[n_heads*head_dim, hidden]`) and fail loudly on mismatch — catching a wrong file or a misread header early.

Unit 10 — Milestone check

MILESTONE CHECK

The engine can now read a real model from disk:

- a **GGUF parser** — header, typed metadata, and tensor descriptors — over a **memory-mapped** file, so weights page in on demand;
- **tensor decode** for F32, F16, and the Q4_0/Q8_0 formats of Unit 9, with the dimension reversal that aligns GGUF to our row-major layout;
- a **loader** that fills `ModelConfig`, the tokenizer vocabulary, and `ModelWeights` from metadata and named tensors; and
- a **boss-grade round-trip** proving a model written to GGUF and loaded back yields an identical forward pass.

The engine is now *alive*: it runs a real, trained network rather than random weights. But it still only computes **logits** — it does not yet produce **text**. **Unit 11** adds the last piece of a usable model: **sampling and generation** — turning a logit vector into a chosen token (greedy, temperature, top- k , top- p) and wrapping it in the prefill-then-decode loop from Unit 7, so the engine reads a prompt and *writes* a continuation, token by token, until it decides to stop. It is the moment the engine finally talks.

UNIT 11

Sampling and Generation

What you will build. The piece that turns a model into a writer: **sampling** and the **generation loop**. You build the samplers — **greedy**, **temperature**, **top- k** , and **top- p** (nucleus) — that pick a token from a logit vector, then wrap them in the prefill-then-decode loop from Unit 7 so the engine reads a prompt and *writes* a continuation, token by token, stopping at an end-of-sequence token or a length cap. The boss proves the loop is correct by matching a from-scratch reference.

Where this fits. Everything so far produces **logits** — a score per vocabulary token — not text. This unit closes the gap: choose a token from the logits, append it, feed it back, and repeat. With the real model of Unit 10 and the tokenizer of Unit 4, the result is a complete, working, single-sequence inference engine: prompt in, generated text out. This is the moment the engine finally talks.

A language model predicts the next token, but a *forward pass* only scores the possibilities — it returns logits, one per vocabulary entry, for the token that should come next. Generation is the act of repeatedly **choosing** from those scores and **continuing**: pick a token, append it to the sequence, run the model again (cheaply, via the cache) to get the next logits, and pick again, until the model emits an end-of-sequence marker or a length limit is reached. Two things make this work well. The first is the **sampler** — how a logit vector becomes a single chosen token — which is where the trade-off between coherence and diversity is set. The second is the **loop** — prefill the prompt, then decode one token at a time — which is exactly the prefill/decode structure the kv cache was built for.

The sampler matters more than it first appears. Always taking the highest-scoring token (greedy) is deterministic and often coherent, but it tends toward bland, repetitive text and can fall into loops. Drawing randomly from the model's

distribution adds variety, but unguarded it occasionally picks a low-probability “tail” token that derails the whole continuation. The standard samplers — temperature to set the overall randomness, top- k and top- p to clip the unreliable tail — give control over exactly this balance, and they compose into the knobs every chat interface exposes. We build them in order, simplest first, then assemble the generation loop with a boss that certifies it against a reference. Add `include/llm/sampler.hpp`.

11.1 Greedy and Temperature Sampling

GOAL

The two foundational choices: **greedy** (take the argmax — deterministic) and **temperature** (scale the logits, softmax to a distribution, and draw a sample). Plus a seeded random generator so sampling is reproducible.

BACKGROUND

The simplest sampler is **greedy**: return the token with the largest logit. It is deterministic and fast, and for many tasks — especially short, factual ones — perfectly good, but because it always takes the single most likely token it produces flat, predictable text and is prone to repetition loops. To introduce variety we treat the logits as defining a **probability distribution** and draw from it. The softmax converts logits to probabilities, and we sample a token in proportion to its probability — so a token the model finds twice as likely is twice as likely to be chosen.

The knob that controls *how* random this is, is **temperature**. Before the softmax, divide the logits by a temperature T :

$$p_i = \frac{\exp(\text{logit}_i/T)}{\sum_j \exp(\text{logit}_j/T)}.$$

At $T = 1$ this is the model’s raw distribution. With $T < 1$ the logits are amplified, sharpening the distribution toward the most likely tokens — more focused, more deterministic — and as $T \rightarrow 0$ it becomes greedy. With $T > 1$ the logits are flattened, spreading probability toward less likely tokens — more diverse, more surprising, and eventually incoherent. Temperature is the primary creativity dial: low for precise tasks, around 0.7–1.0 for natural text, higher for brainstorming.

Drawing the sample itself is the **inverse-CDF** method: compute the cumulative sum of the probabilities, draw a uniform random number $r \in [0, 1)$, and return the first token whose cumulative probability exceeds r . This selects each token with exactly its probability. The randomness comes from a **seeded** pseudo-random generator — a small, fast one like xorshift suffices — so that a fixed seed reproduces a generation exactly, which matters for testing and for users who want repeatable output. (Greedy needs no randomness; it is the $T \rightarrow 0$ limit and we expose it directly.)

THE TASK

In `sampler.hpp`, implement a seeded RNG (xorshift, with `next_u32` and a `next_float` in $[0, 1)$); `argmax(logits, n)`; `softmax_temperature(logits, probs, n, temp)` computing the temperature-scaled stable softmax; and `sample_categorical(probs, n, rng)` drawing a token by the inverse-CDF method.

HINTS

Hint 1 — a small xorshift RNG

```
uint64_t s; uint32_t next_u32() { s ^= s<<13; s ^= s>>7; s ^=
s<<17; return (uint32_t)(s>>32); } and float next_float() { return
(next_u32() >> 8) * (1.0f/16777216.0f); }. Seed with a nonzero value.
```

Hint 2 — temperature is a pre-scale on the stable softmax

Scale by `inv_t = 1/temp`: find `m = max(logit[i]*inv_t)`, then `probs[i] = exp(logit[i]*inv_t - m)`, sum, and normalize. This is Unit 2's stable softmax with the logits scaled first.

Hint 3 — inverse-CDF sampling

```
float r = rng.next_float(); float cum = 0; for i: cum += probs[i];
if (r < cum) return i;. Return n-1 as a rounding safety net. The
cumulative walk picks token i with probability probs[i].
```

TESTS

```

// tests/test_sampler.cpp
#include "test_framework.hpp"
#include "llm/sampler.hpp"
#include <vector>
#include <cmath>
using namespace llm;

TEST(argmax_picks_largest) {
    std::vector<float> l = {0.1f, 2.5f, -1.0f, 2.4f};
    CHECK(argmax(l.data(), (int)l.size()) == 1);
}

TEST(low_temperature_sharpens_toward_greedy) {
    std::vector<float> l = {1.0f, 2.0f, 3.0f};    // token 2 is the max
    std::vector<float> p(3);
    softmax_temperature(l.data(), p.data(), 3, 0.1f);    // very low T
    CHECK(p[2] > 0.99f);    // nearly all mass on the argmax
    ↪
    float sum = p[0] + p[1] + p[2];
    CHECK_CLOSE(sum, 1.0f, 1e-5);
}

TEST(sampling_is_seed_reproducible) {
    std::vector<float> l = {1.0f, 1.0f, 1.0f, 1.0f};
    std::vector<float> p(4); softmax_temperature(l.data(), p.data(), 4, 1.0f);
    RNG a(42), b(42);
    for (int i = 0; i < 10; ++i)
        CHECK(sample_categorical(p.data(), 4, a) == sample_categorical(p.data(), 4,
    ↪ b));
}

TEST(sampling_distribution_matches_probs) {
    std::vector<float> l = {std::log(0.6f), std::log(0.3f), std::log(0.1f)};    //
    ↪ probs 0.6/0.3/0.1
    std::vector<float> p(3); softmax_temperature(l.data(), p.data(), 3, 1.0f);
    int count[3] = {0, 0, 0}; RNG rng(7);
    const int N = 20000;
    for (int i = 0; i < N; ++i) count[sample_categorical(p.data(), 3, rng)]++;
    CHECK(std::fabs(count[0] / (float)N - 0.6f) < 0.03f);    // empirical ~
    ↪ probabilities
    CHECK(std::fabs(count[1] / (float)N - 0.3f) < 0.03f);
}

```

REFERENCE SOLUTION

include/llm/sampler.hpp:

```

// include/llm/sampler.hpp
#pragma once
#include <cstdint>
#include <cmath>
#include <vector>
#include <algorithm>

namespace llm {

struct RNG {
    uint64_t s;
    explicit RNG(uint64_t seed = 0x2545F4914F6CDD1DULL) : s(seed ? seed : 1) {}
    uint32_t next_u32() { s ^= s << 13; s ^= s >> 7; s ^= s << 17; return (
    ↪ uint32_t)(s >> 32); }
    float next_float() { return (next_u32() >> 8) * (1.0f / 16777216.0f); } //
    ↪ [0, 1)
};

inline int argmax(const float* logits, int n) {
    int best = 0;
    for (int i = 1; i < n; ++i) if (logits[i] > logits[best]) best = i;
    return best;
}

inline void softmax_temperature(const float* logits, float* probs, int n, float
    ↪ temp) {
    const float inv_t = 1.0f / temp;
    float m = -INFINITY;
    for (int i = 0; i < n; ++i) m = std::max(m, logits[i] * inv_t);
    float sum = 0.0f;
    for (int i = 0; i < n; ++i) { float e = std::exp(logits[i] * inv_t - m); probs
    ↪ [i] = e; sum += e; }
    const float is = 1.0f / sum;
    for (int i = 0; i < n; ++i) probs[i] *= is;
}

inline int sample_categorical(const float* probs, int n, RNG& rng) {
    const float r = rng.next_float();
    float cum = 0.0f;
    for (int i = 0; i < n; ++i) { cum += probs[i]; if (r < cum) return i; }
    return n - 1;
    // rounding safety net
}

```

```
} // namespace llm
```

These are the two ends of the spectrum: greedy takes the single best token, and temperature sampling draws from a distribution you can sharpen or flatten at will. The reproducibility and distribution tests pin both the determinism (same seed, same draws) and the correctness (empirical frequencies track the probabilities) — the properties a sampler must have to be trustworthy. But raw temperature sampling still has a weakness: at any temperature, it can occasionally draw a genuinely bad token from the long tail. The next challenge adds the standard truncation that prevents it.

GOING FURTHER

- **Gumbel-max sampling.** Add a Gumbel noise vector to the logits and take the argmax — an equivalent way to sample that needs no softmax and composes nicely with top- k . Useful for some speculative-decoding schemes (Unit 12).
- **Repetition penalty.** Down-weight the logits of tokens already generated (or seen in the prompt) before sampling — the standard fix for degenerate loops, and a one-line pre-pass over the logits.
- **Logit bias.** Allow per-token additive biases (ban a token with $-\infty$, force one with $+\infty$) — the mechanism behind constrained decoding and “forbidden word” lists.

11.2 Top- k and Top- p Sampling

GOAL

The standard tail-truncation samplers: **top- k** (keep the k highest-probability tokens) and **top- p** (keep the smallest set whose cumulative probability reaches p). Compose them with temperature into one `sample_token` that every generation call uses.

BACKGROUND

Temperature sampling has a structural weakness: the **tail**. A vocabulary has tens of thousands of tokens, and the softmax gives every one of them nonzero probability. Individually each tail token is unlikely, but collectively the tail carries real mass, so a pure draw will — occasionally — select a genuinely wrong token, and a single bad token can derail an entire continuation. The remedy is to **truncate** the distribution before sampling, keeping only the candidates the model actually finds plausible.

Top- k keeps the k highest-probability tokens, discards the rest, renormalizes, and samples from what remains. It is simple and effective — k around 40 is a common choice. Its limitation is that k is *fixed*: when the model is confident (one obvious next token) k candidates is too many, admitting noise; when the model is genuinely uncertain (many plausible continuations) k may be too few, cutting off good options.

Top- p , or **nucleus** sampling, fixes that by truncating on cumulative *probability* rather than count: sort tokens by probability and keep the smallest prefix whose probabilities sum to at least p (say 0.9), discarding the rest. The candidate set is now **adaptive** — just one or two tokens when the model is peaked, dozens when it is flat — which is why nucleus sampling has become the default for open-ended generation. The two are not exclusive; the standard pipeline applies them in sequence: **temperature** to set the overall sharpness, then **top- k** to cap the count, then **top- p** to cap the cumulative mass, then a categorical draw from whatever survives. Each stage only narrows the set, and any stage can be disabled (`top_k = 0`, `top_p = 1`). Greedy is just the limit of this machine with the candidate set forced to one.

Implementation is a sort and two cutoffs. Convert logits to probabilities with temperature; order the token indices by probability descending; compute the keep-count as the smaller of the top- k cap and the nucleus cutoff; then sample within the kept prefix, renormalized over its mass. The sort dominates the cost; for a large vocabulary a partial selection (`nth_element`) is faster, which we leave to Going Further.

THE TASK

Add a `SamplingParams` (temperature, top_k, top_p, greedy) and `sample_token(logits, n, params, rng)`: if greedy (or $T \leq 0$) return `argmax`; otherwise build temperature probabilities, sort indices by probability descend-

ing, reduce the keep-count by top_k and then by the top_p nucleus, and sample (inverse-CDF) within the kept, renormalized prefix.

HINTS

Hint 1 — sort indices by probability

```
std::vector<int> idx(n); std::iota(...); std::sort(idx.begin(),
idx.end(), [&](int a,int b){ return probs[a] > probs[b]; });.    You
sort indices, not the probabilities, so you can map back to token ids.
```

Hint 2 — two cutoffs on the keep-count

Start keep = n. If top_k > 0, keep = min(keep, top_k). If top_p < 1, walk the sorted prefix accumulating probability and stop once it reaches top_p, setting keep to that count. The nucleus is the prefix up to keep.

Hint 3 — sample within the kept mass

Sum the kept probabilities, draw $r = \text{rng.next_float()} * \text{sum}$, and walk the kept prefix accumulating until you pass r , returning that token id. Renormalizing by the kept sum is what makes the truncated distribution proper.

TESTS

```
// add to tests/test_sampler.cpp
#include <set>

TEST(top_k_restricts_to_k_tokens) {
    std::vector<float> l = {3.0f, 2.0f, 1.0f, 0.0f, -1.0f}; // tokens 0,1 are
    ↪ the top two
    SamplingParams p; p.temperature = 1.0f; p.top_k = 2;
    RNG rng(5); std::set<int> seen;
    for (int i = 0; i < 2000; ++i) seen.insert(sample_token(l.data(), 5, p, rng));
    for (int t : seen) CHECK(t == 0 || t == 1); // never a token
    ↪ outside top-2
}

TEST(top_p_keeps_only_the_nucleus) {
```

```

std::vector<float> l = {std::log(0.95f), std::log(0.03f), std::log(0.02f)};
SamplingParams p; p.temperature = 1.0f; p.top_p = 0.9f; // 0.95 >= 0.9 ->
↳ keep just token 0
RNG rng(3);
for (int i = 0; i < 1000; ++i) CHECK(sample_token(l.data(), 3, p, rng) == 0);
}

TEST(greedy_param_returns_argmax) {
std::vector<float> l = {0.5f, 3.0f, 1.0f};
SamplingParams p; p.greedy = true; RNG rng(0);
CHECK(sample_token(l.data(), 3, p, rng) == 1);
}

```

REFERENCE SOLUTION

Add to `sampler.hpp`:

```

// sampler.hpp
#include <numeric>

struct SamplingParams {
    float temperature = 1.0f;
    int top_k = 0; // 0 = disabled
    float top_p = 1.0f; // 1 = disabled
    bool greedy = false;
};

inline int sample_token(const float* logits, int n, const SamplingParams& sp, RNG&
↳ rng) {
    if (sp.greedy || sp.temperature <= 0.0f) return argmax(logits, n);

    std::vector<float> probs(n);
    softmax_temperature(logits, probs.data(), n, sp.temperature);

    const bool truncate = (sp.top_k > 0 && sp.top_k < n) || (sp.top_p < 1.0f);
    if (!truncate) return sample_categorical(probs.data(), n, rng);

    std::vector<int> idx(n);
    std::iota(idx.begin(), idx.end(), 0);
    std::sort(idx.begin(), idx.end(), [&](int a, int b){ return probs[a] > probs[b]
↳ ]; });

    int keep = n;
    if (sp.top_k > 0) keep = std::min(keep, sp.top_k); // top-k cap

```

```

    if (sp.top_p < 1.0f) {                                     // top-p nucleus
        float cum = 0.0f; int np = 0;
        for (int i = 0; i < keep; ++i) { cum += probs[idx[i]]; ++np; if (cum >= sp.
        ↪ top_p) break; }
        keep = np;
    }

    float sum = 0.0f;
    for (int i = 0; i < keep; ++i) sum += probs[idx[i]];      // mass of the
    ↪ kept set
    float r = rng.next_float() * sum, c = 0.0f;
    for (int i = 0; i < keep; ++i) { c += probs[idx[i]]; if (r < c) return idx[i];
    ↪ }
    return idx[keep - 1];
}

```

sample_token is the one entry point generation uses, and it spans the whole spectrum: set greedy for deterministic decoding, a temperature for free sampling, a top_k or top_p (or both) for the tail-clipped sampling that real chat models use by default. The tests confirm the truncation does what it claims — top- k never escapes the k best, top- p collapses to the nucleus when the model is confident — so the candidate set is always exactly what the parameters specify. With a trustworthy sampler in hand, the only thing left is the loop that calls it.

GOING FURTHER

- **Partial selection.** Replace the full sort with `std::nth_element` (or a max-heap) to find the top candidates in $O(n)$ — a real saving when the vocabulary is large and k is small, which is the common case.
- **Min- p and typical sampling.** Add newer truncation schemes (min- p keeps tokens within a factor of the top probability; typical sampling targets a entropy band) — a few lines each, and increasingly offered alongside top- p .
- **Order sensitivity.** Experiment with applying top- k /top- p *before* vs *after* temperature; the conventions differ across runtimes and subtly change behavior, worth understanding when matching another engine's output.

11.3 The Generation Loop

BOSS CHALLENGE

Assemble the whole engine into `generate()`: prefill the prompt, sample the next token, decode it through the cache, and repeat until end-of-sequence or a length cap. Prove it correct — greedy generation is deterministic and matches, token for token, a from-scratch reference that recomputes the full sequence at every step.

GOAL

The payoff: a `generate()` that reads a prompt and writes a continuation. Prefill into the kv cache, sample from the last position, then decode one token at a time — appending, re-running, re-sampling — until the model emits `eos` or the limit is reached. Certify it by matching a reference loop exactly.

BACKGROUND

The generation loop is where everything you have built finally runs together: the forward pass (Unit 5), the kv cache (Unit 7), and the sampler (this unit). Its shape follows the prefill/decode split. **Prefill**: run the whole prompt through `forward_cached` in one pass, filling the cache, and take the logits at the **last** position — those predict the first new token. Sample it. **Decode**: then loop — if the sampled token is `eos`, stop; otherwise append it to the output, run `forward_cached` on that single token (a cheap cached step) to get the next logits, and sample again — until `eos` or a maximum count.

That description is short, but the loop is a notorious home for off-by-one bugs, and naming them is the point of the boss. You must sample from the *last* position's logits, not the first. The cache cursor must advance exactly once per token. `eos` must *stop* generation without being appended to the output. The first token comes from prefill, the rest from decode, and conflating them drops or duplicates a token. Any of these is easy to get subtly wrong and hard to see by eye.

So the certificate is **equivalence to a reference**, made sharp by using **greedy** decoding. Greedy is deterministic — it always takes the `argmax` — so two things follow that a sampler with randomness could not give us. First, the *same prompt produces the same output*, which we check directly. Second, and decisively, greedy

generation with the cache must produce **exactly the same tokens** as a reference loop that ignores the cache entirely — recomputing the full sequence from scratch at every step and taking the argmax of the last position. If the two token sequences agree, then the prefill, the cache advance, the last-position indexing, the eos handling, and the sampler are all correct together; if any link is wrong, they diverge at the first mistake. This re-proves the cache’s exactness (Unit 7) in the actual generation setting and validates the loop around it in a single comparison.

With this in place, the engine is functionally complete. Hand `generate()` a prompt encoded by the tokenizer (Unit 4), backed by a model loaded from GGUF (Unit 10), and it returns generated token ids that decode to text. The engine talks.

THE TASK

Implement `generate(model, cfg, prompt, max_new, eos_id, params, rng)`: allocate a KVCache; `forward_cached` the prompt and `sample_token` from the last position; then loop up to `max_new` times — break on `eos_id`, else append the token, `forward_cached` the single token, and `sample_token` the next — returning the generated ids. Then the boss tests: (1) greedy output is deterministic; (2) greedy generation equals a full-recompute reference loop; (3) an eos as the first token yields empty output.

The bar (the boss): greedy `generate()` is deterministic, stops at eos or the cap without emitting eos, and produces the identical token sequence to a from-scratch recompute-and-argmax reference.

HINTS

Hint 1 — sample from the last position

After `Tensor logits = forward_cached(m, cfg, prompt, cache);`, the row to sample is `logits.data_ptr<float>() + (int64_t)(prompt.size()-1) * vocab` — the *last* prompt position. Sampling the first row is the classic bug.

Hint 2 — the loop structure

```
for (step < max_new) { if (next == eos_id) break;
out.push_back(next); Tensor l = forward_cached(m, cfg, {next},
cache); next = sample_token(l..., vocab, params, rng); }. The eos
check precedes the append, so eos is never emitted.
```

Hint 3 — size the cache for the whole run

KVCache::allocate(cfg, prompt.size() + max_new + 8) — enough rows for the prompt and every token you might generate, so no step exceeds capacity.

TESTS

```
// add to tests/test_sampler.cpp
// (reuses tiny_model/tiny_cfg from the model tests, forward/forward_cached, argmax
  ↳ )
#include "llm/model.hpp"

TEST(boss_generate_greedy_deterministic) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    SamplingParams sp; sp.greedy = true;
    const std::vector<int> prompt = {3, 7, 1};
    RNG r1(123), r2(123);
    auto a = generate(m, c, prompt, /*max_new=*/5, /*eos=*/-1, sp, r1);
    auto b = generate(m, c, prompt, 5, -1, sp, r2);
    CHECK(a == b);
    CHECK((int)a.size() == 5); // no EOS (eos=-1) -> runs to
  ↳ the cap
}

TEST(boss_generate_matches_full_recompute) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 7);
    SamplingParams sp; sp.greedy = true;
    const std::vector<int> prompt = {5, 2, 8};
    RNG rng(0);
    auto gen = generate(m, c, prompt, /*max_new=*/6, /*eos=*/-1, sp, rng);

    std::vector<int> seq = prompt, ref; // reference: recompute + argmax
  ↳ each step
    const int V = c.vocab_size;
    for (int step = 0; step < 6; ++step) {
        Tensor lg = forward(m, c, seq);
        int nxt = argmax(lg.data_ptr<float>()) + (int64_t)(seq.size() - 1) * V, V);
        ref.push_back(nxt); seq.push_back(nxt);
    }
    CHECK(gen == ref); // cache + loop == from-scratch
  ↳ reference
}
```

```

}

TEST(boss_generate_stops_at_eos) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    SamplingParams sp; sp.greedy = true;
    const std::vector<int> prompt = {3, 7, 1};
    RNG r0(0);
    auto full = generate(m, c, prompt, 5, /*eos=*/-1, sp, r0);
    REQUIRE(!full.empty());
    const int first = full[0];           // make THAT token the EOS
    RNG r1(0);
    auto stopped = generate(m, c, prompt, 5, /*eos=*/first, sp, r1);
    CHECK(stopped.empty());             // EOS first -> nothing emitted
}

```

REFERENCE SOLUTION

Declare in `sampler.hpp` (or a `generate.hpp`), define alongside:

```

// generation
#include "llm/model.hpp"

inline std::vector<int> generate(const ModelWeights& model, const ModelConfig& cfg,
    ↪
                                const std::vector<int>& prompt, int max_new,
                                int eos_id, const SamplingParams& sp, RNG& rng) {
    const int V = cfg.vocab_size;
    KVCache cache = KVCache::allocate(cfg, (int)prompt.size() + max_new + 8);

    Tensor logits = forward_cached(model, cfg, prompt, cache);           // prefill
    int next = sample_token(logits.data_ptr<float>() + (int64_t)(prompt.size() - 1)
    ↪ * V,
                                V, sp, rng);                             // from
    ↪ the LAST position

    std::vector<int> out;
    for (int step = 0; step < max_new; ++step) {
        if (next == eos_id) break;                                       // stop,
    ↪ do not emit EOS
        out.push_back(next);
        Tensor l = forward_cached(model, cfg, {next}, cache);           // one
    ↪ cached decode step
        next = sample_token(l.data_ptr<float>(), V, sp, rng);
    }
}

```

```

    }
    return out;
}

```

Run the boss. Greedy generation is bit-stable across runs, an EOS first token produces nothing, and — the heart of it — the cached generation loop emits exactly the tokens the from-scratch recompute-and-argmax reference does. **Boss cleared.**

What you have proven. Not “a loop that emits tokens,” but “a generation loop that is *correct*”: it samples from the right position, advances the cache exactly once per token, stops at EOS without emitting it, and — proven against a reference that shares none of its machinery — produces precisely the sequence the model dictates. With the sampler beneath it, the same loop spans greedy decoding and tail-clipped sampling. Point it at a real model and a real tokenizer and it reads a prompt and writes a continuation: a complete, working inference engine.

GOING FURTHER

- **Streaming output.** Yield each token (and its decoded text, with the UTF-8 buffering from Unit 4) as it is produced, instead of returning the whole list — the interactive, token-by-token experience users expect.
- **Stop sequences and limits.** Add multiple stop tokens, stop *strings*, and a maximum total length; halt generation when any is hit — the controls a chat API exposes.
- **A real conversation.** Combine `Tokenizer::encode`, a chat template, `generate`, and `decode` into a tiny CLI that talks to a loaded GGUF model — the first end-to-end run of the whole engine.

Unit 11 — Milestone check

MILESTONE CHECK

The engine can now *generate text*:

- **samplers** spanning the full spectrum — greedy, temperature, top-*k*, and top-*p* (nucleus) — composed into one `sample_token`;

- a seeded **RNG** for reproducible sampling;
- the **generation loop** — prefill the prompt, sample, decode through the cache, repeat until `EOS` or a length cap; and
- a **boss-grade proof** that greedy generation is deterministic and matches a from-scratch recompute reference token for token.

With a model loaded from GGUF, a tokenizer to encode and decode, and this loop to drive it, the engine is a **complete, working, single-sequence inference engine**: prompt in, text out. What remains is to make it *faster* and serve it at *scale*. **Unit 12** (★) tackles the speed of sequential decoding with **speculative decoding** — using a small draft model to propose several tokens that the large model verifies in a single pass, accelerating generation two- to three-fold *without* changing the output distribution. Then Unit 13 serves many sequences at once, and Unit 14 brings the whole engine to life on a real model.

UNIT 12

Speculative Decoding

What you will build. The technique that breaks decode’s sequential bottleneck: **speculative decoding**. A small, fast **draft** model proposes several tokens; the large **target** model **verifies** them all in a single forward pass, accepting the longest correct prefix and filling in its own token at the first disagreement. You build the greedy verification, the **distribution-preserving** acceptance rule for sampling, and the speculative loop. The boss proves it changes *speed but not output*.

Where this fits. Decoding is one token per forward pass, and each pass streams the entire model from memory to compute almost nothing — it is **memory-bound**, and the hardware sits idle. Yet a single pass can score many positions for nearly the same memory cost. Speculative decoding exploits this: guess the next few tokens cheaply, verify them in one expensive pass, and emit several tokens at once — a two- to three-fold decode speedup with the output distribution *provably unchanged*.

The economics of decoding are lopsided. To produce one token, a forward pass must read *every* weight in the model — gigabytes — and with it perform a tiny amount of arithmetic. The bytes dominate: decoding is **memory-bandwidth-bound**, the compute units mostly waiting on weights to arrive. The galling part is that loading those weights costs almost the same whether the pass processes one new token or ten — prefill already exploits this, scoring a whole prompt in one pass. If only we could process *several new tokens* per pass during decode, we would amortize the weight-loading over all of them.

The obstacle is that decode is sequential: token $t+1$ depends on token t , so we cannot run the model on tokens we have not chosen yet. Speculative decoding’s move is to **guess them** — cheaply, with a small **draft** model — and then **verify the guesses in parallel** with one pass of the large **target** model. The target processes

the proposed tokens all at once, and we accept the longest prefix the target agrees with; at the first disagreement, the target’s own prediction supplies the correct token and the round ends. When the draft is good, most guesses are accepted, so each expensive target pass yields multiple tokens.

The remarkable part is that this is **free of quality cost**. With the right acceptance rule, the emitted tokens are distributed *exactly* as if drawn from the target model alone — the draft model influences only how many tokens come out per pass, never which tokens are possible. We build it in three steps: greedy drafting and verification (where “agree” means “same argmax”), the distribution-preserving acceptance that extends this to sampling, and the full loop — with a boss certifying that speculative greedy decoding is *identical* to plain greedy decoding while emitting more than one token per target pass. Add `include/llm/speculative.hpp`.

12.1 Drafting and Parallel Verification

GOAL

One speculative round, greedy: the draft model proposes K tokens; the target model verifies them in a **single forward pass**; we accept the longest prefix where the draft matches the target’s greedy choice and emit one correction at the first mismatch (or a bonus token if all K are accepted).

BACKGROUND

A greedy speculative round has three steps. First, **draft**: run the small draft model to propose K candidate tokens, by greedy decoding from the current committed sequence. This is sequential but cheap — the draft model is small, so its K passes cost far less than one target pass. Second, **verify**: run the target model *once* over the committed sequence followed by the K draft tokens. A single forward pass yields the target’s logits at every one of those positions — and crucially, the logits at position $L-1+i$ give the target’s own prediction for what should follow the first i drafts. Third, **accept**: walk the drafts in order; the target “agrees” with draft i when d_i equals the target’s greedy choice (argmax) at the preceding position. Accept the matching prefix; at the *first* mismatch, emit the target’s choice there instead — the **correction** — and stop. If all K are accepted, the target’s

prediction at the final position is a free **bonus** token.

The payoff is the second step. Verifying K guesses takes **one** target forward pass, not K — and with the kv cache (Unit 7) that pass processes only the K new positions, so it costs about the same as a single ordinary decode step while potentially producing up to $K+1$ tokens. A round thus emits between 1 token (everything rejected — just the correction) and $K+1$ tokens (all accepted plus the bonus), and the average over a generation is the speedup factor.

The correctness of the greedy case is immediate and worth stating, because it is what makes the technique safe. **Every emitted token is the target's own greedy choice** given the committed prefix: accepted drafts are emitted only because they *equal* the target's argmax, and the correction *is* the target's argmax. So speculative greedy decoding produces precisely the same tokens as plain greedy decoding of the target — the draft model cannot alter the output, only the number of target passes needed to produce it. A useless draft costs speed; it never costs correctness. (Here we use full recomputation for clarity; production replaces both the draft's and target's passes with cached ones — the target verifying in a single $O(K)$ cached pass with a rollback when drafts are rejected — which is where the wall-clock win is realized.)

THE TASK

Implement `speculative_round_greedy(target, tc, draft, dc, seq, K)`: draft K tokens greedily from the draft model; run one target forward over `seq` plus the drafts; accept the matching prefix (`d[i]` equals the target argmax at position `L-1+i`); on the first mismatch set the **correction** to the target's token there, or if all K match set it to the target's **bonus** at the final position; return the accepted drafts followed by the correction.

HINTS

Hint 1 — draft greedily

```
std::vector<int> d, dseq = seq; for j in [0,K): forward the draft
on dseq, argmax the last position, push to d and dseq.    The draft
extends its own working sequence as it goes.
```

Hint 2 — verify in one pass

cand = seq + d; Tensor tl = forward(target, tc, cand);. The target's prediction for what follows the first i drafts is argmax of tl row L-1+i, where L = seq.size().

Hint 3 — accept, then one correction

Loop a from 0; if d[a] differs from the target argmax at L-1+a, set correction to that argmax and break. If the loop reaches K, set correction to the argmax at L-1+K (the bonus). Emit d[0..a-1] then correction.

TESTS

```
// tests/test_speculative.cpp
// (reuses tiny_model/tiny_cfg from model tests, forward, argmax)
#include "test_framework.hpp"
#include "llm/speculative.hpp"
#include "llm/model.hpp"
#include "llm/sampler.hpp"
using namespace llm;

// the target's greedy continuation of a sequence
static int target_greedy(const ModelWeights& m, const ModelConfig& c, const std::
    ↪ vector<int>& s) {
    Tensor l = forward(m, c, s);
    return argmax(l.data_ptr<float>() + (int64_t)(s.size() - 1) * c.vocab_size, c.
    ↪ vocab_size);
}

TEST(spec_round_same_model_accepts_all) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    const std::vector<int> seq = {3, 7, 1};
    auto emit = speculative_round_greedy(m, c, m, c, seq, /*K=*/4); // draft ==
    ↪ target
    CHECK((int)emit.size() == 5); // K
    ↪ accepted + bonus
}

TEST(spec_round_emits_target_greedy_regardless_of_draft) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights target = tiny_model(c, 4);
```

```

    const ModelWeights draft = tiny_model(c, 99); // a poor,
    ↪ different draft
    const std::vector<int> seq = {5, 2, 8};
    auto emit = speculative_round_greedy(target, c, draft, c, seq, 4);
    REQUIRE(!emit.empty());
    std::vector<int> s = seq; // every emit
    ↪ == target greedy
    for (int tok : emit) { CHECK(tok == target_greedy(target, c, s)); s.push_back(
    ↪ tok); }
}

```

REFERENCE SOLUTION

include/llm/speculative.hpp:

```

// include/llm/speculative.hpp
#pragma once
#include <vector>
#include "llm/model.hpp"
#include "llm/sampler.hpp" // argmax

namespace llm {

// One greedy speculative round: returns the tokens to emit (accepted drafts + one
    ↪ correction).
inline std::vector<int> speculative_round_greedy(
    const ModelWeights& target, const ModelConfig& tc,
    const ModelWeights& draft, const ModelConfig& dc,
    const std::vector<int>& seq, int K) {
    // 1. draft K tokens greedily from the draft model
    std::vector<int> d, dseq = seq;
    for (int j = 0; j < K; ++j) {
        Tensor dl = forward(draft, dc, dseq);
        int dt = argmax(dl.data_ptr<float>()) + (int64_t)(dseq.size() - 1) * dc.
        ↪ vocab_size,
            dc.vocab_size);
        d.push_back(dt); dseq.push_back(dt);
    }
    // 2. verify all K in ONE target forward pass
    std::vector<int> cand = seq;
    cand.insert(cand.end(), d.begin(), d.end());
    Tensor tl = forward(target, tc, cand);
    const int V = tc.vocab_size, L = (int)seq.size();
}

```

```

// 3. accept the matching prefix; one correction (or bonus) at the end
int a = 0, correction = 0;
for (a = 0; a < K; ++a) {
    int ti = argmax(tl.data_ptr<float>() + (int64_t)(L - 1 + a) * V, V);
    if (d[a] != ti) { correction = ti; break; }           // first mismatch
}
if (a == K)                                             // all accepted ->
    ↪ bonus token
    correction = argmax(tl.data_ptr<float>() + (int64_t)(L - 1 + K) * V, V);

std::vector<int> emit(d.begin(), d.begin() + a);
emit.push_back(correction);
return emit;
}

} // namespace llm

```

A round turns one target forward pass into up to $K+1$ committed tokens, and — as the second test insists — those tokens are exactly the target’s greedy continuation no matter how bad the draft is. That invariance is the soul of speculative decoding: the verification is a filter that lets through only what the target itself would have produced. Greedy makes “agree” mean “same argmax,” which is clean but limited to deterministic decoding. To accelerate *sampling* without distorting it takes a subtler acceptance rule — the next challenge.

GOING FURTHER

- **Cached verification.** Replace the full forward calls with cached ones: the target verifies in a single pass over the K new positions, and on a rejection the cache is rolled back to the accepted length. This is where the real wall-clock speedup comes from.
- **Self-speculation.** Use a few of the target’s own early layers (or a smaller quantization) as the draft, avoiding a separate model entirely — “Medusa” and “self-speculative” decoding.
- **n-gram drafts.** Draft by looking up recent context in an n-gram table or the prompt itself — zero model cost, and surprisingly effective for repetitive or retrieval-style text.

12.2 Distribution-Preserving Acceptance

GOAL

The acceptance rule that makes speculation *exact* for sampling: accept a draft token drawn from the draft distribution q with probability $\min(1, p(x)/q(x))$ against the target distribution p , and on rejection resample from the residual $(p-q)_+$. The emitted token is then distributed **exactly** as p — the target — no matter what the draft does.

BACKGROUND

Greedy verification accepted a draft token when it matched the target’s argmax. For **sampling** that rule is wrong: only ever emitting draft tokens that hit the target’s mode would starve the tail and distort the distribution. We need an acceptance rule that accelerates sampling while leaving the output distributed *exactly* as the target would sample it. Remarkably, one exists, and it is simple.

The draft token x is drawn from the draft’s distribution q at this position. Let p be the target’s distribution at the same position. **Accept** x with probability

$$\alpha = \min\left(1, \frac{p(x)}{q(x)}\right).$$

If the target likes x at least as much as the draft did ($p(x) \geq q(x)$), accept unconditionally; otherwise accept proportionally to how much less. If x is **rejected**, resample a token from the **residual** distribution — the part of the target’s mass the draft undersampled —

$$p'(i) = \frac{\max(0, p(i) - q(i))}{\sum_j \max(0, p(j) - q(j))}.$$

The token finally emitted — accepted x , or a residual resample — is the round’s output for this position.

The theorem behind this (Leviathan et al.; Chen et al., 2023) is the crux of the whole technique: **the emitted token is distributed exactly as p , for any draft distribution q .** The two pieces conspire to make it so. The probability of drafting and accepting token i is $q(i) \cdot \min(1, p(i)/q(i)) = \min(p(i), q(i))$ — the overlap of the two distributions. The probability mass lost to rejection is exactly

$\sum_i \max(0, p(i) - q(i))$, and the residual distribution redistributes it in proportion to $\max(0, p(i) - q(i))$. Add the accepted overlap to the resampled residual and every token i comes out with total probability $p(i)$. So sampling through speculative acceptance is mathematically identical to sampling from the target directly: the draft changes the *speed* (how often tokens are accepted) but provably never the *distribution*. This is the precise sense in which speculative decoding is free.

The greedy case of 12.1 is the limit of this rule when both distributions are point masses at their argmax: acceptance becomes “same token,” and the residual becomes “the target’s argmax” — which is exactly the correction we used. So one mechanism covers both regimes; sampling just needs the probabilistic version.

THE TASK

Implement `speculative_accept(p, q, n, x, rng)`: accept x with probability $\min(1, p[x]/q[x])$ (returning x); otherwise sample and return a token from the normalized residual $\max(0, p[i]-q[i])$. (p and q are probability vectors of length n ; reuse the RNG from Unit 11.)

HINTS

Hint 1 — the accept test

```
float a = (q[x] > 0) ? std::min(1.0f, p[x]/q[x]) : 1.0f; if
(rng.next_float() < a) return x;. Guard q[x] == 0 (accept, since
the target could only have more mass there).
```

Hint 2 — the residual is $(p - q)$ clipped

On rejection: $\text{sum} = \text{sum of } \max(0, p[i]-q[i])$; $r = \text{rng.next_float()} * \text{sum}$; then inverse-CDF over the clipped residual, returning the first i whose cumulative residual passes r .

Hint 3 — the test that matters

Draw x from q (with `sample_categorical`) and pass it through `speculative_accept` many times; the histogram of emitted tokens must match p , not q . That is the distribution-preservation theorem made empirical.

TESTS

```

// tests/test_speculative.cpp (continued; uses sampler.hpp)
#include "llm/sampler.hpp"
#include <cmath>

TEST(speculative_accept_emits_target_distribution) {
    const int n = 4;
    float p[4] = {0.4f, 0.3f, 0.2f, 0.1f}; // target
    float q[4] = {0.1f, 0.2f, 0.3f, 0.4f}; // draft -- deliberately very
    ↪ different
    int count[4] = {0, 0, 0, 0};
    RNG rng(1);
    const int N = 200000;
    for (int t = 0; t < N; ++t) {
        int x = sample_categorical(q, n, rng); // draft draws from q
        int emit = speculative_accept(p, q, n, x, rng);
        count[emit]++;
    }
    for (int i = 0; i < n; ++i)
        CHECK(std::fabs(count[i] / (float)N - p[i]) < 0.02f); // emitted ~ p,
    ↪ not q
}

TEST(speculative_accept_certain_when_p_equals_q) {
    const int n = 3;
    float p[3] = {0.5f, 0.3f, 0.2f}, q[3] = {0.5f, 0.3f, 0.2f}; // identical
    RNG rng(4);
    for (int t = 0; t < 1000; ++t) {
        int x = t % n;
        CHECK(speculative_accept(p, q, n, x, rng) == x); // p==q ->
    ↪ always accept
    }
}

TEST(speculative_accept_probability_matches_min_ratio) {
    const int n = 3;
    float p[3] = {0.5f, 0.3f, 0.2f}, q[3] = {0.25f, 0.5f, 0.25f};
    // token 1: p/q = 0.6; residual mass at 1 is max(0, 0.3-0.5)=0, so emit==1 only
    ↪ if accepted
    int got1 = 0; RNG rng(2); const int N = 50000;
    for (int t = 0; t < N; ++t) if (speculative_accept(p, q, n, 1, rng) == 1) got1
    ↪ ++;
    CHECK(std::fabs(got1 / (float)N - 0.6f) < 0.02f); // accept prob =
    ↪ min(1, 0.6)
}

```

REFERENCE SOLUTION

Add to `speculative.hpp`:

```
// speculative.hpp
#include "llm/sampler.hpp" // RNG
#include <algorithm>

// Accept draft token x (drawn from q) against target p, or resample from the
// ↪ residual.
// The returned token is distributed exactly as p.
inline int speculative_accept(const float* p, const float* q, int n, int x, RNG&
// ↪ rng) {
    const float a = (q[x] > 0.0f) ? std::min(1.0f, p[x] / q[x]) : 1.0f;
    if (rng.next_float() < a) return x; // accept

    float sum = 0.0f; // residual (p -
// ↪ q)_+
    for (int i = 0; i < n; ++i) sum += std::max(0.0f, p[i] - q[i]);
    if (sum <= 0.0f) return x; // degenerate
// ↪ guard
    float r = rng.next_float() * sum, c = 0.0f;
    for (int i = 0; i < n; ++i) {
        c += std::max(0.0f, p[i] - q[i]);
        if (r < c) return i;
    }
    return n - 1;
}
```

The distribution test is the whole theorem in one assertion: tokens are *drawn* from q yet *emitted* according to p , because the accept-or-resample rule corrects the mismatch exactly. This is what lets speculative decoding accelerate temperature/top- p sampling without changing a model's behavior by even a fraction of a percent — the output is, distributionally, the target model's own. With greedy verification (12.1) and this sampling rule in hand, we can build the loop and prove the end-to-end guarantee.

GOING FURTHER

- **Chain acceptance.** Apply the rule down a chain of K drafts: accept token by token until the first rejection, resample there, and discard the rest. Prove the accepted prefix is distributed as the target's first tokens — the sampling analogue of 12.1's prefix.

- **Typical acceptance rate.** Show the expected number of accepted tokens equals $\sum_i \min(p_i, q_i)$ summed appropriately — the overlap of target and draft — which quantifies why a *well-aligned* draft accelerates most.
- **Temperature coupling.** Examine how the acceptance rate changes with temperature: at low temperature both distributions sharpen toward the argmax and acceptance approaches the greedy case; at high temperature drafts are rejected more often.

12.3 The Speculative Loop

BOSS CHALLENGE

Wrap drafting and verification into a generation loop, and prove the two things that matter together: speculative greedy decoding produces the **identical** token sequence to plain greedy decoding — for any draft, even a bad one — and with a good draft it emits **more than one token per target forward pass**. Same output, fewer expensive passes.

GOAL

The complete technique: a loop that drafts K tokens, verifies them in one target pass, emits the accepted prefix plus a correction, and repeats until `eos` or a length cap — certified to match plain greedy decoding exactly while reducing the number of target passes.

BACKGROUND

The speculative loop is the round of 12.1 run repeatedly: draft K , verify in a single target pass, emit the accepted drafts and the correction, and continue from the new committed sequence until the model emits `eos` or the limit is reached. (For sampling, the round uses 12.2’s acceptance rule in place of the argmax comparison; the loop is otherwise identical. We certify the greedy case, where everything is deterministic and the check can be exact.)

The boss measures the two properties that, *together*, define a correct speculative decoder — and either alone would be worthless. The first is **exactness**: speculative greedy decoding must produce the same tokens as plain greedy decoding of the target. We test this with a deliberately *poor* draft — a different model entirely

— and demand bit-identical output anyway. This is the strongest correctness statement available: the draft is a pure speed device that *cannot* change what the engine produces. (The sampling analogue is the distribution-equivalence proven in 12.2: emitted tokens are distributed exactly as the target.) A “speedup” that altered the output would be a bug, not a feature, so exactness is non-negotiable.

The second is **acceleration**: the mechanism must actually save expensive passes. We test this with a *perfect* draft — the target model itself — so every proposal matches and all K drafts plus the bonus are accepted each round. Then a round emits $K+1$ tokens per target forward pass, and the number of target passes is far below the number of tokens generated. Measuring **tokens emitted per target pass** and showing it exceeds one proves the speculative structure delivers its multiplication. An “exact” method that still needed one target pass per token would be pointless, so acceleration is equally essential.

Real speedups — the two- to threefold seen in practice — come from a draft that is *both* cheap (a small model, so its passes are nearly free) *and* well-aligned (so acceptance is high). The boss isolates the two factors: the poor-draft test shows correctness is independent of draft quality, and the perfect-draft test shows the acceleration the mechanism is capable of. A production draft lands between them — imperfect but fast and mostly right — and the average tokens per target pass is the realized speedup.

THE TASK

Implement `spec_generate_greedy(target, tc, draft, dc, prompt, max_new, K, eos_id, passes)`: loop the greedy round, emitting its tokens (honoring `eos_id` and `max_new`) and counting one target pass per round, until done. Then the boss tests: (1) with a different (poor) draft, the output equals plain greedy generate; (2) with the target as its own draft, tokens-per-target-pass exceeds 1.5.

The bar (the boss): speculative greedy output is identical to plain greedy output regardless of draft, and a perfect draft yields more than one emitted token per target forward pass.

HINTS

Hint 1 — loop the round

```
while (out.size() < max_new) { auto emit =
speculative_round_greedy(...); ++passes; for (tok : emit) { if
(tok==eos_id) stop; out.push_back(tok); if (out.size())>=max_new)
stop; } }. One target pass happens inside each round.
```

Hint 2 — count target passes, not draft passes

The speedup metric is *target* forward passes (the expensive ones). Increment passes once per round; the draft's K cheap passes do not count.

Hint 3 — compare against Unit 11's generate

Plain `greedy` is `generate(target, cfg, prompt, max_new, eos, {greedy=true}, rng)`. With `eos=-1` both loops run to `max_new`, making the sequences directly comparable.

TESTS

```
// add to tests/test_speculative.cpp (reuses tiny_model/tiny_cfg, generate)

TEST(boss_spec_greedy_matches_plain_greedy) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights target = tiny_model(c, 4);
    const ModelWeights draft = tiny_model(c, 99); // a poor, unrelated draft
    const std::vector<int> prompt = {3, 7, 1};

    SamplingParams sp; sp.greedy = true; RNG rng(0);
    auto plain = generate(target, c, prompt, /*max_new=*/8, /*eos=*/-1, sp, rng);
    auto spec = spec_generate_greedy(target, c, draft, c, prompt, 8, /*K=*/4, /*
↳ eos=*/-1, nullptr);
    CHECK(spec == plain); // identical, whatever the
↳ draft
}

TEST(boss_spec_perfect_draft_accelerates) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    const std::vector<int> prompt = {3, 7, 1};
    int passes = 0;
    auto out = spec_generate_greedy(m, c, m, c, prompt, /*max_new=*/12, /*K=*/4, /*
```

```

    ↪ eos=*/-1, &passes);
    REQUIRE(!out.empty());
    REQUIRE(passes > 0);
    CHECK((double)out.size() / passes > 1.5);           // > 1 token per target pass
}

```

REFERENCE SOLUTION

Add to `speculative.hpp`:

```

// speculative.hpp
inline std::vector<int> spec_generate_greedy(
    const ModelWeights& target, const ModelConfig& tc,
    const ModelWeights& draft,  const ModelConfig& dc,
    const std::vector<int>& prompt, int max_new, int K, int eos_id,
    int* target_passes) {
    std::vector<int> seq = prompt, out;
    int passes = 0; bool done = false;

    while (!done && (int)out.size() < max_new) {
        std::vector<int> emit = speculative_round_greedy(target, tc, draft, dc,
    ↪ seq, K);
        ++passes;                                     // one target forward pass
    ↪ per round
        for (int tok : emit) {
            if (tok == eos_id) { done = true; break; }
            out.push_back(tok); seq.push_back(tok);
            if ((int)out.size() >= max_new) { done = true; break; }
        }
    }
    if (target_passes) *target_passes = passes;
    return out;
}

```

Run the boss. With an unrelated draft, speculative greedy decoding still emits exactly the plain greedy sequence — the draft changed nothing but the path taken to get there. With the target as its own draft, every round accepts all K proposals plus the bonus, so a dozen tokens come out of a handful of target passes: more than one token per expensive pass. **Boss cleared.**

What you have proven. Not “a faster decoder,” but “a faster decoder that is *the same* decoder”: speculative greedy decoding produces token-for-token the output of plain greedy decoding, for any draft whatsoever, while a well-aligned draft

cuts the number of target forward passes below the token count. Paired with the distribution-preservation of 12.2, this is the full guarantee of speculative decoding — accelerate generation two- to threefold and change what the model says *not at all*. The draft is leverage on speed with zero leverage on output.

GOING FURTHER

- **The cached, sampling loop.** Combine cached verification (12.1's Going Further) with the sampling acceptance (12.2) into a production speculative loop, and confirm its output distribution matches plain sampling — the full technique at full speed.
- **Tune K .** Sweep the draft length K and measure realized tokens-per-pass; too small wastes the parallel verify, too large wastes drafting on tokens that will be rejected. The optimum depends on the draft's acceptance rate.
- **Tree drafts.** Have the draft propose a *tree* of continuations and verify many branches in one target pass ("Medusa"/"Eagle"), raising the accepted-token count per pass beyond a single chain.

Unit 12 — Milestone check

MILESTONE CHECK

The engine can now decode faster without changing what it produces:

- **greedy verification** — a draft proposes K tokens, the target accepts the longest matching prefix in one forward pass and corrects the first mismatch;
- **distribution-preserving acceptance** — the rejection-sampling rule that makes speculative *sampling* emit tokens distributed exactly as the target;
- the **speculative loop** tying drafting, verification, and emission together; and
- a **boss-grade proof** that speculative greedy decoding is identical to plain greedy decoding for any draft, while a good draft emits more than one token per target pass.

The engine generates correct text (Unit 11) and now generates it faster (Unit 12) — but always for *one* sequence at a time. Serving real traffic means many users

at once, and running their sequences one by one would waste the hardware utterly. **Unit 13** (★) is the serving unit: **batching** multiple sequences through the model together, **continuous batching** to admit and retire requests mid-flight, and a **paged kv cache** to pack many sequences' caches into memory without waste — turning the single-stream engine into one that serves a crowd. Then Unit 14 brings the whole engine to life on a real model.

UNIT 13

Serving and Batching

What you will build. The systems layer that serves many users at once: **batching**. You build **batched decode** (process several sequences in one forward pass, so the weights are loaded once and applied to all), a **paged kv cache** (pack many sequences' caches into a shared memory pool without waste), and a **continuous-batching** scheduler (admit and retire requests mid-flight to keep the batch full). The boss proves the server is both *correct* per request and *efficient* in total.

Where this fits. The engine so far serves one sequence at a time, and decode is memory-bound — each step streams the whole model to produce a single token, leaving the compute units idle. Running many users sequentially multiplies that waste. Batching amortizes the weight-loading across many sequences, raising throughput severalfold; paged caching and continuous batching make it practical at scale. This is the architecture of production serving systems like vLLM and TGI.

The inefficiency of single-stream decoding is, at its root, an idle-hardware problem. Each decode step reads every weight from memory and does a tiny amount of arithmetic, so the expensive resource — memory bandwidth — is spent loading weights that compute just one token. The crucial observation is that loading those weights costs *the same* whether the forward pass processes one sequence's token or a hundred: a matmul against a weight matrix reads the matrix once regardless of how many input rows it multiplies. So if we run **many sequences' tokens through one forward pass**, the weight-loading is amortized across all of them, and throughput climbs with the batch size until the model finally becomes compute-bound.

Serving real traffic, then, is the art of keeping a large, full batch flowing through the model. Three mechanisms make it work. **Batched decode** runs B sequences

in one pass — the projections and feed-forward become larger matmuls over B rows, while attention remains per-sequence (each sequence attends its own history). A **paged kv cache** solves the memory problem that batching creates: many sequences of unknown, varying length cannot each preallocate a maximum-length cache without enormous waste, so we page the cache into fixed-size blocks drawn from a shared pool, like virtual memory. And **continuous batching** keeps the batch full by admitting new requests and retiring finished ones *between* iterations, rather than running fixed batches to completion. We build them in that order, with a boss that certifies the server returns each request’s correct output while doing far less total work than serving the requests one by one. Add `include/llm/serving.hpp`.

13.1 Batched Decode

GOAL

Process a batch of sequences in a single forward pass: embed B tokens (one per sequence) into a $[B, \text{hidden}]$ activation, run the projections and FFN as batched matmuls, attend each sequence over *its own* cache, and produce $[B, \text{vocab}]$ logits — amortizing the weight-loading across the whole batch.

BACKGROUND

A batched decode step looks almost like the single-sequence step, with one distinction that drives the whole design: **what batches and what does not**. The **linear** operations — the query, key, value, and output projections, and the feed-forward matmuls — batch trivially. Stacking B sequences’ hidden vectors into a $[B, \text{hidden}]$ matrix and multiplying by a weight matrix is just a larger matmul over more rows, reading the weights exactly once; this is where the throughput comes from, and it is why the matmul work of Unit 3 was worth doing well. **Attention**, however, does *not* batch across sequences: each sequence has a different cache, a different length, and a different position, so the score/softmax/weighted-sum must be computed per sequence against its own keys and values.

So a batched step is: embed the B tokens into $[B, \text{hidden}]$; then for each layer, normalize the batch, run the **batched** Q/K/V projections, **loop** over sequences to apply RoPE, append to each sequence’s cache, and attend within it, then run

the **batched** output projection and residual, and the **batched** FFN and residual; advance every cache by one; finish with a batched final norm and output projection to $[B, \text{vocab}]$. The expensive matmuls process all B rows at once; only the (comparatively cheap, and memory-light after Flash Attention) attention loops per sequence.

The property that matters — and that the test pins down — is that batching changes **throughput, not results**. A batched decode of B sequences produces exactly what decoding each sequence independently would: the matmuls compute the same per-row outputs whether the rows are processed together or apart, and each sequence’s attention is unchanged because it still sees only its own cache. Batching is an efficiency transformation with no effect on any sequence’s output, which is what lets a server batch arbitrary unrelated requests together without consequence.

THE TASK

Implement `rope_inplace(vec, n_heads, head_dim, theta, pos)` (rotate one token’s head vectors); `batched_decode_block(x, w, cfg, caches, layer)` (batched norms/projections/FFN, per-sequence RoPE/append/attention); and `batched_decode(model, cfg, caches, tokens)` (embed the batch, run the blocks, advance each cache, final norm + `lm_head`, return the greedy next token per sequence). Each sequence carries its own KVCache.

HINTS

Hint 1 — batch the matmuls, loop the attention

`linear(q, normed, w.wq)` processes all B rows at once. Then for $(b \text{ in } 0..B)$: RoPE row b of q/k , memcpy into `caches[b]` at `caches[b].length`, and run the per-head score/softmax/weighted-sum over `caches[b]` into row b of the attention output.

Hint 2 — single-token RoPE

`rope_inplace` is `apply_rope` for one token: for each head, rotate the `head_dim/2` planes by `pos * theta^(-2i/head_dim)`. Apply it to row b of q (n_heads) and k (n_kv_heads) at `pos = caches[b].length`.

Hint 3 — advance once per sequence, after the layers

Each layer appends at the same `caches[b].length`; after all layers, `caches[b].advance(1)` for every sequence. Then the batched final norm and `lm_head` give `[B, vocab]`.

TESTS

```
// tests/test_serving.cpp
// (reuses tiny_model/tiny_cfg, forward_cached, argmax)
#include "test_framework.hpp"
#include "llm/serving.hpp"
#include "llm/model.hpp"
#include "llm/sampler.hpp"
using namespace llm;

TEST(batched_decode_equals_independent_decode) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    const std::vector<std::vector<int>> prompts = {{3,7,1}, {5,2}, {9,4,6,8}};
    const int B = 3, V = c.vocab_size;

    std::vector<KVCache> bcache, rcache;    // batched caches and reference
    ↪ caches
    std::vector<int> feed(B), ref_next(B);
    for (int b = 0; b < B; ++b) {
        bcache.push_back(KVCache::allocate(c, 64));
        rcache.push_back(KVCache::allocate(c, 64));
        Tensor lb = forward_cached(m, c, prompts[b], bcache[b]);    // prefill
        ↪ both caches
        Tensor lr = forward_cached(m, c, prompts[b], rcache[b]);
        feed[b] = argmax(lb.data_ptr<float>()) + (int64_t)(prompts[b].size()-1)*V, V
        ↪ );
        Tensor rn = forward_cached(m, c, {feed[b]}, rcache[b]);    // reference:
        ↪ decode alone
        ref_next[b] = argmax(rn.data_ptr<float>(), V);
    }

    std::vector<KVCache*> ptrs;
    for (auto& ca : bcache) ptrs.push_back(&ca);
    std::vector<int> bnext = batched_decode(m, c, ptrs, feed);    // one
    ↪ batched step

    for (int b = 0; b < B; ++b) CHECK(bnext[b] == ref_next[b]);    // batch ==
```

```

    ↪ independent
}

```

REFERENCE SOLUTION

include/llm/serving.hpp:

```

// include/llm/serving.hpp
#pragma once
#include <vector>
#include <cmath>
#include <cstring>
#include "llm/model.hpp"
#include "llm/ops.hpp"
#include "llm/sampler.hpp"

namespace llm {

inline void rope_inplace(float* v, int n_heads, int head_dim, float theta, int pos)
    ↪ {
    const int half = head_dim / 2;
    for (int h = 0; h < n_heads; ++h) {
        float* hv = v + h * head_dim;
        for (int i = 0; i < half; ++i) {
            const float freq = std::pow(theta, -2.0f * i / head_dim);
            const float ang = pos * freq, c = std::cos(ang), s = std::sin(ang);
            const float x0 = hv[i], x1 = hv[i + half];
            hv[i] = x0 * c - x1 * s; hv[i + half] = x1 * c + x0 * s;
        }
    }
}

inline void batched_decode_block(Tensor& x, const LayerWeights& w, const
    ↪ ModelConfig& cfg,
                                std::vector<KVCache*>& caches, int layer) {
    const int B = (int)x.shape()[0], hidden = cfg.hidden_size;
    const int H = cfg.n_heads, KVH = cfg.n_kv_heads, D = cfg.head_dim;
    const int qd = H * D, kvd = KVH * D, group = H / KVH;
    const float scale = 1.0f / std::sqrt((float)D);

    Tensor normed = Tensor::empty({B, hidden}, DType::F32);
    rmsnorm(normed, x, w.attn_norm, cfg.rms_norm_eps);
    Tensor q = Tensor::empty({B, qd}, DType::F32);
    Tensor k = Tensor::empty({B, kvd}, DType::F32);

```

```

Tensor v = Tensor::empty({B, kvd}, DType::F32);
linear(q, normed, w.wq); linear(k, normed, w.wk); linear(v, normed, w.wv);  //
↳ BATCHED

Tensor attn = Tensor::empty({B, qd}, DType::F32);
for (int b = 0; b < B; ++b) {  // per-
↳ sequence attention
    KVCache& cache = *caches[b];
    const int pos0 = cache.length;
    float* qb = q.data_ptr<float>() + (int64_t)b * qd;
    float* kb = k.data_ptr<float>() + (int64_t)b * kvd;
    float* vb = v.data_ptr<float>() + (int64_t)b * kvd;
    rope_inplace(qb, H, D, cfg.rope_theta, pos0);
    rope_inplace(kb, KVH, D, cfg.rope_theta, pos0);
    std::memcpy(cache.k[layer].data_ptr<float>() + (int64_t)pos0 * kvd, kb,
↳ sizeof(float)*kvd);
    std::memcpy(cache.v[layer].data_ptr<float>() + (int64_t)pos0 * kvd, vb,
↳ sizeof(float)*kvd);
    const float* Kc = cache.k[layer].data_ptr<float>();
    const float* Vc = cache.v[layer].data_ptr<float>();
    float* ob = attn.data_ptr<float>() + (int64_t)b * qd;
    std::vector<float> sc(pos0 + 1);
    for (int h = 0; h < H; ++h) {
        const int kvh = h / group; const float* qh = qb + h * D;
        float mx = -INFINITY;
        for (int j = 0; j <= pos0; ++j) {
            const float* kj = Kc + ((int64_t)j * KVH + kvh) * D;
            float dot = 0; for (int d = 0; d < D; ++d) dot += qh[d] * kj[d];
            dot *= scale; sc[j] = dot; if (dot > mx) mx = dot;
        }
        float sum = 0; for (int j = 0; j <= pos0; ++j) { float e = std::exp(sc
↳ [j]-mx); sc[j]=e; sum+=e; }
        float inv = 1.0f / sum; float* oh = ob + h * D;
        for (int d = 0; d < D; ++d) oh[d] = 0;
        for (int j = 0; j <= pos0; ++j) {
            const float wj = sc[j]*inv; const float* vj = Vc + ((int64_t)j*KVH
↳ + kvh)*D;
            for (int d = 0; d < D; ++d) oh[d] += wj * vj[d];
        }
    }
}
Tensor ao = Tensor::empty({B, hidden}, DType::F32);
linear(ao, attn, w.wo); add_inplace(x, ao);  // BATCHED
rmsnorm(normed, x, w.ffn_norm, cfg.rms_norm_eps);
Tensor ff = Tensor::empty({B, hidden}, DType::F32);
ffn(ff, normed, w, cfg); add_inplace(x, ff);  // BATCHED
}

```

```

inline std::vector<int> batched_decode(const ModelWeights& m, const ModelConfig&
    ↪ cfg,
                                   std::vector<KVCache*>& caches, const std::
    ↪ vector<int>& tokens) {
    const int B = (int)tokens.size(), V = cfg.vocab_size, hidden = cfg.hidden_size;
    ↪
    Tensor x = Tensor::empty({B, hidden}, DType::F32);
    for (int b = 0; b < B; ++b)
        std::memcpy(x.data_ptr<float>() + (int64_t)b * hidden,
                    m.token_embedding.data_ptr<float>() + (int64_t)tokens[b] *
    ↪ hidden,
                    sizeof(float) * hidden);
    for (int l = 0; l < cfg.n_layers; ++l) batched_decode_block(x, m.layers[l],
    ↪ cfg, caches, l);
    for (int b = 0; b < B; ++b) caches[b]->advance(1);

    Tensor normed = Tensor::empty({B, hidden}, DType::F32);
    rmsnorm(normed, x, m.final_norm, cfg.rms_norm_eps);
    Tensor logits = Tensor::empty({B, V}, DType::F32);
    linear(logits, normed, m.lm_head);

    std::vector<int> next(B);
    for (int b = 0; b < B; ++b) next[b] = argmax(logits.data_ptr<float>() + (
    ↪ int64_t)b * V, V);
    return next;
}

} // namespace llm

```

The test makes the central guarantee concrete: three sequences decoded together produce exactly the tokens they would have produced apart. The batched projections did the heavy arithmetic for all three at once — reading the weights a single time — while each sequence’s attention stayed private to its own cache. That is the whole of batched inference: amortize the matmuls, isolate the attention, change nothing about any result. What batching needs next is a way to hold all those per-sequence caches in memory without waste — paging.

GOING FURTHER

- **Batched (ragged) attention.** The per-sequence attention loop is the part that does not batch; a real engine fuses it with a Flash-Attention kernel (Unit 8) that handles variable lengths (“ragged” batches) in one launch. Wire

flash_attention into the per-sequence step.

- **Prefill batching.** Batch the prompts' prefill too, packing several variable-length prompts into one padded (or ragged) pass — the other half of a server's work besides decode.
- **Per-sequence sampling.** Return $[B, \text{vocab}]$ logits and apply each request's own SamplingParams and RNG, so different users can use different temperatures in the same batch.

13.2 Paged KV Cache

GOAL

A **paged** kv cache: divide each sequence's cache into fixed-size pages drawn from a shared pool, mapped by a per-sequence **block table**. Allocate pages on demand as sequences grow, free them on completion, and address any logical position through the table — eliminating the per-sequence preallocation waste that caps batch size.

BACKGROUND

Batching creates a memory problem. Unit 7's cache preallocated a full $[\text{max_seq}, \text{n_kv} \times \text{head_dim}]$ buffer per layer for one sequence — fine for a single stream, ruinous for a server. With many concurrent sequences of unknown, varying length, reserving the maximum for each wastes most of the memory, because most sequences are far shorter than the maximum and you cannot know any sequence's final length when it begins. That reserved-but-empty memory is pure loss, and it directly limits how many sequences fit at once — which limits the batch size, which limits throughput. The waste is the bottleneck.

The fix, introduced by vLLM as **PagedAttention**, is to manage the cache like **virtual memory**. Divide the logical cache into fixed-size **pages** (blocks) of, say, 16 tokens. Keep a shared **pool** of physical pages and a **free list**. Each sequence holds a **block table** — a list mapping its logical pages $(0, 1, 2, \dots)$ to physical page ids in the pool. As a sequence grows across a page boundary, allocate one fresh physical page from the pool and append it to the table; when the sequence finishes, return all its pages to the pool. Attention reads through the table: logical position p lives in physical page $\text{block_table}[p / \text{page_size}]$ at slot $p \bmod \text{page_size}$.

The sequence sees a contiguous cache; physically its pages are scattered, and the block table is the page table that hides the scattering.

The payoff is the elimination of waste. Memory is handed out one page at a time, exactly as sequences actually grow, and a freed page is immediately reusable by any other sequence. The only internal fragmentation is the partially filled last page of each sequence — at most one page wasted per sequence, versus an entire max-length buffer. In practice this packs several times as many sequences into the same memory, which is what enables the large batches that make batching pay. A physical page holds the keys and values for its tokens across *all* layers, so the block table is shared by every layer — one mapping per sequence, not one per layer.

THE TASK

Implement `PagedKVCache` (`create`, `alloc_page/free_page` over a free list, `row_index(page, layer, slot)` into flat `kpool/vpool`); a `SequenceCache` (`block_table`, `length`); `ensure_page` (allocate when length crosses a page boundary); `write_kv` (store one token's K/V at a layer); `read_k/read_v` (address a logical position); and `free_sequence` (return pages to the pool).

HINTS

Hint 1 — the address arithmetic

A physical page stores $n_layers * page_size$ rows of kvd floats. $row_index(page, layer, slot) = (((page * n_layers + layer) * page_size + slot) * kvd)$ indexes into the flat `kpool`. The same formula serves `vpool`.

Hint 2 — allocate at page boundaries

`ensure_page`: $lp = length / page_size$; if lp is beyond the block table, pop a page from the free list and push it. Call once per token *before* writing layers; the page is shared by all layers of that token.

Hint 3 — free list as a stack

`alloc_page` pops the back, `free_page` pushes it — last-in-first-out, so a just-freed page is the next allocated. Initialize the list so page 0 is allocated first (push ids in reverse).

TESTS

```

// tests/test_serving.cpp (continued)

TEST(paged_cache_stores_and_reads_back) {
    const ModelConfig c = tiny_cfg(); // n_layers=2, kvd = 1*4
    ↪ = 4
    PagedKVCache pool = PagedKVCache::create(c, /*page_size=*/4, /*num_pages=*/8);
    SequenceCache seq;
    const int kvd = c.n_kv_heads * c.head_dim;
    for (int t = 0; t < 6; ++t) { // 6 tokens -> 2 pages of
    ↪ size 4
        REQUIRE(ensure_page(pool, seq));
        std::vector<float> k0(kvd, (float)t), v0(kvd, (float)(100 + t));
        std::vector<float> k1(kvd, (float)(t+10)), v1(kvd, (float)(200 + t));
        write_kv(pool, seq, 0, k0.data(), v0.data());
        write_kv(pool, seq, 1, k1.data(), v1.data());
        seq.length++;
    }
    CHECK((int)seq.block_table.size() == 2);
    for (int t = 0; t < 6; ++t) { // read back, both layers
        CHECK_CLOSE(read_k(pool, seq, 0, t)[0], (float)t, 1e-6);
        CHECK_CLOSE(read_v(pool, seq, 0, t)[0], (float)(100+t), 1e-6);
        CHECK_CLOSE(read_k(pool, seq, 1, t)[0], (float)(t+10), 1e-6);
        CHECK_CLOSE(read_v(pool, seq, 1, t)[0], (float)(200+t), 1e-6);
    }
}

TEST(paged_cache_reuses_freed_pages) {
    const ModelConfig c = tiny_cfg();
    PagedKVCache pool = PagedKVCache::create(c, 2, 4);
    int a = pool.alloc_page(), b = pool.alloc_page(); // 0, 1
    CHECK(a == 0); CHECK(b == 1);
    pool.free_page(a);
    CHECK(pool.alloc_page() == a); // freed page reused (
    ↪ LIFO)
}

TEST(paged_cache_two_sequences_share_pool) {
    const ModelConfig c = tiny_cfg();
    PagedKVCache pool = PagedKVCache::create(c, 2, 8);
    SequenceCache s1, s2;
    const int kvd = c.n_kv_heads * c.head_dim;
    for (int t = 0; t < 3; ++t) { // interleaved growth
        ensure_page(pool, s1);
        std::vector<float> a(kvd, (float)t); write_kv(pool, s1, 0, a.data(),

```

```

    ↪ a.data()); s1.length++;
        ensure_page(pool, s2);
        std::vector<float> b(kvd, (float)(t+50)); write_kv(pool, s2, 0, b.data(),
    ↪ b.data()); s2.length++;
    }
    for (int t = 0; t < 3; ++t) {
        CHECK_CLOSE(read_k(pool, s1, 0, t)[0], (float)t, 1e-6);
        CHECK_CLOSE(read_k(pool, s2, 0, t)[0], (float)(t+50), 1e-6); // no cross-
    ↪ contamination
    }
    for (int p1 : s1.block_table) for (int p2 : s2.block_table) CHECK(p1 != p2);
    ↪ // disjoint
}

```

REFERENCE SOLUTION

Add to serving.hpp:

```

// serving.hpp
struct PagedKVCache {
    int page_size = 0, n_layers = 0, kvd = 0, num_pages = 0;
    std::vector<float> kpool, vpool; // [num_pages * n_layers * page_size *
    ↪ kvd]
    std::vector<int> free_list; // available physical page ids (a stack)

    static PagedKVCache create(const ModelConfig& cfg, int page_size, int
    ↪ num_pages) {
        PagedKVCache p;
        p.page_size = page_size; p.n_layers = cfg.n_layers;
        p.kvd = cfg.n_kv_heads * cfg.head_dim; p.num_pages = num_pages;
        const int64_t cells = (int64_t)num_pages * cfg.n_layers * page_size * p.
    ↪ kvd;
        p.kpool.assign(cells, 0.0f); p.vpool.assign(cells, 0.0f);
        p.free_list.resize(num_pages);
        for (int i = 0; i < num_pages; ++i) p.free_list[i] = num_pages - 1 - i; //
    ↪ page 0 on top
        return p;
    }
    int alloc_page() { if (free_list.empty()) return -1; int p = free_list.back();
    ↪ free_list.pop_back(); return p; }
    void free_page(int p) { free_list.push_back(p); }
    int64_t row_index(int page, int layer, int slot) const {
        return (((int64_t)page * n_layers + layer) * page_size + slot) * kvd;
    }
}

```

```

};

struct SequenceCache {
    std::vector<int> block_table;           // logical page -> physical page id
    int length = 0;
};

inline bool ensure_page(PagedKVCache& pool, SequenceCache& seq) {
    const int lp = seq.length / pool.page_size;
    if (lp < (int)seq.block_table.size()) return true;
    const int pg = pool.alloc_page();
    if (pg < 0) return false;             // pool exhausted
    seq.block_table.push_back(pg);
    return true;
}

inline void write_kv(PagedKVCache& pool, const SequenceCache& seq, int layer,
                    const float* k_row, const float* v_row) {
    const int lp = seq.length / pool.page_size, slot = seq.length % pool.page_size;
    ↪
    const int page = seq.block_table[lp];
    std::memcpy(pool.kpool.data() + pool.row_index(page, layer, slot), k_row,
    ↪ sizeof(float)*pool.kvd);
    std::memcpy(pool.vpool.data() + pool.row_index(page, layer, slot), v_row,
    ↪ sizeof(float)*pool.kvd);
}

inline const float* read_k(const PagedKVCache& pool, const SequenceCache& seq, int
    ↪ layer, int p) {
    return pool.kpool.data() + pool.row_index(seq.block_table[p/pool.page_size],
    ↪ layer, p%pool.page_size);
}

inline const float* read_v(const PagedKVCache& pool, const SequenceCache& seq, int
    ↪ layer, int p) {
    return pool.vpool.data() + pool.row_index(seq.block_table[p/pool.page_size],
    ↪ layer, p%pool.page_size);
}

inline void free_sequence(PagedKVCache& pool, SequenceCache& seq) {
    for (int pg : seq.block_table) pool.free_page(pg);
    seq.block_table.clear(); seq.length = 0;
}

```

The tests confirm the three properties paging must have: a sequence's logical positions read back exactly what was written, across page boundaries and all layers; freed pages return to the pool and are reused; and independent sequences draw

disjoint pages from the same pool without collision. With this, a server no longer reserves a max-length cache per sequence — it hands out pages as sequences grow and reclaims them as sequences finish, fitting many more requests into the same memory. The pieces are now in place to schedule a stream of requests through the model continuously.

GOING FURTHER

- **Prefix sharing (copy-on-write).** Refcount physical pages so two sequences with a shared prompt prefix point at the *same* early pages, copying only when they diverge — the basis of cheap parallel sampling and shared system prompts.
- **Swapping and eviction.** When the pool is exhausted, swap a paused sequence's pages out to host memory (or drop and recompute them on resume), handling more concurrent sequences than fit in device memory — exactly OS demand paging.
- **Page-size trade-off.** Smaller pages cut the wasted tail but enlarge block tables and gather overhead; sweep the page size against memory utilization and attention speed to find the sweet spot for a workload.

13.3 Continuous Batching

BOSS CHALLENGE

Build a scheduler that admits and retires requests between iterations, running one batched decode step per iteration over whatever sequences are active. Prove the two things a server owes: every request's output is **correct** (identical to running it alone), and the whole workload finishes in **fewer model iterations** than serving the requests one by one.

GOAL

A continuous-batching server: a request queue, an active batch, and a loop that each iteration retires finished sequences, admits waiting ones to fill the freed slots, and runs a single batched decode step — keeping the batch full so the model is never idle and no request waits on an unrelated long one.

BACKGROUND

The naive way to batch is **static** batching: gather a fixed batch of requests, run them all to completion together, then start the next batch. It wastes the hardware in two ways. First, **head-of-line blocking** — the batch runs until its *longest* sequence finishes, so short requests sit idle inside the batch waiting on a long one. Second, **draining** — as sequences finish they leave empty slots that are not re-filled until the whole batch completes, so the effective batch shrinks over time and utilization falls. A batch that starts at 32 and drains to 3 spends most of its life far from full.

Continuous batching — iteration-level scheduling, from Orca and vLLM — makes decisions *between* iterations instead. Each iteration: **retire** the sequences that just finished (returning their cache pages to the pool), **admit** waiting requests to fill the freed slots (prefilling their prompts), then run *one* batched decode step over the current active set. Finished requests leave immediately and new ones join immediately, so the batch is continuously topped up to its capacity and the model runs a full batch on (almost) every step. Short requests finish and depart without waiting on long ones, and long ones do not hold empty slots hostage. The effect on throughput is dramatic, and it is why every modern serving stack works this way.

The boss measures exactly what a server must guarantee. **Correctness**: each request's generated tokens must be identical to what it would produce running alone — the interleaving and batching must not let one sequence's processing perturb another's. This follows from the foundations already laid: batched decode equals independent decode (13.1), and each sequence owns its cache (13.2), so a sequence's output depends only on itself no matter who shares its batch. **Efficiency**: the workload must complete in fewer model iterations than serving the requests sequentially. Serving B requests one at a time costs the *sum* of their token counts in decode iterations; continuous batching costs closer to the *max*, because each iteration emits one token for every active sequence at once. Measuring total iterations and showing it falls below the total token count proves the batching is doing real work — more than one token produced per model step.

Together these certify a real server: it returns each user exactly the output they would get alone, while doing a fraction of the total work. This is the loop at the heart of vLLM and TGI — batched decode for the step, a paged cache for the memory, and this scheduler to keep the batch full.

THE TASK

Implement `Request (prompt, max_new, eos, output, done)` and `serve(model, cfg, reqs, max_batch, iters)`: loop — retire active sequences whose next token is eos or that hit `max_new`; admit waiting requests up to `max_batch` (prefill each to its first token); if none active, stop; else run one `batched_decode` over the active set, commit each fed token to its request's output, and set the returned token as the next — counting one iteration per batched step. Then the boss tests: (1) each output equals standalone greedy generate; (2) total iterations is below the total token count.

The bar (the boss): every request's output is identical to running it alone, and the workload completes in fewer batched-decode iterations than the total number of tokens generated.

HINTS

Hint 1 — retire, admit, decode

Each iteration in that order: drop Active entries with `next == eos` or `generated >= max_new` (mark done); push new prefilled requests until `active.size() == max_batch` or the queue empties; if active is empty, break; otherwise one batched step.

Hint 2 — the fed token is the committed token

`batched_decode(feed)` returns the *next* token per sequence. Commit the token you *fed* (`active[i].next`) to output, increment `generated`, then set `active[i].next` to the returned token — mirroring how single-sequence generate commits then decodes.

Hint 3 — one iteration per batched step

Increment `iters` once per `batched_decode` call, not per token. With `max_batch` active sequences, that one increment produced `max_batch` tokens — which is the whole point of the metric.

TESTS

```

// tests/test_serving.cpp (continued; reuses generate)

TEST(boss_serve_output_matches_standalone) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    std::vector<Request> reqs = {
        {{3,7,1}, 5, -1, {}, false},
        {{5,2}, 3, -1, {}, false},
        {{9,4,6}, 6, -1, {}, false},
        {{1,1}, 4, -1, {}, false},
    };
    serve(m, c, reqs, /*max_batch=*/2, nullptr);           // continuous
    ↪ admission at batch 2
    for (auto& r : reqs) {
        SamplingParams sp; sp.greedy = true; RNG rng(0);
        auto solo = generate(m, c, r.prompt, r.max_new, r.eos, sp, rng);
        CHECK(r.output == solo);                           // identical to
    }
    ↪ running alone
}

TEST(boss_serve_fewer_iterations_than_tokens) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    std::vector<Request> reqs = {
        {{3,7,1}, 6, -1, {}, false}, {{5,2}, 6, -1, {}, false},
        {{9,4,6}, 6, -1, {}, false}, {{1,1}, 6, -1, {}, false},
    };
    int iters = 0;
    serve(m, c, reqs, /*max_batch=*/4, &iters);
    int total = 0; for (auto& r : reqs) total += (int)r.output.size();
    CHECK(total == 24);                                     // 4 requests x 6
    ↪ tokens
    REQUIRE(iters > 0);
    CHECK(iters < total);                                   // < 1 iteration per
    ↪ token
}

```

REFERENCE SOLUTION

Add to serving.hpp:

```

// serving.hpp
struct Request {

```

```

    std::vector<int> prompt;
    int max_new = 0;
    int eos = -1;
    std::vector<int> output;    // filled by serve()
    bool done = false;
};

inline void serve(const ModelWeights& m, const ModelConfig& cfg,
                  std::vector<Request>& reqs, int max_batch, int* iters) {
    struct Active { int req; KVCache cache; int next; int generated; };
    std::vector<Active> active;
    const int V = cfg.vocab_size;
    size_t next_req = 0;
    int iterations = 0;

    while (true) {
        // 1. retire finished sequences (free their slots)
        std::vector<Active> keep;
        for (auto& a : active) {
            Request& r = reqs[a.req];
            if (a.next == r.eos || a.generated >= r.max_new) { r.done = true;
↳ continue; }
            keep.push_back(std::move(a));
        }
        active = std::move(keep);

        // 2. admit waiting requests to fill the batch (prefill each)
        while ((int)active.size() < max_batch && next_req < reqs.size()) {
            Request& r = reqs[next_req];
            KVCache cache = KVCache::allocate(cfg, (int)r.prompt.size() + r.max_new
↳ + 8);
            Tensor lg = forward_cached(m, cfg, r.prompt, cache);
            int first = argmax(lg.data_ptr<float>()) + (int64_t)(r.prompt.size()-1)*
↳ V, V);
            active.push_back({(int)next_req, std::move(cache), first, 0});
            ++next_req;
        }

        if (active.empty()) break;    // nothing left to
↳ serve

        // 3. one batched decode step over the active set
        std::vector<KVCache*> ptrs; std::vector<int> feed;
        ptrs.reserve(active.size()); feed.reserve(active.size());
        for (auto& a : active) { ptrs.push_back(&a.cache); feed.push_back(a.next);
↳ }
        std::vector<int> nxt = batched_decode(m, cfg, ptrs, feed);

```

```

    ++iterations;

    for (size_t i = 0; i < active.size(); ++i) {           // commit fed token,
    ↪ advance
        reqs[active[i].req].output.push_back(active[i].next);
        active[i].generated++;
        active[i].next = nxt[i];
    }
}
if (iters) *iters = iterations;
}

```

Run the boss. Four requests of different lengths flow through a batch of two — short ones finishing and departing, waiting ones admitted into the freed slots — and each emerges with exactly the tokens it would have produced alone. With a full batch of four, twenty-four tokens come out of six model iterations: four tokens per step, a fourfold reduction in model invocations versus serving them one at a time. **Boss cleared.**

What you have proven. Not “a server that runs many requests,” but “a server that is *correct* and *efficient* at once”: every request receives its own exact output regardless of who shares its batch, and the workload completes in a fraction of the model steps sequential serving would need. The interleaving cannot corrupt a sequence because batched decode equals independent decode and each sequence owns its cache; the efficiency is real because each model step emits a token for the whole batch. This is the serving loop of a production engine, built from the batched decode, the paged cache, and the scheduler together.

GOING FURTHER

- **Page the server’s cache.** Replace the per-request KVCache with the PagedKVCache of 13.2 so admission draws pages from the shared pool and retirement returns them — the configuration that lets the batch grow far past what max-length caches would allow.
- **Prefill/decode scheduling.** Treat prefill (compute-bound) and decode (memory-bound) differently: chunk long prefills, or interleave them with decode steps, to keep latency low under load — the “chunked prefill” of modern schedulers.
- **Admission policy and fairness.** Add priorities, per-request deadlines, and a cap on concurrent prefills; measure tail latency under a bursty arrival pattern

— the difference between a demo and a service.

Unit 13 — Milestone check

MILESTONE CHECK

The engine can now *serve a crowd*:

- **batched decode** — many sequences through one forward pass, the matmuls amortized across the batch and attention kept per-sequence;
- a **paged kv cache** — fixed-size pages from a shared pool, mapped by per-sequence block tables, with no per-sequence preallocation waste;
- a **continuous-batching scheduler** — retire and admit between iterations to keep the batch full; and
- a **boss-grade proof** that each request's output is identical to running it alone while the workload finishes in fewer model iterations than serving sequentially.

The engine is now functionally complete and serves at scale: it loads real models, generates correct text, decodes faster by speculation, and batches many requests with production-grade scheduling. One question remains — is it *fast*? **Unit 14** is the performance unit: profile the whole engine on a real Llama-family model, fix the memory layout, thread the hot loops, and tune the kernels until it runs at interactive speed on a CPU. That is the **engine-alive** milestone the entire book has been building toward — the moment all fourteen units' work becomes one fast, working program.

UNIT 14

Performance

What you will build. The work that makes the engine *fast*, and the assembly that brings it to life. You build a **profiler** to find where time actually goes, reason with the **roofline** to understand why, **thread** the hot loops across all cores, and then assemble the whole engine — loader, tokenizer, optimized forward pass, sampler, generation loop — into one program that runs a real model at interactive speed. The boss is the **engine-alive** milestone: a measured speedup over the naive baseline, same output, real tokens per second.

Where this fits. Every unit so far chased *correctness*. This one chases *speed*, and speed in LLM inference is not guesswork — it follows from a single fact about the workload, the roofline, which says exactly which optimizations matter. Decode is memory-bandwidth-bound; prefill is compute-bound; and knowing which is which tells you that quantization, batching, threading, and SIMD are the levers — and roughly how much each can buy. This is the unit where fourteen units of work become one fast, working program.

Performance work is empirical, and intuition about it is usually wrong. The discipline is to **measure** where time goes, **understand** why through the roofline model, and **remove** the dominant bottleneck — then measure again. The reason this converges quickly for LLM inference is that one fact dominates everything: a decode step reads every weight of the model from memory and does only a little arithmetic with each, so it is limited by **memory bandwidth**, not compute. The processor spends its time waiting for weights to arrive. Almost every important optimization is a way to move fewer bytes or to do more useful work per byte moved.

That single fact organizes the whole toolkit. **Quantization** (Unit 9) shrinks the weights, so fewer bytes cross the bus per token — a near-direct decode speedup.

Batching (Unit 13) reuses each loaded weight across many sequences, converting a bandwidth-bound problem into a compute-bound one as the batch grows. **Threading** puts every core's share of memory bandwidth and arithmetic to work at once, and is decisive for the compute-bound **prefill** (a large matmul over the whole prompt). **SIMD** (Unit 3) widens each core's arithmetic. None of these is a guess; the roofline says in advance which will help and by how much. We build the profiler that exposes the breakdown, the threading that exploits the cores, and then the assembly — the engine, alive, running a real model at speed. Add `include/llm/profile.hpp` and `include/llm/threadpool.hpp`.

14.1 Profiling and the Roofline

GOAL

Measure before optimizing: a scoped **profiler** that accumulates wall time per section so the forward pass's breakdown is visible, and the **roofline** test — arithmetic intensity against the machine's FLOP-per-byte ratio — that classifies each operation as memory-bound or compute-bound, telling you which optimization will pay.

BACKGROUND

The first rule is to instrument, not guess. A scoped (`RAII`) timer that records the elapsed time of a code region into a named accumulator, dropped into each phase of the forward pass — embedding, attention, feed-forward, the final projection — produces a breakdown of where time is spent. For LLM inference the result is always the same shape: the **matmuls** (the Q/K/V and output projections, the feed-forward, and the vocabulary projection) dominate, typically well over ninety percent of decode time. That breakdown is the instruction: optimize the matmul, and ignore everything else until it no longer dominates.

Why the matmuls dominate, and what to do about it, comes from the **roofline**. Every operation has an **arithmetic intensity**: the ratio of floating-point operations it performs to the bytes it moves from memory, FLOPs/bytes. Every machine has a characteristic ratio of its own — peak FLOPs divided by peak memory bandwidth, the “ridge point.” If an operation's intensity is *below* the machine's ratio, it is starved for data and runs at memory-bandwidth speed: **memory-bound**. If

above, it saturates the arithmetic units: **compute-bound**. The roofline is the rule that turns the breakdown into a plan.

Apply it to a matmul and the two regimes of inference appear. A **decode** step multiplies one token's vector by a weight matrix W of shape $[\text{out}, \text{in}]$: it does $2 \cdot \text{out} \cdot \text{in}$ FLOPS and moves about $4 \cdot \text{out} \cdot \text{in}$ bytes (the weight, read once), an intensity of roughly 0.5 — far below any real machine's ratio of tens to hundreds. Decode is deeply memory-bound: its speed is essentially the model's size in bytes divided by the memory bandwidth. This is the quantitative reason quantization helps (Q4 cuts the bytes nearly eightfold, so decode runs nearly eightfold faster) and why batching helps (reusing W across B tokens multiplies the intensity by B). A **prefill** step multiplies M tokens at once, doing M times the FLOPS for the same weight read, so its intensity is about $M/2$ — compute-bound once M exceeds the ridge point, which is why prefill rewards SIMD and threading with near-linear speedup. One model explains the entire optimization strategy.

THE TASK

Implement a Profiler (named accumulators of seconds and call counts, with a report); a ScopedTimer (RAII: records elapsed time into the profiler on destruction) and a PROFILE macro; and is_memory_bound(flops, bytes, machine_flop_per_byte) with a helper matmul_intensity(M , N , K) returning the FLOPS-per-byte of an $[M, K] \times [K, N]$ matmul.

HINTS

Hint 1 — RAI timing

Capture `std::chrono::high_resolution_clock::now()` in the constructor; in the destructor add the elapsed seconds to `profiler.totals[name]`. A `PROFILE(p, "name")` macro declaring a uniquely named ScopedTimer times the enclosing scope.

Hint 2 — matmul intensity

$\text{flops} = 2.0 * M * N * K$; $\text{bytes} = 4.0 * (M*K + K*N + M*N)$ (inputs, weights, output, in floats). The ratio is the arithmetic intensity; weights ($K \times N$) dominate the bytes when M is small.

Hint 3 — the classification

$\text{is_memory_bound} = (\text{flops} / \text{bytes}) < \text{machine_flop_per_byte}$. For a CPU, the ratio is often in the tens (e.g. peak GFLOPS over peak GB/s); a decode matmul ($M = 1$) falls far below it, a large prefill far above.

TESTS

```
// tests/test_profile.cpp
#include "test_framework.hpp"
#include "llm/profile.hpp"
#include <thread>
#include <chrono>
using namespace llm;

TEST(scoped_timer_measures_elapsed) {
    Profiler p;
    { PROFILE(p, "sleep"); std::this_thread::sleep_for(std::chrono::milliseconds
    ↪ (20)); }
    CHECK(p.totals["sleep"] >= 0.015);           // at least ~15 ms recorded
    CHECK(p.counts["sleep"] == 1);
}

TEST(decode_matmul_is_memory_bound) {
    const double ratio = 30.0;                   // a representative CPU FLOP:
    ↪ byte ridge point
    // decode: one token (M=1) times a 4096x4096 weight
    double di = matmul_intensity(/*M=*/1, /*N=*/4096, /*K=*/4096);
    CHECK(is_memory_bound(di * (4.0*(1*4096 + 4096*4096 + 1*4096)),
    4.0*(1*4096 + 4096*4096 + 1*4096), ratio));
    CHECK(di < ratio);                           // intensity ~ 0.5, well below
    ↪ the ridge
}

TEST(prefill_matmul_is_compute_bound) {
    const double ratio = 30.0;
    double pi = matmul_intensity(/*M=*/512, /*N=*/4096, /*K=*/4096); // 512-
    ↪ token prefill
    CHECK(pi > ratio);                           // many tokens per weight read ->
    ↪ compute-bound
}
```


REFERENCE SOLUTION

include/llm/profile.hpp:

```

// include/llm/profile.hpp
#pragma once
#include <string>
#include <unordered_map>
#include <chrono>
#include <cstdio>
#include <vector>
#include <algorithm>

namespace llm {

struct Profiler {
    std::unordered_map<std::string, double> totals; // section -> seconds
    std::unordered_map<std::string, int> counts;
    void add(const std::string& name, double secs) { totals[name] += secs; counts[
        ↪ name] += 1; }
    void report() const {
        std::vector<std::pair<std::string, double>> rows(totals.begin(), totals.end
        ↪ ());
        std::sort(rows.begin(), rows.end(), [](auto& a, auto& b){ return a.second >
        ↪ b.second; });
        for (auto& r : rows)
            std::printf("%-24s %8.3f ms  (x%d)\n", r.first.c_str(), r.second*1e3,
        ↪ counts.at(r.first));
    }
};

struct ScopedTimer {
    Profiler& p; std::string name;
    std::chrono::high_resolution_clock::time_point t0;
    ScopedTimer(Profiler& prof, std::string n)
        : p(prof), name(std::move(n)), t0(std::chrono::high_resolution_clock::now()
        ↪ ) {}
    ~ScopedTimer() {
        auto t1 = std::chrono::high_resolution_clock::now();
        p.add(name, std::chrono::duration<double>(t1 - t0).count());
    }
};

#define LLM_CONCAT_(a, b) a##b
#define LLM_CONCAT(a, b) LLM_CONCAT_(a, b)
#define PROFILE(prof, name) llm::ScopedTimer LLM_CONCAT(_timer_, __LINE__)(prof),
        ↪ (name))

```

```

// arithmetic intensity (FLOPs per byte) of an [M,K] x [K,N] matmul, fp32
inline double matmul_intensity(int M, int N, int K) {
    const double flops = 2.0 * M * N * K;
    const double bytes = 4.0 * ((double)M*K + (double)K*N + (double)M*N);
    return flops / bytes;
}

inline bool is_memory_bound(double flops, double bytes, double
    ↪ machine_flop_per_byte) {
    return (flops / bytes) < machine_flop_per_byte;
}

} // namespace llm

```

With the profiler dropped into the forward pass, the breakdown confirms what the roofline predicts: the matmuls are the cost, and they are memory-bound at decode and compute-bound at prefill. That is not a detail — it is the entire optimization plan in one measurement. It says to spend effort on the matmul and nowhere else; it says quantization and batching attack the decode bottleneck while threading and SIMD attack prefill; and it says, quantitatively, how much each can return. The next challenge takes the most broadly applicable of these levers — threading — and applies it to the kernel the profiler just indicted.

GOING FURTHER

- **Per-layer breakdown.** Profile each layer and each sub-operation separately to see attention versus feed-forward, and prefill versus decode — the map a real optimization effort works from.
- **Measured bandwidth and FLOPs.** Benchmark your machine's actual memory bandwidth and peak FLOPs, compute its true ridge point, and plot your kernels on the roofline to see how close to the roof they run.
- **Counters.** Add hardware performance counters (cache misses, instructions per cycle) via perf or PAPI to confirm a memory-bound kernel really is stalled on memory.

14.2 Threading the Hot Loops

GOAL

Put every core to work: a persistent **thread pool** with a blocking `parallel_for`, and a threaded `linear` that splits the output features across the pool. Because every projection and feed-forward matmul flows through `linear`, this one change parallelizes the entire forward pass — with the speedup the roofline predicts.

BACKGROUND

The kernel the profiler indicted is embarrassingly parallel. A matmul’s output elements are independent — each is its own dot product — so the output features (and rows) split cleanly across cores with no coordination. The same is true of attention heads and feed-forward rows. The only thing standing between that parallelism and a speedup is the cost of *dispatching* it.

That cost is why a **persistent** thread pool matters. Creating a thread takes microseconds; a decode step also takes microseconds; so spawning fresh threads per call would cost more than it saves. A pool creates its worker threads once and parks them on a condition variable, waking them to run tasks and sleeping them again — dispatch becomes a queue push and a notify. A `parallel_for` splits a range $[0, n)$ into chunks, hands all but one to the workers, runs the remaining chunk on the calling thread so no core sits idle, and blocks until every chunk reports done. With that in hand, parallelizing `linear` is a few lines: split the output-feature dimension across the pool. For decode, that dimension is the hidden size or the vocabulary — thousands of independent dot products, ample work for any core count. And because the whole forward pass routes its matmuls through `linear`, threading that single function threads everything.

The speedup follows the roofline, exactly as predicted. **Prefill** is compute-bound, so spreading its arithmetic across cores gives near-linear speedup until some other ceiling is reached — this is the big win, often the difference between a sluggish and a snappy prompt. **Decode** is memory-bound, so threading helps only up to the point where the cores’ combined memory traffic saturates the bus, then plateaus; but it compounds with quantization, which lowers the traffic so that more cores can be fed before the plateau. The roofline told us in advance that prefill would scale and decode would not scale as far, and measurement confirms it.

Correctness is the non-negotiable constraint: threading must change speed and

nothing else. Each output element is computed by exactly one thread, with the same arithmetic as the serial version, so the result is bit-identical. This requires the `parallel_for` to partition the range cleanly — every index covered once, no index covered twice — which is the first thing the tests check.

THE TASK

Implement a `ThreadPool` (worker threads parked on a condition variable; a `parallel_for(n, fn)` that chunks $[0, n)$ across workers plus the caller and blocks until done); a global pool with `set_num_threads(n)`; and `linear_mt(y, x, w)` parallelizing `linear` over output features through the global pool (falling back to serial when unset).

HINTS

Hint 1 — a parked-worker pool

Workers loop: `cv.wait` until stop or a task is queued, pop a task, run it. The destructor sets stop, notifies all, and joins. Dispatch is push a `std::function` and notify.

Hint 2 — `parallel_for` keeps the caller busy

Chunk size $(n + P - 1) / P$ with $P = \text{workers} + 1$. Queue chunks $1..P-1$ for the workers, run chunk 0 on the calling thread, then wait on an atomic counter until the workers finish their chunks.

Hint 3 — thread `linear` over output features

`linear_mt` splits the out dimension: `parallel_for(out, [&](o0,o1){ for o in [o0,o1): dot each of the M input rows with weight row o } )`. Same result as serial `linear`; only the loop is split.

TESTS

```
// tests/test_threadpool.cpp
// (reuses randt from the model tests)
#include "test_framework.hpp"
```

```

#include "llm/threadpool.hpp"
#include "llm/tensor.hpp"
#include <atomic>
#include <vector>
#include <chrono>
using namespace llm;

TEST(parallel_for_covers_range_exactly_once) {
    ThreadPool pool(4);
    const int n = 1000;
    std::vector<std::atomic<int>> hits(n);
    for (auto& h : hits) h.store(0);
    pool.parallel_for(n, [&](int b, int e) { for (int i = b; i < e; ++i) hits[i].
        ↪ fetch_add(1); });
    for (int i = 0; i < n; ++i) CHECK(hits[i].load() == 1);    // every index
        ↪ once, none twice
}

TEST(threaded_linear_matches_serial) {
    const int M = 3, in = 64, out = 128;
    Tensor x = randt({M, in}, 1), w = randt({out, in}, 2);
    Tensor ys = Tensor::empty({M, out}, DType::F32);
    Tensor yp = Tensor::empty({M, out}, DType::F32);
    set_num_threads(1); linear_mt(ys, x, w);    // serial
    set_num_threads(4); linear_mt(yp, x, w);    // threaded
    set_num_threads(1);
    for (int i = 0; i < M * out; ++i)
        CHECK_CLOSE(ys.data_ptr<float>()[i], yp.data_ptr<float>()[i], 1e-5);
}

TEST(threaded_matmul_is_faster) {    // on a multicore
    ↪ CPU
    const int M = 256, in = 1024, out = 1024;
    Tensor x = randt({M, in}, 1), w = randt({out, in}, 2);
    Tensor y = Tensor::empty({M, out}, DType::F32);
    auto bench = [&](int threads) {
        set_num_threads(threads);
        auto t0 = std::chrono::high_resolution_clock::now();
        for (int r = 0; r < 5; ++r) linear_mt(y, x, w);
        return std::chrono::duration<double>(std::chrono::high_resolution_clock::
            ↪ now() - t0).count();
    };
    double serial = bench(1), threaded = bench(8);
    set_num_threads(1);
    CHECK(threaded < serial);    // parallelism pays
    ↪ off
}

```

REFERENCE SOLUTION

include/llm/threadpool.hpp:

```

// include/llm/threadpool.hpp
#pragma once
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <functional>
#include <atomic>
#include <vector>
#include "llm/tensor.hpp"

namespace llm {

class ThreadPool {
public:
    explicit ThreadPool(int nthreads) : stop_(false) {
        for (int i = 0; i < nthreads; ++i)
            workers_.emplace_back([this] {
                for (;;) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lk(mu_);
                        cv_.wait(lk, [this] { return stop_ || !tasks_.empty(); });
                        if (stop_ && tasks_.empty()) return;
                        task = std::move(tasks_.front()); tasks_.pop();
                    }
                    task();
                }
            });
    }

    ~ThreadPool() {
        { std::unique_lock<std::mutex> lk(mu_); stop_ = true; }
        cv_.notify_all();
        for (auto& t : workers_) t.join();
    }

    int size() const { return (int)workers_.size(); }

    // run fn(begin,end) over [0,n) split across the pool and the caller; block
    // until done
    void parallel_for(int n, const std::function<void(int, int)>& fn) {
        const int P = size() + 1; // workers + calling
        // thread
        if (P <= 1 || n <= 1) { fn(0, n); return; }

```

```

    const int chunk = (n + P - 1) / P;
    std::atomic<int> done(0);
    int launched = 0;
    for (int b = chunk; b < n; b += chunk) {           // workers take chunks 1..
        ↪ P-1
        const int e = std::min(n, b + chunk);
        { std::unique_lock<std::mutex> lk(mu_);
          tasks_.push([&fn, b, e, &done] { fn(b, e); done.fetch_add(1); }); }
          ++launched;
        }
        cv_.notify_all();
        fn(0, std::min(n, chunk));                      // caller runs chunk 0
        while (done.load() < launched) std::this_thread::yield();
    }
private:
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;
    std::mutex mu_;
    std::condition_variable cv_;
    bool stop_;
};

inline ThreadPool*& global_pool() { static ThreadPool* p = nullptr; return p; }
inline void set_num_threads(int n) {
    delete global_pool();
    global_pool() = (n > 1) ? new ThreadPool(n - 1) : nullptr;    // n-1 workers +
    ↪ caller = n
}

// threaded linear:  $y[M, out] = x[M, in] * w[out, in]^T$ , parallel over output
    ↪ features
inline void linear_mt(Tensor& y, const Tensor& x, const Tensor& w) {
    const int M = (int)x.shape()[0], in = (int)x.shape()[1], out = (int)w.shape()
    ↪ [0];
    const float* X = x.data_ptr<float>();
    const float* W = w.data_ptr<float>();
    float* Y = y.data_ptr<float>();
    auto block = [&](int o0, int o1) {
        for (int o = o0; o < o1; ++o) {
            const float* wr = W + (int64_t)o * in;
            for (int m = 0; m < M; ++m) {
                const float* xr = X + (int64_t)m * in;
                float s = 0.0f; for (int k = 0; k < in; ++k) s += xr[k] * wr[k];
                Y[(int64_t)m * out + o] = s;
            }
        }
    };
};

```

```

    if (global_pool()) global_pool()->parallel_for(out, block);
    else block(0, out);
}

} // namespace llm

```

The tests pin the two properties threading must have: the `parallel_for` covers its range exactly once (so no output is computed twice or skipped), and the threaded linear returns bit-identical results to the serial one (so optimization never changes output). The speed test confirms the payoff on a multicore machine. Swap `linear_mt` in for `linear` throughout the forward pass and every `matmul` — the dominant cost — now runs on all cores, with prefill scaling nearly linearly just as the roofline promised. The engine is now fast. It remains to assemble it, point it at a real model, and watch it run.

GOING FURTHER

- **Condition-variable completion.** Replace the `yield` spin with a counter and a condition variable so the calling thread sleeps until the workers finish — lower overhead under contention.
- **Thread affinity and NUMA.** Pin workers to cores and allocate weights on the right NUMA node; on multi-socket machines this alone can move the memory-bound plateau substantially.
- **Fused quantized matmul.** Combine threading with the on-the-fly dequantization of Unit 9 in one kernel, so each thread dequantizes its slice of the weights and multiplies in a single pass — the configuration that makes memory-bound decode as fast as it can be on a CPU.

14.3 Engine Alive

BOSS CHALLENGE

Assemble every unit into one program — load a real model, tokenize a prompt, run the optimized forward pass, sample, generate, detokenize — and prove it works and is fast: the optimized path produces byte-identical greedy output to the baseline while generating at measurably higher tokens per second. The engine is alive.

GOAL

The culmination: a single `run_engine` that takes a prompt string and returns generated text — encode (Unit 4), generate through the threaded, cached, quantization-ready forward pass (Units 5–14), decode (Unit 4) — certified to match the reference output exactly while running measurably faster than the naive baseline.

BACKGROUND

This is the moment every unit has been building toward: the components become one program. The **loader** (Unit 10) maps a model’s weights from a GGUF file. The **tokenizer** (Unit 4) turns the prompt string into token ids. The **forward pass** (Units 5–8), driven by the **kv cache** (Unit 7), now **threaded** (14.2) and able to run **quantized** weights (Unit 9), computes the logits. The **sampler** and **generation loop** (Unit 11) turn logits into tokens, and the tokenizer decodes them back to text. `run_engine` is that entire pipeline in a single call: text in, text out.

The boss measures the two claims the whole book has been making. **Correctness end to end**: the assembled, optimized engine must produce deterministic greedy output identical to the reference path. This is a real test of the optimizations, threading above all — a parallelized matmul that changed the result by even one bit would fail it — and of the wiring, since every unit’s component must connect correctly for the pipeline to reproduce a from-scratch generate. **Speed**: the optimized path must generate at measurably higher tokens per second than the naive single-threaded baseline, with byte-identical output. The bar is a clear speedup — on a representative model, twofold or more from threading alone; combined with quantization on a real model, far more — and the test reports the realized rate.

Why this is *the* milestone: the engine now does the whole job, correctly and at usable speed, entirely from the code in this book. Point `run_engine` at a real model — Llama-3.2-1B-Instruct in Q4 is about two-thirds of a gigabyte — and it maps the file, encodes a chat prompt, and streams a coherent reply at interactive speed on a laptop CPU: quantization moving roughly eight times fewer weight-bytes per token, threading spreading the work across every core, the two together pushing decode against the memory-bandwidth roofline that Challenge 14.1 identified. There is no external runtime, no framework — a tokenizer, a loader, a forward pass, a sampler, a scheduler, all built here. The engine is alive.

THE TASK

Implement `run_engine(model, cfg, tok, prompt, max_new, params, rng)`: `tok.encode` the prompt (with `bos`), `generate` (through `linear_mt`) to `tok.eos_id()`, `tok.decode` the result. Then the boss tests: (1) `run_engine` is deterministic and equals `encode+generate+decode` end to end; (2) on a representative model, threaded generation yields higher tokens per second than single-threaded with identical output.

The bar (the boss): the assembled engine generates byte-identical greedy output through the optimized path while reaching measurably higher tokens per second than the naive baseline (a clear speedup on a representative model; interactive rates on a real one).

HINTS

Hint 1 — the whole pipeline is three calls

```
auto ids = tok.encode(prompt, /*bos=*/true); auto out =
generate(model, cfg, ids, max_new, tok.eos_id(), params, rng);
return tok.decode(out);. Every unit lives behind those three lines.
```

Hint 2 — optimize by swapping one kernel

Route the forward pass's matmuls through `linear_mt`; with `set_num_threads(N)` set, the entire forward pass parallelizes. No other code changes — the speedup rides on the single kernel from 14.2.

Hint 3 — measure tokens per second

Time `generate` over a fixed `max_new` and divide: `tokens / seconds`. Compare the rate at one thread to the rate at many; assert the output is identical and the rate is higher.

TESTS

```
// tests/test_engine.cpp
// (reuses tiny_model/tiny_cfg, a byte-level Tokenizer from Unit 4, generate)
#include "test_framework.hpp"
```

```

#include "llm/engine.hpp"
#include "llm/threadpool.hpp"
#include <cstdio>
#include <chrono>
using namespace llm;

TEST(boss_engine_alive_end_to_end) {
    ModelConfig c = tiny_cfg(); c.vocab_size = 256;           // match a byte-level
    ↪ tokenizer
    const ModelWeights m = tiny_model(c, 4);
    Tokenizer tok = Tokenizer::with_byte_vocab();             // 256 single-byte
    ↪ tokens
    SamplingParams sp; sp.greedy = true;

    set_num_threads(4);
    RNG r1(0), r2(0);
    std::string a = run_engine(m, c, tok, "hi", 8, sp, r1);
    std::string b = run_engine(m, c, tok, "hi", 8, sp, r2);
    CHECK(a == b);                                           // deterministic end to
    ↪ end

    std::vector<int> ids = tok.encode("hi", /*bos=*/true); // == encode +
    ↪ generate + decode
    RNG r3(0);
    auto ref = generate(m, c, ids, 8, tok.eos_id(), sp, r3);
    CHECK(tok.decode(ref) == a);
    set_num_threads(1);
}

TEST(boss_optimized_path_faster_same_output) {
    ModelConfig c = tiny_cfg();                               // a model big enough
    ↪ to time
    c.hidden_size = 512; c.n_heads = 8; c.head_dim = 64; c.n_kv_heads = 8;
    c.intermediate_size = 1376; c.n_layers = 4; c.vocab_size = 4096;
    const ModelWeights m = tiny_model(c, 7);
    const std::vector<int> prompt = {3, 7, 1, 5, 2};
    SamplingParams sp; sp.greedy = true;

    auto run = [&](int threads) {
        set_num_threads(threads); RNG rng(0);
        auto t0 = std::chrono::high_resolution_clock::now();
        auto out = generate(m, c, prompt, /*max_new=*/32, /*eos=*/-1, sp, rng);
        double secs = std::chrono::duration<double>(std::chrono::
    ↪ high_resolution_clock::now()-t0).count();
        return std::make_pair(out, 32.0 / secs);             // tokens, tokens/sec
    };
    auto serial = run(1);

```

```

    auto threaded = run(8);
    set_num_threads(1);

    CHECK(threaded.first == serial.first);           // byte-identical output
    CHECK(threaded.second > serial.second);         // higher tokens/sec
    std::printf("decode: %.1f tok/s serial, %.1f tok/s threaded (%.2fx)\n",
                serial.second, threaded.second, threaded.second / serial.second);
}

```

REFERENCE SOLUTION

include/llm/engine.hpp:

```

// include/llm/engine.hpp
#pragma once
#include <string>
#include <vector>
#include "llm/model.hpp"
#include "llm/tokenizer.hpp"
#include "llm/sampler.hpp" // generate, SamplingParams, RNG
#include "llm/threadpool.hpp" // linear_mt is used inside the forward pass

namespace llm {

// The whole engine: prompt text in, generated text out.
inline std::string run_engine(const ModelWeights& model, const ModelConfig& cfg,
    ↪ const Tokenizer& tok,
                                const std::string& prompt, int max_new,
                                const SamplingParams& sp, RNG& rng) {
    std::vector<int> ids = tok.encode(prompt, /*add_bos=*/true); // Unit 4
    ↪
    std::vector<int> out = generate(model, cfg, ids, max_new, tok.eos_id(), sp,
    ↪ rng); // Units 5-14
    return tok.decode(out); // Unit 4
}

} // namespace llm

```

Run the boss. The assembled engine round-trips a prompt to text deterministically and reproduces the reference pipeline exactly, and the threaded path generates the same tokens at a measurably higher rate — the optimization delivering speed and changing nothing else. **Boss cleared.**

What you have proven. Not a pile of correct components, but *one working engine*:

a prompt enters as text and leaves as a generated continuation, computed by a forward pass that runs on every core, through a path you can quantize for another large step, fast enough to be interactive on real hardware. The correctness held end to end — the optimizations preserved the model’s exact output — and the speed was real and measured. Built from nothing but C++ and the fourteen units behind it, the engine reads and writes language at usable speed. It is alive.

GOING FURTHER

- **Run a real model.** Download Llama-3.2-1B-Instruct in Q4 GGUF, load it with Unit 10, encode a chat-templated prompt, and stream the reply token by token — the first time the whole engine runs end to end on a real model. Measure the tokens per second.
- **Stream and serve.** Combine `run_engine` with the streaming output of Unit 11 and the continuous-batching scheduler of Unit 13 into a small server that handles concurrent prompts — the engine as a service.
- **Close the roofline gap.** Profile the real-model run, fuse the quantized matmul with threading, tune block sizes and SIMD width, and push decode as close to the memory-bandwidth roof as your machine allows.

Unit 14 — Milestone check

MILESTONE CHECK

The engine is alive. Assembled from all fourteen units and made fast:

- **profiling and the roofline** (14.1) — measure where time goes, and know that decode is memory-bound and prefill compute-bound, which dictates every optimization;
- **threading** (14.2) — a persistent pool and a threaded linear that parallelizes the whole forward pass across all cores;
- **the assembled engine** (14.3) — load, tokenize, forward, sample, generate, decode in one `run_engine`; and
- **a boss-grade proof** that the optimized path produces byte-identical output while generating at measurably higher tokens per second.

This is the destination the book set out for: a complete, correct, fast LLM inference engine, written from scratch in C++, running real Llama-family models on a

cpu at interactive speed — no framework beneath it, every part built and proven here. What remains is optional and exhilarating. **Unit 15** (★) is the capstone: port the engine to the GPU with CUDA, where the same roofline thinking applies against vastly more bandwidth and parallelism, for another order of magnitude in throughput. The engine lives on the CPU; now we let it fly.

UNIT 15

GPU and CUDA

What you will build. The capstone: a port of the engine to the GPU with CUDA. You build the foundations — **device memory** and your **first kernel** (elementwise ops run by thousands of threads), then a **tiled GEMM** that uses shared memory the way the CPU version used cache — and finally a **GPU-resident forward pass** that keeps the weights and activations on the device and computes logits with GPU kernels. The boss proves the port is faithful and far faster.

Where this fits. The engine lives on the CPU at interactive speed. A GPU offers an order of magnitude or two more memory bandwidth and thousands of cores — exactly what memory-bound decode and compute-bound prefill crave. The same roofline thinking applies, against far higher ceilings, so the same kernels run dramatically faster and the engine can serve far larger models and batches. This unit is optional and exhilarating: the engine lives; now we let it fly.

The CPU engine is complete and usable, but it is leaving most of the available performance on the table, because the hardware best suited to this workload is the GPU. A modern accelerator has hundreds of gigabytes per second to terabytes per second of memory bandwidth — many times a CPU's — and thousands of arithmetic cores. Memory-bound decode wants bandwidth; compute-bound prefill wants cores; the GPU has an abundance of both. Porting the engine to CUDA is how we reach the next order of magnitude in throughput, and the encouraging part is that nothing about the *reasoning* changes. The roofline still governs — decode memory-bound, prefill compute-bound — only the ceilings move up, so the same kernels, written for the GPU, simply run much faster.

What changes is the execution model, and it must be learned directly. In CUDA, the **host** (CPU) and **device** (GPU) have separate memories; computation is expressed

as a **kernel**, a function run simultaneously by thousands of **threads** in the **SIMT** (single-instruction, multiple-thread) model; and the device has its own **memory hierarchy** — global, shared, registers — that, exactly like the CPU cache hierarchy, must be used well to approach the roofline. The port follows the shape of the CPU work: move the weights to device memory, write kernels for the hot operations, and orchestrate the forward pass on the device. We build a first kernel to learn the model, a tiled GEMM to learn the memory hierarchy, and a resident forward pass to put them together. Production engines lean on CUBLAS, Tensor Cores, and CUDA Flash-Attention kernels — but writing these yourself is what makes those libraries comprehensible. Add `src/cuda/` to the project.

15.1 Your First Kernel: Device Memory and Elementwise Ops

GOAL

The CUDA essentials: allocate **device memory**, copy data across the host/device boundary, and launch a **kernel** run by thousands of threads. Port the elementwise operations — vector add, **SILU** — as your first GPU kernels, each thread handling one element.

BACKGROUND

A GPU program is split across two memories. The **host** allocates and fills arrays in CPU memory; to compute on them, it allocates matching buffers in **device** memory (`cudaMalloc`), copies the inputs over (`cudaMemcpy`, host-to-device), launches a kernel, and copies the results back. Managing those device buffers with an **RAII** wrapper — allocate in the constructor, free in the destructor — keeps the bookkeeping from leaking, just as it did for CPU memory in Unit 1.

A **kernel** is the unit of GPU computation: a function, marked `__global__`, that runs once per **thread**, with thousands of threads executing it at once. Threads are organized into **blocks**, blocks into a **grid**; each thread computes its global index from `blockIdx`, `blockDim`, and `threadIdx`, and uses that index to select the element it works on. The launch syntax names the grid and block dimensions. Because a launch may have fewer threads than elements (or be sized for occupancy rather than data), the standard idiom is a **grid-stride loop**: each thread starts at its global index and strides by the total thread count, covering as many elements as

needed. This makes one kernel correct for any input size.

The simplest kernels are **elementwise**, and they map perfectly onto this model: every output element is independent, so thread i computes output i with no coordination at all. A vector add reads two inputs and writes their sum; `SILU` reads one input and writes $x \cdot \sigma(x)$. These are trivial as GPU kernels precisely because they are embarrassingly parallel — which is also why they are the right place to learn the launch mechanics, the memory transfers, and the indexing before tackling the matmul. The roofline note: like their CPU versions, elementwise ops are memory-bound (a couple of FLOPS per element loaded), so on the GPU they run at its very high memory bandwidth — fast, but bandwidth-limited, exactly as the model predicts.

THE TASK

Implement a `DeviceBuffer<T>` RAII wrapper (`cudaMalloc/cudaFree`, with `from_host/to_host` transfers); a `vector_add_kernel` and a `silu_kernel`, each a grid-stride loop; and host wrappers `gpu_vector_add` and `gpu_silu` that upload, launch, and download.

HINTS

Hint 1 — the global thread index

`int i = blockIdx.x * blockDim.x + threadIdx.x;` is this thread's element. A grid-stride loop `for (; i < n; i += blockDim.x * gridDim.x)` lets a fixed launch cover any n .

Hint 2 — launch configuration

`int threads = 256; int blocks = (n + threads - 1) / threads;` then `kernel<<<blocks, threads>>>(...)`. Follow an async launch with `cudaDeviceSynchronize()` before reading results back.

Hint 3 — RAII device memory

`cudaMalloc(&ptr, n*sizeof(T))` in the constructor, `cudaFree(ptr)` in the destructor; delete the copy operations so a buffer is never double-freed. `from_host/to_host` wrap `cudaMemcpy` with the right direction.

TESTS

```
// tests/test_cuda_elementwise.cu
#include "test_framework.hpp"
#include "llm/cuda/elementwise.cuh"
#include <vector>
#include <cmath>
using namespace llm;

TEST(gpu_vector_add_matches_cpu) {
    const int n = 10000;
    std::vector<float> a(n), b(n), c(n);
    for (int i = 0; i < n; ++i) { a[i] = i * 0.5f; b[i] = -i * 0.25f; }
    gpu_vector_add(a.data(), b.data(), c.data(), n);
    for (int i = 0; i < n; ++i) CHECK_CLOSE(c[i], a[i] + b[i], 1e-5);
}

TEST(gpu_silu_matches_cpu) {
    const int n = 5000;
    std::vector<float> x(n), y(n);
    for (int i = 0; i < n; ++i) x[i] = (i - 2500) * 0.01f;
    gpu_silu(x.data(), y.data(), n);
    for (int i = 0; i < n; ++i) { float v = x[i]; CHECK_CLOSE(y[i], v / (1.0f +
    ↪ std::exp(-v)), 1e-5); }
}
```

REFERENCE SOLUTION

src/cuda/elementwise.cuh:

```
// include/llm/cuda/elementwise.cuh
#pragma once
#include <cuda_runtime.h>
#include <cstdint>

namespace llm {

template <class T>
struct DeviceBuffer {
    T* ptr = nullptr; size_t n = 0;
    explicit DeviceBuffer(size_t count) : n(count) { cudaMalloc(&ptr, count *
    ↪ sizeof(T)); }
    ~DeviceBuffer() { if (ptr) cudaFree(ptr); }
    DeviceBuffer(const DeviceBuffer&) = delete;
};
```

```

    DeviceBuffer& operator=(const DeviceBuffer&) = delete;
    void from_host(const T* src) { cudaMemcpy(ptr, src, n * sizeof(T),
    ↪ cudaMemcpyHostToDevice); }
    void to_host(T* dst) const { cudaMemcpy(dst, ptr, n * sizeof(T),
    ↪ cudaMemcpyDeviceToHost); }
};

__global__ void vector_add_kernel(const float* a, const float* b, float* c, int n)
    ↪ {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    for (; i < n; i += blockDim.x * gridDim.x) c[i] = a[i] + b[i];    // grid-
    ↪ stride
}

__global__ void silu_kernel(const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    for (; i < n; i += blockDim.x * gridDim.x) { float v = x[i]; y[i] = v / (1.0f +
    ↪ expf(-v)); }
}

inline void gpu_vector_add(const float* a, const float* b, float* c, int n) {
    DeviceBuffer<float> da(n), db(n), dc(n);
    da.from_host(a); db.from_host(b);
    const int threads = 256, blocks = (n + threads - 1) / threads;
    vector_add_kernel<<<blocks, threads>>>>(da.ptr, db.ptr, dc.ptr, n);
    cudaDeviceSynchronize();
    dc.to_host(c);
}

inline void gpu_silu(const float* x, float* y, int n) {
    DeviceBuffer<float> dx(n), dy(n);
    dx.from_host(x);
    const int threads = 256, blocks = (n + threads - 1) / threads;
    silu_kernel<<<blocks, threads>>>>(dx.ptr, dy.ptr, n);
    cudaDeviceSynchronize();
    dy.to_host(y);
}

} // namespace llm

```

These first kernels are deliberately trivial, because the point is the scaffolding around them: the device buffers, the host/device transfers, the launch configuration, and the grid-stride indexing that makes a kernel correct for any size. Run them and the GPU reproduces the CPU's elementwise results exactly — the same `SILU` from Unit 2, now computed by thousands of threads. With the mechanics

in hand, we can write the kernel that actually matters: the matmul, where the device's memory hierarchy makes all the difference.

GOING FURTHER

- **Error checking.** Wrap every CUDA call in a macro that checks the returned `cudaError_t` and reports the file and line — indispensable once kernels grow, since a silent launch failure otherwise surfaces as wrong numbers much later.
- **Fused elementwise.** Combine the RMSNORM, residual add, and activation into single kernels so each element is read and written once — the elementwise half of closing the roofline gap on the GPU.
- **Streams.** Overlap host/device transfers with computation using CUDA streams, so the next batch's data uploads while the current one computes — the throughput trick a server depends on.

15.2 Tiled GEMM on the GPU

GOAL

The kernel that matters: a **tiled** matrix multiply that uses the GPU's **shared memory** the way the CPU version used cache. Each thread block loads tiles of the inputs into fast on-chip memory, then computes its output tile from there — reusing every loaded value across the block to slash global-memory traffic.

BACKGROUND

Matmul dominates the GPU forward pass exactly as it dominated the CPU one, and the naive GPU version has the same flaw the naive CPU version did: it re-reads data from slow memory far too often. If each thread computes one output element by streaming a whole row of A and a whole column of B from **global memory**, then every element of A and B is read from global memory many times over — and global memory, though fast, is the GPU's slowest tier. The kernel is bound by that traffic.

The fix is cache blocking (Unit 3), realized through the GPU's **shared memory**: a small, fast, on-chip memory shared by all threads in a block. Each thread **block**

is assigned one **tile** of the output C . The block cooperatively loads a tile of A and a tile of B from global memory into shared memory; every thread then computes partial products by reading from shared memory, so each value loaded once is reused by an entire tile's worth of threads. The tiles march along the K dimension, each thread accumulating into its output element, until the dot products are complete. Threads must **synchronize** (`__syncthreads`) after loading a tile, so no thread reads shared memory before it is filled, and again before the next tile overwrites it.

This is the GPU analogue of cache blocking, with one difference that makes it sharper: the cache is **explicit**. Shared memory is a scratchpad the programmer fills and drains by hand, and `__syncthreads` is the barrier that coordinates the cooperative loading — there is no hardware guessing which lines to keep. Tiling raises the kernel's arithmetic intensity (each global load is reused `TILE` times), pushing it from memory-bound toward the GPU's very high compute roof. For `FP32` a well-tiled kernel approaches that roof; with `FP16` and Tensor Cores the roof is far higher still. In production you would call `CUBLAS` or `CUTLASS`, which are hand-tuned to the last detail — but writing the tiled kernel is what makes their structure, and the roofline they chase, make sense. (The engine's linear computes $xW^\top$; the teaching kernel below computes the plain AB , and the transpose is handled by reading the weight as `[out, in]` or transposing it on upload.)

THE TASK

Implement `gemm_tiled_kernel`: `TILE`×`TILE` shared-memory tiles for A and B ; each block computes one output tile; loop over K -tiles, loading cooperatively, `__syncthreads`, accumulating from shared memory, `__syncthreads`; write the result with boundary guards. Plus a host wrapper `gpu_gemm` computing $C[M, N] = A[M, K] B[K, N]$.

HINTS

Hint 1 — block computes a tile, thread an element

```
int row = blockIdx.y*TILE + threadIdx.y; int col = blockIdx.x*TILE
+ threadIdx.x; is this thread's output element; the block covers a TILE x
TILE patch of C. Launch dim3 threads(TILE, TILE) and a grid sized to cover
M x N.
```

Hint 2 — cooperative load, then sync

Each thread loads one element of the current A-tile and one of the B-tile into `__shared__` arrays (zero-padding out-of-range), then `__syncthreads()` before any thread reads the tile — and `__syncthreads()` again after the inner product, before the next tile overwrites shared memory.

Hint 3 — accumulate across K-tiles

Loop `t` over $(K + \text{TILE} - 1) / \text{TILE}$ tiles; in each, add `TILE` products `As[ty][k]*Bs[k][tx]` into a register accumulator. After all tiles, write `acc` to `C` if `row < M && col < N`.

TESTS

```
// tests/test_cuda_gemm.cu
// (reuses a CPU matmul_blocked from Unit 3 as the reference)
#include "test_framework.hpp"
#include "llm/cuda/gemm.cuh"
#include <vector>
#include <chrono>
#include <cstdio>
using namespace llm;

TEST(gpu_gemm_matches_cpu) {
    const int M = 64, K = 48, N = 80;
    std::vector<float> A(M*K), B(K*N), C(M*N), ref(M*N, 0.0f);
    for (int i = 0; i < M*K; ++i) A[i] = (i % 7) * 0.1f - 0.3f;
    for (int i = 0; i < K*N; ++i) B[i] = (i % 5) * 0.2f - 0.4f;
    for (int i = 0; i < M; ++i) // CPU reference
        for (int j = 0; j < N; ++j) {
            float s = 0; for (int k = 0; k < K; ++k) s += A[i*K+k] * B[k*N+j];
            ref[i*N+j] = s;
        }
    gpu_gemm(A.data(), B.data(), C.data(), M, N, K);
    for (int i = 0; i < M*N; ++i) CHECK_CLOSE(C[i], ref[i], 1e-3);
}

TEST(gpu_gemm_throughput_exceeds_cpu) { // measured on the reader's
    ↪ GPU
    const int M = 1024, N = 1024, K = 1024;
    std::vector<float> A(M*K, 0.5f), B(K*N, 0.25f), C(M*N);
    const double flops = 2.0 * M * N * K;
```

```

    auto now = []{ return std::chrono::high_resolution_clock::now(); };

    auto t0 = now();
    for (int r = 0; r < 10; ++r) gpu_gemm(A.data(), B.data(), C.data(), M, N, K);
    double gpu_gflops = flops / (std::chrono::duration<double>(now()-t0).count()
    ↪ /10) / 1e9;

    auto t1 = now();
    matmul_blocked(C.data(), A.data(), B.data(), M, N, K); // Unit 3 CPU
    ↪ baseline
    double cpu_gflops = flops / std::chrono::duration<double>(now()-t1).count() / 1
    ↪ e9;

    std::printf("GEMM 1024^3: %.1f GFLOP/s CPU, %.1f GFLOP/s GPU (%.1fx)\n",
                cpu_gflops, gpu_gflops, gpu_gflops / cpu_gflops);
    CHECK(gpu_gflops > cpu_gflops * 5.0); // the GPU wins by a wide
    ↪ margin
}

```

REFERENCE SOLUTION

src/cuda/gemm.cuh:

```

// include/llm/cuda/gemm.cuh
#pragma once
#include <cuda_runtime.h>
#include "llm/cuda/elementwise.cuh" // DeviceBuffer

namespace llm {

constexpr int TILE = 16;

// C[M,N] = A[M,K] * B[K,N], shared-memory tiled
__global__ void gemm_tiled_kernel(const float* A, const float* B, float* C, int M,
    ↪ int N, int K) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];
    const int ty = threadIdx.y, tx = threadIdx.x;
    const int row = blockIdx.y * TILE + ty;
    const int col = blockIdx.x * TILE + tx;
    float acc = 0.0f;

    const int ntiles = (K + TILE - 1) / TILE;
    for (int t = 0; t < ntiles; ++t) {

```

```

    const int aCol = t * TILE + tx;
    const int bRow = t * TILE + ty;
    As[ty][tx] = (row < M && aCol < K) ? A[(int64_t)row * K + aCol] : 0.0f;
    Bs[ty][tx] = (bRow < K && col < N) ? B[(int64_t)bRow * N + col] : 0.0f;
    __syncthreads(); // tiles filled before
    ↪ use
    for (int k = 0; k < TILE; ++k) acc += As[ty][k] * Bs[k][tx];
    __syncthreads(); // done before overwrite
}
if (row < M && col < N) C[(int64_t)row * N + col] = acc;
}

inline void gpu_gemm(const float* A, const float* B, float* C, int M, int N, int K)
    ↪ {
    DeviceBuffer<float> dA((size_t)M*K), dB((size_t)K*N), dC((size_t)M*N);
    dA.from_host(A); dB.from_host(B);
    dim3 threads(TILE, TILE);
    dim3 blocks((N + TILE - 1) / TILE, (M + TILE - 1) / TILE);
    gemm_tiled_kernel<<<blocks, threads>>>(dA.ptr, dB.ptr, dC.ptr, M, N, K);
    cudaDeviceSynchronize();
    dC.to_host(C);
}

} // namespace llm

```

The correctness test confirms the tiled kernel computes the same product as the CPU, and the throughput test — run on real hardware — shows it doing so many times faster, the GPU’s bandwidth and core count turned loose on the operation that dominates inference. The shared-memory tiling is the whole reason: it is cache blocking made explicit, with `__syncthreads` where the hardware’s automatic cache used to be. With elementwise kernels and a fast GEMM, the building blocks of the forward pass are all on the device. The capstone is to keep the model there and run the whole pass.

GOING FURTHER

- **Register tiling.** Have each thread compute a small $r \times r$ block of outputs from registers, not just one element — the next step toward CUBLAS-level efficiency, multiplying the reuse again.
- **Tensor Cores.** Use the WMMA API (or FP16/BF16 with the Tensor-Core path) to tap the matrix-multiply units directly, where the real peak FLOPS live.
- **Just call CUBLAS.** Benchmark your kernel against `cublasSgemv` to see the gap

a vendor library closes — and decide, as production engines do, where hand-written kernels still earn their place (fused, quantized, attention-specific paths).

15.3 The GPU Forward Pass

BOSS CHALLENGE

Keep the model on the device and run the whole forward pass there: weights uploaded once, activations never leaving the GPU, logits computed by GPU kernels. Prove the port is faithful — logits matching the CPU within floating-point tolerance — and that the dominant kernel runs an order of magnitude faster.

GOAL

The capstone: a **GPU-resident** forward pass. Upload the weights once, gather embeddings, and run every layer — projections and feed-forward as tiled GEMMS, attention and norms and activations as kernels — with the activations staying on the device, copying only the final logits back. Certify it matches the CPU forward within tolerance, far faster.

BACKGROUND

The decisive idea of a GPU forward pass is **residency**: data that stays on the device. Uploading the weights once and keeping them there means the host/device boundary — the slow part of Challenge 15.1 — is crossed only at the edges: token ids in, logits out. Everything between is computed by kernels reading and writing *device* memory, with no round-trips. If each layer copied its activations back to the host and forward again, the transfers would dwarf the computation; keeping the activations resident is what lets the GPU's bandwidth and cores actually be used.

So the forward pass becomes an orchestration of kernels over resident buffers. Upload the token ids and **gather** their embeddings with a kernel, producing the activation x on the device. For each layer, launch the kernels in sequence: an RM-SNORM kernel; a tiled GEMM (15.2) for the Q/K/V projections; a RoPE kernel; an attention kernel; a GEMM for the output projection; an elementwise residual add;

another RMSNORM; GEMMS and a SWIGLU kernel for the feed-forward; and a final residual add. Then a closing RMSNORM and the vocabulary-projection GEMM produce the logits, of which only the needed rows return to the host. Each kernel follows the patterns already built — elementwise ops from 15.1, the tiled matmul from 15.2, attention as a per-head kernel — and operates entirely on device pointers, so the activation flows from kernel to kernel without ever leaving the GPU.

The boss measures the two claims of any port. **Faithfulness:** the GPU logits must match the CPU logits within floating-point tolerance — not bit-exact, because the GPU accumulates in a different order and may use fused multiply-adds, but close, confirming the kernels compute the same mathematics the CPU did. **Throughput:** the operation that dominates, the GEMM, must run far faster on the GPU — an order of magnitude is the routine result, and more with FP16 and Tensor Cores. Together they certify the port: same model, vastly more speed. This is the ceiling the whole book has been climbing toward — the same engine, the same roofline reasoning, now on the hardware built for it.

THE TASK

Implement `upload_model` (each weight to a resident `DeviceBuffer`); a device-pointer `gemm_dev` (the tiled kernel on buffers already on the device, no transfer); and `gpu_forward` orchestrating the resident kernels — embedding gather, per-layer norm/GEMM/RoPE/attention/ residual/FFN, final norm and `lm_head` — returning the last position's logits. Then the boss tests: (1) `gpu_forward` logits match forward within tolerance; (2) the GPU GEMM exceeds the CPU by an order of magnitude.

The bar (the boss): the GPU forward pass reproduces the CPU logits within floating-point tolerance, and the dominant GEMM runs at least an order of magnitude faster than the optimized CPU path.

HINTS

Hint 1 — residency is the whole point

`upload_model` copies weights to `DeviceBuffers` once. `gpu_forward` allocates device buffers for activations and runs every kernel on device pointers; only the final logits use `to_host`. No per-layer transfers.

Hint 2 — a device-pointer GEMM

`gemm_dev(dC, dA, dB, M, N, K)` launches `gemm_tiled_kernel` directly on device buffers — the resident-data version of `gpu_gemm`, without the `cudaMemcpy`. The forward pass uses this between layers.

Hint 3 — tolerance, not equality

The GPU sums in a different order and fuses multiply-adds, so compare logits with `CHECK_CLOSE(..., 1e-2)`, not exact equality. A faithful port lands well within that.

TESTS

```
// tests/test_cuda_forward.cu
// (reuses tiny_model/tiny_cfg, the CPU forward, matmul_blocked)
#include "test_framework.hpp"
#include "llm/cuda/forward.cuh"
#include "llm/model.hpp"
#include <vector>
#include <chrono>
#include <cstdio>
using namespace llm;

TEST(boss_gpu_forward_matches_cpu) {
    const ModelConfig c = tiny_cfg();
    const ModelWeights m = tiny_model(c, 4);
    GpuModel gm = upload_model(m, c);
    const std::vector<int> tokens = {3, 7, 1, 5};
    const int V = c.vocab_size;

    Tensor cpu = forward(m, c, tokens); // CPU reference
    std::vector<float> gpu = gpu_forward(gm, c, tokens); // GPU, last-
    ↪ position logits

    const float* cpu_last = cpu.data_ptr<float>() + (int64_t)(tokens.size() - 1) *
    ↪ V;
    for (int i = 0; i < V; ++i) CHECK_CLOSE(gpu[i], cpu_last[i], 1e-2); //
    ↪ faithful
}

TEST(boss_gpu_gemm_order_of_magnitude) { // measured on the
    ↪ reader's GPU
```

```

const int M = 2048, N = 2048, K = 2048;
std::vector<float> A(M*K, 0.5f), B(K*N, 0.25f), C(M*N);
const double flops = 2.0 * M * N * K;
auto now = []{ return std::chrono::high_resolution_clock::now(); };

auto t0 = now();
for (int r = 0; r < 10; ++r) gpu_gemm(A.data(), B.data(), C.data(), M, N, K);
double gpu_gflops = flops / (std::chrono::duration<double>(now()-t0).count()
↳ /10) / 1e9;

auto t1 = now();
matmul_blocked(C.data(), A.data(), B.data(), M, N, K);
double cpu_gflops = flops / std::chrono::duration<double>(now()-t1).count() / 1
↳ e9;

std::printf("GEMM 2048^3: %.0f GFLOP/s CPU, %.0f GFLOP/s GPU (%.1fx)\n",
            cpu_gflops, gpu_gflops, gpu_gflops / cpu_gflops);
CHECK(gpu_gflops > cpu_gflops * 10.0);           // an order of
↳ magnitude
}

```

REFERENCE SOLUTION

src/cuda/forward.cuh (orchestration; kernels follow 15.1–15.2 and a per-head attention kernel):

```

// include/llm/cuda/forward.cuh (sketch -- device-resident orchestration)
#pragma once
#include "llm/cuda/gemm.cuh"
#include "llm/model.hpp"
#include <vector>

namespace llm {

struct GpuModel {                                // weights resident on the device (
↳ uploaded once)
    int n_layers, hidden, vocab;
    DeviceBuffer<float> tok_emb, final_norm, lm_head;
    std::vector<DeviceBuffer<float>> wq, wk, ww, wo, w_gate, w_up, w_down,
↳ attn_norm, ffn_norm;
    GpuModel() : tok_emb(0), final_norm(0), lm_head(0) {}
};

// launch the tiled GEMM on buffers ALREADY on the device (no transfer)

```

```

inline void gemm_dev(float* dC, const float* dA, const float* dB, int M, int N,
    ↪ int K) {
    dim3 threads(TILE, TILE);
    dim3 blocks((N + TILE - 1) / TILE, (M + TILE - 1) / TILE);
    gemm_tiled_kernel<<<blocks, threads>>>(dA, dB, dC, M, N, K);
}

// gpu_rmsnorm, gpu_rope, gpu_attention, gpu_swiglu, gpu_residual, gpu_embed:
//   elementwise / per-head kernels following the patterns of 15.1, all on device
    ↪ pointers.

// device-resident forward pass; returns the last position's logits on the host.
inline std::vector<float> gpu_forward(GpuModel& gm, const ModelConfig& cfg,
    const std::vector<int>& tokens) {
    const int S = (int)tokens.size(), H = cfg.hidden_size, V = cfg.vocab_size;
    DeviceBuffer<float> x((size_t)S*H), tmp((size_t)S*H), logits((size_t)S*V);
    gpu_embed(gm.tok_emb.ptr, tokens, x.ptr, S, H);           // gather embeddings
    ↪ on device
    for (int l = 0; l < cfg.n_layers; ++l) {
        // attention block: norm -> QKV gemm -> rope -> attention -> wo gemm ->
        ↪ residual
        gpu_rmsnorm(x.ptr, gm.attn_norm[l].ptr, tmp.ptr, S, H, cfg.rms_norm_eps);
        // ... gemm_dev for q,k,v; gpu_rope; gpu_attention; gemm_dev for wo;
        ↪ gpu_residual into x
        // feed-forward block: norm -> gate/up gemm -> swiglu -> down gemm ->
        ↪ residual
        gpu_rmsnorm(x.ptr, gm.ffn_norm[l].ptr, tmp.ptr, S, H, cfg.rms_norm_eps);
        // ... gemm_dev gate/up; gpu_swiglu; gemm_dev down; gpu_residual into x
    }
    gpu_rmsnorm(x.ptr, gm.final_norm.ptr, tmp.ptr, S, H, cfg.rms_norm_eps);
    gemm_dev(logits.ptr, tmp.ptr, gm.lm_head.ptr, S, V, H); // lm_head
    cudaDeviceSynchronize();
    std::vector<float> last(V);                               // copy only the last
    ↪ row back
    cudaMemcpy(last.data(), logits.ptr + (int64_t)(S-1)*V, V*sizeof(float),
    ↪ cudaMemcpyDeviceToHost);
    return last;
}

} // namespace llm

```

Run the boss on a GPU. The resident forward pass reproduces the CPU's logits to within floating-point tolerance — the same model, faithfully ported — and the GEMM that dominates runs an order of magnitude faster, the device's bandwidth and cores finally turned loose on the whole computation. **Boss cleared.**

What you have proven. The engine is not bound to the CPU. Its mathematics port faithfully to the GPU — logits matching within tolerance — and there the dominant kernel runs an order of magnitude faster, with the headroom of FP16 and Tensor Cores still above. The same roofline that guided the CPU work guided this, against far higher ceilings; the same blocking idea that tamed the CPU matmul, realized in shared memory, tamed the GPU one. The engine that came alive on the CPU now flies on the GPU.

GOING FURTHER

- **Fuse and quantize.** Fuse RMSNORM+residual and dequantization into matmul kernels, and run Q4/Q8 weights on the device — the configuration that makes GPU decode approach its memory-bandwidth roof.
- **Flash Attention on the GPU.** Implement the online-softmax attention of Unit 8 as a CUDA kernel with shared-memory tiles — the kernel that makes long-context inference practical at scale.
- **Paged kv and continuous batching on the device.** Port Unit 13's paged cache and scheduler to GPU memory and serve many sequences at once — the full production engine, on the hardware it was designed for.

Unit 15 — Milestone check

MILESTONE CHECK

The engine runs on the GPU:

- **CUDA foundations** (15.1) — device memory, host/device transfers, and kernels run by thousands of threads, with the elementwise ops ported;
- **a tiled GEMM** (15.2) — shared-memory blocking, the explicit-cache analogue of Unit 3, on the kernel that dominates;
- **a GPU-resident forward pass** (15.3) — weights and activations on the device, only logits crossing back; and
- **a boss-grade proof** that the GPU forward matches the CPU within tolerance while the dominant kernel runs an order of magnitude faster.

Afterword

You began with an empty directory and a single command that had to print a green line. You end with a complete, correct, fast LLM inference engine — one that loads real Llama-family models, reads a prompt, and writes a coherent continuation, on the CPU at interactive speed and on the GPU an order of magnitude faster — with no framework beneath it. Every part of it you built yourself, and every part you proved: the **tensors** and aligned memory that hold the numbers; the **kernels**, **matmul**, and **simd** that move them; the **tokenizer** that turns text into ids; the **transformer** — attention, **ROPE**, **FFN**, the residual stream — that is the model itself; **mixture-of-experts** routing; the **kv cache** and **Flash Attention** that make generation tractable; **quantization** and **GGUF loading** that bring real weights down to size and into memory; **sampling** and the **generation loop** that make it speak; **speculative decoding** that makes it speak faster; **paged caching** and **continuous batching** that let it serve a crowd; the **profiling**, **roofline**, and **threading** that made it fast; and the **CUDA** port that let it fly.

The method was always the same. Each unit posed challenges, and each challenge a measured bar a correct solution had to clear — zero-copy proven by a byte counter, the cache proven equal to recomputation, Flash Attention proven exact at scale, speculative decoding proven identical to plain decoding, the server proven correct per request and cheaper in total. Nothing was taken on faith; everything was made to pass a test that could fail. That discipline is the real lesson, more durable than any one kernel: build the smallest thing that works, prove it, and only then build on it.

What you have is not a toy and not a black box. It is an engine you understand at every level, from the byte layout of a quantized weight block to the scheduling of a batched server — which means you can change it. Bring up a new architecture by editing the forward pass; add a sampler; write a fused quantized kernel; port the paged cache to the GPU; chase the roofline until decode runs at the speed of memory. The engine is complete, but it is yours now, and it is a starting point as much as an ending. You know how inference works because you built it. Go further.